BlindSpy

Classifying encrypted traffic (ISCXVPN2016) from 23 flow timing features

Final report, standalone edition

Readable without the code repository: git details and file paths left out. Built 2026-11-01.

How to read this report

Every experiment in this project has an ID, E00 to E17, and the report uses these IDs throughout: in headings ("4.6 E04: KNN with a signed log transform"), in the text ("E04 lifted KNN by ..."), in table rows and in run names such as E07-xgb-B-15s, which reads "experiment E07, XGBoost, task B, 15 s files". The table below says what each ID is, in which order the experiments were run and where the report describes them.

ID	What it is	Order run	Section	Status
E00	Data check: column names, file families, odd values, duplicates, sessions; data version v1	1	2, 4.0	done
E01	Majority-class baseline, the floor every model must beat	2	4.1	done
E02	Decision tree (C4.5 approximation); also the paper benchmark with J48 settings	2	4.2, 4.3	done
E03	KNN, logistic regression and SVM-RBF baselines; also the paper benchmark with IBk settings	2	4.2, 4.3	done
E04	KNN with a signed log transform and two scalers	3	4.6	done
E05	Random Forest, 300 trees	4	4.7	done
E06	Gradient boosting: XGBoost and LightGBM with default settings	5	4.8	done
E07	Hyperparameter tuning with Optuna of the best ensembles and KNN	6	4.9	done
E08	Class balancing: class weights, oversampling, SMOTE	7	4.10	done
E09	Feature-group ablation of the final models (analysis only)	11	4.14	done
E10	Task B as one 14-class model or two steps	8	4.11	done
E11	Session-aware split	-	4.17	not run: the files hold no session information
E12	Final test: the 30 final models scored once on the test set	17	4.21, 5	done
E13	Learning curve of the final models (analysis only)	12	4.15	done
E14	Split-seed stability of the final models (analysis only)	13	4.16	done
E15	Calibration of the final models' confidence, on validation	16	4.20	done
E16	Reproducing Isnan et al. (2026) on the Kaggle copy, then correcting it	15	4.18	done
E17	The same tuning budget for every method type	9	4.12	done

The IDs are not in the order the work was done. Each ID was given when the experiment was planned. The plan changed as results came in: E17 was added after E10, and the analyses E09, E13 and E14 had to wait until the final models were chosen. The real order was:

1. E00 (data check, data version v1)
2. E01, E02 and E03 (baselines), with the paper benchmark
3. E04 (KNN with a signed log)
4. E05 (Random Forest)
5. E06 (gradient boosting)
6. E07 (tuning)
7. E08 (class balancing)
8. E10 (one model or two steps for task B)
9. E17 (equal tuning budget)
10. final choice of models and the comparison set (Section 4.13)
11. E09 (feature-group ablation)
12. E13 (learning curve)
13. E14 (split-seed stability)
14. E11 recorded as not run (no session information)

15. E16 (reproducing Isnan et al.)
16. E15 (calibration, on validation, before the test set)
17. E12 (final test: the test set opened once)

Three steps have no ID of their own: the paper benchmark (part of E02 and E03, Section 4.3), the storage checks (Sections 4.4 and 4.19) and the final choice of models with the comparison set (Section 4.13). Section 4 is ordered by ID; the order list at its start repeats the sequence above.

1 Introduction

1.1 Problem

Most network traffic is now encrypted. Deep packet inspection, which classifies traffic by reading packet contents, no longer works when the contents are hidden. Network operators still need to know what kind of traffic a connection carries (for quality of service, access control and security monitoring) and whether it is routed through a VPN.

Applications leave a timing pattern even when their contents are encrypted. A voice call sends small packets at a steady rhythm, video streaming arrives in bursts, and a file transfer runs at full speed with almost no idle time. BlindSpy trains classifiers on these patterns only: 23 flow timing statistics per connection, never the payload.

We use the ISCXVPN2016 dataset [2] published with Draper-Gil et al. [1]. Each row is one flow described by 23 time-related features and a label. The dataset is provided in four versions, built with flow timeouts of 15, 30, 60 and 120 seconds. The timeout is a setting of the flow builder (it decides when an inactive flow is closed); it is not the duration of each flow.

1.2 Tasks

We follow the three tasks of the original paper [1], each scored on its own:

Task	Description	Scored by
A1	One binary classifier: VPN vs non-VPN	Precision and recall per class, macro F1, ROC-AUC
A2	Two separate 7-class classifiers, one trained on VPN flows and one on non-VPN flows	Each model scored separately; never merged into one score
B	One 14-class classifier over VPN and non-VPN traffic together	Macro F1 and per-class scores over all 14 labels

The seven traffic types are browsing, email, chat, streaming, file transfer, VoIP and P2P; task B uses each type twice (VPN and non-VPN).

1.3 What BlindSpy can and cannot claim

- It predicts the traffic type from aggregate timing statistics and never reads packet contents.
- It does not prove that traffic is encrypted.
- The data was captured in 2016, in a lab, with specific applications and one VPN setup. Results may not hold for current applications, VPN providers or networks. The conclusion repeats this.
- The files carry no session or capture identifiers, so flows recorded in one session can fall on both sides of a random split. Holding out contiguous blocks of the files lowers out-of-fold accuracy by 0.07-0.26 (Section 7.2), so our scores may be optimistic for traffic from captures the models have never seen.
- Every score is measured on held-out rows of the same captures; none on independently captured traffic.

1.4 Related work

Study	Data and setup	Models	Reported result
Draper-Gil et al. 2016 [1]	ISCXVPN2016 ARFF files, tasks A1, A2, B, each timeout separately; Weka, 10-fold cross-validation	C4.5, KNN	A1 at 15 s, C4.5: precision 0.89 (VPN) and 0.906 (non-VPN); KNN: 0.848 and 0.846. A2 at 15 s, C4.5: average precision 0.84 (VPN model), 0.89 (non-VPN model). B, best timeout: average precision 0.783 (C4.5), 0.711 (KNN)
Bagui et al. 2017 [12]	Same time-related features	Random Forest, gradient boosting and others	About 90% (cited from the public abstract only; the full paper was not available to us, so no further detail is claimed)
Jorgensen et al. 2023 [14]	ISCXVPN2016 captures	Uncertainty-aware traffic labelling	Documents quality problems in the dataset's captures (see Section 7)
Isnan et al. 2026 [13]	Kaggle copy of the dataset, 59,706 rows; 18,719 duplicates removed, leaving 40,987; 14 classes; 80/20 split	Random Forest, 300 trees	88.01% accuracy

Each paper value is recorded with its page and whether it is stated in the text or read from a figure.

Why published scores (80–99%) are not directly comparable. Studies on this dataset differ in the split (k-fold vs a single hold-out), the class setup (binary, 7-class or 14-class), whether duplicates are removed, and whether flows from different timeout versions are mixed. Each of these can move accuracy by several points, so we compare only under matched protocols.

The Isnan et al. data. The Kaggle file used by Isnan et al. is the eight A2 files (VPN and non-VPN at all four timeouts) stacked together: compared as a multiset, its 59,706 rows equal the rows of those files (5,151 + 5,631 + 8,965 + 9,793 + 6,917 + 7,734 + 8,580 + 6,935), with no row left over and the same labels. It has 40,987 unique rows, exactly their count. 7,093 distinct flows appear in two to four timeout files, while only 3,676 rows are duplicates within one file (E00, data check). So most of their 18,719 "duplicates" are the same flows repeated across timeout versions, and a random 80/20 split of the stacked file mixes the four flow-building settings in both training and test data. E16 (reproducing Isnan et al.) reproduces their 88.01% (0.8796 on the same split) and shows that with the timeouts kept apart the same method scores 0.82-0.87 (mean 0.8430). A size control in E16 attributes that drop to training-set size, not to the mixing itself: stacking the four files gives 2.3-4.3 times as many training rows, and at equal size the mixed setup scores no higher (0.8395). Their score is therefore not comparable with a single-timeout score. The file also uses the A2 feature set (`total_fiat` and `total_biat` in place of the paper's `std_fiat` and `std_biat`; Section 2).

1.5 Contributions

1. A reproduction of the paper's C4.5 and KNN results under its own 10-fold protocol, compared precision with precision and recall with recall. [E02 (decision-tree baseline), E03 (KNN, logistic regression and SVM baselines), paper benchmark]
2. A leakage-safe protocol (stratified 70/15/15 split, cross-validation inside the training set only, test set scored once) applied to all three tasks and all four timeouts.
3. Stronger models (Random Forest, XGBoost, LightGBM) and a fair comparison of seven method types: every type tuned with the same search budget, plus class balancing and a two-step model for task B. [E04–E08, E10 (one model or two steps for task B), E17 (equal tuning budget)]
4. A reproduction of a recent published result on a copy of this dataset (Isnan et al.), then a corrected version that keeps the timeout files apart, with a control that separates the effect of training-set size from the effect of mixing timeouts. [E16]
5. A check of the dataset itself: whether the ARFF column names match the data, duplicates, odd values and the class mix across timeouts, recorded as versioned data with a step-by-step cleaning log. [E00]
6. Analyses of what the models learned and how far the results can be trusted: feature-group ablation, SHAP, learning curve, split-seed stability and calibration. [E09 (feature-group ablation), E13–E15]
7. Final models scored once on an untouched test set, with 95% bootstrap intervals and 26 paired tests fixed in advance: test macro F1 A1 0.9434, A2-VPN 0.8652, A2-NonVPN 0.9403, B 0.8480. [E12 (final test), Section 5]
8. A command-line predictor (`predict.py`) and a Streamlit demo that classify a new flow file with the final models, report a confidence (calibrated for A1 and A2-VPN, E15 (calibration)) and show the features behind each prediction, and reject a malformed file. [Sections 5.10-5.11]
9. A full history: every run, data version and experiment, traceable to the exact code that produced it.

2 Dataset

2.1 Source and format

We use ISCXVPN2016 [2], published with Draper-Gil et al. [1] by the Canadian Institute for Cybersecurity, University of New Brunswick. The traffic was captured in 2016 in a lab, with and without a VPN, for seven traffic types: browsing, chat, streaming, mail, VoIP, P2P and file transfer. The authors turned the captures into flows with ISCXFlowMeter and published the flows as 20 ARFF files (Weka's text format). Each row is one flow: 23 numeric time-related features (times in microseconds, rates per second) and a label. We use only these files. They hold no payload, addresses, ports or timestamps. The SHA-256 hash of every file is recorded so anyone can check their copy.

Each scenario is published at four flow timeouts (15, 30, 60 and 120 s). The timeout is a setting of the flow builder, not a flow duration. Flows longer than the timeout exist (up to 646 s) because the timeout closes a flow only after it has been inactive for that long.

2.2 Tasks and files

Task	Files	Labels
A1	`Scenario A1-ARFF`, 4 files	VPN, Non-VPN
A2-VPN	`Scenario A2-ARFF`, 4 `-VPN` files	7 VPN traffic types
A2-NonVPN	`Scenario A2-ARFF`, 4 `-NO-VPN` files	7 traffic types
B	`Scenario B-ARFF`, 4 files	14 classes (7 types x VPN / non-VPN)
(B-AllinOne)	`Scenario B-ARFF`, 4 `-AllinOne` files	7 types, VPN and non-VPN merged; inventoried, not a task

file	task	timeout	header	rows	classes	core_task
Scenario A1-ARFF/TimeBasedFeatures-Dataset-15s-VPN.arff	A1	15	T	18,758	2	1
Scenario A1-ARFF/TimeBasedFeatures-Dataset-30s-VPN.arff	A1	30	T	14,651	2	1
Scenario A1-ARFF/TimeBasedFeatures-Dataset-60s-VPN.arff	A1	60	S	15,515	2	1
Scenario A1-ARFF/TimeBasedFeatures-Dataset-120s-VPN.arff	A1	120	T	10,782	2	1
Scenario A2-ARFF/TimeBasedFeatures-Dataset-15s-VPN.arff	A2-VPN	15	T	9,793	7	1
Scenario A2-ARFF/TimeBasedFeatures-Dataset-30s-VPN.arff	A2-VPN	30	T	7,734	7	1
Scenario A2-ARFF/TimeBasedFeatures-Dataset-60s-VPN.arff	A2-VPN	60	T	6,935	7	1
Scenario A2-ARFF/TimeBasedFeatures-Dataset-120s-VPN.arff	A2-VPN	120	T	5,631	7	1
Scenario A2-ARFF/TimeBasedFeatures-Dataset-15s-NO-VPN.arff	A2-NonVPN	15	T	8,965	7	1
Scenario A2-ARFF/TimeBasedFeatures-Dataset-30s-NO-VPN.arff	A2-NonVPN	30	T	6,917	7	1
Scenario A2-ARFF/TimeBasedFeatures-Dataset-60s-NO-VPN.arff	A2-NonVPN	60	T	8,580	7	1
Scenario A2-ARFF/TimeBasedFeatures-Dataset-120s-NO-VPN.arff	A2-NonVPN	120	T	5,151	7	1
Scenario B-ARFF/TimeBasedFeatures-Dataset-15s.arff	B	15	T	18,758	14	1

file	task	timeout	header	rows	classes	core_task
Scenario B-ARFF/TimeBasedFeatures-Dataset-30s.arff	B	30	T	14,651	14	1
Scenario B-ARFF/TimeBasedFeatures-Dataset-60s.arff	B	60	S	15,515	14	1
Scenario B-ARFF/TimeBasedFeatures-Dataset-120s.arff	B	120	T	10,782	14	1
Scenario B-ARFF/TimeBasedFeatures-Dataset-15s-AllinOne.arff	B-AllinOne	15	T	18,758	7	0
Scenario B-ARFF/TimeBasedFeatures-Dataset-30s-AllinOne.arff	B-AllinOne	30	T	14,651	7	0
Scenario B-ARFF/TimeBasedFeatures-Dataset-60s-AllinOne.arff	B-AllinOne	60	S	15,515	7	0
Scenario B-ARFF/TimeBasedFeatures-Dataset-120s-AllinOne.arff	B-AllinOne	120	T	10,782	7	0

Two features of the raw files matter for everything below. A1, A2 and B hold **the same flows** at each timeout: every A1 row matches an A2 row on seven integer features at all four timeouts (100%), and the labels agree key by key (0 disagreements). B and A1 are identical cell for cell except for one corrupt column (Section 2.4). BROWSING and VPN-BROWSING hold exactly 2,500 flows in every raw A2 file, which suggests the dataset authors capped those classes.

2.3 Column-name check

Problem. The first row of A1 15 s has `duration` = 9,368,711 us (9.4 s) but `total_fiat` = 16 and `total_biat` = 4. A total forward gap of 16 us in a 9-second flow is impossible. The paper's Table 2 [1] and the authors' 2016 poster [15] describe the forward and backward inter-arrival features as min, mean, max and std, with no totals. Yet 17 of the 20 headers list `total_fiat` and `total_biat` and no std. The three 60 s files of A1 and B have a second header that matches the paper.

Method. We read every file under both name lists. On every row we tested identities that hold whatever the traffic: min <= mean <= max for each gap statistic, 0 <= std <= max - min, mean flow gap x (packets - 1) = duration, and, for totals, total / mean + 1 per direction = packet count. Means of 10^7 or more are stored to 3 significant figures in the files, so a tolerance of 0.5% is allowed. We also tested the reading that `total_fiat` is a forward packet count.

check	A1 15 s (T)	A1 15 s (S)	A2-VPN 15 s (T)	A2-VPN 15 s (S)	A2-NonVPN 15 s (T)	A2-NonVPN 15 s (S)	B 15 s (T)	B 15 s (S)
fiat: min <= mean <= max	0.0	100.0	100.0	18.8	100.0	5.24	0.0	100.0
biat: min <= mean <= max	0.01	100.0	100.0	14.07	100.0	4.06	0.01	100.0
flowiat: min <= mean <= max	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
active: min <= mean <= max	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
idle: min <= mean <= max	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
mean_flowiat x (packets - 1) = duration (+-1%)	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
bytes per packet in 40-1500	97.72	97.72	99.26	99.26	96.06	96.06	97.72	97.72
min_flowiat <= min(min_fiat, min_biat)	100.0	100.0	100.0	80.98	100.0	83.1	100.0	100.0
total_fiat <= duration	100.0	-	100.0	-	100.0	-	100.0	-
total/mean + 1, fwd + bwd = packets (+-2%)	0.0	-	100.0	-	100.0	-	0.0	-
total_fiat as packet count (+-10%)	1.98	-	0.0	-	0.04	-	1.98	-
0 <= std_fiat <= max - min	-	100.0	-	0.0	-	0.0	-	100.0
0 <= std_biat <= max - min	-	100.0	-	0.0	-	0.0	-	100.0

The same pattern holds at every timeout (`column_name_check` in the manifest has all 16 files).

Result. The headers are wrong for 9 of the 12 A1/B files, and the fault is a shift, not a shuffle. These files hold the paper's features in the order min, max, mean, std for fiat and biat, but their header names them total, min, max, mean. Under the corrected names every identity passes on 100% of rows; under the header names, min <= mean <= max passes on at most 0.02%. A2 files are named correctly and hold a different pair of features: `total_fiat` and `total_biat` (the sum of the forward and backward gaps) instead of `std_fiat` and `std_biat`. The packet-count reading passes on at most 2% of rows in any file, so it is ruled out. The column map (`COLUMN_MAP`) renames the 9 shifted files. It maps `total_fiat`->`min_fiat`, `total_biat`->`min_biat`, `min_fiat`->`max_fiat`, `min_biat`->`max_biat`, `max_fiat`->`mean_fiat`, `max_biat`->`mean_biat`, `mean_fiat`->`std_fiat`, `mean_biat`->`std_biat`; the other 15 names are unchanged. All feature names in this report (importances, SHAP, ablation) are the corrected ones.

Units and value ranges. Table 2 gives no units. The data does: with packets = flow packets per second x duration / 10^6, the mean flow gap x (packets - 1) equals the duration within 1% on 100% of rows in all 16 files, which holds only if times are in microseconds and rates per second. Every measured gap, active or idle value is at most the flow's duration (100% of values), as Table 2's definitions require. Under the corrected names the A1/B feature list is exactly Table 2's 23 features; the A2 list differs only in `total_fiat` and `total_biat` .

feature	min	median	p99	max	exactly_-1	other_negative	% <= duration
duration	0	14,422,310	119,998,194	646,565,983	0	0	-
min_fiat	-1	112	45,211,908	119,999,708	6,308	0	100.0
min_biat	-1	45	45,213,098	119,999,620	10,522	0	100.0
max_fiat	-1	1,650,890	104,244,407	646,287,731	6,308	0	100.0
max_biat	-1	2,294,484	63,987,999	600,109,667	10,522	0	100.0
mean_fiat	0	158,066	45,200,000	152,000,000	0	0	100.0
mean_biat	0	83,359	45,200,000	120,000,000	0	0	100.0
std_fiat	0	27,808	34,000,000	264,000,000	0	0	-
std_biat	0	24,804	25,700,000	98,000,000	0	0	-
flowPktsPerSecond	0	8.96615	57,143	2,000,000	0	0	-
flowBytesPerSecond	0	1,695	6,140,974	617,000,000	0	0	-
min_flowiat	-1,247	21	1,076,195	59,947,704	224	405	100.0
max_flowiat	-1	962,728	109,547,639	646,287,731	45	0	100.0
mean_flowiat	0	144,230	19,900,000	64,700,000	0	0	100.0
std_flowiat	0	192,278	33,700,000	204,000,000	0	0	-
min_active	-1	5,949,131	120,988,572	646,565,983	28,663	0	100.0
mean_active	0	0	110,000,000	647,000,000	0	0	100.0
max_active	-1	10,316,202	122,147,146	646,565,983	28,663	0	100.0
std_active	0	0	23,200,000	184,000,000	0	0	-
min_idle	-1	4,993,314	120,661,941	646,287,731	28,663	0	100.0
mean_idle	0	0	109,000,000	646,000,000	0	0	100.0
max_idle	-1	10,000,610	121,615,880	646,287,731	28,663	0	100.0
std_idle	0	0	23,100,000	185,000,000	0	0	-

This also explains the earlier puzzle that A1 and A2 rows never match. As published (names ignored), 0 A1 rows equal an A2 row in all 23 values at any timeout, and only 2,364 of 18,758 at 15 s (1,501, 2,190 and 494 at 30, 60 and 120 s) contain the same set of values in another order. Two of the 23 values mean different things in the two families, and 9 of the 12 A1/B headers shift the names.

2.4 File-family decision

Because A1/B and A2 hold the same flows with two different feature pairs, we had three options: use each task's own files, rebuild A1 and B from the A2 files, or merge both into 25 features. We use **each task's own files**: A1 and B with the paper's 23 features, A2 with its 23. These are the files the paper trained on for each scenario, so our comparison with it stays matched. A rebuild would have replaced two of the paper's features, and a merge is ambiguous wherever a key repeats. For the two-step classifier in E10 (one model or two steps for task B), both steps use the A1/B layout: the 7-class step is trained on B rows split by VPN (the same flows as A2).

One column needed repair. In A1 120 s, `flowPktsPerSecond` disagrees with A2 on 99.97% of matched rows, while B 120 s, identical to A1 120 s in the other 22 columns of every row, agrees with A2 on 100%. We restore the column from B 120 s, which changes 10,288 of 10,782 rows.

2.5 Cleaning rules

Every rule is written in the project's configuration and is applied by the preparation script in this order. Each file gets one line per rule in the cleaning log, including rules that change nothing.

#	Step	Rule	Reason
1	parse	Strip trailing commas from header lines; skip comma-only data lines	Export glitch; a comma-only line (one in each 60 s file of A1 and B) is not a flow
2	column map	Rename with `COLUMN_MAP`	Section 2.3
3	restore	A1 120 s `flowPktsPerSecond` from B 120 s	Section 2.4
4-5	missing, infinite	Count; none found	No imputation needed
6-9	-1 codes	Keep	-1 means "not measurable": fewer than 2 packets in a direction, a single-packet flow, or no active/idle period. The matching mean or total is 0 on the same rows
10	single-packet flows	Keep `duration` = 0 flows	Real flows; dropping them would bias the data toward long flows
11	negative `min_flowiat`	Keep (down to -1,247 us)	Out-of-order capture timestamps
12	rounding	Keep means, stds and rates >= 10^7 as stored	Already rounded to 3 significant figures in the source; can't be undone
13	packet size	Keep flows outside 40-1,500 B per packet	Header-only packets (down to 30 B) and segments merged at capture (up to about 7 kB) are real
14	exact duplicates	Drop within each file before splitting, keep the first copy	A copy in both train and test would score the model on rows it trained on
15	conflicting duplicates	Keep (one row per label)	Real ambiguity; removing it would make the test set easier than reality
16-19	not done	No normalization, no feature selection, no outlier removal, no class balancing	Scaling and resampling run inside the model pipeline on training folds only; all 23 features are kept

Rows each kept-as-is rule applies to:

rule	A1	A2-VPN	A2-NonVPN	B
fiat_not_measurable	8,846	5,120	3,726	8,846
biat_not_measurable	13,857	8,056	5,801	13,857
flowiat_not_measurable	2,108	1,205	903	2,108
active_idle_not_measurable	32,228	15,753	16,475	32,228
single_packet	2,110	1,207	903	2,110
negative_flowiat	586	300	286	586
rounding	15,217	7,979	7,234	15,217
packet_size	1,380	397	983	1,380

2.6 Dataset summary

file	initial_rows	comma_only_lines_skipped	exact_duplicates_removed	conflicting_rows_kept	conflicting_groups	final_rows	missing_values	features	classes	largest_to_smallest_class	train	val	test
A1 15 s	18,758	0	720	46	23	18,038	0	23	2	1.08:1	12,626	2,706	2,706
A1 30 s	14,651	0	1,041	44	22	13,610	0	23	2	1.03:1	9,526	2,042	2,042
A1 60 s	15,516	1	838	94	47	14,677	0	23	2	1.26:1	10,273	2,202	2,202
A1 120 s	10,782	0	1,258	62	31	9,524	0	23	2	1.08:1	6,666	1,429	1,429
A2-VPN 15 s	9,793	0	413	53	24	9,380	0	23	7	5.79:1	6,566	1,407	1,407
A2-VPN 30 s	7,734	0	790	39	17	6,944	0	23	7	8.39:1	4,860	1,042	1,042
A2-VPN 60 s	6,935	0	412	61	28	6,523	0	23	7	12.50:1	4,565	979	979
A2-VPN 120 s	5,631	0	983	111	52	4,648	0	23	7	15.48:1	3,252	698	698
A2-NonVPN 15 s	8,965	0	271	10	3	8,694	0	23	7	11.85:1	6,085	1,304	1,305
A2-NonVPN 30 s	6,917	0	223	9	3	6,694	0	23	7	32.47:1	4,685	1,004	1,005
A2-NonVPN 60 s	8,580	0	374	34	15	8,206	0	23	7	11.31:1	5,744	1,231	1,231
A2-NonVPN 120 s	5,151	0	210	9	3	4,941	0	23	7	14.37:1	3,458	741	742
B 15 s	18,758	0	684	100	41	18,074	0	23	14	11.85:1	12,651	2,711	2,712
B 30 s	14,651	0	1,013	83	33	13,638	0	23	14	32.47:1	9,546	2,046	2,046
B 60 s	15,516	1	786	174	75	14,729	0	23	14	12.69:1	10,309	2,210	2,210
B 120 s	10,782	0	1,193	173	77	9,589	0	23	14	17.24:1	6,711	1,439	1,439

2.7 Data funnel (data v1)

Only two steps remove rows: the parser (two comma-only lines in total) and exact de-duplication (11,209 rows). Over the 16 core files, 179,120 raw data lines become 167,909 cleaned flows; no value is changed except the restored A1 120 s column.

file	raw data lines	parsed	after duplicates	final	train	val	test
A1 15 s	18,758	18,758	18,038	18,038	12,626	2,706	2,706
A1 30 s	14,651	14,651	13,610	13,610	9,526	2,042	2,042
A1 60 s	15,516	15,515	14,677	14,677	10,273	2,202	2,202
A1 120 s	10,782	10,782	9,524	9,524	6,666	1,429	1,429
A2-VPN 15 s	9,793	9,793	9,380	9,380	6,566	1,407	1,407
A2-VPN 30 s	7,734	7,734	6,944	6,944	4,860	1,042	1,042
A2-VPN 60 s	6,935	6,935	6,523	6,523	4,565	979	979
A2-VPN 120 s	5,631	5,631	4,648	4,648	3,252	698	698
A2-NonVPN 15 s	8,965	8,965	8,694	8,694	6,085	1,304	1,305
A2-NonVPN 30 s	6,917	6,917	6,694	6,694	4,685	1,004	1,005
A2-NonVPN 60 s	8,580	8,580	8,206	8,206	5,744	1,231	1,231
A2-NonVPN 120 s	5,151	5,151	4,941	4,941	3,458	741	742
B 15 s	18,758	18,758	18,074	18,074	12,651	2,711	2,712
B 30 s	14,651	14,651	13,638	13,638	9,546	2,046	2,046
B 60 s	15,516	15,515	14,729	14,729	10,309	2,210	2,210

file	raw data lines	parsed	after duplicates	final	train	val	test
B 120 s	10,782	10,782	9,589	9,589	6,711	1,439	1,439
All 16 files	179,120	179,118	167,909	167,909	117,523	25,191	25,195

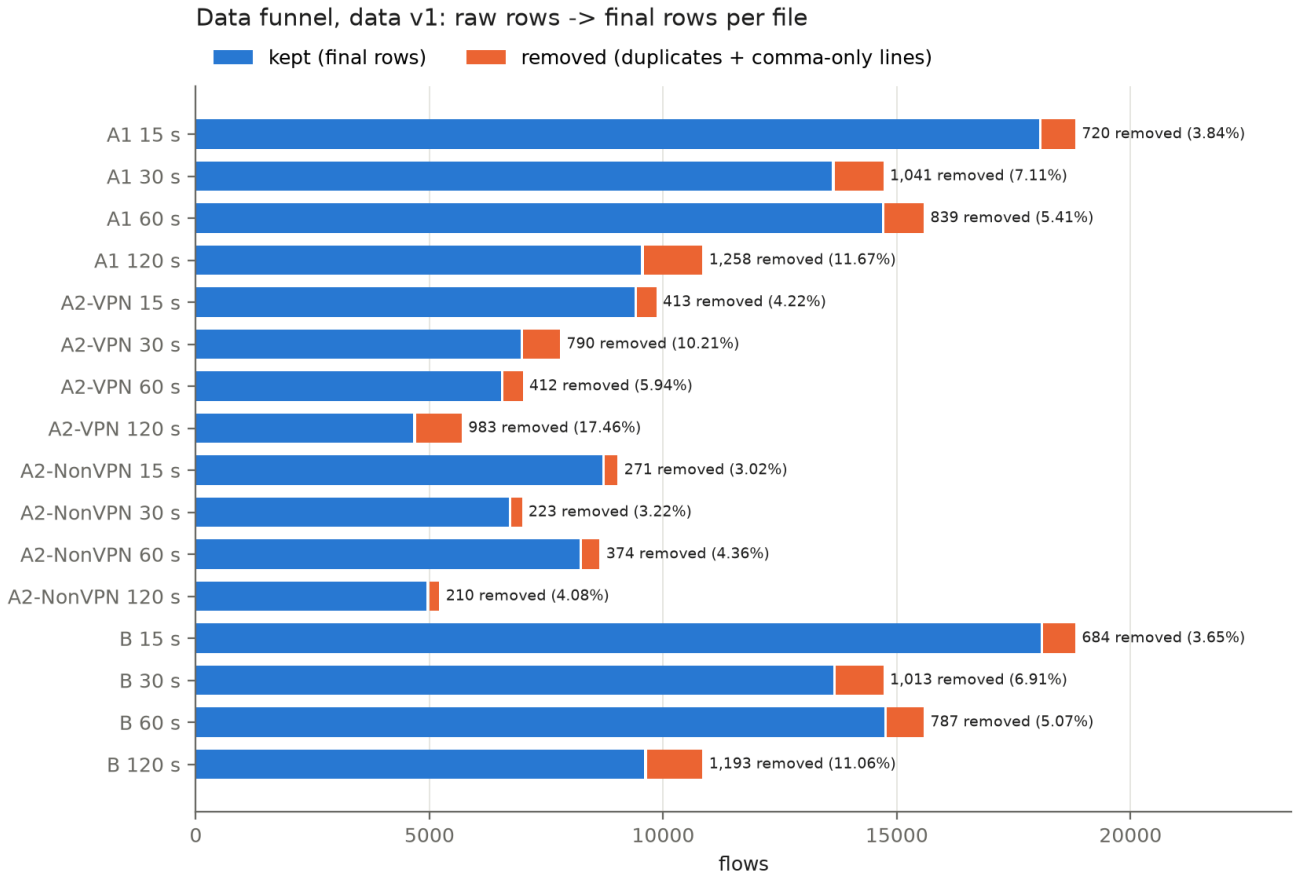


Figure 1. Data funnel

Removals are uneven by class, so they change the class balance.

A1

class	15 s	30 s	60 s	120 s
Non-VPN	278 of 8,965	229 of 6,917	393 of 8,580	216 of 5,151
VPN	442 of 9,793	812 of 7,734	445 of 6,935	1,042 of 5,631

A2-VPN

class	15 s	30 s	60 s	120 s
VPN-BROWSING	0 of 2,500	0 of 2,500	37 of 2,500	256 of 2,500
VPN-CHAT	38 of 1,196	38 of 780	36 of 514	39 of 349
VPN-STREAMING	0 of 475	0 of 298	0 of 197	0 of 145
VPN-MAIL	59 of 491	483 of 848	59 of 383	420 of 722
VPN-VOIP	5 of 2,271	5 of 1,299	9 of 1,620	5 of 386
VPN-P2P	262 of 928	230 of 851	218 of 823	215 of 813
VPN-FT	49 of 1,932	34 of 1,158	53 of 898	48 of 716

A2-NonVPN

class	15 s	30 s	60 s	120 s
BROWSING	0 of 2,500	0 of 2,500	0 of 2,500	0 of 2,500
CHAT	3 of 890	7 of 595	24 of 778	3 of 242
STREAMING	32 of 482	31 of 342	31 of 252	31 of 208
MAIL	11 of 249	4 of 81	117 of 849	11 of 185
VOIP	5 of 2,826	3 of 1,438	36 of 1,829	3 of 392
P2P	59 of 1,000	7 of 1,000	4 of 1,000	2 of 1,000
FT	161 of 1,018	171 of 961	162 of 1,372	160 of 624

B

class	15 s	30 s	60 s	120 s
BROWSING	0 of 2,500	0 of 2,500	0 of 2,500	0 of 2,500
CHAT	3 of 890	7 of 595	24 of 778	3 of 242
STREAMING	32 of 482	31 of 342	31 of 252	31 of 208
MAIL	11 of 249	4 of 81	117 of 849	11 of 185
VOIP	5 of 2,826	3 of 1,438	36 of 1,829	3 of 392
P2P	59 of 1,000	7 of 1,000	4 of 1,000	2 of 1,000
FT	161 of 1,018	171 of 961	162 of 1,372	160 of 624
VPN-BROWSING	0 of 2,500	0 of 2,500	37 of 2,500	256 of 2,500
VPN-CHAT	38 of 1,196	38 of 780	36 of 514	39 of 349
VPN-STREAMING	0 of 475	0 of 298	0 of 197	0 of 145
VPN-MAIL	59 of 491	483 of 848	59 of 383	420 of 722
VPN-VOIP	5 of 2,271	5 of 1,299	9 of 1,620	5 of 386
VPN-P2P	262 of 928	230 of 851	218 of 823	215 of 813
VPN-FT	49 of 1,932	34 of 1,158	53 of 898	48 of 716

file	groups	rows	test rows with a conflicting twin in train
A1 15 s	23	46	4
A1 30 s	22	44	8
A1 60 s	47	94	9
A1 120 s	31	62	7
A2-VPN 15 s	24	53	9
A2-VPN 30 s	17	39	4
A2-VPN 60 s	28	61	6
A2-VPN 120 s	52	111	11
A2-NonVPN 15 s	3	10	3
A2-NonVPN 30 s	3	9	2
A2-NonVPN 60 s	15	34	6
A2-NonVPN 120 s	3	9	1
B 15 s	41	100	15
B 30 s	33	83	15
B 60 s	75	174	25
B 120 s	77	173	21

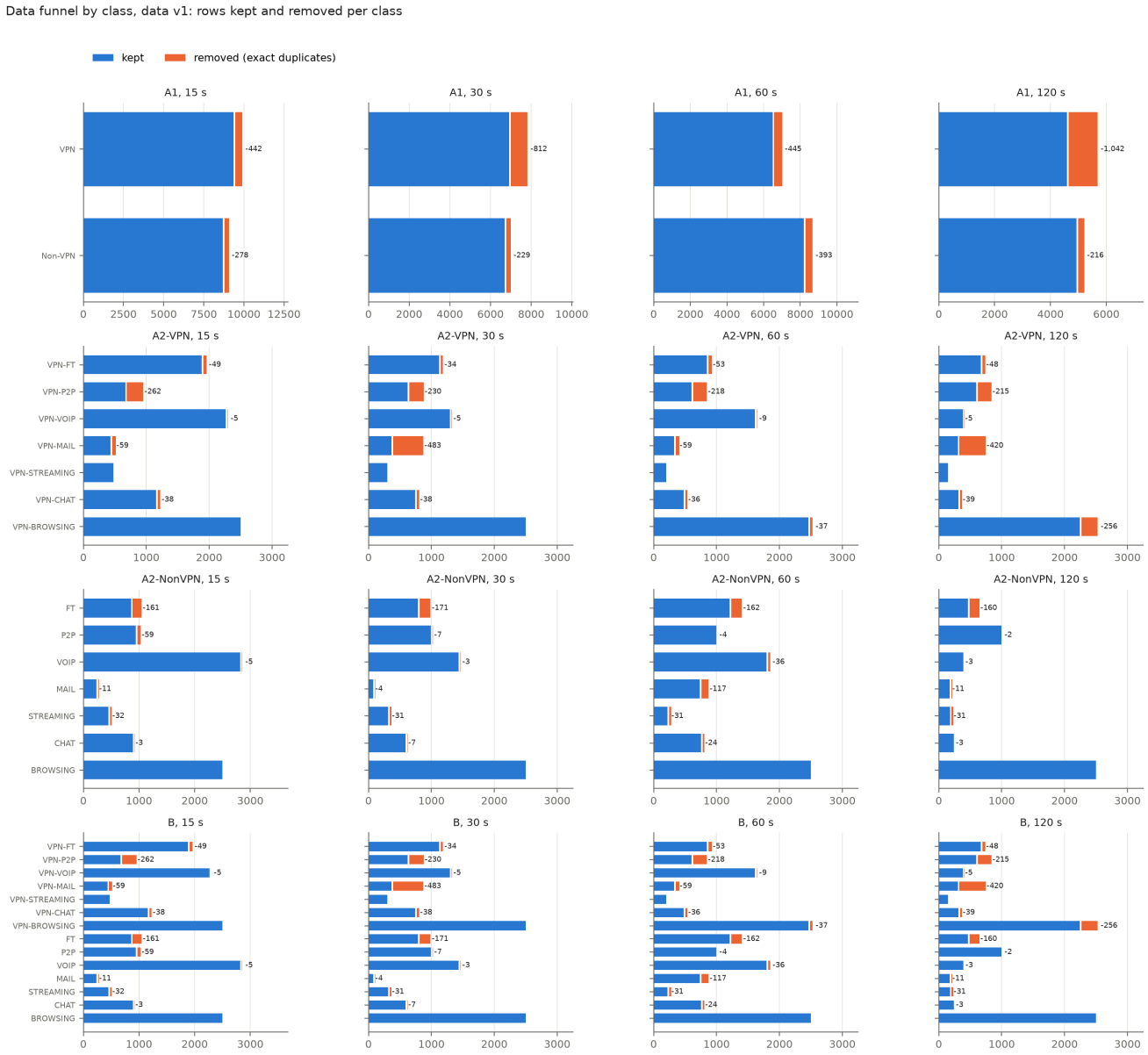


Figure 2. Data funnel by class

Per class, from raw rows to the final train / validation / test sets:

A1

class	timeout	raw	final	train	val	test
Non-VPN	15 s	8,965	8,687	6,081	1,303	1,303
VPN	15 s	9,793	9,351	6,545	1,403	1,403
Non-VPN	30 s	6,917	6,688	4,681	1,004	1,003
VPN	30 s	7,734	6,922	4,845	1,038	1,039
Non-VPN	60 s	8,580	8,187	5,731	1,228	1,228
VPN	60 s	6,935	6,490	4,542	974	974
Non-VPN	120 s	5,151	4,935	3,454	741	740
VPN	120 s	5,631	4,589	3,212	688	689

A2-VPN

class	timeout	raw	final	train	val	test
VPN-BROWSING	15 s	2,500	2,500	1,750	375	375

class	timeout	raw	final	train	val	test
VPN-CHAT	15 s	1,196	1,158	810	174	174
VPN-STREAMING	15 s	475	475	333	71	71
VPN-MAIL	15 s	491	432	302	65	65
VPN-VOIP	15 s	2,271	2,266	1,586	340	340
VPN-P2P	15 s	928	666	466	100	100
VPN-FT	15 s	1,932	1,883	1,319	282	282
VPN-BROWSING	30 s	2,500	2,500	1,750	375	375
VPN-CHAT	30 s	780	742	520	111	111
VPN-STREAMING	30 s	298	298	208	45	45
VPN-MAIL	30 s	848	365	255	55	55
VPN-VOIP	30 s	1,299	1,294	906	194	194
VPN-P2P	30 s	851	621	435	93	93
VPN-FT	30 s	1,158	1,124	786	169	169
VPN-BROWSING	60 s	2,500	2,463	1,724	369	370
VPN-CHAT	60 s	514	478	334	72	72
VPN-STREAMING	60 s	197	197	139	29	29
VPN-MAIL	60 s	383	324	227	49	48
VPN-VOIP	60 s	1,620	1,611	1,127	242	242
VPN-P2P	60 s	823	605	423	91	91
VPN-FT	60 s	898	845	591	127	127
VPN-BROWSING	120 s	2,500	2,244	1,570	337	337
VPN-CHAT	120 s	349	310	216	47	47
VPN-STREAMING	120 s	145	145	101	22	22
VPN-MAIL	120 s	722	302	212	45	45
VPN-VOIP	120 s	386	381	267	57	57
VPN-P2P	120 s	813	598	418	90	90
VPN-FT	120 s	716	668	468	100	100

A2-NonVPN

class	timeout	raw	final	train	val	test
BROWSING	15 s	2,500	2,500	1,750	375	375
CHAT	15 s	890	887	621	133	133
STREAMING	15 s	482	450	315	67	68
MAIL	15 s	249	238	166	36	36
VOIP	15 s	2,826	2,821	1,975	423	423
P2P	15 s	1,000	941	659	141	141
FT	15 s	1,018	857	599	129	129
BROWSING	30 s	2,500	2,500	1,750	375	375
CHAT	30 s	595	588	412	88	88
STREAMING	30 s	342	311	217	47	47
MAIL	30 s	81	77	53	12	12

class	timeout	raw	final	train	val	test
VOIP	30 s	1,438	1,435	1,005	215	215
P2P	30 s	1,000	993	695	149	149
FT	30 s	961	790	553	118	119
BROWSING	60 s	2,500	2,500	1,750	375	375
CHAT	60 s	778	754	528	113	113
STREAMING	60 s	252	221	155	33	33
MAIL	60 s	849	732	512	110	110
VOIP	60 s	1,829	1,793	1,255	269	269
P2P	60 s	1,000	996	697	150	149
FT	60 s	1,372	1,210	847	181	182
BROWSING	120 s	2,500	2,500	1,750	375	375
CHAT	120 s	242	239	167	36	36
STREAMING	120 s	208	177	124	26	27
MAIL	120 s	185	174	122	26	26
VOIP	120 s	392	389	273	58	58
P2P	120 s	1,000	998	698	150	150
FT	120 s	624	464	324	70	70

B

class	timeout	raw	final	train	val	test
BROWSING	15 s	2,500	2,500	1,750	375	375
CHAT	15 s	890	887	621	133	133
STREAMING	15 s	482	450	315	68	67
MAIL	15 s	249	238	166	36	36
VOIP	15 s	2,826	2,821	1,975	423	423
P2P	15 s	1,000	941	659	141	141
FT	15 s	1,018	857	600	128	129
VPN-BROWSING	15 s	2,500	2,500	1,750	375	375
VPN-CHAT	15 s	1,196	1,158	810	174	174
VPN-STREAMING	15 s	475	475	333	71	71
VPN-MAIL	15 s	491	432	302	65	65
VPN-VOIP	15 s	2,271	2,266	1,586	340	340
VPN-P2P	15 s	928	666	466	100	100
VPN-FT	15 s	1,932	1,883	1,318	282	283
BROWSING	30 s	2,500	2,500	1,750	375	375
CHAT	30 s	595	588	412	88	88
STREAMING	30 s	342	311	217	47	47
MAIL	30 s	81	77	54	11	12
VOIP	30 s	1,438	1,435	1,005	215	215
P2P	30 s	1,000	993	695	149	149
FT	30 s	961	790	553	119	118

class	timeout	raw	final	train	val	test
VPN-BROWSING	30 s	2,500	2,500	1,750	375	375
VPN-CHAT	30 s	780	742	520	111	111
VPN-STREAMING	30 s	298	298	208	45	45
VPN-MAIL	30 s	848	365	255	55	55
VPN-VOIP	30 s	1,299	1,294	906	194	194
VPN-P2P	30 s	851	621	435	93	93
VPN-FT	30 s	1,158	1,124	786	169	169
BROWSING	60 s	2,500	2,500	1,750	375	375
CHAT	60 s	778	754	528	113	113
STREAMING	60 s	252	221	155	33	33
MAIL	60 s	849	732	512	110	110
VOIP	60 s	1,829	1,793	1,255	269	269
P2P	60 s	1,000	996	698	149	149
FT	60 s	1,372	1,210	847	182	181
VPN-BROWSING	60 s	2,500	2,463	1,724	369	370
VPN-CHAT	60 s	514	478	334	72	72
VPN-STREAMING	60 s	197	197	138	30	29
VPN-MAIL	60 s	383	324	227	48	49
VPN-VOIP	60 s	1,620	1,611	1,127	242	242
VPN-P2P	60 s	823	605	423	91	91
VPN-FT	60 s	898	845	591	127	127
BROWSING	120 s	2,500	2,500	1,750	375	375
CHAT	120 s	242	239	167	36	36
STREAMING	120 s	208	177	124	26	27
MAIL	120 s	185	174	122	26	26
VOIP	120 s	392	389	273	58	58
P2P	120 s	1,000	998	698	150	150
FT	120 s	624	464	324	70	70
VPN-BROWSING	120 s	2,500	2,244	1,570	337	337
VPN-CHAT	120 s	349	310	217	47	46
VPN-STREAMING	120 s	145	145	101	22	22
VPN-MAIL	120 s	722	302	212	45	45
VPN-VOIP	120 s	386	381	267	57	57
VPN-P2P	120 s	813	598	418	90	90
VPN-FT	120 s	716	668	468	100	100

In sentences. For task A1 at 15 s, 18,758 flows were parsed. After removing 720 exact duplicates (442 VPN, 278 non-VPN), 18,038 unique flows remained. All 23 features were kept without feature selection or normalization, and the flows were split 12,626 / 2,706 / 2,706 into training, validation and test sets. Duplicates are most common at the 120 s timeout: 1,258 of 10,782 A1 flows (11.67%) and 983 of 5,631 A2-VPN flows (17.46%). They hit some classes much harder than others: VPN-MAIL lost 483 of 848 flows at 30 s and 420 of 722 at 120 s, FT lost 160-171 flows at every timeout, and BROWSING lost none. A1 loses slightly more rows than B at the same timeout (720 vs 684 at 15 s) because flows that share features but differ in traffic type

become exact duplicates once both are labelled with the same VPN / non-VPN label. After cleaning, no exact duplicate crosses train and test in any file; 1-25 test flows per file have a twin in training with the same features but a different label.

Deliberately not done, and why. No values were imputed (none are missing). No -1 code was replaced and nothing was set to zero, because -1 has a meaning. No flag columns were added, since the matching zero mean already marks the same rows and flags would change the paper's 23-feature set. No rows were dropped as outliers, because long, bursty or single-packet flows are real traffic. No features were scaled or selected and no classes were rebalanced at data level; all of that happens inside model pipelines, fitted on training data only (Section 3).

2.8 Timeouts

Within each task the feature names, their order and their units are the same at all four timeouts. The number of flows falls as the timeout grows (A1: 18,038 at 15 s to 9,524 at 120 s), and the class mix moves a lot. One class's share changes by up to 24.6 percentage points in A2-NonVPN (VOIP 32.5% at 15 s vs 7.9% at 120 s; BROWSING 28.8% vs 50.6%), 21.6 in A2-VPN, 12.2 in B and 7.6 in A1. A difference in scores between timeouts is therefore confounded by class mix, and we do not attribute it to the timeout alone.



Figure 3. Class mix per timeout

The 60 s files differ in what some classes look like. At 15, 30 and 120 s, non-VPN VOIP flows send about 100 packets per second with a 20 ms median forward gap, and VPN-VOIP flows about 2 with a gap of about 960 ms. At 60 s the two are almost reversed (2.1 packets/s and 962 ms for VOIP, 16.5 packets/s and 18 ms for VPN-VOIP), and non-VPN FT drops from 1,237-3,739 to 56.5 packets/s. It is not a label swap. Of 7,041 flows found identically in more than one timeout file, 210 change label between timeouts; they are mostly 2-packet flows, and no VoIP pair is among them. The 60 s files are also the only ones exported with the paper-style header, so they may come from a different export run. The ARFF files can't settle the cause. Results at 60 s are therefore read with care and the issue is listed under Limitations (Section 7).

class, timeout	flows	median packets per second	median mean_fiat (us)	% mean_fiat 15-25 ms	% no active/idle period
VOIP 15s	2,821	100.1	19994.9	78.1	99.4
VOIP 30s	1,435	100.0	19998.1	77.1	98.9
VOIP 60s	1,793	2.1	961624.6	31.2	51.5
VOIP 120s	389	100.0	19999.8	73.5	95.4
FT 15s	857	1236.7	1042.8	1.2	69.4
FT 30s	790	2155.9	768.0	1.5	72.0
FT 60s	1,210	56.5	34444.3	7.8	80.1

class, timeout	flows	median packets per second	median mean_fiat (us)	% mean_fiat 15-25 ms	% no active/idle period
FT 120s	464	3738.5	507.5	0.4	78.9
VPN-VOIP 15s	2,266	2.2	960175.4	42.9	64.4
VPN-VOIP 30s	1,294	2.1	960284.3	37.7	56.6
VPN-VOIP 60s	1,611	16.5	18295.3	15.7	71.6
VPN-VOIP 120s	381	2.1	960800.7	32.3	49.3
VPN-FT 15s	1,883	7.1	192693.4	0.7	55.1
VPN-FT 30s	1,124	4.0	416771.2	0.8	35.0
VPN-FT 60s	845	6.1	286898.0	1.4	45.1
VPN-FT 120s	668	7.5	187957.4	2.1	48.8

2.9 The Kaggle copy used by Isnan et al.

The Kaggle file used by Isnan et al. [13] has 59,706 rows and the A2 header. Compared as a multiset of rows, it equals the eight A2 files (VPN and non-VPN at all four timeouts) stacked: no row is left over at 12 significant digits, and the labels are the same. It has 40,987 unique rows, their count after removing 18,719 duplicates. Only 3,676 of those duplicates lie within one file; 7,093 distinct flows appear in two (2,475), three (1,286) or four (3,332) timeout files. Most of their "duplicates" are therefore the same flow built with different timeouts, and a random 80/20 split of the stacked file mixes all four timeout settings in training and test. The file also carries the A2 feature pair (`total_*`), not the paper's (`std_*`). E16 (reproducing Isnan et al.) reproduces their setup and measures the effect of keeping timeouts apart (Section 4): the drop it finds is explained by training-set size, not by the mixing itself.

2.10 Data versions

Version	Date	Author	What changed	Files	Rows (raw lines -> final)
v1	2026-10-27	utsav-s-r	First cleaned version	16 core files (A1, A2-VPN, A2-NonVPN, B x 15/30/60/120 s)	179,120 -> 167,909

The exploration notebook adds class counts per task and timeout, feature distributions by class at every timeout (16 figures), feature correlations for all four tasks, and the data funnel by class. It records two findings that matter later. Classes differ by orders of magnitude on rate and gap features (VPN-STREAMING: median 202 packets/s; VPN-VOIP: 2.2). Many features are near-duplicates: 19 of 253 pairs in A1 and B (27 in A2-VPN, 20 in A2-NonVPN) have Spearman |rho| >= 0.9, mostly within the active/idle group. Feature importances will therefore be shared out among correlated features.

3 Method

3.1 Pipeline overview

All code is in the project repository (Python 3.12). Dependencies are managed with uv; exact package versions are fixed in `uv.lock`, so every team member and every rerun uses identical library versions. The main libraries are pandas and NumPy (data), liac-arff and pyarrow (reading ARFF, writing Parquet), scikit-learn [3] (models, pipelines, metrics), MLflow [8] (run tracking), matplotlib and seaborn (figures) and SHAP [6] (explanations). XGBoost [4], LightGBM [5], Optuna [7] and imbalanced-learn [9] were added together in one change before the first Phase 3 run, so every Phase 3 run uses the same locked library versions.

The flow of data and results is:

1. **Raw data.** The 20 ARFF files of ISCXVPN2016 [2] are kept unchanged; their SHA-256 hashes are recorded and checked on every machine.
2. **Preparation.** A preparation script parses the files, applies the verified column-name map and the cleaning rules, and writes one Parquet file per task and timeout to a versioned folder. [Column-name check and cleaning rules: E00 (data check).]
3. **Training.** A run script reads one YAML configuration file, builds a scikit-learn Pipeline (preprocessing + model), runs 5-fold cross-validation on the training set, fits on the training set and scores the validation set.
4. **Tracking.** Every run is logged to MLflow (Section 3.6).
5. **Reporting.** every run is exported to one history table; every result table is built from it by code, and the notebooks draw the figures from it, so no number in this report is copied by hand.

3.2 Data versions

Any change that alters the cleaned data (a cleaning rule, the column map, the parser, row filtering or the split seed) creates a new data version (v1, v2, …) in its own folder, which is never overwritten. Each version has a manifest (file hashes, row and label counts, column map, cleaning rules) and a step-by-step cleaning log listing, for every step, the rows before and after, rows removed or changed (overall and per class), the rule and its reason. Steps that change nothing and decisions not to do something are logged too. Every run records the data version and its hash. After a version change, the current best model is rerun as its own experiment so the effect of the change is measured. [Data versions: see Section 2 and the data changelog.]

3.3 Splits and evaluation protocol

For each task and timeout the cleaned data is split once into 70% training, 15% validation and 15% test sets, stratified by class, with random seed 42. The row indices are saved inside the data version, so every run on that version uses exactly the same rows.

- Hyperparameters are chosen by 5-fold stratified cross-validation inside the training set only.
- Candidate models are compared on the validation set.
- The chosen configuration is retrained on training + validation data and scored **once** on the test set, at the final milestone, with 95% percentile intervals from 10,000 stratified bootstrap [18] resamples (rows drawn with replacement within each true class, so every resample keeps the test set's class counts; seed 42). A comparison set (the best validation variant of each method type per task, Section 3.8) is fixed before the test set is opened and scored the same way, once; it never changes the chosen configuration. 26 differences are tested, fixed before the test set is opened (E12, final test): per task the chosen model against each of the other six method types, and against the old-rule pick (Section 3.8) in A1 and A2-VPN. Each test is a two-sided paired bootstrap on macro F1 (the same resampled rows for both models), Holm-corrected [17] over all 26 at alpha 0.05; other differences and metrics are only

described. 10,000 resamples are needed because with 1,000 the smallest possible p-value (0.0020) is above Holm's first threshold (0.05 / 26 = 0.0019).

- Calibration (E15) is decided on the validation set before the test set is opened; it changes only the reported confidence, never the predicted class. The isotonic calibrator [19] learns from 5-fold out-of-fold predictions on the training rows, with the same shuffled folds (seed 42) as every other cross-validation here: unshuffled folds over the file-ordered rows hold out whole blocks of the files and give far too poor out-of-fold predictions (E15).
- Scaling, transforms and resampling are steps inside the pipeline, so they are fitted on training folds only and never see validation or test rows.
- Every random step (split, models, tuning sampler, oversampling, bootstrap) uses seed 42. Only the learning-curve and seed-stability experiments (E13 (learning curve), E14 (split-seed stability)) use seeds 42–51, on purpose.

Paper benchmark. To compare with Draper-Gil et al. [1], the decision tree and KNN are also run with the paper's own protocol: 10-fold cross-validation on the full file with Weka's default settings (KNN with k = 1; C4.5 with pruning confidence 0.25 and at least 2 samples per leaf). These results are reported in their own table. scikit-learn's decision tree with the entropy criterion is a close substitute for C4.5 [11], not an exact copy, and is called a C4.5 approximation.

Session-aware check. Flows from one recording session may fall on both sides of a random split and inflate scores. A group-aware split would keep each session on one side, but the ARFF files hold only the 23 timing features and the label: no capture-file name, address, port or timestamp identifies a session (E00). E11 (session-aware split) therefore cannot run, and this is listed as a limitation (Section 7).

3.4 Models

Model	Settings	Experiment
Majority class	Always predicts the most frequent class (the floor)	E01
Decision tree	Entropy criterion (C4.5 approximation); then tuned	E02, E17
KNN	k = 5, standard scaling; signed log + standard or robust scaling; then tuned	E03, E04, E07
Logistic Regression	Standard scaling; then tuned, signed log optional	E03, E17
SVM, RBF kernel	Standard scaling; then tuned, signed log optional	E03, E17
Random Forest	300 trees, default settings; then tuned	E05, E07, E17
XGBoost, LightGBM	Default settings; then tuned	E06, E07, E17
Class balancing	Class weights, random oversampling, SMOTE (inside the pipeline)	E08
Two-step model (task B)	VPN / non-VPN model, then one 7-class model per group	E10

Every baseline is run on all tasks (A1; A2-VPN and A2-NonVPN as separate models; B) at all four timeouts. Improvement experiments change one thing at a time against the current best model: preprocessing for KNN (E04, KNN with a signed log), tuning with Optuna, 50 trials per model with the same budget for the best baseline (E07, tuning), class imbalance handling (E08: none, class weights, random oversampling, SMOTE [10]), feature-group ablation (E09), direct vs two-step classification for task B (E10, one model or two steps for task B), and the same 50-trial tuning budget for every remaining method type (E17, equal tuning budget). The chosen models are then described by a learning curve (E13) and a split-seed stability check (E14), and a published result on a copy of this dataset is reproduced and corrected (E16, reproducing Isnan et al.). Each experiment states its hypothesis and a numeric pass/fail threshold before it is run.

3.5 Metrics

The main metric is **macro F1**, which weights every class equally, so small classes count as much as large ones. We also report per-class precision, recall and F1; weighted F1; accuracy; balanced accuracy; confusion matrices and the five most frequent confusion pairs; ROC-AUC (task A1); cross-validation mean and standard deviation; the gap between cross-validation and held-out scores; training time, prediction time per 1,000

flows, model size and peak memory; and, for the final models, Expected Calibration Error [20, 21] with a reliability diagram (E15). Comparisons with the paper use precision against precision and recall against recall.

3.6 Tracking and reproducibility

Every run is logged automatically to MLflow with: the experiment ID, the person and machine that ran it (operating system, CPU, memory), the exact code version, whether there were unsaved changes, the hash of the dependency lock file, the configuration file, the data version and hash, all settings, all metrics, timing, model size and memory, plots, and the trained model file (every official run except individual tuning trials). Official runs are made on one machine, so timings are comparable, and only from saved code shared with the team; the run script refuses otherwise. Each experiment also has a written entry (hypothesis, threshold, change, result, verdict, decision), and each milestone (baselines, tuned models, final models) is marked so its exact code can be recovered.

3.7 Preprocessing for distance-based models: the signed log

KNN, logistic regression and the RBF-kernel SVM depend on distances or linear scales, and the raw timing features span six or more orders of magnitude (microseconds up to about 1.2e8), so a few very large values dominate. A log compresses them, but plain log1p is not usable here: the features hold -1 codes meaning "not measurable" (log1p(-1) is minus infinity) and 586 negative minimum inter-arrival times (E00). We therefore apply the **signed log** sign(x)·log1p(|x|) as a step inside the pipeline, before scaling. It is defined for every value, keeps the order of values, keeps -1 apart from 0, and is a fixed formula: nothing is learned from data, so it cannot leak information from validation or test rows. Data v1 is not changed. Alternatives not chosen: shifting each column so its minimum is 0 before the log (a shift learned from the training data, different per column, and -1 merges into small values); logging only the non-negative columns (inconsistent, and large values stay uncompressed); replacing -1 by 0 plus indicator columns (changes the feature set, so E04 would change two things at once). E04 compares the signed log with StandardScaler and with RobustScaler; in E17 logistic regression and the SVM choose it inside their search (both chose it in every task). Tree models are not affected by any order-keeping transform, so they get none.

3.8 Phase 3 rules and decisions

These rules were fixed before the Phase 3 runs (28 Oct) unless stated otherwise; each experiment entry repeats the ones it uses before it runs.

- **One change at a time.** Each experiment changes one thing against the **current best** model of each task: the highest 15 s validation macro F1 so far, whatever the margin.
- **Winner rule.** Every experiment runs first at 15 s for all four tasks. A model whose 15 s validation macro F1 beats the task's current best by at least 0.03 is rerun at 30, 60 and 120 s with its 15 s settings (nothing is re-tuned per timeout). Why 0.03: it is above the largest Phase 2 cross-validation standard deviation (0.025), so a smaller gain could be noise from the split; a fixed number set before running keeps the choice honest. Not chosen: rerunning the top two per task, or running everything at every timeout (about four times the runs, and no screen-then-confirm design).
- **Tuning (E07, E17).** Optuna with the TPE sampler (seed 42), 50 trials per search, trials run one after another (parallel trials are not reproducible). Each trial is scored by 5-fold cross-validation macro F1 on the training rows only. E07 tunes, per task, the best two of Random Forest, XGBoost and LightGBM plus KNN with the same budget, so no model wins only because it was tuned more. E17 (added 28 Oct after E10) gives the same budget to every other method type, so the comparison of method types is between tuned models. Search ranges were written into the configurations before running. The best trial is retrained and scored like any other run; trials keep their settings and scores only.
- **Balancing (E08).** Inside the pipeline (imbalanced-learn), so only training folds are resampled. XGBoost has no class-weight option, so its "balanced" step uses per-row sample weights. SMOTE uses 5 neighbours; the smallest training class in any task file has 53 rows (MAIL, A2-NonVPN 30 s), about 42 per cross-validation fold, so it is always possible.

- **Two-step model (E10).** Both steps train on task B's training rows only; the VPN / non-VPN label comes from the B label. The A1 split is never used, because A1 and B hold the same flows split differently, so an A1-trained model would have seen B validation flows.
- **Analysis experiments (E09, E13, E14)** describe the chosen models and change nothing. If a feature set in E09 had scored at least 0.03 above all 23 features, the work would have stopped for a decision, because choosing features on validation scores risks overfitting the choice. E13 and E14 use other seeds on purpose and never touch the seed-42 test rows.
- **Reproducing Isnan et al. (E16)** uses its own data version (kaggle-k1, built by a script with its own manifest and hashes); data v1 is untouched, and no E16 result feeds model selection.
- **Cost and resources.** Every model runs on one thread (XGBoost and LightGBM included), with the five cross-validation folds in parallel. This keeps results deterministic and timings comparable with Phase 2. Peak memory is measured twice: with Python's tracemalloc (comparable with Phase 2) and as the peak memory of a fresh process doing the fit, which also sees the native memory of XGBoost and LightGBM. Every model is saved with joblib compression level 3. This is lossless and cuts a 300-tree Random Forest from 171.9 MiB to 22.4 MiB. Both the compressed and the uncompressed size are logged.
- **Final choice (decided 28 Oct after E10, a deviation from the rule set before Phase 3).** The rule fixed before Phase 3 treated any gap below 0.03 as a tie, broken in favour of the simpler model. Applied as written, it would have chosen KNN or the SVM at 0.0146-0.0286 below the best model. The cross-validation standard deviations of the chosen models (0.0028-0.0058) are far below 0.03, and the goal is the best score on unseen traffic. The final configuration per task is therefore the highest validation macro F1 among all model-selection runs (any experiment, any timeout). The only fallback is a saved model file above 50 MB , which none of the chosen models reaches. The old rule's picks were scored on the test set too (E12), so the cost of either rule is visible (Section 5.3).
- **Comparison set.** Fixed before the test set is opened: per task, the best 15 s validation variant of each of 7 method types (decision tree, KNN, logistic regression, SVM-RBF, Random Forest, XGBoost, LightGBM), 28 models. A type's variants are its default, tuned and balanced runs; KNN includes the E03 and E04 preprocessings; the E10 two-step model counts as XGBoost; the majority baseline is a reference, not a member. Only the four headline models are kept as final model files; the others stay in the run store.

4 Experiments

Each experiment states a hypothesis and a numeric pass/fail threshold before it runs, changes one thing, and ends with a verdict (supported / not supported). Unless stated otherwise, scores are on the **validation** set of the fixed 70/15/15 split (seed 42); models are chosen with 5-fold cross-validation on the training set; the test set is not used until the final experiment (E12, final test).

The order in which the experiments were run (this section is ordered by ID; see the key before Section 1):

1. E00 (data check, data version v1)
2. E01, E02 and E03 (baselines), with the paper benchmark
3. E04 (KNN with a signed log)
4. E05 (Random Forest)
5. E06 (gradient boosting)
6. E07 (tuning)
7. E08 (class balancing)
8. E10 (one model or two steps for task B)
9. E17 (equal tuning budget)
10. final choice of models and the comparison set (Section 4.13)
11. E09 (feature-group ablation)
12. E13 (learning curve)
13. E14 (split-seed stability)
14. E11 recorded as not run (no session information)
15. E16 (reproducing Isnan et al.)
16. E15 (calibration, on validation, before the test set)
17. E12 (final test: the test set opened once)

4.0 E00: data findings and data version v1

E00 (data check) checked the dataset before any model was trained: the corrected column names, the file-family decision, the odd-value and duplicate rules, the session question and the Kaggle finding. It produced data version v1 (167,909 flows in 16 task files). Section 2 reports it in full.

4.1 E01: majority-class baseline

What. A model that always predicts the largest training class (`DummyClassifier`), for every task and timeout (16 runs). **Why.** It is the floor every model must beat, and its scores are known in advance, so it also tests the metrics code: accuracy must equal the largest class's share p of the validation set, and macro F1 must equal 2p / ((1 + p) k) for k classes. **Result.** Supported: all 16 runs match both values to 4 decimals (threshold 0.001). The floor in macro F1 is 0.34-0.36 for A1, 0.06-0.10 for A2 and 0.02-0.03 for B. Accuracy alone would look much better (up to 0.56 for A1 and 0.51 for A2-NonVPN 120 s), which is why macro F1 is the main score.

task	timeout	classes	largest_class	p	val_accuracy	expected_macro_f1	val_macro_f1	ok
A1	15 s	2	VPN	0.5185	0.5185	0.3414	0.3414	yes
A1	30 s	2	VPN	0.5083	0.5083	0.337	0.337	yes
A1	60 s	2	Non-VPN	0.5577	0.5577	0.358	0.358	yes
A1	120 s	2	Non-VPN	0.5185	0.5185	0.3415	0.3415	yes
A2-VPN	15 s	7	VPN-BROWSING	0.2665	0.2665	0.0601	0.0601	yes
A2-VPN	30 s	7	VPN-BROWSING	0.3599	0.3599	0.0756	0.0756	yes
A2-VPN	60 s	7	VPN-BROWSING	0.3769	0.3769	0.0782	0.0782	yes

task	timeout	classes	largest_class	p	val_accuracy	expected_macro_f1	val_macro_f1	ok
A2-VPN	120 s	7	VPN-BROWSING	0.4828	0.4828	0.093	0.093	yes
A2-NonVPN	15 s	7	VOIP	0.3244	0.3244	0.07	0.07	yes
A2-NonVPN	30 s	7	BROWSING	0.3735	0.3735	0.0777	0.0777	yes
A2-NonVPN	60 s	7	BROWSING	0.3046	0.3046	0.0667	0.0667	yes
A2-NonVPN	120 s	7	BROWSING	0.5061	0.5061	0.096	0.096	yes
B	15 s	14	VOIP	0.156	0.156	0.0193	0.0193	yes
B	30 s	14	BROWSING	0.1833	0.1833	0.0221	0.0221	yes
B	60 s	14	BROWSING	0.1697	0.1697	0.0207	0.0207	yes
B	120 s	14	BROWSING	0.2606	0.2606	0.0295	0.0295	yes

4.2 E02 and E03: decision tree, KNN, Logistic Regression and SVM-RBF

What. Four baselines, each a scikit-learn pipeline, run on all 16 task/timeout files (64 runs): an unpruned entropy decision tree (E02 (decision-tree baseline); the closest scikit-learn model to C4.5, so a "C4.5 approximation"), KNN with k = 5 and standard scaling, Logistic Regression and an RBF-kernel SVM, both with standard scaling (E03, KNN, logistic regression and SVM baselines). Scaling is a pipeline step, fitted on training rows only. **Why.** The tree and KNN are the paper's two algorithms. Logistic Regression and SVM-RBF match the baselines of Isnan et al. [13]. Together they set the bar for the improvements in Phase 3.
Hypotheses and thresholds (written before running). E02: the tree beats the floor by at least 0.30 in every config and reaches at least 0.80 on A1 at every timeout. E03: all three beat the floor by at least 0.10; KNN is within 0.10 of the tree in at least 12 of 16 configs; Logistic Regression and SVM-RBF score below the tree in at least 12 of 16 configs, because the features are heavy-tailed (microsecond times up to about 10^8 next to -1 and 0 codes). **Result.** All supported. The tree is the best baseline in all 16 configs (macro F1 0.706-0.890), 0.499-0.765 above the floor. KNN is second everywhere, 0.039-0.101 below the tree (within 0.10 in 15 of 16). Logistic Regression (0.270-0.598) and SVM-RBF (0.274-0.621) are below the tree in all 16.

Validation macro F1 (E01-E03):

task	timeout	majority	tree	knn	logreg	svm_rbf
A1	15 s	0.341	0.890	0.813	0.585	0.602
A1	30 s	0.337	0.864	0.825	0.598	0.610
A1	60 s	0.358	0.857	0.813	0.541	0.545
A1	120 s	0.341	0.857	0.807	0.576	0.621
A2-VPN	15 s	0.060	0.825	0.774	0.432	0.468
A2-VPN	30 s	0.076	0.828	0.788	0.536	0.542
A2-VPN	60 s	0.078	0.775	0.730	0.339	0.330
A2-VPN	120 s	0.093	0.780	0.727	0.332	0.332
A2-NonVPN	15 s	0.070	0.829	0.773	0.449	0.487
A2-NonVPN	30 s	0.078	0.828	0.783	0.486	0.468
A2-NonVPN	60 s	0.067	0.825	0.750	0.447	0.441
A2-NonVPN	120 s	0.096	0.839	0.769	0.482	0.471
B	15 s	0.019	0.759	0.660	0.307	0.296
B	30 s	0.022	0.748	0.647	0.320	0.311
B	60 s	0.021	0.706	0.617	0.271	0.278
B	120 s	0.030	0.716	0.637	0.270	0.274

5-fold cross-validation on the training set, mean +- standard deviation:

task	timeout	majority	tree	knn	logreg	svm_rbf
A1	15 s	0.341 +- 0.000	0.888 +- 0.005	0.821 +- 0.002	0.600 +- 0.011	0.608 +- 0.005
A1	30 s	0.337 +- 0.000	0.873 +- 0.006	0.814 +- 0.010	0.600 +- 0.012	0.619 +- 0.023
A1	60 s	0.358 +- 0.000	0.852 +- 0.010	0.808 +- 0.009	0.551 +- 0.008	0.552 +- 0.006
A1	120 s	0.341 +- 0.000	0.857 +- 0.009	0.803 +- 0.002	0.569 +- 0.008	0.608 +- 0.011
A2-VPN	15 s	0.060 +- 0.000	0.810 +- 0.007	0.767 +- 0.008	0.410 +- 0.014	0.448 +- 0.024
A2-VPN	30 s	0.076 +- 0.000	0.796 +- 0.009	0.769 +- 0.025	0.495 +- 0.016	0.507 +- 0.021
A2-VPN	60 s	0.078 +- 0.000	0.760 +- 0.016	0.708 +- 0.019	0.334 +- 0.022	0.331 +- 0.021
A2-VPN	120 s	0.093 +- 0.000	0.725 +- 0.023	0.692 +- 0.016	0.343 +- 0.019	0.317 +- 0.021
A2-NonVPN	15 s	0.070 +- 0.000	0.862 +- 0.009	0.792 +- 0.010	0.459 +- 0.013	0.479 +- 0.018
A2-NonVPN	30 s	0.078 +- 0.000	0.817 +- 0.015	0.783 +- 0.017	0.459 +- 0.007	0.426 +- 0.008
A2-NonVPN	60 s	0.067 +- 0.000	0.829 +- 0.016	0.740 +- 0.013	0.454 +- 0.015	0.432 +- 0.022
A2-NonVPN	120 s	0.096 +- 0.000	0.807 +- 0.015	0.731 +- 0.013	0.472 +- 0.016	0.441 +- 0.023
B	15 s	0.019 +- 0.000	0.757 +- 0.013	0.663 +- 0.008	0.299 +- 0.010	0.278 +- 0.008
B	30 s	0.022 +- 0.000	0.733 +- 0.008	0.654 +- 0.016	0.323 +- 0.009	0.292 +- 0.008
B	60 s	0.021 +- 0.000	0.703 +- 0.013	0.607 +- 0.010	0.275 +- 0.002	0.261 +- 0.009
B	120 s	0.030 +- 0.000	0.684 +- 0.020	0.607 +- 0.015	0.250 +- 0.005	0.251 +- 0.017

What the tables show:

- **Cross-validation predicts validation.** For every run, |CV mean - validation| is at most 0.054, and CV standard deviations are at most 0.025. A gap of several points between two models is therefore larger than fold-to-fold noise.
- **Small classes.** Logistic Regression and SVM-RBF score F1 = 0.00 on MAIL / VPN-MAIL in most A2 and B configs. So their macro F1 is far below their weighted F1 (gap up to 0.225); the tree's gap is 0.000-0.069. The SVM keeps 55-88% of the training rows as support vectors, a sign that the RBF kernel separates the standard-scaled features poorly. A log transform or a robust scaler is tested in E04 (KNN with a signed log).
- **Easiest and hardest classes.** VoIP is the easiest type (tree F1 0.92-0.99 for VOIP and VPN-VOIP). The weakest are CHAT, VPN-CHAT, STREAMING, MAIL and VPN-MAIL (tree F1 0.42-0.71).
- **Most frequent confusion.** In task B the largest confusion for the tree and KNN, at every timeout, is BROWSING vs VPN-BROWSING in both directions (tree: 29-45 validation flows each way). The model recognises browsing, but not whether it went through the VPN.
- **Timeouts.** B and A2-VPN score lower at 60 s and 120 s. The class mix differs between timeouts (Section 2.8), and the 60 s files hold differently shaped VOIP and FT flows (E00 addendum), so this is not read as an effect of the timeout.
- **Cost.** All baselines train in under 3 s. To predict 1,000 flows, the tree needs 0.30 ms (median), KNN 8.0 ms and the SVM 210 ms. Model files range from 1.4 KB (majority) and 4-5 KB (Logistic Regression) to 4.2 MB (SVM).

Per-class extremes for the tree (E02): macro vs weighted F1, their gap, the generalisation gap, and the best and worst class by F1 and by recall:

task	timeout	macro F1	weighted F1	gap	CV - val	best F1	worst F1	best recall	worst recall
A1	15 s	0.89	0.89	0.0	-0.002	VPN 0.89	Non-VPN 0.89	Non-VPN 0.89	VPN 0.89
A1	30 s	0.864	0.864	0.0	0.009	VPN 0.87	Non-VPN 0.86	VPN 0.87	Non-VPN 0.86

task	timeout	macro F1	weighted F1	gap	CV - val	best F1	worst F1	best recall	worst recall
A1	60 s	0.857	0.859	0.002	-0.005	Non-VPN 0.87	VPN 0.84	Non-VPN 0.87	VPN 0.85
A1	120 s	0.857	0.857	0.0	0.0	Non-VPN 0.86	VPN 0.85	Non-VPN 0.86	VPN 0.85
A2-VPN	15 s	0.825	0.849	0.024	-0.014	VPN-VOIP 0.99	VPN-CHAT 0.65	VPN-VOIP 0.98	VPN-CHAT 0.66
A2-VPN	30 s	0.828	0.847	0.02	-0.031	VPN-VOIP 0.98	VPN-CHAT 0.67	VPN-STREAMING 0.98	VPN-CHAT 0.70
A2-VPN	60 s	0.775	0.831	0.056	-0.014	VPN-VOIP 0.92	VPN-CHAT 0.63	VPN-VOIP 0.92	VPN-FT 0.65
A2-VPN	120 s	0.78	0.825	0.045	-0.054	VPN-VOIP 0.98	VPN-CHAT 0.61	VPN-VOIP 0.98	VPN-CHAT 0.57
A2-NonVPN	15 s	0.829	0.891	0.061	0.033	VOIP 0.99	STREAMING 0.71	VOIP 0.99	STREAMING 0.70
A2-NonVPN	30 s	0.828	0.889	0.061	-0.011	VOIP 0.98	STREAMING 0.68	VOIP 0.98	STREAMING 0.57
A2-NonVPN	60 s	0.825	0.878	0.053	0.004	VOIP 0.97	STREAMING 0.58	VOIP 0.97	STREAMING 0.55
A2-NonVPN	120 s	0.839	0.893	0.054	-0.033	VOIP 0.95	CHAT 0.56	BROWSING 0.95	CHAT 0.56
B	15 s	0.759	0.81	0.051	-0.001	VOIP 0.98	VPN-MAIL 0.55	VOIP 0.99	VPN-MAIL 0.52
B	30 s	0.748	0.8	0.053	-0.014	VPN-VOIP 0.98	MAIL 0.55	VPN-VOIP 0.99	MAIL 0.55
B	60 s	0.706	0.775	0.069	-0.002	VOIP 0.92	VPN-CHAT 0.42	VOIP 0.92	VPN-CHAT 0.42
B	120 s	0.716	0.767	0.051	-0.031	VOIP 0.97	STREAMING 0.47	VOIP 1.00	CHAT 0.42

Top 5 confusion pairs of the tree on the validation set (E02):

task	timeout	top 5 confusion pairs (true -> predicted: flows)
A1	15 s	VPN -> Non-VPN: 158; Non-VPN -> VPN: 139
A1	30 s	Non-VPN -> VPN: 142; VPN -> Non-VPN: 135
A1	60 s	Non-VPN -> VPN: 161; VPN -> Non-VPN: 150
A1	120 s	Non-VPN -> VPN: 104; VPN -> Non-VPN: 100
A2-VPN	15 s	VPN-FT -> VPN-CHAT: 28; VPN-CHAT -> VPN-BROWSING: 26; VPN-BROWSING -> VPN-CHAT: 25; VPN-CHAT -> VPN-FT: 20; VPN-FT -> VPN-BROWSING: 19
A2-VPN	30 s	VPN-BROWSING -> VPN-CHAT: 26; VPN-FT -> VPN-BROWSING: 22; VPN-BROWSING -> VPN-FT: 15; VPN-CHAT -> VPN-BROWSING: 12; VPN-CHAT -> VPN-FT: 12
A2-VPN	60 s	VPN-FT -> VPN-BROWSING: 17; VPN-BROWSING -> VPN-FT: 16; VPN-FT -> VPN-CHAT: 14; VPN-BROWSING -> VPN-VOIP: 13; VPN-CHAT -> VPN-FT: 12
A2-VPN	120 s	VPN-FT -> VPN-BROWSING: 18; VPN-BROWSING -> VPN-FT: 16; VPN-CHAT -> VPN-BROWSING: 11; VPN-FT -> VPN-P2P: 9; VPN-P2P -> VPN-BROWSING: 9

task	timeout	top 5 confusion pairs (true -> predicted: flows)
A2-NonVPN	15 s	BROWSING -> CHAT: 18; CHAT -> BROWSING: 15; BROWSING -> STREAMING: 12; FT -> BROWSING: 12; FT -> CHAT: 11
A2-NonVPN	30 s	CHAT -> BROWSING: 12; STREAMING -> BROWSING: 11; BROWSING -> CHAT: 10; BROWSING -> FT: 10; FT -> CHAT: 9
A2-NonVPN	60 s	CHAT -> BROWSING: 20; BROWSING -> CHAT: 19; MAIL -> FT: 8; FT -> BROWSING: 7; BROWSING -> MAIL: 6
A2-NonVPN	120 s	CHAT -> BROWSING: 12; P2P -> BROWSING: 8; FT -> BROWSING: 7; BROWSING -> P2P: 6; BROWSING -> CHAT: 5
B	15 s	VPN-BROWSING -> BROWSING: 41; BROWSING -> VPN-BROWSING: 39; VPN-CHAT -> VPN-FT: 25; VPN-FT -> VPN-CHAT: 24; VPN-BROWSING -> VPN-CHAT: 18
B	30 s	VPN-BROWSING -> BROWSING: 44; BROWSING -> VPN-BROWSING: 36; VPN-BROWSING -> VPN-CHAT: 13; VPN-FT -> VPN-CHAT: 12; VPN-CHAT -> VPN-BROWSING: 11
B	60 s	BROWSING -> VPN-BROWSING: 37; VPN-BROWSING -> BROWSING: 29; VPN-MAIL -> MAIL: 15; VPN-P2P -> P2P: 14; VPN-VOIP -> VOIP: 14
B	120 s	BROWSING -> VPN-BROWSING: 45; VPN-BROWSING -> BROWSING: 32; VPN-BROWSING -> VPN-FT: 17; VPN-FT -> VPN-BROWSING: 12; VPN-BROWSING -> VPN-P2P: 10

Model complexity (E02 tree, E03 SVM):

task	timeout	n_train	tree depth	tree nodes	tree leaves	SVM support vectors
A1	15 s	12,626	27	1,789	895	10,292
A1	30 s	9,526	35	1,601	801	7,976
A1	60 s	10,273	27	1,917	959	7,882
A1	120 s	6,666	25	1,243	622	5,501
A2-VPN	15 s	6,566	23	1,407	704	4,497
A2-VPN	30 s	4,860	20	1,067	534	3,108
A2-VPN	60 s	4,565	24	1,249	625	3,265
A2-VPN	120 s	3,252	21	985	493	2,368
A2-NonVPN	15 s	6,085	16	743	372	3,329
A2-NonVPN	30 s	4,685	19	751	376	2,751
A2-NonVPN	60 s	5,744	24	1,001	501	3,803
A2-NonVPN	120 s	3,458	18	591	296	1,907
B	15 s	12,651	30	3,409	1,705	11,076
B	30 s	9,546	23	2,901	1,451	8,368
B	60 s	10,309	26	3,437	1,719	8,796
B	120 s	6,711	22	2,481	1,241	5,813

Cost per model over the 16 configs:

model	train time (s) median	train time (s) max	5-fold CV time (s) median	5-fold CV time (s) max	predict ms / 1,000 flows median	predict ms / 1,000 flows max	model file (KB) median	model file (KB) max	peak fit memory (MB) median	peak fit memory (MB) max
majority	0.004	0.008	0.157	2.854	0.091	0.18	1.414	1.523	0.623	1.182
tree	0.112	0.304	0.293	0.652	0.298	0.531	134.49	592.916	1.257	2.392
knn	0.01	0.021	0.182	0.281	8.031	13.338	1242.967	2374.631	2.999	5.722
logreg	0.084	0.428	0.212	0.603	0.378	0.69	3.829	5.196	3.433	8.144
svm_rbf	0.626	2.913	1.61	3.784	209.808	596.678	1175.091	4288.896	3.552	7.781

Peak fit memory is measured with Python's tracemalloc, which counts NumPy arrays but not memory allocated inside compiled libraries (for example libsvm), so SVM memory is a lower bound. Training time comes from a fit without tracemalloc and peak memory from a separate fit of a clone. The first Phase 2 runs timed the fit with tracemalloc on, which slowed it (x1.05 tree, x1.00 SVM, x1.19 Logistic Regression, x2.3 KNN in a recheck); all 144 configs were rerun from the fixed code with identical scores, and these tables use the

rerun. The first runs also logged process_max_rss_mb, a high-water mark of the whole batch process rather than a per-run figure; it is not used.

The same tables were also produced for KNN, Logistic Regression and SVM-RBF, with accuracy, balanced accuracy, weighted F1 and A1 ROC-AUC for every run.

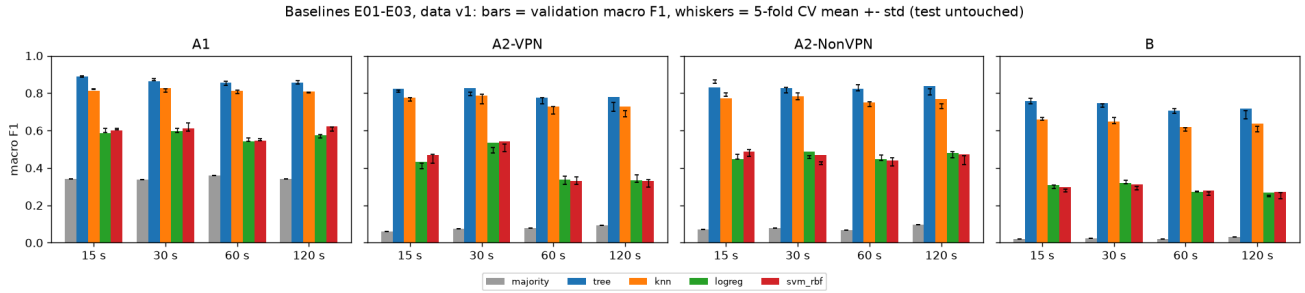


Figure 4. Baselines E01-E03

4.3 Paper benchmark (part of E02 and E03)

What. The paper's own protocol [1]: 10-fold stratified cross-validation on the whole file, with predictions pooled over the folds as Weka reports them, for its two algorithms with Weka's default settings. J48 (C4.5) defaults are a pruning confidence of 0.25 and at least 2 instances per leaf; scikit-learn has no C4.5 pruning, so our stand-in is an unpruned entropy tree with at least 2 rows per leaf. IBk (KNN) defaults are k = 1 with a Euclidean distance that normalises each attribute to its range; our stand-in is min-max scaling followed by 1-NN. The defaults were checked in the Weka documentation [16] . Each algorithm ran on two row sources per file (64 runs): every row as published (duplicates kept, values untouched), which is closest to the paper's input, and data v1 (deduplicated). **Why.** It tests whether we reproduce the paper's results before we try to improve on them. We compare precision with precision and recall with recall, never the paper's precision with our F1. These scores are never used to choose a model. **Hypotheses and thresholds.** On the published rows, at least 12 of the 16 A1 values of the paper's Figure 2 are matched within +-0.05, for each algorithm. Because duplicates can put a copy of a flow in the training folds, removing them lowers 10-fold macro F1 in at least 12 of 16 files. **Result.** Reproduction supported: our C4.5 approximation matches all 16 A1 values (largest difference 0.021, close to the +-0.01 reading error of the chart), and 1-NN matches 15 of 16 (the miss is VPN recall at 120 s: 0.803 vs 0.860). The duplicate hypothesis is not supported (6 of 16 files lower for the tree, 2 of 16 for 1-NN).

A1, paper (Figure 2, read to +-0.01) vs ours (`diff` = ours - paper):

algorithm	class	timeout	metric	paper	paper_source	ours_published	ours_v1	diff_published	diff_v1
C4.5	Non-VPN	15	precision	0.906	chart +-0.01 and text	0.886	0.882	-0.02	-0.024
C4.5	Non-VPN	30	precision	0.885	chart +-0.01	0.866	0.866	-0.019	-0.019
C4.5	Non-VPN	60	precision	0.864	chart +-0.01	0.86	0.876	-0.004	0.012
C4.5	Non-VPN	120	precision	0.887	chart +-0.01 and text	0.87	0.863	-0.017	-0.024
C4.5	Non-VPN	15	recall	0.877	chart +-0.01	0.882	0.903	0.005	0.026
C4.5	Non-VPN	30	recall	0.846	chart +-0.01	0.857	0.886	0.011	0.04
C4.5	Non-VPN	60	recall	0.888	chart +-0.01	0.889	0.898	0.001	0.01
C4.5	Non-VPN	120	recall	0.843	chart +-0.01	0.853	0.89	0.01	0.047

algorithm	class	timeout	metric	paper	paper_source	ours_published	ours_v1	diff_published	diff_v1
C4.5	VPN	15	precision	0.89	chart +-0.01 and text	0.892	0.908	0.002	0.018
C4.5	VPN	30	precision	0.868	chart +-0.01	0.874	0.888	0.006	0.02
C4.5	VPN	60	precision	0.856	chart +-0.01	0.857	0.867	0.001	0.011
C4.5	VPN	120	precision	0.863	chart +-0.01 and text	0.868	0.877	0.005	0.014
C4.5	VPN	15	recall	0.917	chart +-0.01	0.896	0.888	-0.021	-0.029
C4.5	VPN	30	recall	0.901	chart +-0.01	0.881	0.867	-0.02	-0.034
C4.5	VPN	60	recall	0.827	chart +-0.01	0.821	0.84	-0.006	0.013
C4.5	VPN	120	recall	0.902	chart +-0.01	0.884	0.849	-0.018	-0.053
KNN	Non-VPN	15	precision	0.846	chart +-0.01 and text	0.821	0.846	-0.025	0.0
KNN	Non-VPN	30	precision	0.84	chart +-0.01	0.805	0.83	-0.035	-0.01
KNN	Non-VPN	60	precision	0.838	chart +-0.01	0.831	0.854	-0.007	0.016
KNN	Non-VPN	120	precision	0.837	chart +-0.01 and text	0.794	0.83	-0.043	-0.007
KNN	Non-VPN	15	recall	0.832	chart +-0.01	0.857	0.848	0.025	0.016
KNN	Non-VPN	30	recall	0.808	chart +-0.01	0.839	0.831	0.031	0.023
KNN	Non-VPN	60	recall	0.851	chart +-0.01	0.857	0.849	0.006	-0.002
KNN	Non-VPN	120	recall	0.787	chart +-0.01	0.827	0.825	0.04	0.038
KNN	VPN	15	precision	0.848	chart +-0.01 and text	0.863	0.859	0.015	0.011
KNN	VPN	30	precision	0.835	chart +-0.01	0.85	0.836	0.015	0.001
KNN	VPN	60	precision	0.812	chart +-0.01	0.816	0.811	0.004	-0.001
KNN	VPN	120	precision	0.815	chart +-0.01 and text	0.836	0.813	0.021	-0.002
KNN	VPN	15	recall	0.861	chart +-0.01	0.83	0.857	-0.031	-0.004
KNN	VPN	30	recall	0.862	chart +-0.01	0.818	0.835	-0.044	-0.027
KNN	VPN	60	recall	0.795	chart +-0.01	0.784	0.817	-0.011	0.022
KNN	VPN	120	recall	0.86	chart +-0.01	0.803	0.818	-0.057	-0.042

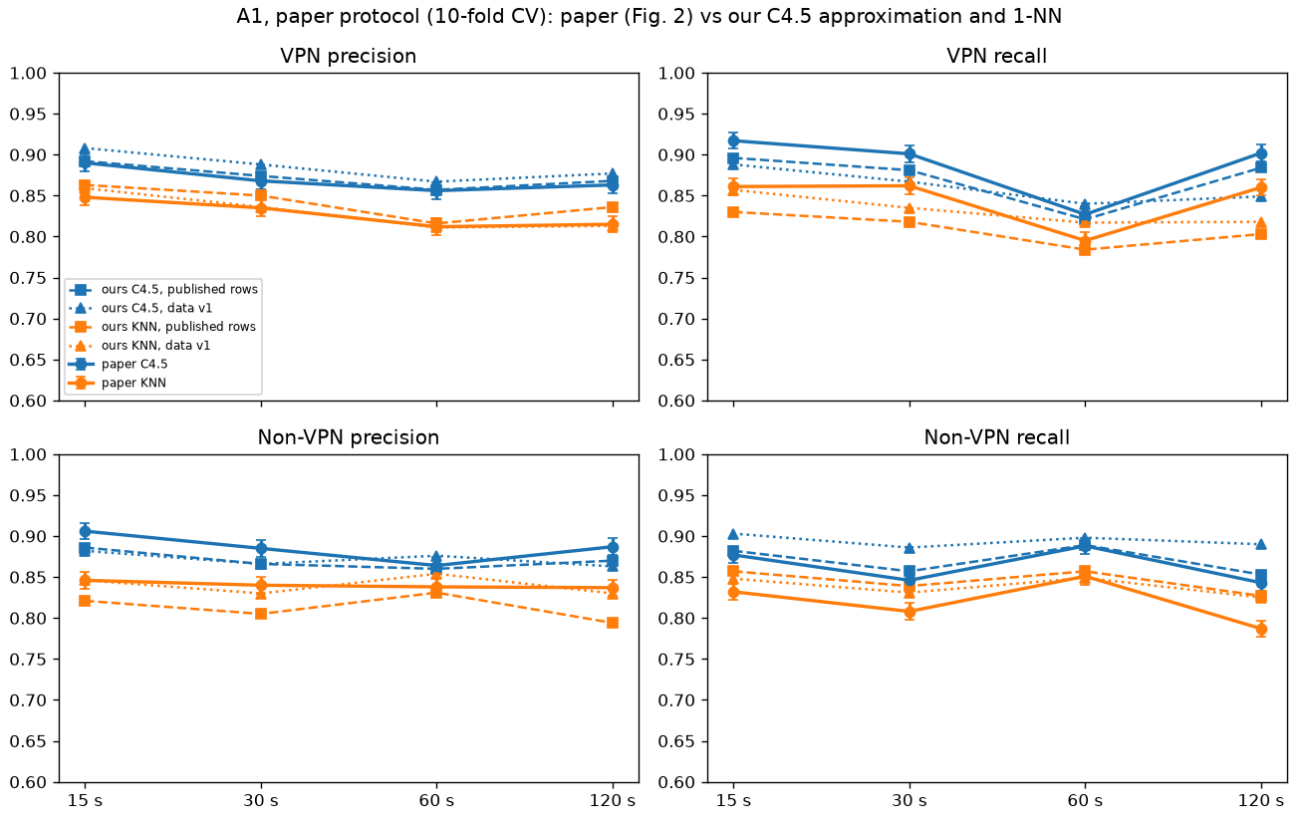


Figure 5. Paper vs ours, A1

The plan asks for the comparison as one table per task. The tables below give our 10-fold precision and recall for every class, both algorithms and all timeouts, on the published rows (closest to the paper's input), with the paper's value in brackets wherever the paper gives one (C = read from its chart, +-0.01; T = stated in its text). The same tables were also produced on data v1.

A2-VPN:

class	metric	C4.5 15 s	C4.5 30 s	C4.5 60 s	C4.5 120 s	KNN 15 s	KNN 30 s	KNN 60 s	KNN 120 s
VPN-BROWSING	precision	0.872	0.878	0.873	0.878	0.830	0.874	0.881	0.795
VPN-BROWSING	recall	0.884	0.916	0.918	0.906	0.846	0.893	0.902	0.888
VPN-CHAT	precision	0.661	0.647	0.568	0.532	0.511	0.490	0.427	0.576
VPN-CHAT	recall	0.687	0.628	0.562	0.473	0.651	0.667	0.658	0.562
VPN-FT	precision	0.792	0.744	0.676	0.649	0.773	0.764	0.714	0.674
VPN-FT	recall	0.770	0.690	0.610	0.605	0.735	0.689	0.648	0.595
VPN-MAIL	precision	0.808	0.891	0.778	0.864	0.766	0.912	0.753	0.859
VPN-MAIL	recall	0.762	0.887	0.715	0.852	0.766	0.947	0.734	0.904
VPN-P2P	precision	0.827	0.831	0.793	0.810	0.736	0.806	0.727	0.771
VPN-P2P	recall	0.863	0.870	0.848	0.862	0.517	0.585	0.527	0.567
VPN-STREAMING	precision	0.922	0.926	0.875	0.838	0.906	0.876	0.838	0.815
VPN-STREAMING	recall	0.891	0.876	0.782	0.752	0.914	0.856	0.787	0.759
VPN-VOIP	precision	0.990	0.992	0.939	0.973	0.990	0.993	0.911	0.984

class	metric	C4.5 15 s	C4.5 30 s	C4.5 60 s	C4.5 120 s	KNN 15 s	KNN 30 s	KNN 60 s	KNN 120 s
VPN-VOIP	recall	0.979	0.975	0.914	0.925	0.987	0.981	0.907	0.951
AVERAGE (macro)	precision	0.839 (paper 0.840 T)	0.844	0.786	0.792	0.788	0.816	0.750	0.782
AVERAGE (macro)	recall	0.834	0.835	0.764	0.768	0.774	0.803	0.738	0.747

A2-NonVPN:

class	metric	C4.5 15 s	C4.5 30 s	C4.5 60 s	C4.5 120 s	KNN 15 s	KNN 30 s	KNN 60 s	KNN 120 s
BROWSING	precision	0.904	0.915	0.897	0.928	0.866	0.883	0.865	0.922
BROWSING	recall	0.930	0.934	0.922	0.948	0.880	0.912	0.872	0.932
CHAT	precision	0.813	0.769	0.738	0.656	0.558	0.508	0.483	0.331
CHAT	recall	0.798	0.793	0.753	0.694	0.692	0.662	0.661	0.599
FT	precision	0.810	0.794	0.793	0.796	0.812	0.769	0.847	0.791
FT	recall	0.853	0.824	0.873	0.851	0.670	0.629	0.734	0.571
MAIL	precision	0.849	0.754	0.866	0.859	0.709	0.734	0.762	0.709
MAIL	recall	0.835	0.605	0.786	0.789	0.735	0.716	0.723	0.697
P2P	precision	0.971	0.943	0.948	0.946	0.907	0.897	0.893	0.914
P2P	recall	0.955	0.916	0.919	0.929	0.890	0.883	0.897	0.911
STREAMING	precision	0.837	0.769	0.757	0.768	0.782	0.759	0.655	0.657
STREAMING	recall	0.701	0.664	0.567	0.558	0.685	0.617	0.504	0.524
VOIP	precision	0.992	0.990	0.973	0.989	0.990	0.980	0.968	0.931
VOIP	recall	0.988	0.979	0.946	0.941	0.987	0.977	0.954	0.929
AVERAGE (macro)	precision	0.882 (paper 0.890 T)	0.848	0.853	0.849	0.803	0.790	0.782	0.751
AVERAGE (macro)	recall	0.866	0.817	0.824	0.816	0.791	0.771	0.764	0.738

B:

class	metric	C4.5 15 s	C4.5 30 s	C4.5 60 s	C4.5 120 s	KNN 15 s	KNN 30 s	KNN 60 s	KNN 120 s
BROWSING	precision	0.771	0.790	0.779	0.800	0.690	0.748	0.746	0.807
BROWSING	recall	0.820	0.831	0.827	0.847	0.700	0.758	0.741	0.794
CHAT	precision	0.677	0.620	0.631	0.510	0.349	0.292	0.336	0.164
CHAT	recall	0.727	0.667	0.667	0.554	0.567	0.536	0.586	0.488
FT	precision	0.717	0.666	0.807	0.687	0.744	0.685	0.809	0.683
FT	recall	0.653	0.614	0.743	0.535	0.622	0.552	0.717	0.487
MAIL	precision	0.633 (paper 0.671 T)	0.617	0.741	0.680 (paper 0.790 T)	0.618	0.712	0.674	0.548
MAIL	recall	0.651	0.617	0.727	0.665	0.643	0.642	0.645	0.616
P2P	precision	0.880	0.851	0.842	0.874	0.749	0.743	0.760	0.818
P2P	recall	0.893	0.845	0.863	0.891	0.732	0.724	0.752	0.804
STREAMING	precision	0.713	0.657	0.496	0.510	0.679	0.630	0.536	0.477
STREAMING	recall	0.670	0.623	0.472	0.495	0.629	0.532	0.440	0.394

class	metric	C4.5 15 s	C4.5 30 s	C4.5 60 s	C4.5 120 s	KNN 15 s	KNN 30 s	KNN 60 s	KNN 120 s
VOIP	precision	0.983	0.976	0.912	0.934	0.980	0.974	0.923	0.895
VOIP	recall	0.981	0.971	0.905	0.903	0.983	0.972	0.903	0.895
VPN-BROWSING	precision	0.773 (paper 0.771 T)	0.800	0.797	0.807 (paper 0.809 T)	0.689 (paper 0.691 T)	0.738	0.760	0.742 (paper 0.743 T)
VPN-BROWSING	recall	0.768	0.778	0.802	0.799	0.711	0.759	0.780	0.703
VPN-CHAT	precision	0.616	0.607	0.557	0.534	0.510 (paper 0.501 T)	0.503	0.460	0.466 (paper 0.501 T)
VPN-CHAT	recall	0.571	0.560	0.477	0.444	0.507	0.471	0.442	0.476
VPN-FT	precision	0.759	0.717	0.639	0.661	0.693	0.664	0.584	0.549
VPN-FT	recall	0.712	0.661	0.532	0.539	0.637	0.593	0.528	0.580
VPN-MAIL	precision	0.770 (paper 0.739 T)	0.828	0.702	0.804 (paper 0.786 T)	0.687 (paper 0.688 T)	0.843	0.599	0.820 (paper 0.688 T)
VPN-MAIL	recall	0.670	0.808	0.608	0.802	0.644	0.902	0.554	0.878
VPN-P2P	precision	0.615	0.620	0.547	0.629	0.591	0.666	0.593	0.669
VPN-P2P	recall	0.767	0.771	0.745	0.806	0.428	0.498	0.441	0.514
VPN-STREAMING	precision	0.885	0.871	0.863	0.875	0.836	0.774	0.774	0.713
VPN-STREAMING	recall	0.823	0.812	0.766	0.772	0.848	0.772	0.782	0.703
VPN-VOIP	precision	0.986	0.980	0.893	0.942	0.978	0.963	0.807	0.921
VPN-VOIP	recall	0.973	0.968	0.845	0.889	0.972	0.965	0.809	0.912
AVERAGE (macro)	precision	0.770	0.757	0.729	0.732	0.700	0.710	0.669	0.662
AVERAGE (macro)	recall	0.763	0.752	0.713	0.710	0.687	0.691	0.651	0.660

For A2 and B the paper's Figure 3 draws two unlabelled lines per timeout, which can't be assigned to C4.5 or KNN. So only the values the paper states in its text are compared. The paper doesn't say whether its "average" is a plain or a class-weighted mean, so both are shown. On the published rows the plain (macro) mean matches it within 0.013 in every case, which suggests the paper used the plain mean.

For averages the paper does not say whether it is a plain or a class-weighted mean (Weka prints the weighted one), so both are shown.

task	algorithm	class	timeout	metric	paper	rows	ours	ours_kind	diff
A2-VPN	C4.5	AVERAGE	15	precision	0.84	published	0.839	macro	-0.001
A2-VPN	C4.5	AVERAGE	15	precision	0.84	published	0.853	weighted	0.013
A2-VPN	C4.5	AVERAGE	15	precision	0.84	v1	0.827	macro	-0.013
A2-VPN	C4.5	AVERAGE	15	precision	0.84	v1	0.846	weighted	0.006
A2-NonVPN	C4.5	AVERAGE	15	precision	0.89	published	0.882	macro	-0.008
A2-NonVPN	C4.5	AVERAGE	15	precision	0.89	published	0.914	weighted	0.024
A2-NonVPN	C4.5	AVERAGE	15	precision	0.89	v1	0.886	macro	-0.004
A2-NonVPN	C4.5	AVERAGE	15	precision	0.89	v1	0.92	weighted	0.03
B	C4.5	VPN-BROWSING	15	precision	0.771	published	0.773	class	0.002

task	algorithm	class	timeout	metric	paper	rows	ours	ours_kind	diff
B	C4.5	VPN-BROWSING	15	precision	0.771	v1	0.76	class	-0.011
B	C4.5	VPN-BROWSING	120	precision	0.809	published	0.807	class	-0.002
B	C4.5	VPN-BROWSING	120	precision	0.809	v1	0.76	class	-0.049
B	C4.5	VPN-MAIL	15	precision	0.739	published	0.77	class	0.031
B	C4.5	VPN-MAIL	15	precision	0.739	v1	0.718	class	-0.021
B	C4.5	VPN-MAIL	120	precision	0.786	published	0.804	class	0.018
B	C4.5	VPN-MAIL	120	precision	0.786	v1	0.682	class	-0.104
B	C4.5	MAIL	15	precision	0.671	published	0.633	class	-0.038
B	C4.5	MAIL	15	precision	0.671	v1	0.625	class	-0.046
B	C4.5	MAIL	120	precision	0.79	published	0.68	class	-0.11
B	C4.5	MAIL	120	precision	0.79	v1	0.613	class	-0.177
B	KNN	VPN-BROWSING	15	precision	0.691	published	0.689	class	-0.002
B	KNN	VPN-BROWSING	15	precision	0.691	v1	0.695	class	0.004
B	KNN	VPN-BROWSING	120	precision	0.743	published	0.742	class	-0.001
B	KNN	VPN-BROWSING	120	precision	0.743	v1	0.7	class	-0.043
B	KNN	VPN-CHAT	15	precision	0.501	published	0.51	class	0.009
B	KNN	VPN-CHAT	15	precision	0.501	v1	0.509	class	0.008
B	KNN	VPN-CHAT	120	precision	0.501	published	0.466	class	-0.035
B	KNN	VPN-CHAT	120	precision	0.501	v1	0.494	class	-0.007
B	KNN	VPN-MAIL	15	precision	0.688	published	0.687	class	-0.001
B	KNN	VPN-MAIL	15	precision	0.688	v1	0.633	class	-0.055
B	KNN	VPN-MAIL	120	precision	0.688	published	0.82	class	0.132
B	KNN	VPN-MAIL	120	precision	0.688	v1	0.521	class	-0.167
B	C4.5	AVERAGE_BEST_TIMEOUT	best	precision	0.783	published	0.77	macro	-0.013
B	C4.5	AVERAGE_BEST_TIMEOUT	best	precision	0.783	published	0.808	weighted	0.025
B	C4.5	AVERAGE_BEST_TIMEOUT	best	precision	0.783	v1	0.774	macro	-0.009
B	C4.5	AVERAGE_BEST_TIMEOUT	best	precision	0.783	v1	0.815	weighted	0.032
B	KNN	AVERAGE_BEST_TIMEOUT	best	precision	0.711	published	0.71	macro	-0.001
B	KNN	AVERAGE_BEST_TIMEOUT	best	precision	0.711	published	0.745	weighted	0.034
B	KNN	AVERAGE_BEST_TIMEOUT	best	precision	0.711	v1	0.707	macro	-0.004
B	KNN	AVERAGE_BEST_TIMEOUT	best	precision	0.711	v1	0.755	weighted	0.044

Duplicates. Removing the exact duplicates changed the 10-fold macro F1 by -0.018 to +0.014 for the tree (mean +0.001) and by -0.017 to +0.040 for 1-NN (mean +0.012):

algo	task	timeout	rows_published	rows_v1	f1_published	f1_v1	v1_minus_published
j48	A1	15	18,758	18,038	0.8888	0.8954	0.0067
j48	A1	30	14,651	13,610	0.8696	0.8766	0.0069
j48	A1	60	15,515	14,677	0.8562	0.8702	0.014
j48	A1	120	10,782	9,524	0.8688	0.8694	0.0007
j48	A2-VPN	15	9,793	9,380	0.836	0.8197	-0.0162
j48	A2-VPN	30	7,734	6,944	0.8389	0.821	-0.0179
j48	A2-VPN	60	6,935	6,523	0.7741	0.7702	-0.0039

algo	task	timeout	rows_published	rows_v1	f1_published	f1_v1	v1_minus_published
j48	A2-VPN	120	5,631	4,648	0.779	0.7719	-0.0071
j48	A2-NonVPN	15	8,965	8,694	0.8731	0.8768	0.0037
j48	A2-NonVPN	30	6,917	6,694	0.8303	0.8359	0.0055
j48	A2-NonVPN	60	8,580	8,206	0.8359	0.8483	0.0123
j48	A2-NonVPN	120	5,151	4,941	0.8294	0.8346	0.0052
j48	B	15	18,758	18,074	0.765	0.7707	0.0057
j48	B	30	14,651	13,638	0.7535	0.7488	-0.0047
j48	B	60	15,515	14,729	0.7185	0.7293	0.0107
j48	B	120	10,782	9,589	0.7183	0.718	-0.0003
ibk	A1	15	18,758	18,038	0.8424	0.8524	0.01
ibk	A1	30	14,651	13,610	0.8276	0.8331	0.0055
ibk	A1	60	15,515	14,677	0.8217	0.8326	0.0109
ibk	A1	120	10,782	9,524	0.8146	0.8218	0.0072
ibk	A2-VPN	15	9,793	9,380	0.7766	0.793	0.0164
ibk	A2-VPN	30	7,734	6,944	0.8047	0.81	0.0054
ibk	A2-VPN	60	6,935	6,523	0.7377	0.7552	0.0175
ibk	A2-VPN	120	5,631	4,648	0.7611	0.7441	-0.017
ibk	A2-NonVPN	15	8,965	8,694	0.7949	0.8118	0.0169
ibk	A2-NonVPN	30	6,917	6,694	0.7769	0.7989	0.022
ibk	A2-NonVPN	60	8,580	8,206	0.7687	0.7899	0.0213
ibk	A2-NonVPN	120	5,151	4,941	0.7349	0.7751	0.0402
ibk	B	15	18,758	18,074	0.69	0.7071	0.0171
ibk	B	30	14,651	13,638	0.6958	0.6931	-0.0027
ibk	B	60	15,515	14,729	0.6561	0.6697	0.0137
ibk	B	120	10,782	9,589	0.6531	0.661	0.0079

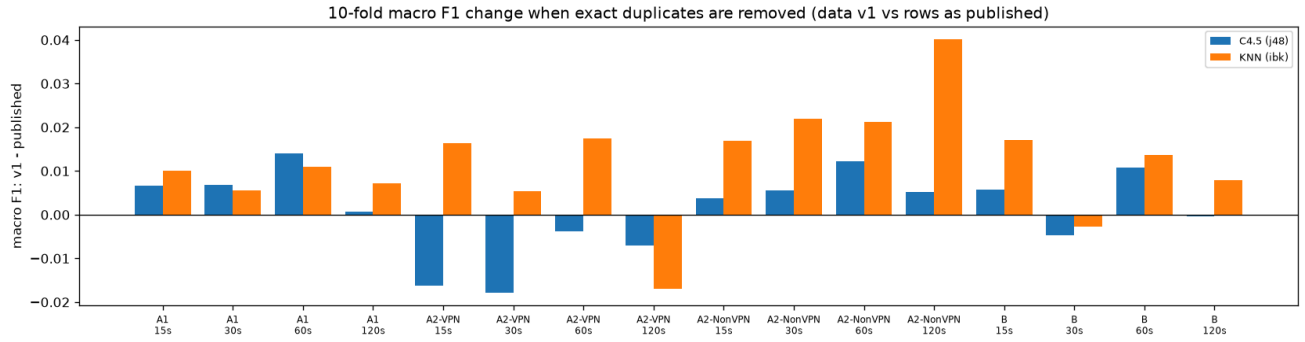


Figure 6. Duplicate effect

The reason the hypothesis failed is in the rows themselves. The removed duplicates are not copies of ordinary flows. Between 39% and 92% of them have an identical twin with a different label, against 2-10% of all rows, and their median flow has 0-2 packets (all flows: 8-43). They are near-empty flows that timing features can't tell apart. Keeping every copy multiplies errors that no model can avoid, which outweighs the advantage of having a test flow's copy in the training data:

Share of removed rows whose features also occur with another label, and packets per flow. A1 120 s packet counts use its published (corrupt) flowPktsPerSecond.

task	timeout	duplicates removed	removed with other-label twin	all rows with other-label twin	median packets, removed	median packets, all
A1	15	720	0.761	0.032	0.0	32.0
A1	30	1,041	0.51	0.039	2.0	22.0
A1	60	838	0.766	0.047	0.0	16.0
A1	120	1,257	0.434	0.056	0.0	0.0
A2-VPN	15	413	0.823	0.04	0.0	27.0
A2-VPN	30	790	0.395	0.045	2.0	17.0
A2-VPN	60	412	0.748	0.053	0.0	8.0
A2-VPN	120	983	0.45	0.098	2.0	8.0
A2-NonVPN	15	271	0.742	0.024	0.0	43.0
A2-NonVPN	30	223	0.897	0.03	0.0	26.0
A2-NonVPN	60	374	0.805	0.039	0.0	23.0
A2-NonVPN	120	210	0.919	0.039	0.0	17.0
B	15	684	0.804	0.035	0.0	32.0
B	30	1,013	0.529	0.042	2.0	22.0
B	60	786	0.818	0.053	0.0	16.0
B	120	1,193	0.562	0.078	2.0	12.0

The duplicate rule of data v1 stays: in our own split it guarantees that no flow is scored after being trained on, and this benchmark shows it changes 10-fold scores by only about +-0.02.

4.4 Storage after the baselines

Every official run saves its trained model (the plan's rule). After the first 147 runs (80 baselines, 64 benchmark, 2 pilot, 1 setup check), the MLflow store held 137.8 MB of artifacts and a 5.8 MB database (275.2 MB and 10.4 MB after the timing rerun of all 144 configs); the largest single run is 4.3 MB (SVM). Per model type:

kind	runs	total_MB	per_run_max_MB	model_file_max_MB
knn (holdout)	16	23.075	2.475	2.319
knn_minmax (paper_10fold)	32	66.742	3.596	3.438
logreg (holdout)	16	1.64	0.167	0.005
majority (holdout)	16	1.395	0.118	0.001
svm_rbf (holdout)	16	27.418	4.345	4.188
tree (holdout)	16	5.457	0.764	0.579
tree (paper_10fold)	32	11.674	0.817	0.629

Whole store now (after the rerun): artifacts 275.2 MB, mlflow.db 10.4 MB.

Tree ensembles in Phase 3 will be much larger: a 300-tree Random Forest, measured on the largest training set (B, 15 s), is a 171.9 MB file (22.4 MB with joblib compression level 3) [E03].

4.5 Phase 3: how the improvement experiments were run

Phase 3 (E04-E17) starts from the Phase 2 baselines and changes **one thing at a time** against the **current best** model of each task: the highest 15 s validation macro F1 so far, whatever the margin. Every experiment runs first at 15 s for all four tasks. A model is a **winner**, and is rerun at 30, 60 and 120 s with its 15 s settings, only if it beats the task's current best by at least 0.03; 0.03 is above the largest Phase 2 cross-validation

standard deviation (0.025), so a smaller gain could be noise from the split. The rule was fixed on 28 Oct, before any Phase 3 run, and written into every experiment entry before it ran. All Phase 3 runs use data v1, the same dependency lock file and one thread per model; every model is saved with joblib compression level 3. Every table below is copied unchanged from tables generated by code from the run history; the figures come from notebooks 03 and 04. Scores are validation scores of the fixed 70/15/15 split (seed 42); the test set is not used in Phase 3.

4.6 E04: KNN with a signed log transform

What. KNN (k = 5, as in E03) with a signed log, sign(x)·log1p(|x|), applied inside the pipeline before scaling, once with StandardScaler and once with RobustScaler (Section 3.7). **Why.** KNN compares flows by distance, and a few very large timing values (up to about 1.2e8 microseconds) dominate every distance on the raw features. **Threshold (pre-stated).** (a) the better variant beats the E03 KNN by at least 0.02 in at least 3 of 4 tasks; (b) the two scalers are within 0.02 of each other in at least 3 of 4 tasks.

task	model	val macro F1	CV macro F1	best before	best before F1	gain	winner (gain >= 0.03)
A1	E04-knn_log_robust	0.8929	0.8874 +- 0.0042	E02-tree	0.8901	0.0028	no
A1	E04-knn_log_standard	0.9048	0.9033 +- 0.0046	E02-tree	0.8901	0.0146	no
A2-VPN	E04-knn_log_robust	0.8372	0.8185 +- 0.0080	E02-tree	0.8247	0.0125	no
A2-VPN	E04-knn_log_standard	0.8544	0.8336 +- 0.0103	E02-tree	0.8247	0.0296	no
A2-NonVPN	E04-knn_log_robust	0.854	0.8688 +- 0.0050	E02-tree	0.8294	0.0247	no
A2-NonVPN	E04-knn_log_standard	0.867	0.8861 +- 0.0068	E02-tree	0.8294	0.0376	yes
B	E04-knn_log_robust	0.742	0.7555 +- 0.0094	E02-tree	0.7585	-0.0166	no
B	E04-knn_log_standard	0.7644	0.7798 +- 0.0078	E02-tree	0.7585	0.0059	no

Result. The signed log lifts KNN by 0.080-0.104 in every task, and StandardScaler beats RobustScaler in every task (by 0.0118-0.0224). The signed-log KNN also passes the E02 tree in all four tasks and becomes the current best; only A2-NonVPN (+0.0376) is a winner (A2-VPN misses the margin by 0.0004). Rerun at the other timeouts, it stays the best A2-NonVPN run at each:

task	timeout	model	val macro F1	CV macro F1	best other run	its F1	difference
A2-NonVPN	15 s	E04-knn_log_standard	0.867	0.8861 +- 0.0068	E04-knn_log_robust	0.854	0.0129
A2-NonVPN	30 s	E04-knn_log_standard	0.8523	0.8487 +- 0.0067	E02-tree	0.8283	0.024
A2-NonVPN	60 s	E04-knn_log_standard	0.859	0.8638 +- 0.0108	E02-tree	0.8246	0.0344
A2-NonVPN	120 s	E04-knn_log_standard	0.8491	0.8243 +- 0.0252	E02-tree	0.8393	0.0098

Verdict. (a) supported, (b) supported. [E04]

4.7 E05: Random Forest

What. A 300-tree Random Forest with default settings. **Why.** The E02 tree is unpruned (depth 16-35) and fits noise; averaging 300 trees grown on bootstrap samples lowers that variance. **Threshold.** (a) the forest beats the E02 tree by at least 0.03 in at least 3 of 4 tasks; (b) it beats the current best (the E04 KNN) in at least 2 of 4 tasks.

task	model	val macro F1	CV macro F1	best before	best before F1	gain	winner (gain >= 0.03)
A1	E05-rf	0.9203	0.9172 +- 0.0050	E04-knn_log_standard	0.9048	0.0156	no
A2-VPN	E05-rf	0.8679	0.8637 +- 0.0066	E04-knn_log_standard	0.8544	0.0135	no
A2-NonVPN	E05-rf	0.8941	0.9048 +- 0.0049	E04-knn_log_standard	0.867	0.0271	no
B	E05-rf	0.8147	0.8102 +- 0.0090	E04-knn_log_standard	0.7644	0.0503	yes

Result. The forest is the current best in all four tasks (+0.0135 to +0.0503 over the E04 KNN; +0.0302 to +0.0647 over the tree). Only B is a winner; its reruns:

task	timeout	model	val macro F1	CV macro F1	best other run	its F1	difference
B	15 s	E05-rf	0.8147	0.8102 +- 0.0090	E04-knn_log_standard	0.7644	0.0503
B	30 s	E05-rf	0.8114	0.7979 +- 0.0119	E02-tree	0.7476	0.0638
B	60 s	E05-rf	0.7771	0.7656 +- 0.0136	E02-tree	0.7058	0.0713
B	120 s	E05-rf	0.7835	0.7562 +- 0.0083	E02-tree	0.7158	0.0677

Verdict. (a) and (b) supported. The forest is the largest model so far: 171.9 MiB uncompressed for B at 15 s, 22.4 MiB as a compressed file. [E05 (Random Forest)]

4.8 E06: gradient boosting (XGBoost and LightGBM)

What. XGBoost and LightGBM with default settings. **Why.** Boosted trees correct the errors of earlier trees, which should help the classes the forest still confuses. **Threshold.** (a) the better booster beats the forest in at least 2 of 4 tasks; (b) the two are within 0.02 of each other in at least 3 of 4 tasks.

task	model	val macro F1	CV macro F1	best before	best before F1	gain	winner (gain >= 0.03)
A1	E06-lgbm	0.9181	0.9175 +- 0.0042	E05-rf	0.9203	-0.0022	no
A1	E06-xgb	0.9271	0.9251 +- 0.0046	E05-rf	0.9203	0.0067	no
A2-VPN	E06-lgbm	0.8694	0.8827 +- 0.0080	E05-rf	0.8679	0.0015	no
A2-VPN	E06-xgb	0.8758	0.8799 +- 0.0025	E05-rf	0.8679	0.0079	no
A2-NonVPN	E06-lgbm	0.8983	0.9224 +- 0.0097	E05-rf	0.8941	0.0042	no
A2-NonVPN	E06-xgb	0.9015	0.9237 +- 0.0117	E05-rf	0.8941	0.0075	no
B	E06-lgbm	0.8201	0.8305 +- 0.0088	E05-rf	0.8147	0.0054	no

task	model	val macro F1	CV macro F1	best before	best before F1	gain	winner (gain >= 0.03)
B	E06-xgb	0.8303	0.8334 +- 0.0053	E05-rf	0.8147	0.0156	no

Result. Default XGBoost is the current best in all four tasks (+0.0067 to +0.0156 over the forest); LightGBM is 0.0032-0.0102 below it. No winner. **Verdict.** (a) and (b) supported. Default XGBoost is also one of the cheapest models to run: 1.13-11.76 ms per 1,000 flows, against 21.91-27.28 ms for the forest. The figures show validation macro F1 with cross-validation mean +- std for every E02-E06 model at 15 s, the F1 of every class, and macro F1 against training and prediction time. [E06 (gradient boosting)]

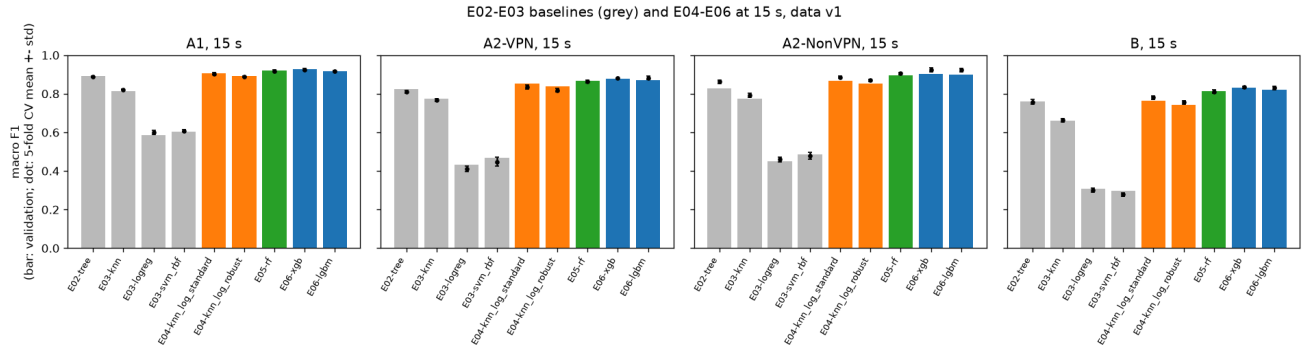


Figure 7. E02-E06 at 15 s

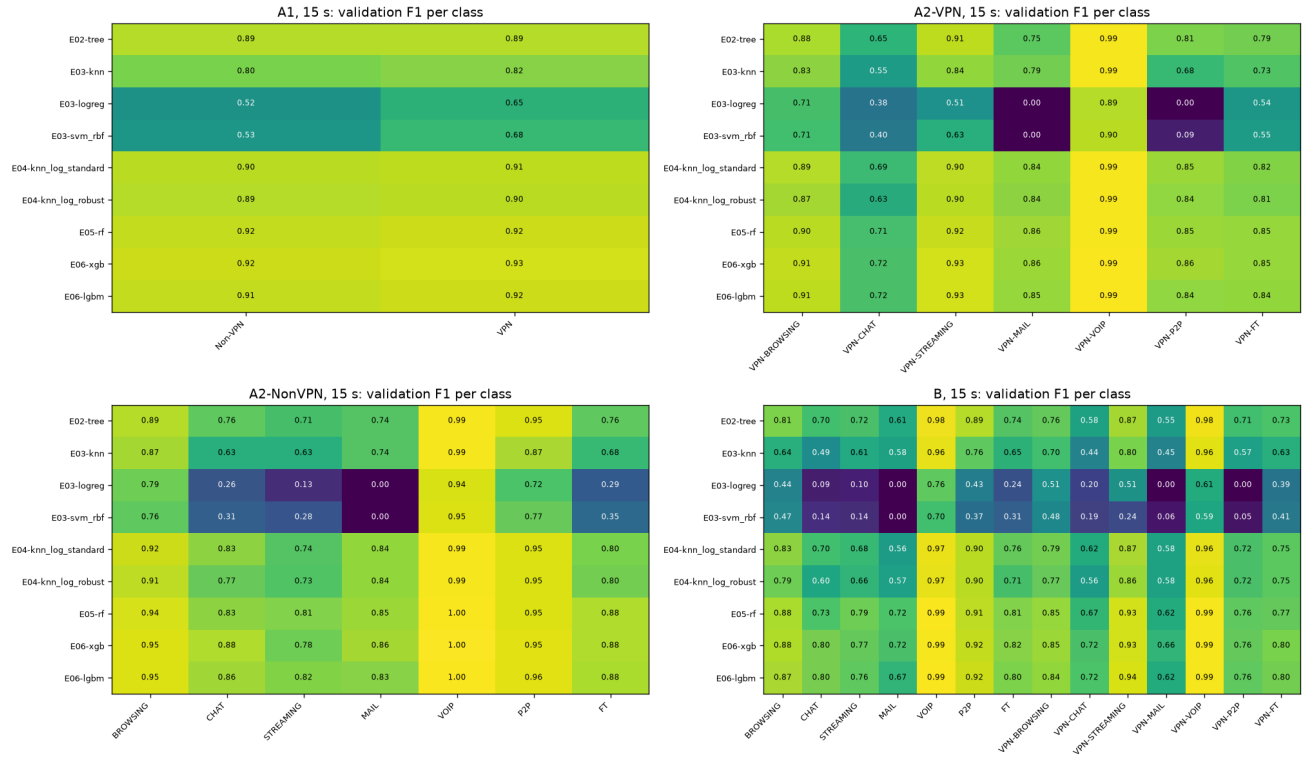


Figure 8. Per-class F1, E02-E06

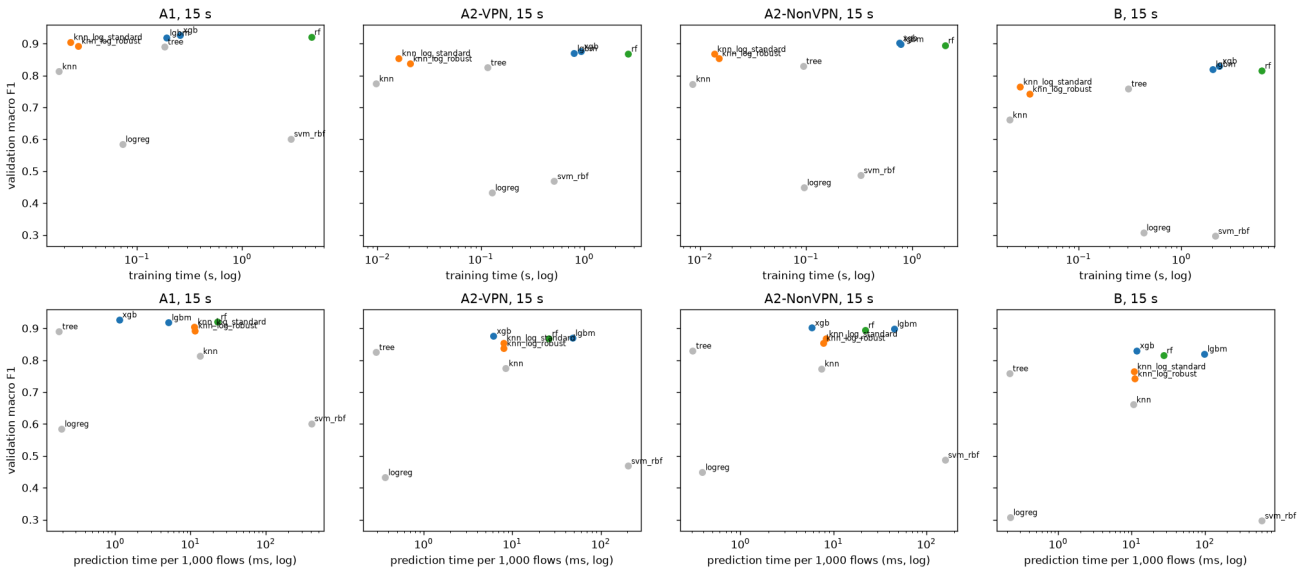


Figure 9. Macro F1 against cost, E02-E06

4.9 E07: hyperparameter tuning with Optuna

What. For each task the best 2 of Random Forest, XGBoost and LightGBM, plus the signed-log KNN, each tuned with the same budget: 50 Optuna trials (TPE sampler, seed 42), each trial scored by 5-fold cross-validation macro F1 on the training rows only. The search ranges were fixed in the plan before running. The best trial is retrained and scored on validation like any other run. **Why.** Defaults are general-purpose; giving every candidate the same budget means no model wins only because it was tuned more. **Threshold.** (a) tuned above untuned in at least 8 of 12 searches; (b) the best E07 (tuning) model beats the current best (default XGBoost) by at least 0.01 in at least 2 of 4 tasks; (c) the tuned KNN stays below the best tuned tree ensemble in all four tasks.

task	ranking (15 s val macro F1)	tuned in E07	KNN preprocessing for E07
A1	xgb 0.9271, rf 0.9203, lgbm 0.9181	xgb, rf	knn_log_standard 0.9048
A2-VPN	xgb 0.8758, lgbm 0.8694, rf 0.8679	xgb, lgbm	knn_log_standard 0.8544
A2-NonVPN	xgb 0.9015, lgbm 0.8983, rf 0.8941	xgb, lgbm	knn_log_standard 0.8670
B	xgb 0.8303, lgbm 0.8201, rf 0.8147	xgb, lgbm	knn_log_standard 0.7644

task	model	val macro F1	CV macro F1	best before	best before F1	gain	winner (gain >= 0.03)
A1	E07-knn_log_standard	0.9163	0.9122 +- 0.0047	E06-xgb	0.9271	-0.0108	no
A1	E07-rf	0.9181	0.9209 +- 0.0025	E06-xgb	0.9271	-0.009	no
A1	E07-xgb	0.9259	0.9293 +- 0.0065	E06-xgb	0.9271	-0.0011	no
A2-VPN	E07-knn_log_standard	0.8625	0.8575 +- 0.0048	E06-xgb	0.8758	-0.0133	no
A2-VPN	E07-lgbm	0.8768	0.8902 +- 0.0023	E06-xgb	0.8758	0.001	no
A2-VPN	E07-xgb	0.8661	0.8838 +- 0.0011	E06-xgb	0.8758	-0.0097	no
A2-NonVPN	E07-knn_log_standard	0.8929	0.9060 +- 0.0071	E06-xgb	0.9015	-0.0086	no

task	model	val macro F1	CV macro F1	best before	best before F1	gain	winner (gain >= 0.03)
A2-NonVPN	E07-lgbm	0.9028	0.9329 +- 0.0072	E06-xgb	0.9015	0.0013	no
A2-NonVPN	E07-xgb	0.9025	0.9294 +- 0.0067	E06-xgb	0.9015	0.001	no
B	E07-knn_log_standard	0.8006	0.8044 +- 0.0111	E06-xgb	0.8303	-0.0297	no
B	E07-lgbm	0.8352	0.8447 +- 0.0058	E06-xgb	0.8303	0.0049	no
B	E07-xgb	0.8428	0.8405 +- 0.0056	E06-xgb	0.8303	0.0125	no

Result. Tuning raised the cross-validation score in all 12 searches but the validation score in only 9: the best of 50 cross-validation scores is optimistic. The largest gains are for KNN (B +0.0361, A2-NonVPN +0.0260); the ensembles gain at most +0.0151. Only B improves by more than 0.01 (tuned XGBoost 0.8428, +0.0125). No winner. In every search the retrained model's cross-validation score equals its best trial's exactly. **Verdict.** (a) supported (9 of 12), (b) not supported (1 of 4 tasks), (c) supported. [E07]

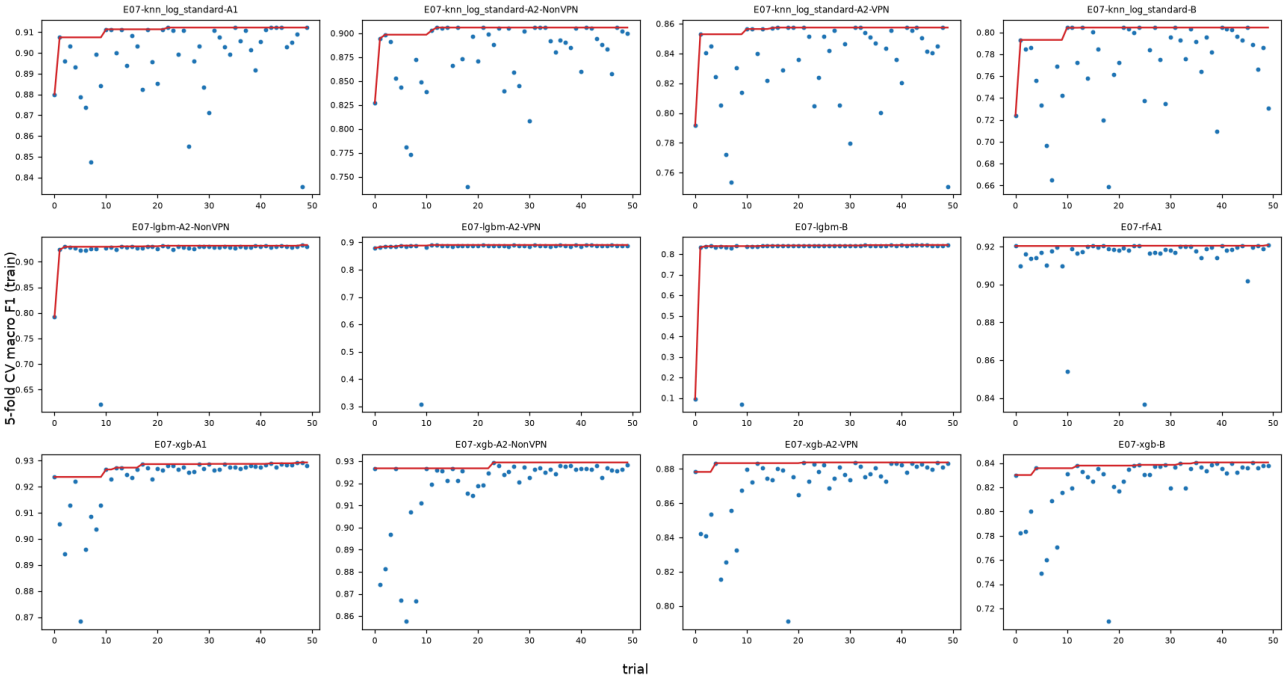


Figure 10. E07 Optuna histories

4.10 E08: class balancing

What. On the current best model per task, a ladder: none, then class weights, then random oversampling, then SMOTE (inside the pipeline, so only training folds are resampled); a step runs only if every earlier step helped. **Why.** Three tasks are imbalanced (largest to smallest class 5.79:1 in A2-VPN, 11.85:1 in A2-NonVPN and B), and macro F1 weights every class equally. **Threshold.** (a) the best step beats "none" by at least 0.01 in at least 2 of the 3 imbalanced tasks; (b) in A1 no step moves macro F1 by more than 0.01; (c) the best step raises the F1 of the weakest class in at least 2 of the 3 imbalanced tasks.

task	step	run	val macro F1	change vs none	F1 of weakest class without balancing	helped	next
A1	none	E06-xgb	0.9271	0.0	Non-VPN 0.9240		
A1	class_weight	E08-xgb_class_weight	0.9237	-0.0034	Non-VPN 0.9202	no	stop: this step did not help

task	step	run	val macro F1	change vs none	F1 of weakest class without balancing	helped	next
A2-VPN	none	E07-lgbm	0.8768	0.0	VPN-CHAT 0.7343		
A2-VPN	class_weight	E08-lgbm_class_weight	0.883	0.0062	VPN-CHAT 0.7463	yes	
A2-VPN	oversample	E08-lgbm_oversample	0.883	0.0062	VPN-CHAT 0.7477	no	stop: this step did not help
A2-NonVPN	none	E07-lgbm	0.9028	0.0	STREAMING 0.8095		
A2-NonVPN	class_weight	E08-lgbm_class_weight	0.9049	0.0022	STREAMING 0.8189	yes	
A2-NonVPN	oversample	E08-lgbm_oversample	0.9075	0.0047	STREAMING 0.8372	yes	
A2-NonVPN	smote	E08-lgbm_smote	0.9046	0.0019	STREAMING 0.8217	no	stop: this step did not help
B	none	E07-xgb	0.8428	0.0	VPN-MAIL 0.6496		
B	class_weight	E08-xgb_class_weight	0.8364	-0.0064	VPN-MAIL 0.6290	no	stop: this step did not help

Result. Every change is within +-0.0064. Balancing helps a little in A2-VPN (class weights +0.0062) and A2-NonVPN (random oversampling +0.0047) and hurts in A1 and B. **Verdict.** (a) not supported, (b) supported, (c) supported. The weak classes (VPN-CHAT, STREAMING, VPN-MAIL) seem to fail because their flows look like those of other classes, not because there are too few of them; E08 (class balancing) does not test this directly. [E08]

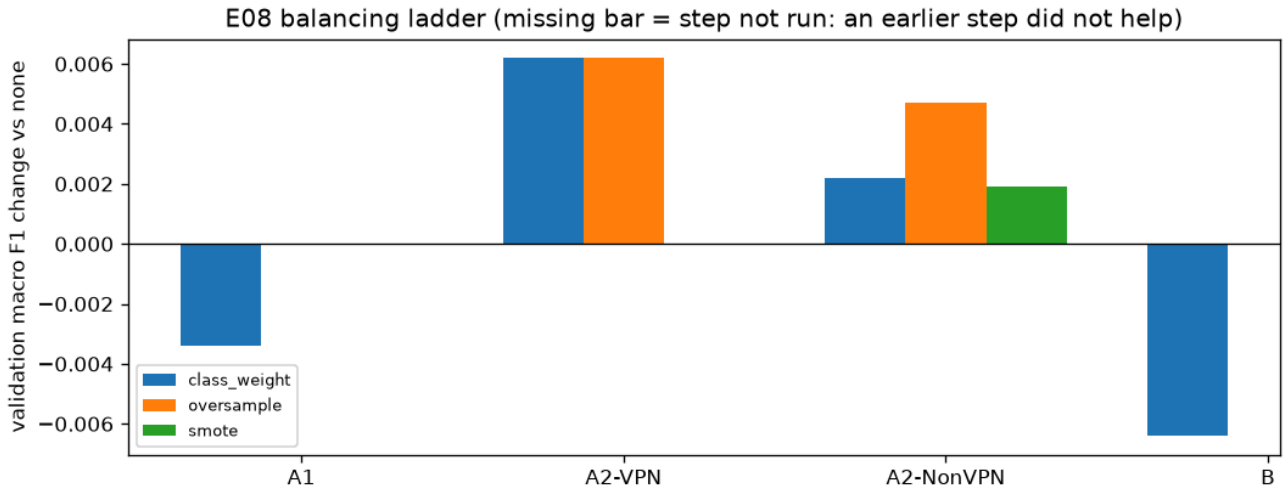


Figure 11. E08 balancing ladder

4.11 E10: task B as one 14-class model or two steps

What. A two-step model for task B: a VPN / non-VPN model routes each flow to one of two 7-class models. All three models use the current best B model type and settings (tuned XGBoost), and all are trained on B's training rows only. They are never trained on the A1 split, because A1 and B hold the same flows split differently. **Why.** Task B mixes two questions; separating them could help. **Threshold.** (a) the two-step score is within 0.01 of the direct model; (b) its first step makes fewer VPN / non-VPN errors than the direct model (180); (c) more than half of its errors come from a wrong first step.

structure	run	val macro F1	CV macro F1	val accuracy	rows	errors	group errors (VPN / non-VPN wrong)	type errors (right group, wrong type)	group accuracy	share of errors from the group
direct 14-class	E07-xgb	0.8428	0.8405 +- 0.0056	0.8794	2,711	327	180	147	0.9336	0.5505

structure	run	val macro F1	CV macro F1	val accuracy	rows	errors	group errors (VPN / non-VPN wrong)	type errors (right group, wrong type)	group accuracy	share of errors from the group
two-step	E10-xgb_two_step	0.8318	0.8274 +- 0.0083	0.8727	2,711	345	191	154	0.9295	0.5536

Result. The two-step model scores 0.0110 lower, makes more group errors (191 vs 180), and 55.36% of its errors come from the first step. The direct model also makes 55.05% of its errors across the VPN / non-VPN line: telling VPN from non-VPN is the hard part either way. **Verdict.** (a) not supported, (b) not supported, (c) supported. Task B stays a direct 14-class model. [E10 (one model or two steps for task B)]

4.12 E17: the same tuning budget for every method type

What. The same 50-trial search as E07 for every method type not yet tuned: decision tree, logistic regression and SVM-RBF in all four tasks, Random Forest in A2-VPN, A2-NonVPN and B, and LightGBM in A1 (16 searches). Logistic regression and SVM may choose the signed log inside the search. **Why.** Without it, a comparison of method types would mix tuned and untuned models, the weakness we point out in earlier studies. Added on 28 Oct after E10, with the ranges agreed before running. **Threshold.** (a) tuned above untuned in at least 12 of 16 searches; (b) the best run per task is still XGBoost or LightGBM in at least 3 of 4 tasks; (c) the signed log is chosen in at least 6 of the 8 logistic-regression and SVM searches.

task	model	val macro F1	CV macro F1	best before	best before F1	gain	winner (gain >= 0.03)
A1	E17-lgbm	0.9308	0.9320 +- 0.0028	E06-xgb	0.9271	0.0037	no
A1	E17-logreg	0.6517	0.6762 +- 0.0051	E06-xgb	0.9271	-0.2754	no
A1	E17-svm_rbf	0.9259	0.9187 +- 0.0044	E06-xgb	0.9271	-0.0011	no
A1	E17-tree	0.8905	0.8915 +- 0.0043	E06-xgb	0.9271	-0.0366	no
A2-VPN	E17-logreg	0.7263	0.7344 +- 0.0074	E08-lgbm_class_weight	0.883	-0.1566	no
A2-VPN	E17-rf	0.8704	0.8707 +- 0.0069	E08-lgbm_class_weight	0.883	-0.0126	no
A2-VPN	E17-svm_rbf	0.8707	0.8668 +- 0.0074	E08-lgbm_class_weight	0.883	-0.0123	no
A2-VPN	E17-tree	0.8292	0.8129 +- 0.0071	E08-lgbm_class_weight	0.883	-0.0538	no
A2-NonVPN	E17-logreg	0.8002	0.8138 +- 0.0120	E08-lgbm_oversample	0.9075	-0.1073	no
A2-NonVPN	E17-rf	0.8829	0.9063 +- 0.0050	E08-lgbm_oversample	0.9075	-0.0246	no
A2-NonVPN	E17-svm_rbf	0.9058	0.9256 +- 0.0042	E08-lgbm_oversample	0.9075	-0.0017	no
A2-NonVPN	E17-tree	0.8294	0.8619 +- 0.0086	E08-lgbm_oversample	0.9075	-0.0781	no
B	E17-logreg	0.5862	0.5909 +- 0.0067	E07-xgb	0.8428	-0.2566	no
B	E17-rf	0.8167	0.8270 +- 0.0092	E07-xgb	0.8428	-0.0261	no

task	model	val macro F1	CV macro F1	best before	best before F1	gain	winner (gain >= 0.03)
B	E17-svm_rbf	0.8242	0.8228 +- 0.0059	E07-xgb	0.8428	-0.0186	no
B	E17-tree	0.7585	0.7575 +- 0.0132	E07-xgb	0.8428	-0.0843	no

Result. SVM-RBF gains the most (+0.3243 to +0.5277), and every logistic-regression and SVM search chose the signed log. The tuned SVM ends 0.0017-0.0186 below each task's best. Logistic regression gains +0.0670 to +0.3515 but stays at 0.5862-0.8002. The tree (0.0000 to +0.0044) and the forest (-0.0111 to +0.0025) barely change. Tuned LightGBM is the new A1 best (0.9308). No winner. **Verdict.** (a) supported (13 of 16), (b) supported (4 of 4), (c) supported (8 of 8). The A2-NonVPN logistic regression stops at its fixed 1,000 iterations before converging (chosen C = 985); this is recorded as a limitation and was not changed after the run. [E17 (equal tuning budget)]

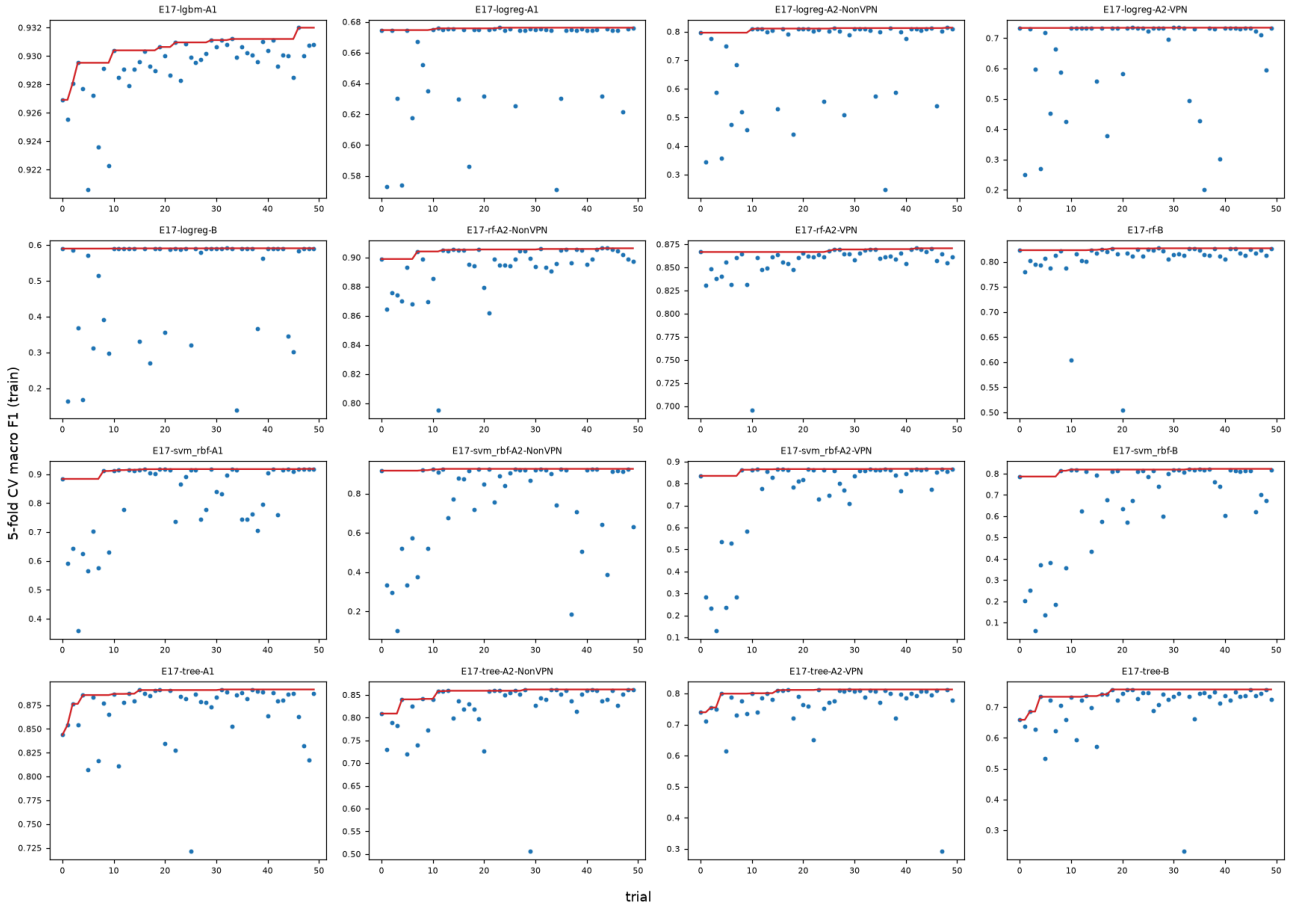


Figure 12. E17 Optuna histories

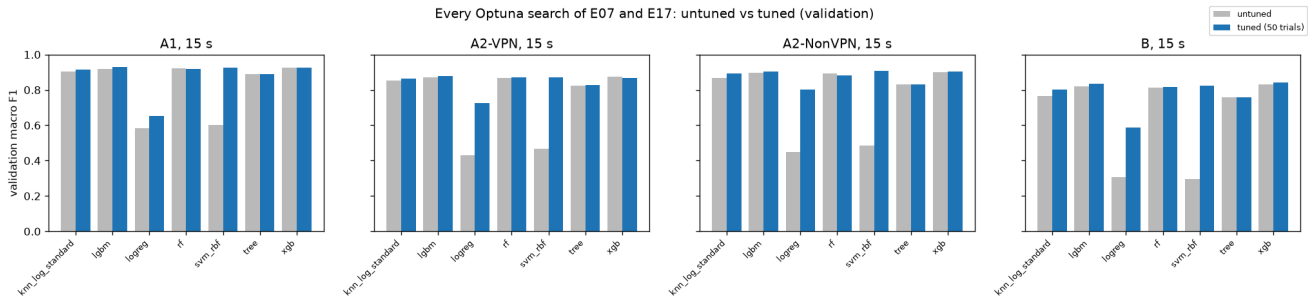


Figure 13. Untuned vs tuned, E07 and E17

4.13 Final choice per task and the comparison set

The final configuration per task is the run with the highest validation macro F1 among all model-selection runs (E02-E17, any timeout), decided on 28 Oct after E10. This replaces the rule set before Phase 3, under which any gap below 0.03 was a tie broken in favour of the simpler model. The cross-validation standard deviations of the headline models (0.0028-0.0058) are far below 0.03, and under the old rule KNN or the SVM would have been chosen at 0.0146-0.0286 below the best. This is a deviation from the pre-set rule and is reported as one (Section 3.8). The models the old rule would pick were also scored on the test set (E12), so the cost of either rule is visible (Section 5.3). The only fallback is a saved model file above 50 MB; none of the four is.

task	run	method type	timeout	val macro F1	next best	next best F1	model file MiB	skipped for file size
A1	E17-lgbm-A1-15s	LightGBM	15	0.9308	E06-xgb-A1-15s	0.9271	7.09	none
A2-VPN	E08-lgbm_class_weight-A2-VPN-15s	LightGBM	15	0.883	E08-lgbm_oversample-A2-VPN-15s	0.883	25.12	none
A2-NonVPN	E08-lgbm_oversample-A2-NonVPN-15s	LightGBM	15	0.9075	E17-svm_rbf-A2-NonVPN-15s	0.9058	4.32	none
B	E07-xgb-B-15s	XGBoost	15	0.8428	E08-xgb_class_weight-B-15s	0.8364	4.31	none

task	run	method type	val macro F1	below the best by	runs within 0.03	in comparison set
A1	E04-knn_log_standard-A1-15s	KNN	0.9048	0.026	10	no
A2-VPN	E04-knn_log_standard-A2-VPN-15s	KNN	0.8544	0.0286	11	no
A2-NonVPN	E07-knn_log_standard-A2-NonVPN-15s	KNN	0.8929	0.0146	11	yes
B	E17-svm_rbf-B-15s	SVM-RBF	0.8242	0.0186	9	yes

The **comparison set** holds, per task, the best 15 s validation variant of each of the 7 method types, 28 models in all. It was fixed before the test set was opened. In Phase 4 each was retrained on training + validation data and scored once on the test set, with 26 paired bootstrap tests on macro F1 (Holm correction), fixed before the test set was opened (Section 3.3, E12; results in Section 5.3).

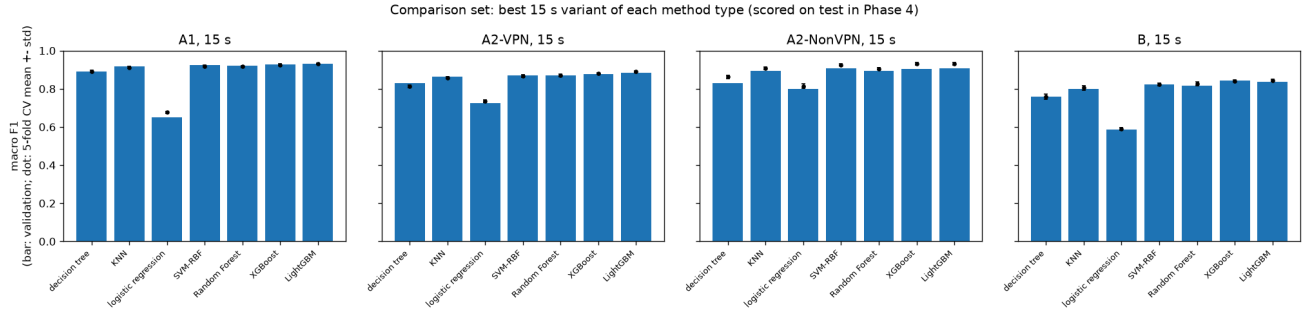


Figure 14. Comparison set at 15 s

task	method type	run	val macro F1	variants compared
A1	decision tree	E17-tree-A1-15s	0.8905	2
A1	KNN	E07-knn_log_standard-A1-15s	0.9163	4
A1	logistic regression	E17-logreg-A1-15s	0.6517	2
A1	SVM-RBF	E17-svm_rbf-A1-15s	0.9259	2
A1	Random Forest	E05-rf-A1-15s	0.9203	2
A1	XGBoost	E06-xgb-A1-15s	0.9271	3

task	method type	run	val macro F1	variants compared
A1	LightGBM	E17-lgbm-A1-15s	0.9308	2
A2-VPN	decision tree	E17-tree-A2-VPN-15s	0.8292	2
A2-VPN	KNN	E07-knn_log_standard-A2-VPN-15s	0.8625	4
A2-VPN	logistic regression	E17-logreg-A2-VPN-15s	0.7263	2
A2-VPN	SVM-RBF	E17-svm_rbf-A2-VPN-15s	0.8707	2
A2-VPN	Random Forest	E17-rf-A2-VPN-15s	0.8704	2
A2-VPN	XGBoost	E06-xgb-A2-VPN-15s	0.8758	2
A2-VPN	LightGBM	E08-lgbm_class_weight-A2-VPN-15s	0.883	4
A2-NonVPN	decision tree	E02-tree-A2-NonVPN-15s	0.8294	2
A2-NonVPN	KNN	E07-knn_log_standard-A2-NonVPN-15s	0.8929	4
A2-NonVPN	logistic regression	E17-logreg-A2-NonVPN-15s	0.8002	2
A2-NonVPN	SVM-RBF	E17-svm_rbf-A2-NonVPN-15s	0.9058	2
A2-NonVPN	Random Forest	E05-rf-A2-NonVPN-15s	0.8941	2
A2-NonVPN	XGBoost	E07-xgb-A2-NonVPN-15s	0.9025	2
A2-NonVPN	LightGBM	E08-lgbm_oversample-A2-NonVPN-15s	0.9075	5
B	decision tree	E02-tree-B-15s	0.7585	2
B	KNN	E07-knn_log_standard-B-15s	0.8006	4
B	logistic regression	E17-logreg-B-15s	0.5862	2
B	SVM-RBF	E17-svm_rbf-B-15s	0.8242	2
B	Random Forest	E17-rf-B-15s	0.8167	2
B	XGBoost	E07-xgb-B-15s	0.8428	4
B	LightGBM	E07-lgbm-B-15s	0.8352	2

4.14 E09: feature-group ablation (analysis only)

What. The four headline models retrained with one feature group only, with one group removed, and with the timing-only set (all but flowBytesPerSecond, the only size-based feature). **Threshold.** (a) every group alone scores below all 23 features in all four tasks; (b) dropping any one group costs at most 0.02 in at least 3 of 4 tasks; (c) the timing-only set is within 0.01 of all 23 features in at least 3 of 4 tasks. If any set scored at least 0.03 above all 23, the work would stop and the user would be asked (it did not happen).

task	feature set	features	val macro F1	change vs all	CV macro F1
A1	all	23	0.9308	0.0	0.9320 +- 0.0028
A1	only_fiat	4	0.8078	-0.1229	0.8111 +- 0.0059
A1	only_biat	4	0.8243	-0.1064	0.8136 +- 0.0100
A1	only_flowiat	4	0.8507	-0.08	0.8456 +- 0.0043
A1	only_active	4	0.5489	-0.3819	0.5618 +- 0.0124
A1	only_idle	4	0.5505	-0.3803	0.5614 +- 0.0107
A1	only_rates_duration	3	0.8392	-0.0915	0.8363 +- 0.0063
A1	no_fiat	19	0.9256	-0.0052	0.9187 +- 0.0050
A1	no_biat	19	0.9152	-0.0155	0.9161 +- 0.0050
A1	no_flowiat	19	0.9322	0.0014	0.9222 +- 0.0019
A1	no_active	19	0.9296	-0.0011	0.9300 +- 0.0024
A1	no_idle	19	0.9304	-0.0004	0.9302 +- 0.0031

task	feature set	features	val macro F1	change vs all	CV macro F1
A1	no_rates_duration	20	0.9185	-0.0122	0.9133 +- 0.0054
A1	timing_only	22	0.9159	-0.0148	0.9140 +- 0.0031
A2-VPN	all	23	0.883	0.0	0.8893 +- 0.0044
A2-VPN	only_fiat	4	0.7593	-0.1237	0.7326 +- 0.0146
A2-VPN	only_biat	4	0.7199	-0.1631	0.6958 +- 0.0108
A2-VPN	only_flowiat	4	0.7861	-0.0969	0.7690 +- 0.0107
A2-VPN	only_active	4	0.3252	-0.5578	0.3222 +- 0.0193
A2-VPN	only_idle	4	0.3067	-0.5763	0.3208 +- 0.0145
A2-VPN	only_rates_duration	3	0.779	-0.104	0.7661 +- 0.0082
A2-VPN	no_fiat	19	0.8651	-0.0179	0.8723 +- 0.0037
A2-VPN	no_biat	19	0.8724	-0.0105	0.8760 +- 0.0037
A2-VPN	no_flowiat	19	0.8752	-0.0078	0.8833 +- 0.0054
A2-VPN	no_active	19	0.886	0.003	0.8895 +- 0.0041
A2-VPN	no_idle	19	0.8826	-0.0004	0.8893 +- 0.0054
A2-VPN	no_rates_duration	20	0.8435	-0.0395	0.8449 +- 0.0083
A2-VPN	timing_only	22	0.849	-0.034	0.8410 +- 0.0095
A2-NonVPN	all	23	0.9075	0.0	0.9310 +- 0.0058
A2-NonVPN	only_fiat	4	0.7535	-0.154	0.7720 +- 0.0087
A2-NonVPN	only_biat	4	0.7271	-0.1804	0.7407 +- 0.0140
A2-NonVPN	only_flowiat	4	0.8165	-0.091	0.8288 +- 0.0095
A2-NonVPN	only_active	4	0.3355	-0.572	0.3208 +- 0.0128
A2-NonVPN	only_idle	4	0.3038	-0.6037	0.3225 +- 0.0177
A2-NonVPN	only_rates_duration	3	0.8471	-0.0604	0.8396 +- 0.0110
A2-NonVPN	no_fiat	19	0.9024	-0.0051	0.9260 +- 0.0087
A2-NonVPN	no_biat	19	0.8993	-0.0082	0.9240 +- 0.0087
A2-NonVPN	no_flowiat	19	0.903	-0.0045	0.9189 +- 0.0065
A2-NonVPN	no_active	19	0.9093	0.0018	0.9283 +- 0.0066
A2-NonVPN	no_idle	19	0.9039	-0.0036	0.9300 +- 0.0045
A2-NonVPN	no_rates_duration	20	0.8774	-0.0301	0.8980 +- 0.0037
A2-NonVPN	timing_only	22	0.8744	-0.0331	0.9000 +- 0.0060
B	all	23	0.8428	0.0	0.8405 +- 0.0056
B	only_fiat	4	0.64	-0.2028	0.6399 +- 0.0089
B	only_biat	4	0.6098	-0.233	0.6147 +- 0.0121
B	only_flowiat	4	0.6716	-0.1712	0.6926 +- 0.0102
B	only_active	4	0.2138	-0.629	0.2262 +- 0.0072
B	only_idle	4	0.2083	-0.6345	0.2118 +- 0.0092
B	only_rates_duration	3	0.7043	-0.1385	0.6886 +- 0.0091
B	no_fiat	19	0.8244	-0.0184	0.8266 +- 0.0072
B	no_biat	19	0.8163	-0.0265	0.8189 +- 0.0067
B	no_flowiat	19	0.8262	-0.0166	0.8284 +- 0.0052
B	no_active	19	0.8479	0.0051	0.8400 +- 0.0071

task	feature set	features	val macro F1	change vs all	CV macro F1
B	no_idle	19	0.8428	0.0	0.8405 +- 0.0058
B	no_rates_duration	20	0.7879	-0.0549	0.7936 +- 0.0076
B	timing_only	22	0.7987	-0.0441	0.7949 +- 0.0062

Result. No single group comes close to all 23 features (the best single group is 0.0604-0.1385 below). Removing rates + duration costs the most in A2-VPN, A2-NonVPN and B (0.0395, 0.0301, 0.0549); in A1 removing the backward gaps costs more (0.0155 against 0.0122). Active or idle features can be removed with almost no effect (at most +-0.0051). The timing-only set loses 0.0148-0.0441, so packet sizes carry information here. **Verdict.** (a) supported, (b) not supported, (c) not supported. Nothing changes in the final configurations. [E09 (feature-group ablation)]

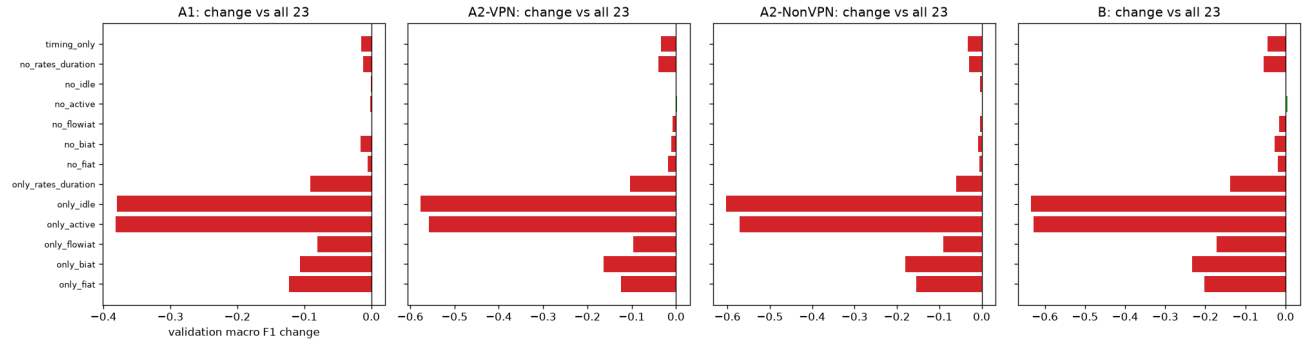


Figure 15. E09 feature-group ablation

4.15 E13: learning curve (analysis only)

What. The headline models trained on 10%, 20%, 40%, 60% and 80% of the training rows (stratified samples, 5 seeds each) and scored on the full validation set; 100% is the headline run. **Threshold.** (a) 100% scores above 10% in all four tasks; (b) the gain from 80% to 100% is below 0.01 in at least 3 of 4 tasks; (c) the spread over seeds is larger at 10% than at 80% in at least 3 of 4 tasks.

task	train fraction	runs	training rows	val macro F1 mean	val macro F1 std
A1	0.1	5	1,262	0.8724	0.0062
A1	0.2	5	2,525	0.8952	0.0041
A1	0.4	5	5,050	0.9106	0.0066
A1	0.6	5	7,575	0.9213	0.0039
A1	0.8	5	10,100	0.927	0.0029
A1	1.0	1	12,626	0.9308	
A2-VPN	0.1	5	656	0.7793	0.0052
A2-VPN	0.2	5	1,313	0.826	0.0101
A2-VPN	0.4	5	2,626	0.8582	0.0041
A2-VPN	0.6	5	3,939	0.8721	0.0053
A2-VPN	0.8	5	5,252	0.8804	0.0036
A2-VPN	1.0	1	6,566	0.883	
A2-NonVPN	0.1	5	608	0.8176	0.0093
A2-NonVPN	0.2	5	1,217	0.8585	0.0118
A2-NonVPN	0.4	5	2,434	0.8806	0.0057
A2-NonVPN	0.6	5	3,651	0.8934	0.0045
A2-NonVPN	0.8	5	4,868	0.9005	0.0033

task	train fraction	runs	training rows	val macro F1 mean	val macro F1 std
A2-NonVPN	1.0	1	6,085	0.9075	
B	0.1	5	1,265	0.6969	0.0097
B	0.2	5	2,530	0.7468	0.0138
B	0.4	5	5,060	0.7975	0.0036
B	0.6	5	7,590	0.8156	0.0016
B	0.8	5	10,120	0.8297	0.0029
B	1.0	1	12,651	0.8428	

Result. Every task gains with more data (10% to 100%: +0.0584 to +0.1459). A1 and both A2 tasks are near their plateau (80% to 100%: +0.0026 to +0.0070), but the 14-class task B still gains +0.0131. **Verdict.** (a), (b) and (c) supported. [E13 (learning curve)]

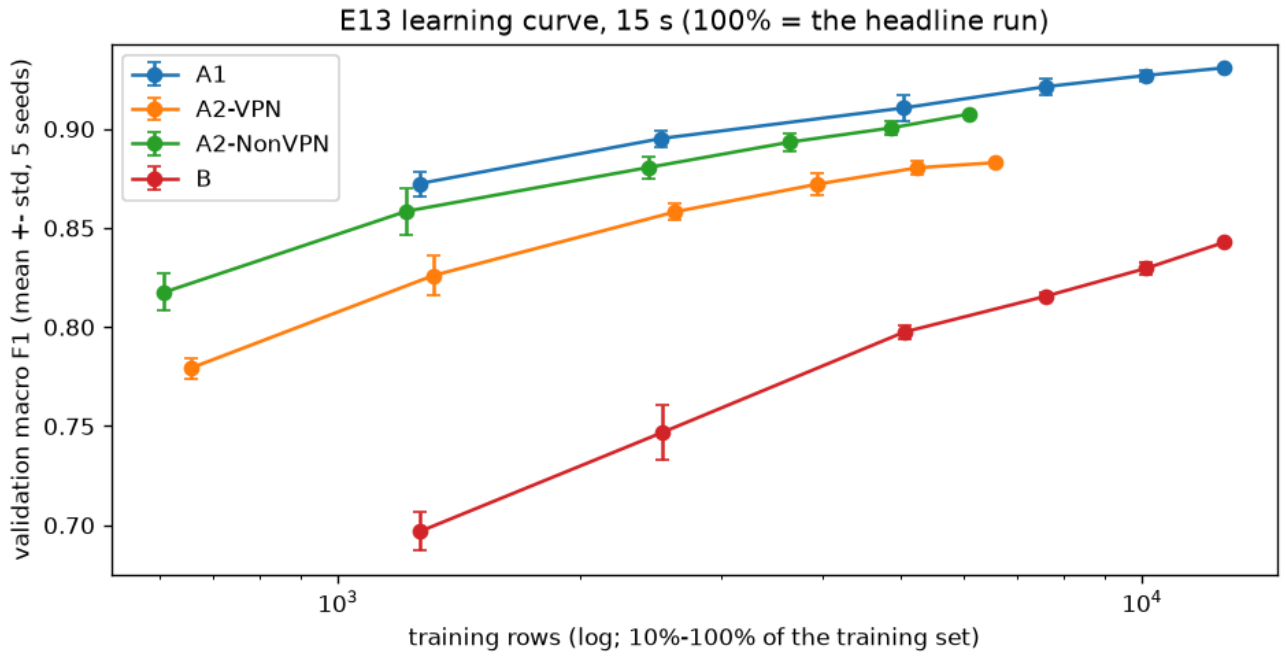


Figure 16. E13 learning curve

4.16 E14: split-seed stability (analysis only)

What. The headline models retrained on 10 new stratified splits of training + validation (seeds 42-51, same sizes). The seed-42 test rows are never pooled or looked at. Seed 42 here is a resplit, not the official split. **Threshold.** (a) the standard deviation is below 0.01 in at least 3 of 4 tasks; (b) mean cross-validation minus mean validation is within 0.01 in at least 3 of 4 tasks; (c) the official validation score lies inside the range of the 10 resplits in all four tasks.

task	seeds	val macro F1 mean	val macro F1 std	val macro F1 min	val macro F1 max	val accuracy mean	val accuracy std	CV macro F1 mean	CV - val mean	official split (seed 42) val macro F1
A1	10	0.9338	0.0042	0.9275	0.9397	0.9339	0.0042	0.9306	-0.0032	0.9308
A2-VPN	10	0.8862	0.0114	0.8689	0.902	0.9037	0.0066	0.8805	-0.0057	0.883
A2-NonVPN	10	0.9324	0.0071	0.9257	0.9468	0.9589	0.0044	0.9283	-0.0041	0.9075
B	10	0.8476	0.0084	0.8344	0.8636	0.8819	0.0074	0.8358	-0.0119	0.8428

Result. Scores move with the split by a standard deviation of 0.0042-0.0114. In A2-NonVPN the official validation score (0.9075) is below all 10 resplits (0.9257-0.9468): its validation rows are unusually hard, mostly for MAIL and P2P. This also explains the large cross-validation minus validation gaps seen for A2-NonVPN in E04-E17. **Verdict.** (a) supported, (b) supported, (c) not supported. [E14 (split-seed stability)]

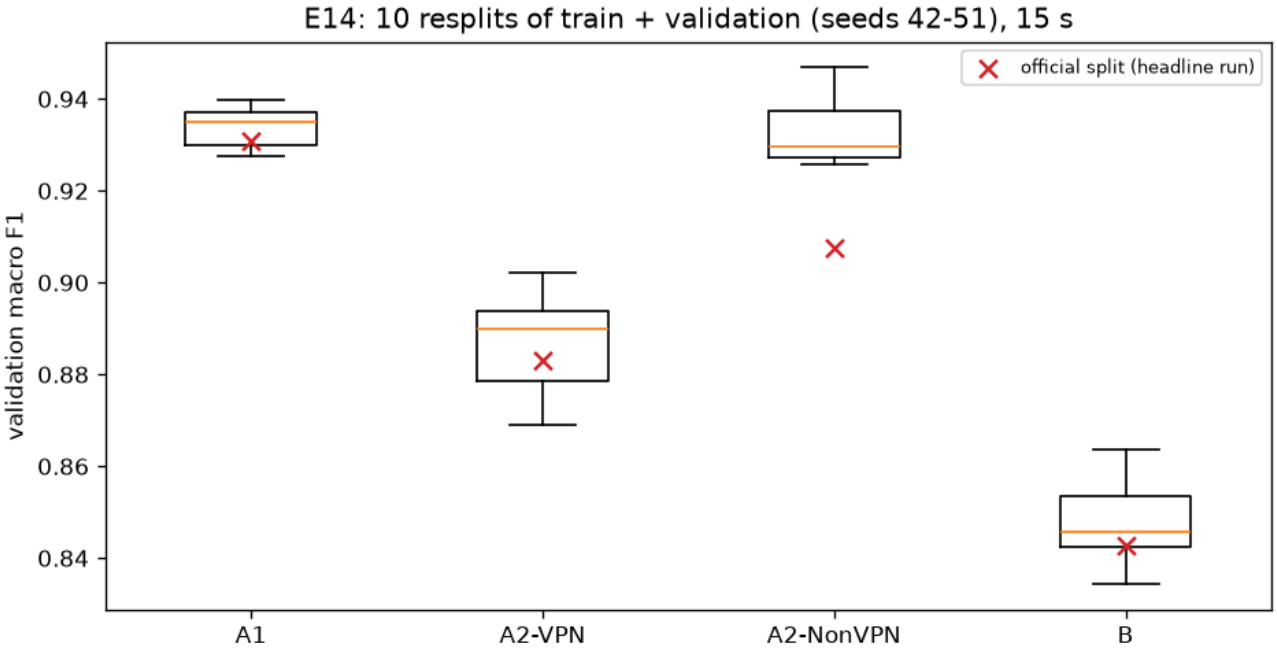


Figure 17. E14 seed stability

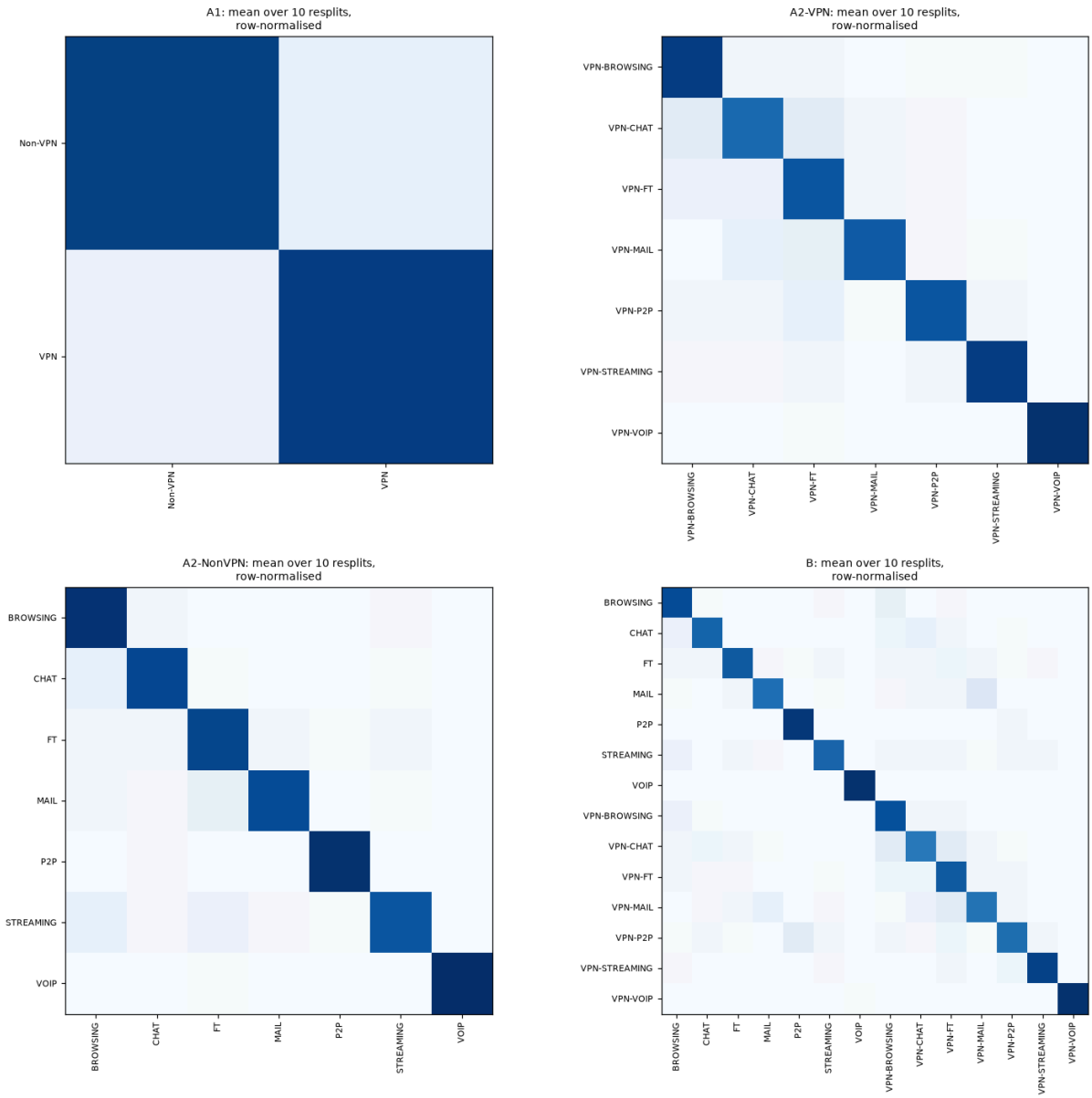


Figure 18. E14 mean confusion matrices

4.17 E11: session-aware split (not run)

The ARFF files hold only the 23 features and the label, with no flow IDs, addresses, ports, timestamps or capture-file names, so flows cannot be grouped by recording session (E00). E11 (session-aware split) therefore does not run. Exact duplicates were removed before the split (E00), and E14 and E16 (reproducing Isnan et al.)(b) measure related effects. The missing session split is a limitation (Section 7). [E11]

4.18 E16: reproducing Isnan et al. (2026), then correcting it

What. Isnan et al. report 88.01% accuracy for a 300-tree Random Forest on the Kaggle copy of the dataset, which is the eight A2 files from all four timeouts stacked (Section 2.9). E16 used its own data version (kaggle-k1) and ran four parts:

- **(a)** their exact setup: duplicates dropped over the whole file, a stratified 80/20 split with seed 42, and their grid search with accuracy as the score;
- **(b)** the same method with each timeout kept apart;
- **(c)** our tuned task-B XGBoost under (b);
- **(d)**, added in a recheck: (a)'s forest trained on mixed-timeout rows cut to each per-timeout training size.

Threshold. (a) accuracy and weighted F1 within 0.01 of theirs; (b) the per-timeout mean is at least 0.01 below (a); (c) our model's per-timeout mean is at least the forest's; (d) at equal training size, mixing timeouts adds less than 0.01 over (b).

run	split	train / test rows	grid pick	grid best CV accuracy	test accuracy	test weighted F1	test macro F1
E16-isnan-rf	isnan	32,789 / 8,198	{"max_depth": null, "min_samples_leaf": 1, "min_samples_split": 2, "n_estimators": 200}	0.8742	0.8796	0.8792	0.8407
E16-per_timeout-rf-120s	per_timeout_120s	7,671 / 1,918	{"max_depth": null, "min_samples_leaf": 1, "min_samples_split": 5, "n_estimators": 300}	0.8178	0.8206	0.817	0.7724
E16-per_timeout-rf-15s	per_timeout_15s	14,459 / 3,615	{"max_depth": 30, "min_samples_leaf": 1, "min_samples_split": 2, "n_estimators": 300}	0.8578	0.8658	0.8652	0.8323
E16-per_timeout-rf-30s	per_timeout_30s	10,910 / 2,728	{"max_depth": 30, "min_samples_leaf": 1, "min_samples_split": 2, "n_estimators": 300}	0.8457	0.8537	0.8527	0.815
E16-per_timeout-rf-60s	per_timeout_60s	11,783 / 2,946	{"max_depth": 30, "min_samples_leaf": 1, "min_samples_split": 2, "n_estimators": 300}	0.8307	0.8316	0.8308	0.7873
E16-per_timeout-xgb-120s	per_timeout_120s	7,671 / 1,918	fixed (final task-B settings)		0.8384	0.8346	0.7881
E16-per_timeout-xgb-15s	per_timeout_15s	14,459 / 3,615	fixed (final task-B settings)		0.8924	0.8919	0.8637
E16-per_timeout-xgb-30s	per_timeout_30s	10,910 / 2,728	fixed (final task-B settings)		0.8728	0.872	0.8327
E16-per_timeout-xgb-60s	per_timeout_60s	11,783 / 2,946	fixed (final task-B settings)		0.8551	0.8544	0.8076

training rows (timeouts mixed)	runs	test accuracy mean	std	min - max	15s test rows	30s test rows	60s test rows	120s test rows	(b) same rows, one timeout
32,789	1	0.8796			0.8913	0.9038	0.8982	0.8222	
14,459	5	0.8504	0.0021	0.8470 - 0.8529	0.8643	0.8798	0.8655	0.7887	15s: 0.8658
11,783	5	0.8435	0.0028	0.8401 - 0.8464	0.8592	0.8742	0.8558	0.7809	60s: 0.8316
10,910	5	0.8396	0.0024	0.8367 - 0.8430	0.8559	0.8708	0.8517	0.7755	30s: 0.8537
7,671	5	0.8243	0.0038	0.8202 - 0.8302	0.8402	0.8576	0.8372	0.7583	120s: 0.8206

Result.

- **(a)** Their result is reproduced: 0.8796 accuracy and 0.8792 weighted F1, against their 0.8801 and 0.8797, on the same split. Their accuracy hides weaker small classes (macro F1 0.8407).
- **(b)** With the timeouts kept apart, the same method scores 0.8206-0.8658 (mean 0.8430).
- **(d)** That drop is explained by training-set size, not by the mixing. Stacking the four files gives 2.3-4.3 times as many training rows. At equal size the mixed setup scores 0.8395 on average, slightly below (b).
- **(c)** Our tuned XGBoost beats their forest at every timeout (mean 0.8647; 0.8924 at 15 s).

The Kaggle rows use the A2 column layout and an 80/20 split, so these scores are not comparable with our validation scores. **Verdict.** (a), (b), (c) and (d) supported. E16 feeds no model selection. [E16]

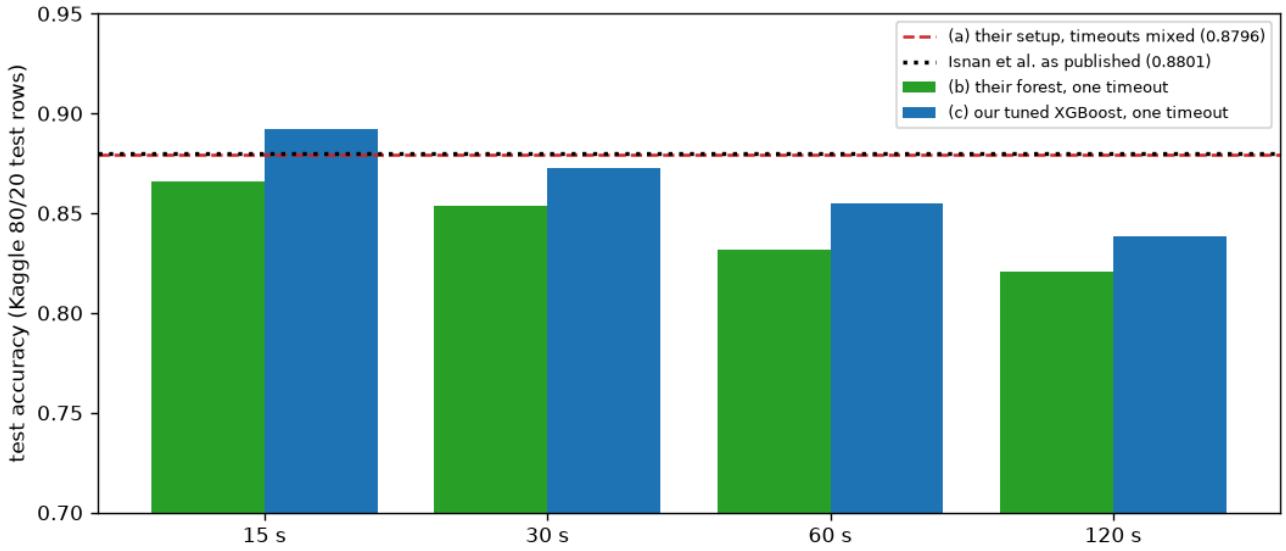


Figure 19. E16 Isnan et al. reproduced and corrected

4.19 Storage after Phase 3

Every official run saves its model (compressed, level 3). Per experiment and model type:

kind	runs	total_MB	per_run_max_MB	model_file_max_MB
E04 knn_log_robust	4	4.678	1.577	1.435
E04 knn_log_standard	7	6.54	1.579	1.437
E05 rf	7	102.403	22.6	22.436
E06 lgbm	4	4.56	2.168	2.014
E06 xgb	4	2.522	1.226	1.064
E07 knn_log_standard	4	4.78	1.604	1.437
E07 lgbm	3	92.646	62.694	62.51
E07 rf	1	3.835	3.835	3.72
E07 xgb	4	9.238	4.508	4.315
E08 lgbm	5	67.775	27.135	27.033
E08 xgb	2	4.724	4.511	4.348
E09 lgbm	39	482.749	27.644	27.549
E09 xgb	13	61.881	5.261	5.092
E10 xgb_two_step	1	3.856	3.856	3.728
E13 lgbm	75	474.7	21.265	21.163
E13 xgb	25	73.528	4.188	4.025
E14 lgbm	30	372.146	25.859	25.759
E14 xgb	10	44.771	4.494	4.336
E16 rf	26	453.605	35.049	34.902
E16 xgb	4	17.765	4.829	4.697
E17 lgbm	1	7.215	7.215	7.092
E17 logreg	4	0.515	0.199	0.004
E17 rf	3	68.011	45.962	45.773
E17 svm_rbf	4	2.944	1.479	1.306

kind	runs	total_MB	per_run_max_MB	model_file_max_MB
E17 tree	4	0.732	0.275	0.07

Whole store at this point: artifacts 2645.3 MiB, mlflow.db 35.8 MiB. This is far below the plan's 20 GB limit. The four headline model files are 4.31-25.12 MiB, all below the 50 MB limit set for final model files.

4.20 E15: calibration of the headline models (on validation, before the test set)

What. The Expected Calibration Error (15 equal-width bins; confidence = the probability of the predicted class) and a reliability diagram for each headline model, fitted on the training rows and measured on the validation rows; then isotonic calibration (`CalibratedClassifierCV`, one base model, mapping learned from 5-fold out-of-fold predictions on the training rows with shuffled folds, seed 42). **Why.** `predict.py` and the demo report a confidence for every prediction, so it must be trustworthy. **Rule and thresholds, fixed before running.** Calibration is adopted for a task if the uncalibrated validation ECE is above 0.05 and isotonic calibration lowers it; it changes only the reported confidence, never the predicted class. (a) uncalibrated ECE above 0.05 in at least 2 of 4 tasks; (b) isotonic ECE lower in all 4; (c) the predicted class unchanged on every row and validation macro F1 equal to the source run. **Correction before the counted runs.** The first pilot used `cv=5`, which means unshuffled folds; the training rows are stored in file order, so each fold held out a block of the file and the out-of-fold predictions were far too poor (A1 out-of-fold accuracy 0.6701 unshuffled against 0.9321 shuffled). The folds were changed to the project's shuffled folds before any counted E15 (calibration) run; the pilot run is kept and superseded.

task	run	headline	val accuracy	mean confidence, uncalibrated	mean confidence, isotonic	ECE uncalibrated	ECE isotonic	adopted	same predicted class (share of rows)	calibrated argmax would differ (share, unused)	val macro F1 = source run
A1	E15-A1-15s	E17-lgbm-A1-15s	0.9309	0.9861	0.9356	0.0552	0.014	yes	1.0	0.0007	yes
A2-VPN	E15-A2-VPN-15s	E08-lgbm_class_weight-A2-VPN-15s	0.8984	0.9692	0.9004	0.0714	0.0168	yes	1.0	0.0078	yes
A2-NonVPN	E15-A2-NonVPN-15s	E08-lgbm_oversample-A2-NonVPN-15s	0.9479	0.991	0.9469	0.0444	0.0152	no	1.0	0.0084	yes
B	E15-B-15s	E07-xgb-B-15s	0.8794	0.9144	0.8787	0.035	0.0099	no	1.0	0.0077	yes

Result. All four models are overconfident on validation. (a), (b) and (c) are supported: calibration was adopted for A1 (ECE 0.0552 to 0.0140) and A2-VPN (0.0714 to 0.0168), and not for A2-NonVPN (0.0444) or B (0.0350), whose uncalibrated ECE is not above 0.05. The test-set ECE, measured once, is in Section 5.4.

4.21 E12: final results on the test set

What. Each of the 30 final models (the 28 comparison-set members, which include the 4 headline models, and the 2 old-rule picks that are not members) was retrained on training + validation data and scored once on the test rows. The models, the 26 paired tests, the 10,000-resample stratified bootstrap and the calibration rule were all fixed before the test set was opened; a rehearsal of the same runs on the validation rows (fitted on the training rows, in a separate run store) passed every check first, and the official runs went in one batch, with no crash and no rerun. **Why.** A score on rows that no choice has seen is the honest estimate of performance on new flows from the same captures. **Hypotheses, fixed in Phase 3.** (a) every headline model within 0.03 of its validation score; (b) the headline model above the old rule's pick in at least 3 of 4 tasks; (c) Spearman's rank correlation [22] >= 0.8 between the validation and test order of the 7 method types in at least 3 of 4 tasks.

task	role	method type	run	source run	val macro F1	test macro F1	95% interval	test - val	test accuracy	accuracy 95% interval	test weighted F1	test balanced accuracy	CV macro F1 (fit rows)	CV - test	calibration
A1	headline	LightGBM	E12-final-A1	E17-lgbm-A1-15s	0.9308	0.9434	0.9348-0.9519	0.0126	0.9435	0.9350-0.9520	0.9435	0.9433	0.9322	-0.0111	isotonic
A2-VPN	headline	LightGBM	E12-final-A2-VPN	E08-lgbm_class_weight-A2-VPN-15s	0.883	0.8652	0.8431-0.8866	-0.0178	0.8955	0.8799-0.9112	0.896	0.8682	0.8827	0.0175	isotonic

task	role	method type	run	source run	val macro F1	test macro F1	95% interval	test - val	test accuracy	accuracy 95% interval	test weighted F1	test balanced accuracy	CV macro F1 (fit rows)	CV - test	calibration
A2-NonVPN	headline	LightGBM	E12-final-A2-NonVPN	E08-lgbm_oversample-A2-NonVPN-15s	0.9075	0.9403	0.9216-0.9573	0.0328	0.9609	0.9510-0.9709	0.9607	0.9343	0.9347	-0.0055	none
B	headline	XGBoost	E12-final-B	E07-xgb-B-15s	0.8428	0.848	0.8305-0.8651	0.0052	0.8798	0.8684-0.8916	0.8797	0.8439	0.8473	-0.0007	none

Result. (a) is not supported (A2-NonVPN is 0.0328 above its validation score; E14 had already shown its validation rows to be unusually hard), (b) and (c) are supported. 14 of the 26 paired differences are clear after Holm; two more sit on the Holm line and are borderline. All results are in Section 5.

5 Results

This section reports the final models on the test rows: 15% of each task's 15 s file (A1 2,706 flows, A2-VPN 1,407, A2-NonVPN 1,305, B 2,712), held out by the seed-42 split and used by no earlier experiment. Each model was retrained on training + validation data with the settings fixed before the test set was opened (Section 3.3, Section 4.13). Every interval is a 95% percentile interval from 10,000 stratified paired bootstrap resamples (seed 42), so it is conditional on the test set's class counts.

5.1 Final test scores

task	role	method type	run	source run	val macro F1	test macro F1	95% interval	test - val	test accuracy	accuracy 95% interval	test weighted F1	test balanced accuracy	CV macro F1 (fit rows)	CV - test	calibration
A1	headline	LightGBM	E12-final-A1	E17-lgbm-A1-15s	0.9308	0.9434	0.9348-0.9519	0.0126	0.9435	0.9350-0.9520	0.9435	0.9433	0.9322	-0.0111	isotonic
A2-VPN	headline	LightGBM	E12-final-A2-VPN	E08-lgbm_class_weight-A2-VPN-15s	0.883	0.8652	0.8431-0.8866	-0.0178	0.8955	0.8799-0.9112	0.896	0.8682	0.8827	0.0175	isotonic
A2-NonVPN	headline	LightGBM	E12-final-A2-NonVPN	E08-lgbm_oversample-A2-NonVPN-15s	0.9075	0.9403	0.9216-0.9573	0.0328	0.9609	0.9510-0.9709	0.9607	0.9343	0.9347	-0.0055	none
B	headline	XGBoost	E12-final-B	E07-xgb-B-15s	0.8428	0.848	0.8305-0.8651	0.0052	0.8798	0.8684-0.8916	0.8797	0.8439	0.8473	-0.0007	none

- Test macro F1: A1 0.9434 (0.9348-0.9519), A2-VPN 0.8652 (0.8431-0.8866), A2-NonVPN 0.9403 (0.9216-0.9573), B 0.8480 (0.8305-0.8651). The two A2 models are scored separately and never merged into one score.
- Against validation, three scores are higher and one lower: +0.0126 (A1), -0.0178 (A2-VPN), +0.0328 (A2-NonVPN), +0.0052 (B). E14 (split-seed stability) had already shown that the official A2-NonVPN validation rows are unusually hard: its validation macro F1 (0.9075) lies below all 10 resplits of training + validation (0.9257-0.9468), and the test score lies inside that range (Section 5.5).
- Weighted and macro F1 differ most where the classes are unequal in size: weighted minus macro F1 is 0.0001 (A1), 0.0308 (A2-VPN), 0.0204 (A2-NonVPN) and 0.0317 (B).
- Cross-validation on the fitting rows agrees with the test score to within 0.0175 (CV - test: -0.0111, 0.0175, -0.0055, -0.0007).

Hypotheses fixed before the test set was opened (E12 (final test), Section 4.21):

- (a) every headline model within 0.03 of its validation score: **not supported**, because A2-NonVPN is 0.0328 higher. The failure is reported as it is; nothing was changed because of it.
- (b) the headline model above the old tie rule's pick in at least 3 of 4 tasks: **supported** (4 of 4; the paired test finds the difference clear in A1 and A2-NonVPN).
- (c) the order of the 7 method types by test score agrees with their order by validation score (Spearman >= 0.8) in at least 3 of 4 tasks: **supported** (0.9643, 0.8214, 0.9286, 0.9643).

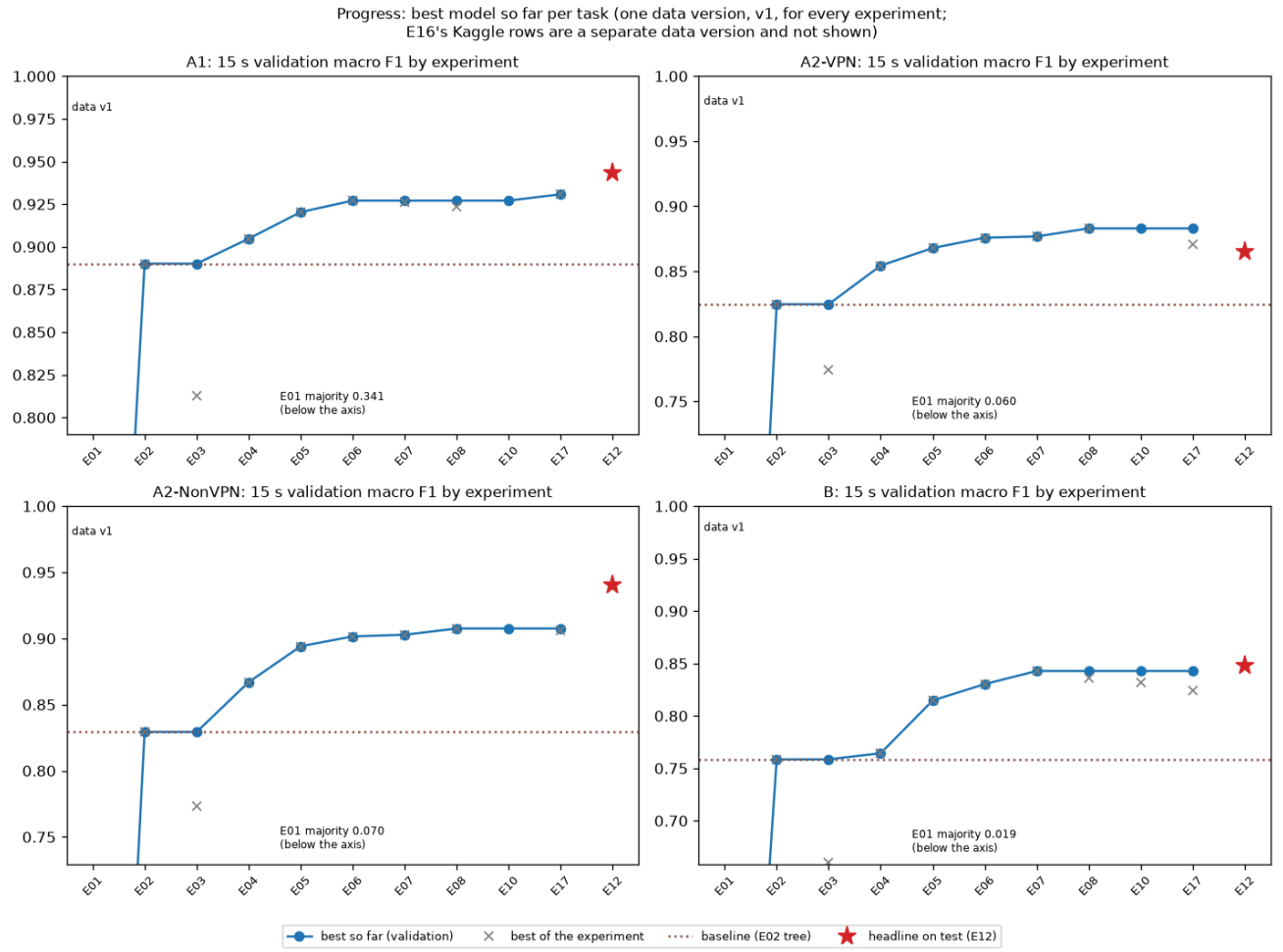


Figure 20. Progress over the experiments

5.2 Per class

class	test rows	precision	recall	F1
Non-VPN	1,303	0.9444	0.9378	0.9411
VPN	1,403	0.9426	0.9487	0.9456

class	test rows	precision	recall	F1
VPN-BROWSING	375	0.9305	0.928	0.9292
VPN-CHAT	174	0.7663	0.8103	0.7877
VPN-FT	282	0.8672	0.8333	0.8499
VPN-MAIL	65	0.7188	0.7077	0.7132
VPN-P2P	100	0.8148	0.88	0.8462
VPN-STREAMING	71	0.9429	0.9296	0.9362
VPN-VOIP	340	1.0	0.9882	0.9941

class	test rows	precision	recall	F1
BROWSING	375	0.9365	0.984	0.9597
CHAT	133	0.9219	0.8872	0.9042
FT	129	0.9504	0.8915	0.92
MAIL	36	0.9167	0.9167	0.9167
P2P	141	0.9929	0.9858	0.9893

class	test rows	precision	recall	F1
STREAMING	68	0.9091	0.8824	0.8955
VOIP	423	1.0	0.9929	0.9964

class	test rows	precision	recall	F1
BROWSING	375	0.8865	0.896	0.8912
CHAT	133	0.7698	0.8045	0.7868
FT	129	0.8992	0.8295	0.8629
MAIL	36	0.8182	0.75	0.7826
P2P	141	0.9214	0.9149	0.9181
STREAMING	67	0.8833	0.791	0.8346
VOIP	423	0.9929	0.9905	0.9917
VPN-BROWSING	375	0.827	0.8667	0.8464
VPN-CHAT	174	0.7607	0.7126	0.7359
VPN-FT	283	0.8027	0.8481	0.8247
VPN-MAIL	65	0.7	0.7538	0.7259
VPN-P2P	100	0.814	0.7	0.7527
VPN-STREAMING	71	0.8861	0.9859	0.9333
VPN-VOIP	340	1.0	0.9706	0.9851

Best and worst classes, calibration on the test rows and cost of the four headline models:

task	run	calibration	test ECE (probabilities used)	test ECE uncalibrated	mean confidence	test accuracy	weighted - macro F1 gap	best class F1	worst class F1	best class recall	worst class recall	top-5 confusion pairs (true -> predicted: flows)	model file MiB	train time s	predict ms per 1,000 flows
A1	E12-final-A1	isotonic	0.009	0.0442	0.9376	0.9435	0.0001	VPN 0.9456	Non-VPN 0.9411	VPN 0.9487	Non-VPN 0.9378	Non-VPN -> VPN: 81; VPN -> Non-VPN: 72	7.63	4.31	192.9
A2-VPN	E12-final-A2-VPN	isotonic	0.0169	0.0709	0.889	0.8955	0.0308	VPN-VOIP 0.9941	VPN-MAIL 0.7132	VPN-VOIP 0.9882	VPN-MAIL 0.7077	VPN-FT -> VPN-CHAT: 19; VPN-BROWSING -> VPN-CHAT: 16; VPN-CHAT -> VPN-FT: 15; VPN-CHAT -> VPN-BROWSING: 11; VPN-FT -> VPN-BROWSING: 10	28.14	14.79	1372.9
A2-NonVPN	E12-final-A2-NonVPN	none	0.0339	0.0339	0.9942	0.9609	0.0204	VOIP 0.9964	STREAMING 0.8955	VOIP 0.9929	STREAMING 0.8824	CHAT -> BROWSING: 14; STREAMING -> BROWSING: 7; BROWSING -> CHAT: 4; FT -> BROWSING: 4; FT -> STREAMING: 4	4.92	6.24	325.6
B	E12-final-B	none	0.042	0.042	0.9213	0.8798	0.0317	VOIP 0.9917	VPN-MAIL 0.7259	VOIP 0.9905	VPN-P2P 0.7000	BROWSING -> VPN-BROWSING: 28; VPN-BROWSING -> BROWSING: 17; VPN-CHAT -> VPN-FT: 16; VPN-BROWSING -> VPN-CHAT: 13; VPN-CHAT -> VPN-BROWSING: 13	4.6	10.56	58.8

The best class in each multi-class task is the voice class (VPN-VOIP 0.9941 in A2-VPN, VOIP 0.9964 in A2-NonVPN and 0.9917 in B); the weakest are VPN-MAIL (0.7132 in A2-VPN, 0.7259 in B) and STREAMING in A2-NonVPN (0.8955), and VPN-P2P has B's lowest recall (0.7000). In A1 both classes are close (VPN 0.9456, Non-VPN 0.9411).

The five largest confusions per task:

task	run	rank	true	predicted	flows	share of true class
A1	E12-final-A1	1	Non-VPN	VPN	81	0.0622
A1	E12-final-A1	2	VPN	Non-VPN	72	0.0513
A2-VPN	E12-final-A2-VPN	1	VPN-FT	VPN-CHAT	19	0.0674
A2-VPN	E12-final-A2-VPN	2	VPN-BROWSING	VPN-CHAT	16	0.0427
A2-VPN	E12-final-A2-VPN	3	VPN-CHAT	VPN-FT	15	0.0862

task	run	rank	true	predicted	flows	share of true class
A2-VPN	E12-final-A2-VPN	4	VPN-CHAT	VPN-BROWSING	11	0.0632
A2-VPN	E12-final-A2-VPN	5	VPN-FT	VPN-BROWSING	10	0.0355
A2-NonVPN	E12-final-A2-NonVPN	1	CHAT	BROWSING	14	0.1053
A2-NonVPN	E12-final-A2-NonVPN	2	STREAMING	BROWSING	7	0.1029
A2-NonVPN	E12-final-A2-NonVPN	3	BROWSING	CHAT	4	0.0107
A2-NonVPN	E12-final-A2-NonVPN	4	FT	BROWSING	4	0.031
A2-NonVPN	E12-final-A2-NonVPN	5	FT	STREAMING	4	0.031
B	E12-final-B	1	BROWSING	VPN-BROWSING	28	0.0747
B	E12-final-B	2	VPN-BROWSING	BROWSING	17	0.0453
B	E12-final-B	3	VPN-CHAT	VPN-FT	16	0.092
B	E12-final-B	4	VPN-BROWSING	VPN-CHAT	13	0.0347
B	E12-final-B	5	VPN-CHAT	VPN-BROWSING	13	0.0747

In B the largest confusion is between a non-VPN class and its VPN counterpart: BROWSING predicted as VPN-BROWSING (28 flows) and the reverse (17). Within the VPN classes, CHAT, FT and BROWSING are mixed up with each other in both A2-VPN and B.

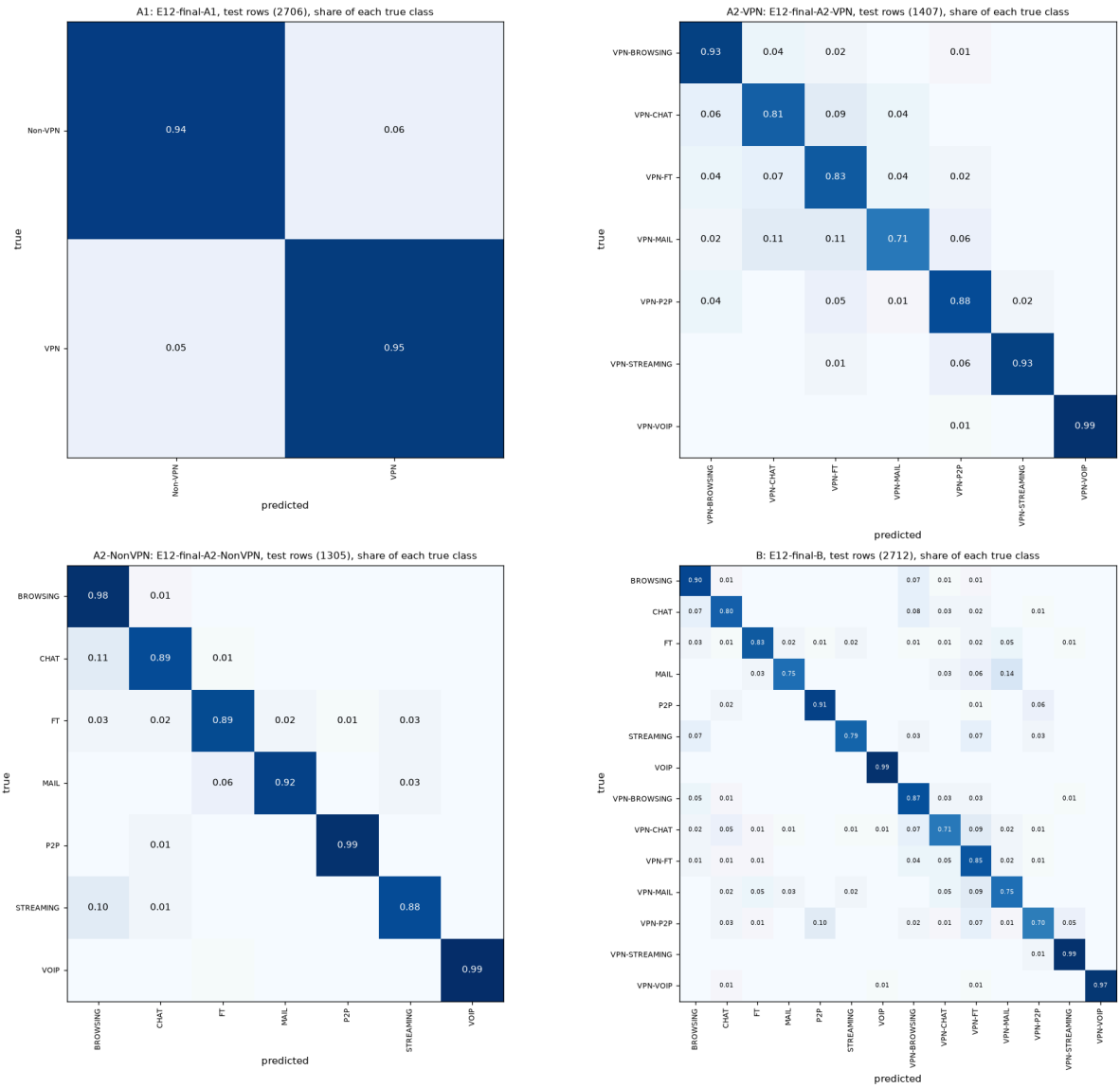


Figure 21. Normalised confusion matrices of the final models

5.3 Comparison of the seven method types

Every model of the comparison set (the best validation variant of each method type, fixed before the test set was opened) and the two old-rule picks that are not members, scored once on the same test rows:

task	role	method type	run	source run	val macro F1	test macro F1	95% interval	test - val	test accuracy	accuracy 95% interval	test weighted F1	test balanced accuracy	CV macro F1 (fit rows)	CV - test	calibration
A1	headline	LightGBM	E12-final-A1	E17-lgbm-A1-15s	0.9308	0.9434	0.9348-0.9519	0.0126	0.9435	0.9350-0.9520	0.9435	0.9433	0.9322	-0.0111	isotonic
A1	comparison	KNN	E12-compare-knn-A1	E07-knn_log_standard-A1-15s	0.9163	0.9208	0.9108-0.9308	0.0045	0.9209	0.9109-0.9309	0.9209	0.9209	0.9174	-0.0034	none
A1	comparison	Random Forest	E12-compare-random_forest-A1	E05-rf-A1-15s	0.9203	0.9289	0.9192-0.9382	0.0085	0.929	0.9194-0.9383	0.929	0.9286	0.9199	-0.009	none
A1	comparison	SVM-RBF	E12-compare-svm_rbf-A1	E17-svm_rbf-A1-15s	0.9259	0.9311	0.9215-0.9404	0.0052	0.9313	0.9217-0.9405	0.9312	0.9309	0.9222	-0.0089	none
A1	comparison	XGBoost	E12-compare-xgboost-A1	E06-xgb-A1-15s	0.9271	0.9304	0.9207-0.9397	0.0033	0.9305	0.9209-0.9398	0.9305	0.9301	0.929	-0.0014	none
A1	comparison	decision tree	E12-compare-decision_tree-A1	E17-tree-A1-15s	0.8905	0.8982	0.8866-0.9093	0.0077	0.8984	0.8869-0.9095	0.8983	0.8979	0.8983	0.0001	none
A1	comparison	logistic regression	E12-compare-logistic_regression-A1	E17-logreg-A1-15s	0.6517	0.6804	0.6628-0.6976	0.0287	0.6811	0.6633-0.6984	0.6809	0.6803	0.673	-0.0074	none
A1	old_rule	KNN	E12-old_rule-A1	E04-knn_log_standard-A1-15s	0.9048	0.9112	0.9004-0.9216	0.0064	0.9113	0.9006-0.9217	0.9113	0.9111	0.9078	-0.0034	none
A2-VPN	headline	LightGBM	E12-final-A2-VPN	E08-lgbm_class_weight-A2-VPN-15s	0.883	0.8652	0.8431-0.8866	-0.0178	0.8955	0.8799-0.9112	0.896	0.8682	0.8827	0.0175	isotonic
A2-VPN	comparison	KNN	E12-compare-knn-A2-VPN	E07-knn_log_standard-A2-VPN-15s	0.8625	0.8431	0.8190-0.8655	-0.0195	0.8735	0.8564-0.8898	0.8731	0.8441	0.8577	0.0146	none
A2-VPN	comparison	Random Forest	E12-compare-random_forest-A2-VPN	E17-rf-A2-VPN-15s	0.8704	0.8594	0.8379-0.8804	-0.011	0.8827	0.8664-0.8984	0.8823	0.8646	0.8688	0.0093	none
A2-VPN	comparison	SVM-RBF	E12-compare-svm_rbf-A2-VPN	E17-svm_rbf-A2-VPN-15s	0.8707	0.867	0.8464-0.8875	-0.0037	0.8863	0.8699-0.9019	0.8869	0.8732	0.8672	0.0002	none
A2-VPN	comparison	XGBoost	E12-compare-xgboost-A2-VPN	E06-xgb-A2-VPN-15s	0.8758	0.8577	0.8349-0.8796	-0.0181	0.8891	0.8728-0.9048	0.8896	0.8606	0.882	0.0243	none
A2-VPN	comparison	decision tree	E12-compare-decision_tree-A2-VPN	E17-tree-A2-VPN-15s	0.8292	0.8308	0.8085-0.8534	0.0016	0.8557	0.8380-0.8735	0.8568	0.8383	0.8219	-0.0089	none
A2-VPN	comparison	logistic regression	E12-compare-logistic_regression-A2-VPN	E17-logreg-A2-VPN-15s	0.7263	0.7179	0.6893-0.7446	-0.0084	0.7633	0.7434-0.7832	0.7584	0.7093	0.7275	0.0096	none
A2-VPN	old_rule	KNN	E12-old_rule-A2-VPN	E04-knn_log_standard-A2-VPN-15s	0.8544	0.8442	0.8216-0.8667	-0.0102	0.8685	0.8515-0.8856	0.8679	0.8538	0.8444	0.0003	none
A2-NonVPN	headline	LightGBM	E12-final-A2-NonVPN	E08-lgbm_oversample-A2-NonVPN-15s	0.9075	0.9403	0.9216-0.9573	0.0328	0.9609	0.9510-0.9709	0.9607	0.9343	0.9347	-0.0055	none
A2-NonVPN	comparison	KNN	E12-compare-knn-A2-NonVPN	E07-knn_log_standard-A2-NonVPN-15s	0.8929	0.916	0.8960-0.9351	0.0231	0.9425	0.9303-0.9540	0.9423	0.9126	0.9091	-0.007	none
A2-NonVPN	comparison	Random Forest	E12-compare-random_forest-A2-NonVPN	E05-rf-A2-NonVPN-15s	0.8941	0.9067	0.8848-0.9276	0.0127	0.9379	0.9249-0.9502	0.9373	0.897	0.908	0.0012	none
A2-NonVPN	comparison	SVM-RBF	E12-compare-svm_rbf-A2-NonVPN	E17-svm_rbf-A2-NonVPN-15s	0.9058	0.9244	0.9036-0.9438	0.0186	0.9525	0.9410-0.9632	0.9521	0.9202	0.923	-0.0014	none
A2-NonVPN	comparison	XGBoost	E12-compare-xgboost-A2-NonVPN	E07-xgb-A2-NonVPN-15s	0.9025	0.9346	0.9151-0.9523	0.032	0.9579	0.9471-0.9678	0.9576	0.9268	0.9293	-0.0053	none
A2-NonVPN	comparison	decision tree	E12-compare-decision_tree-A2-NonVPN	E02-tree-A2-NonVPN-15s	0.8294	0.8825	0.8596-0.9051	0.0532	0.9203	0.9057-0.9341	0.9204	0.8841	0.8686	-0.0139	none
A2-NonVPN	comparison	logistic regression	E12-compare-logistic_regression-A2-NonVPN	E17-logreg-A2-NonVPN-15s	0.8002	0.8053	0.7724-0.8360	0.005	0.8828	0.8674-0.8981	0.8789	0.7814	0.7992	-0.0061	none
B	headline	XGBoost	E12-final-B	E07-xgb-B-15s	0.8428	0.848	0.8305-0.8651	0.0052	0.8798	0.8684-0.8916	0.8797	0.8439	0.8473	-0.0007	none
B	comparison	KNN	E12-compare-knn-B	E07-knn_log_standard-B-15s	0.8006	0.8253	0.8068-0.8432	0.0248	0.861	0.8488-0.8732	0.8607	0.825	0.811	-0.0144	none
B	comparison	LightGBM	E12-compare-lightgbm-B	E07-lgbm-B-15s	0.8352	0.8567	0.8389-0.8735	0.0215	0.885	0.8732-0.8964	0.8846	0.8468	0.8501	-0.0066	none

task	role	method type	run	source run	val macro F1	test macro F1	95% interval	test - val	test accuracy	accuracy 95% interval	test weighted F1	test balanced accuracy	CV macro F1 (fit rows)	CV - test	calibration
B	comparison	Random Forest	E12-compare-random_forest-B	E17-rf-B-15s	0.8167	0.8403	0.8222-0.8576	0.0236	0.8721	0.8603-0.8838	0.8713	0.8339	0.8359	-0.0044	none
B	comparison	SVM-RBF	E12-compare-svm_rbf-B	E17-svm_rbf-B-15s	0.8242	0.8411	0.8224-0.8584	0.0169	0.8728	0.8610-0.8850	0.872	0.8342	0.8248	-0.0163	none
B	comparison	decision tree	E12-compare-decision_tree-B	E02-tree-B-15s	0.7585	0.7886	0.7687-0.8081	0.03	0.8278	0.8145-0.8414	0.8279	0.7892	0.7574	-0.0312	none
B	comparison	logistic regression	E12-compare-logistic_regression-B	E17-logreg-B-15s	0.5862	0.6089	0.5856-0.6313	0.0227	0.6574	0.6412-0.6737	0.6505	0.595	0.5885	-0.0204	none

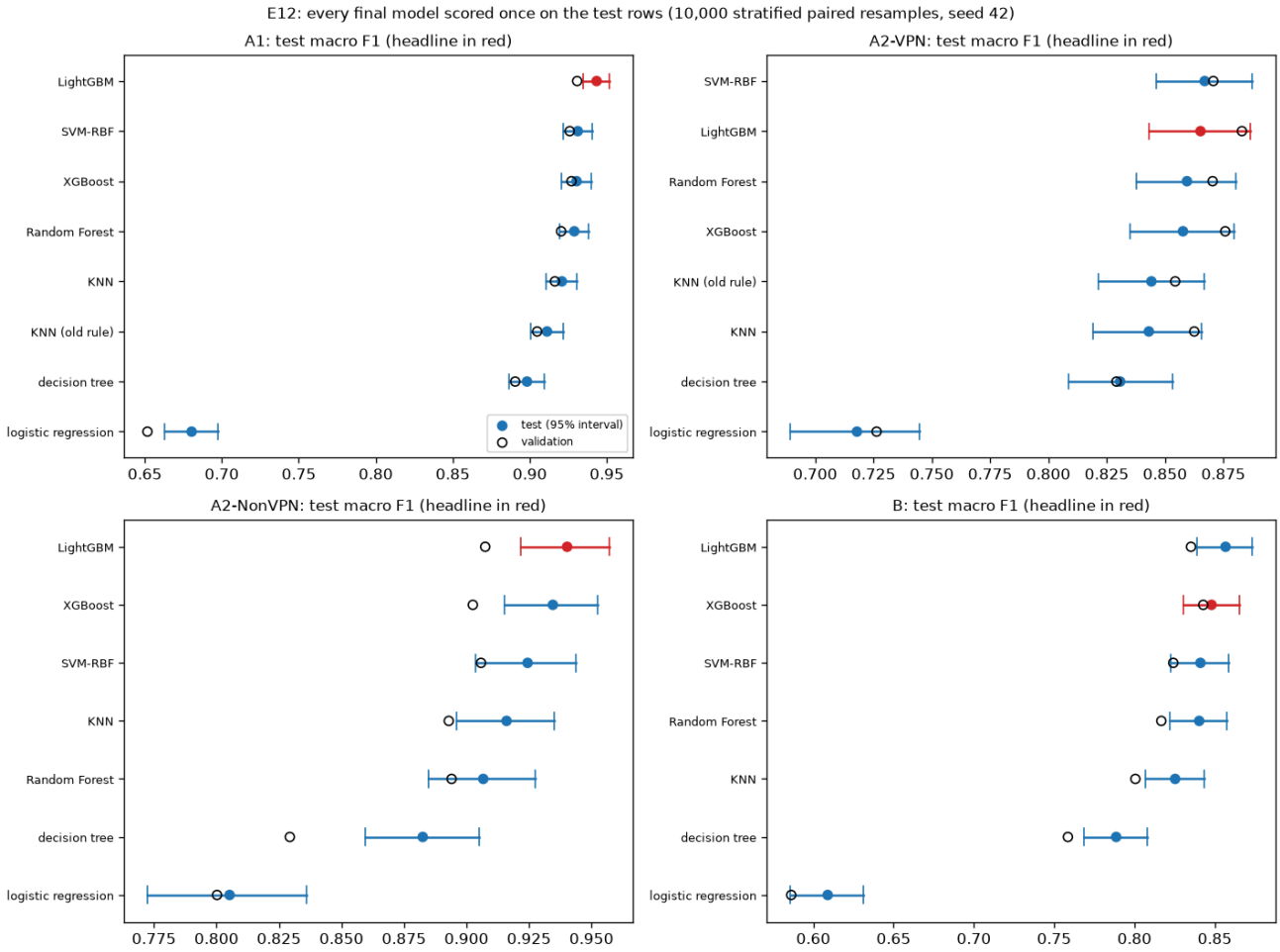


Figure 22. Every final model on the test rows

The 26 paired tests fixed before the test set was opened (headline model minus the other model on the same resampled rows; Holm correction over all 26 at 0.05):

task	headline	other	other role	other type	d (headline - other)	d 95% interval	p	Holm p	different (Holm, 0.05)
A1	E12-final-A1	E12-compare-knn-A1	comparison	KNN	0.0226	0.0140-0.0314	0.0002	0.0052	yes
A1	E12-final-A1	E12-compare-random_forest-A1	comparison	Random Forest	0.0145	0.0071-0.0219	0.0004	0.0064	yes
A1	E12-final-A1	E12-compare-svm_rbf-A1	comparison	SVM-RBF	0.0122	0.0038-0.0207	0.0042	0.0504	no
A1	E12-final-A1	E12-compare-xgboost-A1	comparison	XGBoost	0.013	0.0056-0.0204	0.0004	0.0064	yes
A1	E12-final-A1	E12-compare-decision_tree-A1	comparison	decision tree	0.0452	0.0352-0.0559	0.0002	0.0052	yes
A1	E12-final-A1	E12-compare-logistic_regression-A1	comparison	logistic regression	0.263	0.2446-0.2817	0.0002	0.0052	yes
A1	E12-final-A1	E12-old_rule-A1	old_rule	KNN	0.0322	0.0226-0.0422	0.0002	0.0052	yes

task	headline	other	other role	other type	d (headline - other)	d 95% interval	p	Holm p	different (Holm, 0.05)
A2-VPN	E12-final-A2-VPN	E12-compare-knn-A2-VPN	comparison	KNN	0.0221	0.0052-0.0390	0.0132	0.1320	no
A2-VPN	E12-final-A2-VPN	E12-compare-random_forest-A2-VPN	comparison	Random Forest	0.0058	-0.0117-0.0234	0.5123	1.0000	no
A2-VPN	E12-final-A2-VPN	E12-compare-svm_rbf-A2-VPN	comparison	SVM-RBF	-0.0018	-0.0224-0.0186	0.8525	1.0000	no
A2-VPN	E12-final-A2-VPN	E12-compare-xgboost-A2-VPN	comparison	XGBoost	0.0075	-0.0062-0.0214	0.2918	1.0000	no
A2-VPN	E12-final-A2-VPN	E12-compare-decision_tree-A2-VPN	comparison	decision tree	0.0345	0.0115-0.0577	0.0042	0.0504	no
A2-VPN	E12-final-A2-VPN	E12-compare-logistic_regression-A2-VPN	comparison	logistic regression	0.1473	0.1173-0.1777	0.0002	0.0052	yes
A2-VPN	E12-final-A2-VPN	E12-old_rule-A2-VPN	old_rule	KNN	0.0211	-0.0005-0.0426	0.0566	0.4752	no
A2-NonVPN	E12-final-A2-NonVPN	E12-compare-knn-A2-NonVPN	comparison	KNN	0.0242	0.0106-0.0387	0.0004	0.0064	yes
A2-NonVPN	E12-final-A2-NonVPN	E12-compare-random_forest-A2-NonVPN	comparison	Random Forest	0.0336	0.0181-0.0506	0.0002	0.0052	yes
A2-NonVPN	E12-final-A2-NonVPN	E12-compare-svm_rbf-A2-NonVPN	comparison	SVM-RBF	0.0158	-0.0001-0.0327	0.0528	0.4752	no
A2-NonVPN	E12-final-A2-NonVPN	E12-compare-xgboost-A2-NonVPN	comparison	XGBoost	0.0057	-0.0059-0.0181	0.3406	1.0000	no
A2-NonVPN	E12-final-A2-NonVPN	E12-compare-decision_tree-A2-NonVPN	comparison	decision tree	0.0578	0.0402-0.0763	0.0002	0.0052	yes
A2-NonVPN	E12-final-A2-NonVPN	E12-compare-logistic_regression-A2-NonVPN	comparison	logistic regression	0.135	0.1033-0.1687	0.0002	0.0052	yes
B	E12-final-B	E12-compare-knn-B	comparison	KNN	0.0227	0.0090-0.0364	0.0020	0.0260	yes
B	E12-final-B	E12-compare-lightgbm-B	comparison	LightGBM	-0.0087	-0.0197-0.0028	0.1368	0.9575	no
B	E12-final-B	E12-compare-random_forest-B	comparison	Random Forest	0.0077	-0.0040-0.0196	0.1952	1.0000	no
B	E12-final-B	E12-compare-svm_rbf-B	comparison	SVM-RBF	0.0069	-0.0093-0.0235	0.4078	1.0000	no
B	E12-final-B	E12-compare-decision_tree-B	comparison	decision tree	0.0594	0.0435-0.0762	0.0002	0.0052	yes
B	E12-final-B	E12-compare-logistic_regression-B	comparison	logistic regression	0.2391	0.2147-0.2648	0.0002	0.0052	yes

- 14 of the 26 differences are clear after Holm (A1 6 of 7, A2-VPN 1 of 7, A2-NonVPN 4 of 6, B 3 of 6). The headline model is clearly better than logistic regression in every task and than the decision tree in A1, A2-NonVPN and B.
- It is not clearly different from the SVM in any task, nor from XGBoost or LightGBM except A1 against XGBoost. Two comparison models scored slightly higher on the test rows, neither clearly: the A2-VPN SVM (0.8670 against 0.8652) and the B LightGBM (0.8567 against 0.8480).

Two of the 26 tests sit on the Holm line. The p-value of each test comes from the same 10,000 resamples, so it carries Monte Carlo error; the band below is 2 x the tail share plus or minus two standard errors, times the test's Holm multiplier:

task	other	smaller tail count	p	Holm p	Holm p band (+-2 SE)	different (Holm, 0.05)	settled by B resamples
A1	E12-compare-knn-A1	0	0.0002	0.0052	0.0000-0.0156	yes	yes
A1	E12-compare-decision_tree-A1	0	0.0002	0.0052	0.0000-0.0150	yes	yes
A1	E12-compare-logistic_regression-A1	0	0.0002	0.0052	0.0000-0.0144	yes	yes
A1	E12-old_rule-A1	0	0.0002	0.0052	0.0000-0.0138	yes	yes
A2-VPN	E12-compare-logistic_regression-A2-VPN	0	0.0002	0.0052	0.0000-0.0132	yes	yes

task	other	smaller tail count	p	Holm p	Holm p band (+-2 SE)	different (Holm, 0.05)	settled by B resamples
A2-NonVPN	E12-compare-random_forest-A2-NonVPN	0	0.0002	0.0052	0.0000-0.0126	yes	yes
A2-NonVPN	E12-compare-decision_tree-A2-NonVPN	0	0.0002	0.0052	0.0000-0.0120	yes	yes
A2-NonVPN	E12-compare-logistic_regression-A2-NonVPN	0	0.0002	0.0052	0.0000-0.0114	yes	yes
B	E12-compare-decision_tree-B	0	0.0002	0.0052	0.0000-0.0108	yes	yes
B	E12-compare-logistic_regression-B	0	0.0002	0.0052	0.0000-0.0102	yes	yes
A1	E12-compare-random_forest-A1	1	0.0004	0.0064	0.0000-0.0154	yes	yes
A1	E12-compare-xgboost-A1	1	0.0004	0.0064	0.0000-0.0145	yes	yes
A2-NonVPN	E12-compare-knn-A2-NonVPN	1	0.0004	0.0064	0.0000-0.0135	yes	yes
B	E12-compare-knn-B	9	0.0020	0.0260	0.0096-0.0424	yes	yes
A1	E12-compare-svm_rbf-A1	20	0.0042	0.0504	0.0284-0.0724	no	no
A2-VPN	E12-compare-decision_tree-A2-VPN	20	0.0042	0.0504	0.0261-0.0663	no	no
A2-VPN	E12-compare-knn-A2-VPN	65	0.0132	0.1320	0.0996-0.1644	no	yes
A2-NonVPN	E12-compare-svm_rbf-A2-NonVPN	263	0.0528	0.4752	0.4174-0.5329	no	yes
A2-VPN	E12-old_rule-A2-VPN	282	0.0566	0.4752	0.3997-0.5058	no	yes
B	E12-compare-lightgbm-B	683	0.1368	0.9575	0.8868-1.0000	no	yes
B	E12-compare-random_forest-B	975	0.1952	1.0000	1.0000-1.0000	no	yes
A2-VPN	E12-compare-xgboost-A2-VPN	1,458	0.2918	1.0000	1.0000-1.0000	no	yes
A2-NonVPN	E12-compare-xgboost-A2-NonVPN	1,702	0.3406	1.0000	1.0000-1.0000	no	yes
B	E12-compare-svm_rbf-B	2,038	0.4078	1.0000	1.0000-1.0000	no	yes
A2-VPN	E12-compare-random_forest-A2-VPN	2,561	0.5123	1.0000	0.9898-1.0000	no	yes
A2-VPN	E12-compare-svm_rbf-A2-VPN	4,262	0.8525	1.0000	0.8327-0.8723	no	yes

A1 against the SVM and A2-VPN against the decision tree (Holm p 0.0504) have bands of about 0.03-0.07, so 10,000 resamples cannot settle whether they fall below 0.05. Under the rule fixed in advance both count as not clear; they are reported as borderline. No verdict depends on them, and the other 24 tests are settled either way. An independent recomputation from the saved predictions reproduced every interval, p-value and Holm p.

The deviation from the pre-set tie rule, and what it cost. The rule set before Phase 3 would have chosen a simpler model whenever the gap was below 0.03 (Section 3.8). Its picks were scored on the test set too: the headline model scored higher in all four tasks, by 0.0322 in A1 (against the E04 (KNN with a signed log) KNN, clear), 0.0211 in A2-VPN (E04 KNN, not clear), 0.0242 in A2-NonVPN (E07 (tuning) KNN, clear) and 0.0069 in B (E17 (equal tuning budget) SVM, not clear). Choosing by the highest validation score therefore did not cost anything on the test rows, and in two tasks it gained a clear margin.

5.4 Calibration (E15)

task	run	headline	val accuracy	mean confidence, uncalibrated	mean confidence, isotonic	ECE uncalibrated	ECE isotonic	adopted	same predicted class (share of rows)	calibrated argmax would differ (share, unused)	val macro F1 = source run
A1	E15-A1-15s	E17-lgbm-A1-15s	0.9309	0.9861	0.9356	0.0552	0.014	yes	1.0	0.0007	yes

task	run	headline	val accuracy	mean confidence, uncalibrated	mean confidence, isotonic	ECE uncalibrated	ECE isotonic	adopted	same predicted class (share of rows)	calibrated argmax would differ (share, unused)	val macro F1 = source run
A2-VPN	E15-A2-VPN-15s	E08-lgbm_class_weight-A2-VPN-15s	0.8984	0.9692	0.9004	0.0714	0.0168	yes	1.0	0.0078	yes
A2-NonVPN	E15-A2-NonVPN-15s	E08-lgbm_oversample-A2-NonVPN-15s	0.9479	0.991	0.9469	0.0444	0.0152	no	1.0	0.0084	yes
B	E15-B-15s	E07-xgb-B-15s	0.8794	0.9144	0.8787	0.035	0.0099	no	1.0	0.0077	yes

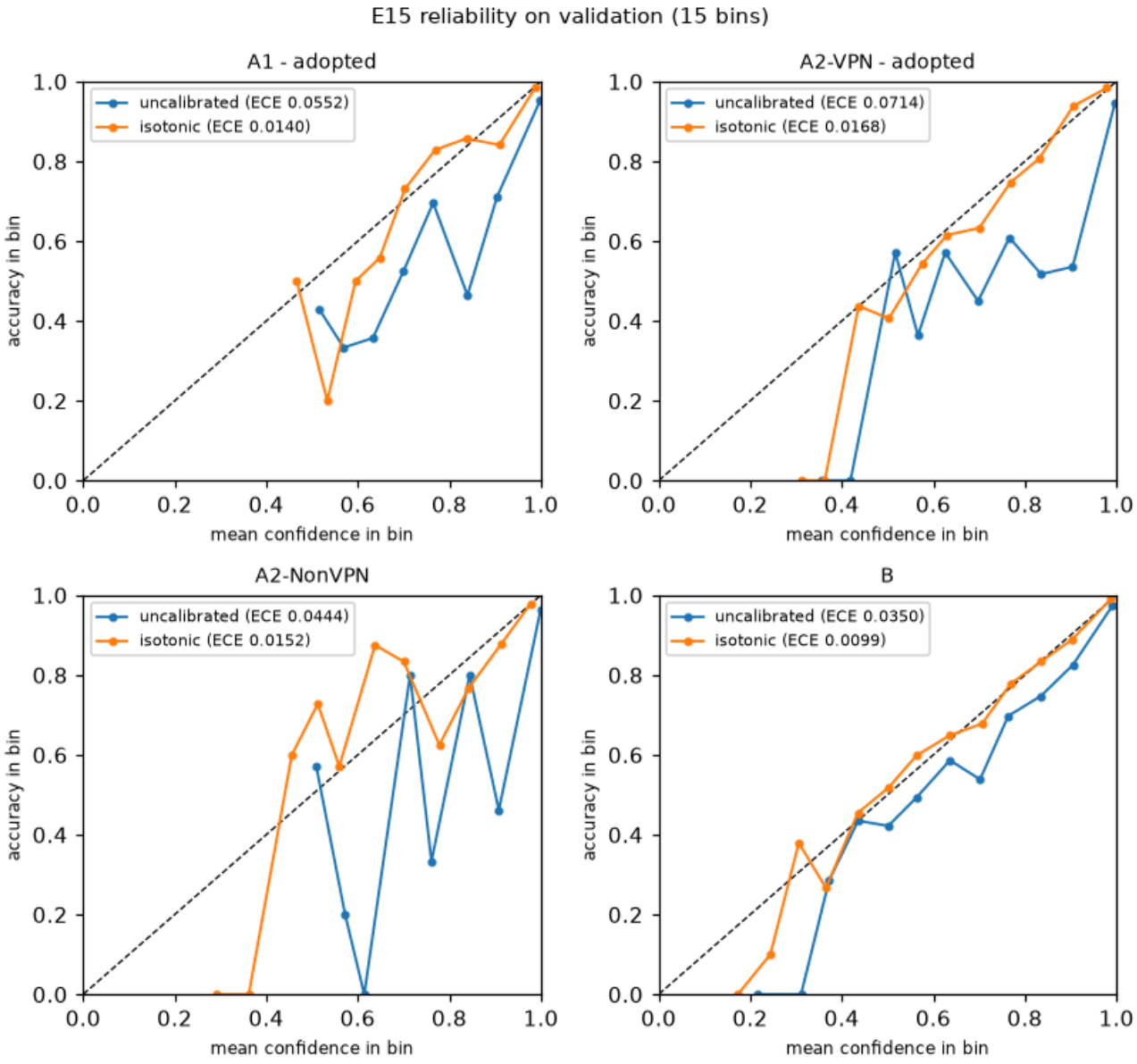


Figure 23. Reliability diagrams (validation)

All four headline models were overconfident on validation (mean confidence above accuracy). By the rule fixed before the test set (calibrate where the uncalibrated validation ECE is above 0.05 and isotonic calibration lowers it), calibration was adopted for A1 and A2-VPN. On the test rows, measured once, it lowers the expected calibration error from 0.0442 to 0.0090 (A1) and from 0.0709 to 0.0169 (A2-VPN); the uncalibrated A2-NonVPN and B models have 0.0339 and 0.0420. Calibration changes only the reported confidence; the predicted class, and so every test score above, is the uncalibrated model's.

5.5 Stability and amount of training data (E14, E13)

Split-seed stability on validation, 10 resplits of training + validation (E14, Section 4.16):

task	seeds	val macro F1 mean	val macro F1 std	val macro F1 min	val macro F1 max	val accuracy mean	val accuracy std	CV macro F1 mean	CV - val mean	official split (seed 42) val macro F1
A1	10	0.9338	0.0042	0.9275	0.9397	0.9339	0.0042	0.9306	-0.0032	0.9308
A2-VPN	10	0.8862	0.0114	0.8689	0.902	0.9037	0.0066	0.8805	-0.0057	0.883
A2-NonVPN	10	0.9324	0.0071	0.9257	0.9468	0.9589	0.0044	0.9283	-0.0041	0.9075
B	10	0.8476	0.0084	0.8344	0.8636	0.8819	0.0074	0.8358	-0.0119	0.8428

The validation macro F1 of the headline models varies by 0.0042-0.0114 (standard deviation) across resplits. The test scores are inside the resplit range for A2-NonVPN (0.9403 in 0.9257-0.9468) and B (0.8480 in 0.8344-0.8636), above it for A1 (0.9434 against at most 0.9397) and below it for A2-VPN (0.8652 against at least 0.8689). The resplit models are trained on the training share of each resplit only, while the final models also use the validation rows, so the two are not the same model; a single 15% test set can also land outside a 10-resplit range by chance.

Learning curve on validation (E13, Section 4.15):

task	train fraction	runs	training rows	val macro F1 mean	val macro F1 std
A1	0.1	5	1,262	0.8724	0.0062
A1	0.2	5	2,525	0.8952	0.0041
A1	0.4	5	5,050	0.9106	0.0066
A1	0.6	5	7,575	0.9213	0.0039
A1	0.8	5	10,100	0.927	0.0029
A1	1.0	1	12,626	0.9308	
A2-VPN	0.1	5	656	0.7793	0.0052
A2-VPN	0.2	5	1,313	0.826	0.0101
A2-VPN	0.4	5	2,626	0.8582	0.0041
A2-VPN	0.6	5	3,939	0.8721	0.0053
A2-VPN	0.8	5	5,252	0.8804	0.0036
A2-VPN	1.0	1	6,566	0.883	
A2-NonVPN	0.1	5	608	0.8176	0.0093
A2-NonVPN	0.2	5	1,217	0.8585	0.0118
A2-NonVPN	0.4	5	2,434	0.8806	0.0057
A2-NonVPN	0.6	5	3,651	0.8934	0.0045
A2-NonVPN	0.8	5	4,868	0.9005	0.0033
A2-NonVPN	1.0	1	6,085	0.9075	
B	0.1	5	1,265	0.6969	0.0097
B	0.2	5	2,530	0.7468	0.0138
B	0.4	5	5,060	0.7975	0.0036
B	0.6	5	7,590	0.8156	0.0016
B	0.8	5	10,120	0.8297	0.0029
B	1.0	1	12,651	0.8428	

Macro F1 still rises between 80% and 100% of the training rows in every task (A1 0.927 to 0.9308, A2-VPN 0.8804 to 0.883, A2-NonVPN 0.9005 to 0.9075, B 0.8297 to 0.8428), most in B, the task with the most classes. Retraining on training + validation for the final models adds 15 percentage points of the file to the 70% used here.

5.6 What the models use (SHAP and E09)

SHAP values of the four final models on the test rows (notebook 05); no score is computed from them.

rank	A1	A2-VPN	A2-NonVPN	B
1	min_flowiat (2.224)	flowBytesPerSecond (1.062)	flowBytesPerSecond (1.409)	flowBytesPerSecond (0.989)
2	min_biat (2.192)	min_flowiat (0.776)	min_flowiat (1.326)	min_flowiat (0.828)
3	flowBytesPerSecond (1.905)	min_fiat (0.688)	min_fiat (0.756)	duration (0.687)
4	min_fiat (1.535)	min_biat (0.537)	flowPktsPerSecond (0.685)	min_fiat (0.589)
5	max_flowiat (1.320)	duration (0.448)	mean_flowiat (0.663)	min_biat (0.495)

task	group	features	mean	SHAP	(sum)	SHAP share	E09 drop this group (val change)	E09 this group alone (val change)
A1	fiat	4	2.9511	0.1654	-0.0052	-0.1229		
A1	biat	4	4.0202	0.2253	-0.0155	-0.1064		
A1	flowiat	4	5.2763	0.2957	0.0014	-0.08		
A1	active	4	0.9953	0.0558	-0.0011	-0.3819		
A1	idle	4	0.7757	0.0435	-0.0004	-0.3803		
A1	rates_duration	3	3.8264	0.2144	-0.0122	-0.0915		
A2-VPN	fiat	4	1.6589	0.2212	-0.0179	-0.1237		
A2-VPN	biat	4	1.4386	0.1918	-0.0105	-0.1631		
A2-VPN	flowiat	4	1.9133	0.2551	-0.0078	-0.0969		
A2-VPN	active	4	0.2794	0.0373	0.003	-0.5578		
A2-VPN	idle	4	0.4264	0.0569	-0.0004	-0.5763		
A2-VPN	rates_duration	3	1.7828	0.2377	-0.0395	-0.104		
A2-NonVPN	fiat	4	2.3124	0.2197	-0.0051	-0.154		
A2-NonVPN	biat	4	1.694	0.1609	-0.0082	-0.1804		
A2-NonVPN	flowiat	4	2.8594	0.2716	-0.0045	-0.091		
A2-NonVPN	active	4	0.593	0.0563	0.0018	-0.572		
A2-NonVPN	idle	4	0.3305	0.0314	-0.0036	-0.6037		
A2-NonVPN	rates_duration	3	2.7372	0.26	-0.0301	-0.0604		
B	fiat	4	1.5248	0.2032	-0.0184	-0.2028		
B	biat	4	1.3158	0.1754	-0.0265	-0.233		
B	flowiat	4	1.9626	0.2616	-0.0166	-0.1712		
B	active	4	0.4103	0.0547	0.0051	-0.629		
B	idle	4	0.2732	0.0364	0.0	-0.6345		
B	rates_duration	3	2.0161	0.2687	-0.0549	-0.1385		

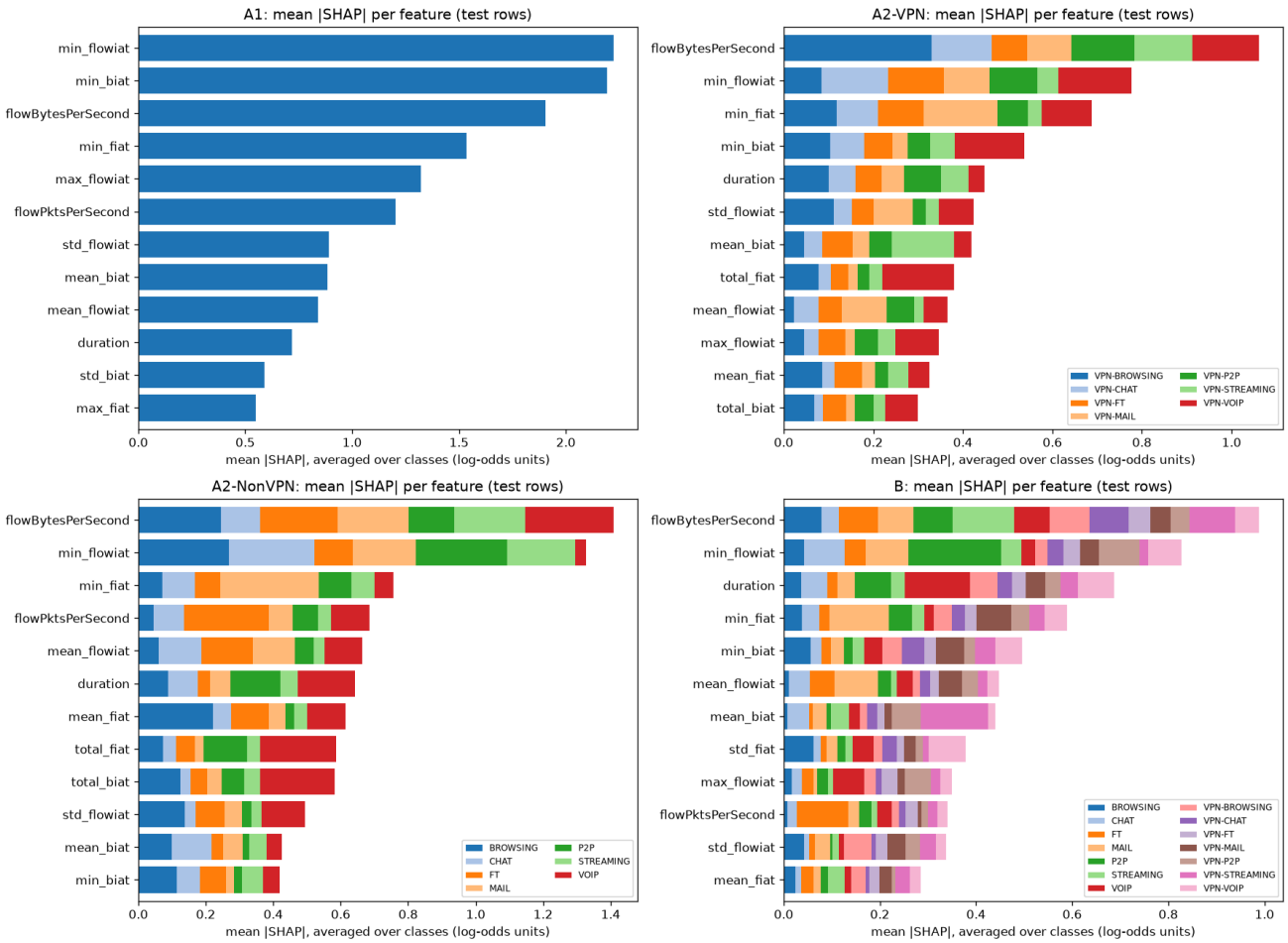


Figure 24. SHAP summary per task

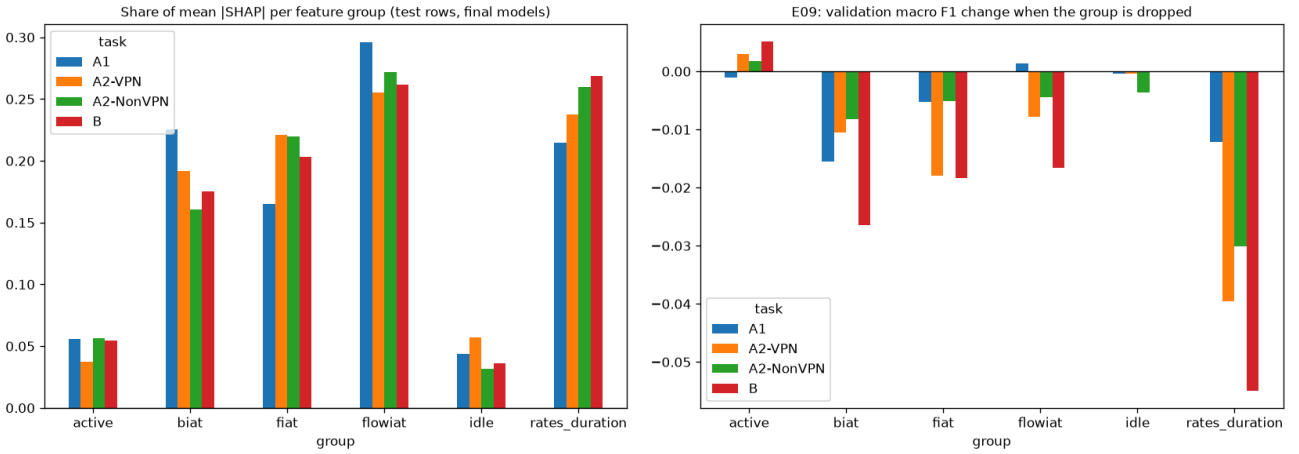


Figure 25. SHAP feature groups next to the E09 ablation

- The byte rate (flowBytesPerSecond) is the most used feature in A2-VPN, A2-NonVPN and B, and the smallest gaps between packets (min_flowiat, min_fiat, min_biat) are in the top 5 of every task.
- SHAP and the ablation (E09, feature-group ablation) agree that the active and idle groups matter least (3.1-5.7% of mean |SHAP|; dropping either changes validation macro F1 by -0.0036 to +0.0051). Rates + duration hold 21.4-26.9% of mean |SHAP| (the largest group share in B), and dropping them costs the most in A2-VPN, A2-NonVPN and B (0.0395, 0.0301, 0.0549); in A1 dropping the backward gaps costs more (0.0155 against 0.0122).
- They differ on the flow gaps (flowiat): the largest SHAP share in A1, A2-VPN and A2-NonVPN (25.5-29.6%), yet dropping the group costs at most 0.0166, because the correlated forward and backward

gaps carry much the same timing. SHAP shows what a model uses; the ablation shows what it cannot do without. Because the features are correlated, neither describes what causes the traffic patterns.

Worked examples (one flow each; picked by a rule fixed before looking at them):

task	example	raw_row	true	predicted	confidence	rank	feature	value	SHAP
A1	most confident correct	797	Non-VPN	Non-VPN	1.0	1	max_flowiat	55123.0	4.2942
A1	most confident correct	797	Non-VPN	Non-VPN	1.0	2	min_flowiat	55123.0	3.4935
A1	most confident correct	797	Non-VPN	Non-VPN	1.0	3	flowPktsPerSecond	36.2825	3.1267
A1	least confident correct	5,497	VPN	VPN	0.4519	1	flowBytesPerSecond	240.3304	2.6886
A1	least confident correct	5,497	VPN	VPN	0.4519	2	flowPktsPerSecond	1.6321	1.3703
A1	least confident correct	5,497	VPN	VPN	0.4519	3	mean_active	4695071.0	0.7458
A1	most confident error	2,218	Non-VPN	VPN	0.9956	1	flowBytesPerSecond	130.9021	4.5879
A1	most confident error	2,218	Non-VPN	VPN	0.9956	2	min_fiat	794487.0	2.3452
A1	most confident error	2,218	Non-VPN	VPN	0.9956	3	max_flowiat	794487.0	1.9669
A1	weakest class, median confidence	2,896	Non-VPN	Non-VPN	0.9946	1	min_flowiat	559.0	6.0851
A1	weakest class, median confidence	2,896	Non-VPN	Non-VPN	0.9946	2	max_flowiat	969065.0	1.6854
A1	weakest class, median confidence	2,896	Non-VPN	Non-VPN	0.9946	3	min_biat	959994.0	1.3987
A2-VPN	most confident correct	9,144	VPN-VOIP	VPN-VOIP	0.9994	1	min_biat	10038206.0	3.9411
A2-VPN	most confident correct	9,144	VPN-VOIP	VPN-VOIP	0.9994	2	flowBytesPerSecond	21.4332	3.4869
A2-VPN	most confident correct	9,144	VPN-VOIP	VPN-VOIP	0.9994	3	min_flowiat	38775.0	3.2343
A2-VPN	least confident correct	4,435	VPN-FT	VPN-FT	0.2259	1	min_flowiat	52.0	1.8895
A2-VPN	least confident correct	4,435	VPN-FT	VPN-FT	0.2259	2	min_fiat	52.0	1.016

task	example	raw_row	true	predicted	confidence	rank	feature	value	SHAP
A2-VPN	least confident correct	4,435	VPN-FT	VPN-FT	0.2259	3	min_biat	60.0	0.5417
A2-VPN	most confident error	1	VPN-BROWSING	VPN-STREAMING	0.9788	1	flowBytesPerSecond	1322206.4391	4.4567
A2-VPN	most confident error	1	VPN-BROWSING	VPN-STREAMING	0.9788	2	mean_biat	998.9129	3.3432
A2-VPN	most confident error	1	VPN-BROWSING	VPN-STREAMING	0.9788	3	max_flowiat	217299.0	2.3032
A2-VPN	weakest class, median confidence	5,962	VPN-MAIL	VPN-MAIL	0.877	1	duration	192.0	3.2163
A2-VPN	weakest class, median confidence	5,962	VPN-MAIL	VPN-MAIL	0.877	2	min_flowiat	192.0	3.0667
A2-VPN	weakest class, median confidence	5,962	VPN-MAIL	VPN-MAIL	0.877	3	max_fiat	192.0	1.4462
A2-NonVPN	most confident correct	1,570	BROWSING	BROWSING	1.0	1	min_flowiat	2.0	2.7472
A2-NonVPN	most confident correct	1,570	BROWSING	BROWSING	1.0	2	std_flowiat	2354206.5587	2.6679
A2-NonVPN	most confident correct	1,570	BROWSING	BROWSING	1.0	3	mean_flowiat	567400.6667	1.7755
A2-NonVPN	least confident correct	4,393	FT	FT	0.4821	1	min_flowiat	961.0	2.639
A2-NonVPN	least confident correct	4,393	FT	FT	0.4821	2	std_flowiat	139858.6502	1.0974
A2-NonVPN	least confident correct	4,393	FT	FT	0.4821	3	max_flowiat	198751.0	0.6833
A2-NonVPN	most confident error	4,356	FT	MAIL	1.0	1	duration	79.0	4.2734
A2-NonVPN	most confident error	4,356	FT	MAIL	1.0	2	min_fiat	79.0	4.1959
A2-NonVPN	most confident error	4,356	FT	MAIL	1.0	3	mean_fiat	79.0	2.9155
A2-NonVPN	weakest class, median confidence	5,721	STREAMING	STREAMING	1.0	1	flowBytesPerSecond	180014.1793	4.7998
A2-NonVPN	weakest class, median confidence	5,721	STREAMING	STREAMING	1.0	2	min_flowiat	0.0	1.7719

task	example	raw_row	true	predicted	confidence	rank	feature	value	SHAP
A2-NonVPN	weakest class, median confidence	5,721	STREAMING	STREAMING	1.0	3	min_fiat	5.0	1.6621
B	most confident correct	3,045	VOIP	VOIP	1.0	1	duration	14991066.0	3.8544
B	most confident correct	3,045	VOIP	VOIP	1.0	2	std_biat	3595.304	1.1211
B	most confident correct	3,045	VOIP	VOIP	1.0	3	max_flowiat	30384.0	0.8372
B	least confident correct	2,091	MAIL	MAIL	0.2948	1	min_fiat	96.0	1.3413
B	least confident correct	2,091	MAIL	MAIL	0.2948	2	duration	96.0	1.3326
B	least confident correct	2,091	MAIL	MAIL	0.2948	3	mean_flowiat	96.0	0.825
B	most confident error	7,408	VPN-P2P	VPN-FT	0.9986	1	flowBytesPerSecond	365.1224	2.4335
B	most confident error	7,408	VPN-P2P	VPN-FT	0.9986	2	max_flowiat	284836.0	0.5198
B	most confident error	7,408	VPN-P2P	VPN-FT	0.9986	3	min_fiat	-1.0	0.4553
B	weakest class, median confidence	9,737	VPN-MAIL	VPN-MAIL	0.9845	1	duration	11544.0	1.7503
B	weakest class, median confidence	9,737	VPN-MAIL	VPN-MAIL	0.9845	2	min_fiat	27.0	1.2891
B	weakest class, median confidence	9,737	VPN-MAIL	VPN-MAIL	0.9845	3	min_flowiat	27.0	0.6031

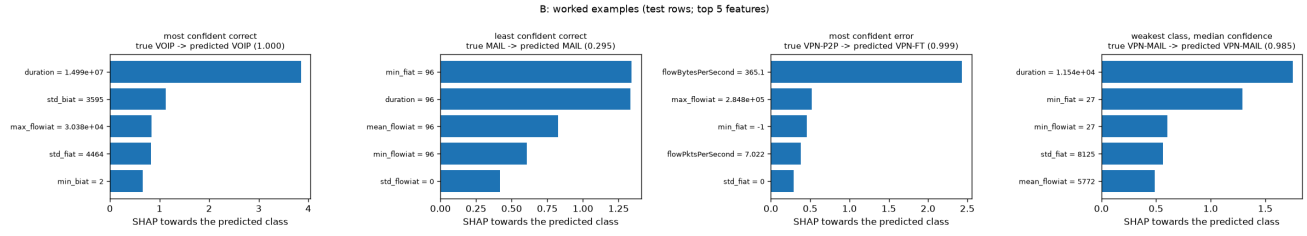


Figure 26. Worked examples, task B

The most confident errors come from a byte rate typical of another class (for example a B VPN-P2P flow at 365.1 bytes per second predicted as VPN-FT with confidence 0.9986), so a high confidence alone does not show that a prediction is right.

5.7 Comparison with the paper

The paper's own protocol (10-fold cross-validation on the full file) is reproduced in Section 4.3: on the published rows our C4.5 approximation matches all 16 A1 values of the paper's Figure 2 within 0.05 and 1-NN matches 15 of 16. The scores in this section come from a different protocol (a held-out test set after deduplication, models chosen on validation) and are not compared with the paper's figures value for value.

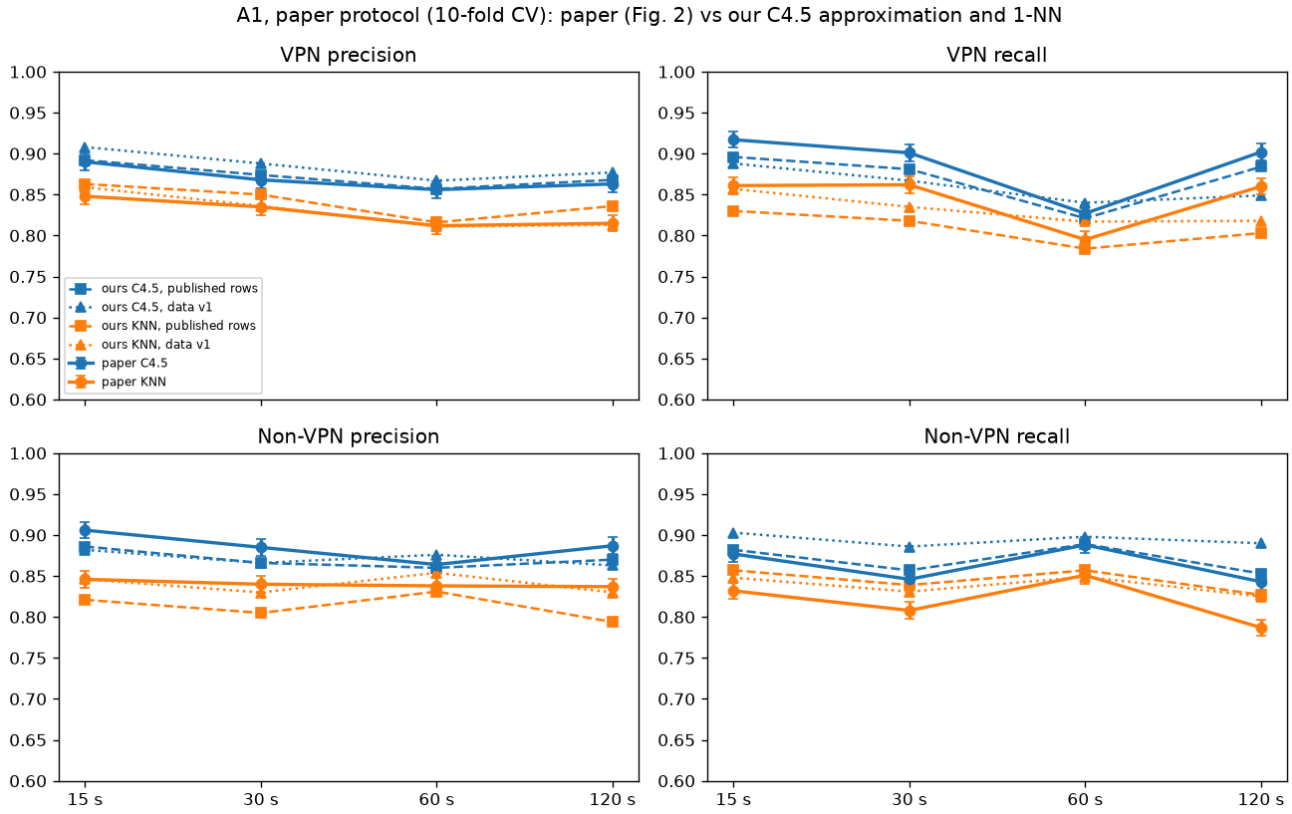


Figure 27. Paper vs ours, A1

5.8 Cost and timeout

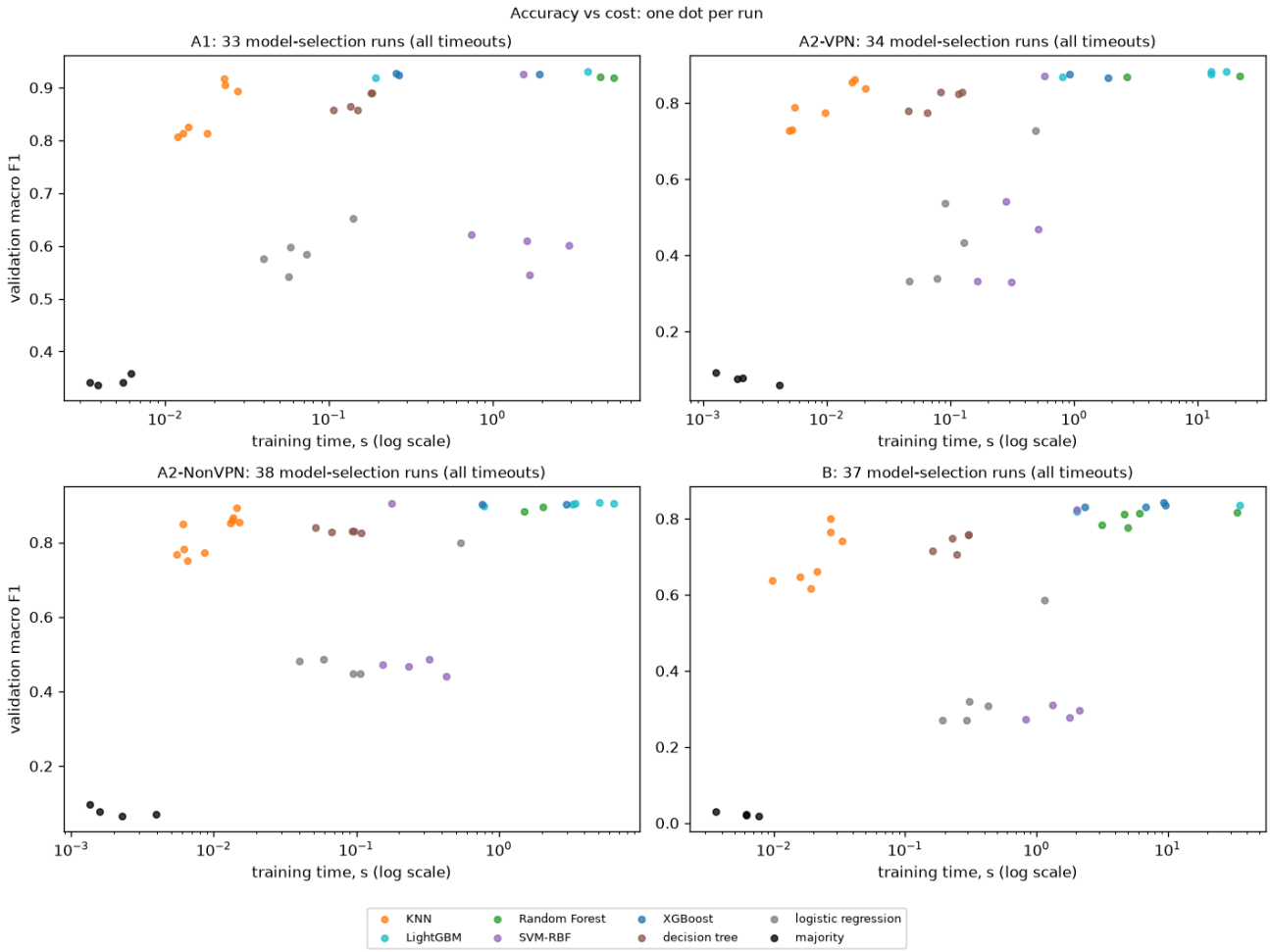


Figure 28. Accuracy against training time, one dot per model-selection run

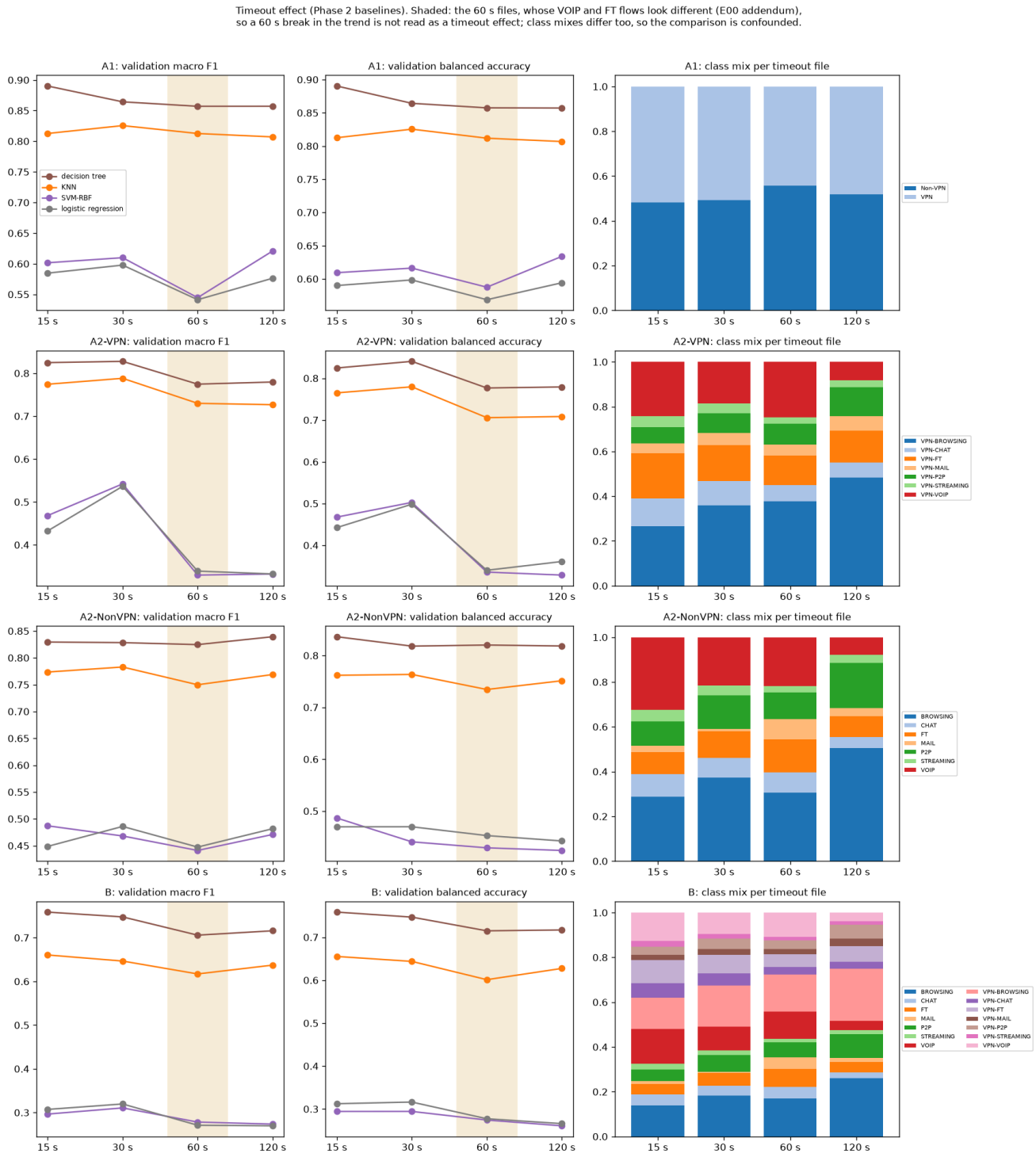


Figure 29. Timeout effect of the Phase 2 baselines with the class mix of each file

The best scores come from the boosted trees, Random Forest and the tuned SVM, which take longer to train than KNN and the decision tree (figure above); prediction with the headline models costs 58.8-1,372.9 ms per 1,000 flows (B and A2-VPN). The timeout comparison uses the Phase 2 baselines, the only models run at all four timeouts. The class mix changes with the timeout and the 60 s files hold different VOIP and FT flows (Section 2), so the comparison is confounded and is not read as a timeout effect.

5.9 Every pre-stated hypothesis

Each experiment stated its hypotheses and numeric thresholds before it ran; the verdicts, read from the generated threshold checks:

experiment	supported	not supported
E01	single check	none
E02	a, b, c	d
E03	a_knn, a_logreg, a_svm_rbf, b, c_logreg, c_svm_rbf, d	e
E04	a, b	none
E05	a, b	none
E06	a, b	none
E07	a, c	b
E08	b, c	a
E09	a	b, c
E10	c	a, b
E12	b, c	a
E13	a, b, c	none
E14	a, b	c
E15	a, b, c	none
E16	a, b, c, d	none
E17	a, b, c	none

5.10 Using a final model

The command-line predictor (`predict.py`) checks a flow file against the model's feature list (names, order, numeric and finite values) and writes, per flow, the predicted label, its confidence and the three features that pushed the model most towards that label. A malformed file stops with an error and exit code 2. The first 8 rows of its output for the task B sample file, from `uv run python scripts/predict.py --task B --input app/samples/B_test_sample.csv --output predictions.csv`:

row	label	true_label	confidence	top_feature_1	top_feature_2	top_feature_3
1	MAIL	MAIL	0.9879	flowBytesPerSecond (+3.048)	max_biat (+1.391)	min_flowiat (+1.088)
2	VOIP	VOIP	0.9998	duration (+3.221)	min_biat (+0.953)	flowBytesPerSecond (+0.756)
3	VOIP	VOIP	0.9998	duration (+3.220)	min_biat (+0.989)	flowBytesPerSecond (+0.760)
4	VOIP	VOIP	1.0	duration (+3.751)	std_biat (+1.102)	max_flowiat (+0.877)
5	VOIP	VOIP	0.9995	duration (+3.618)	mean_flowiat (+0.849)	max_flowiat (+0.743)
6	VOIP	VOIP	1.0	duration (+3.528)	std_biat (+1.169)	std_fiat (+0.747)
7	VPN-FT	VPN-CHAT	0.7308	flowPktsPerSecond (+0.375)	mean_fiat (+0.346)	flowBytesPerSecond (+0.306)
8	VPN-FT	VPN-FT	0.7385	max_flowiat (+0.977)	min_flowiat (+0.556)	min_fiat (+0.353)

5.11 The demo

The demo is a one-page Streamlit [23] app built only on the four final models and the code above (the predictor for the labels and confidences, the explanation code for the SHAP values); it trains nothing. The user chooses a task, picks one of five sample files (20 test rows per task, and one malformed file with a renamed column) or uploads a CSV, and gets a table of predictions with their confidence. Clicking a row shows the 10 features that pushed the model most towards its label, with each feature's value and unit; `−1` is shown as "not measurable" (Section 2.5). A malformed file is rejected with the same message as the command-line predictor. The page is tested headless. A 1-minute video of the demo goes through the steps of its video script on the running app (task A1 and row 10 below, task B and a wrong prediction, the malformed file, task A2-VPN); it was made by a script that drives the app in a headless browser and adds a cursor and captions.

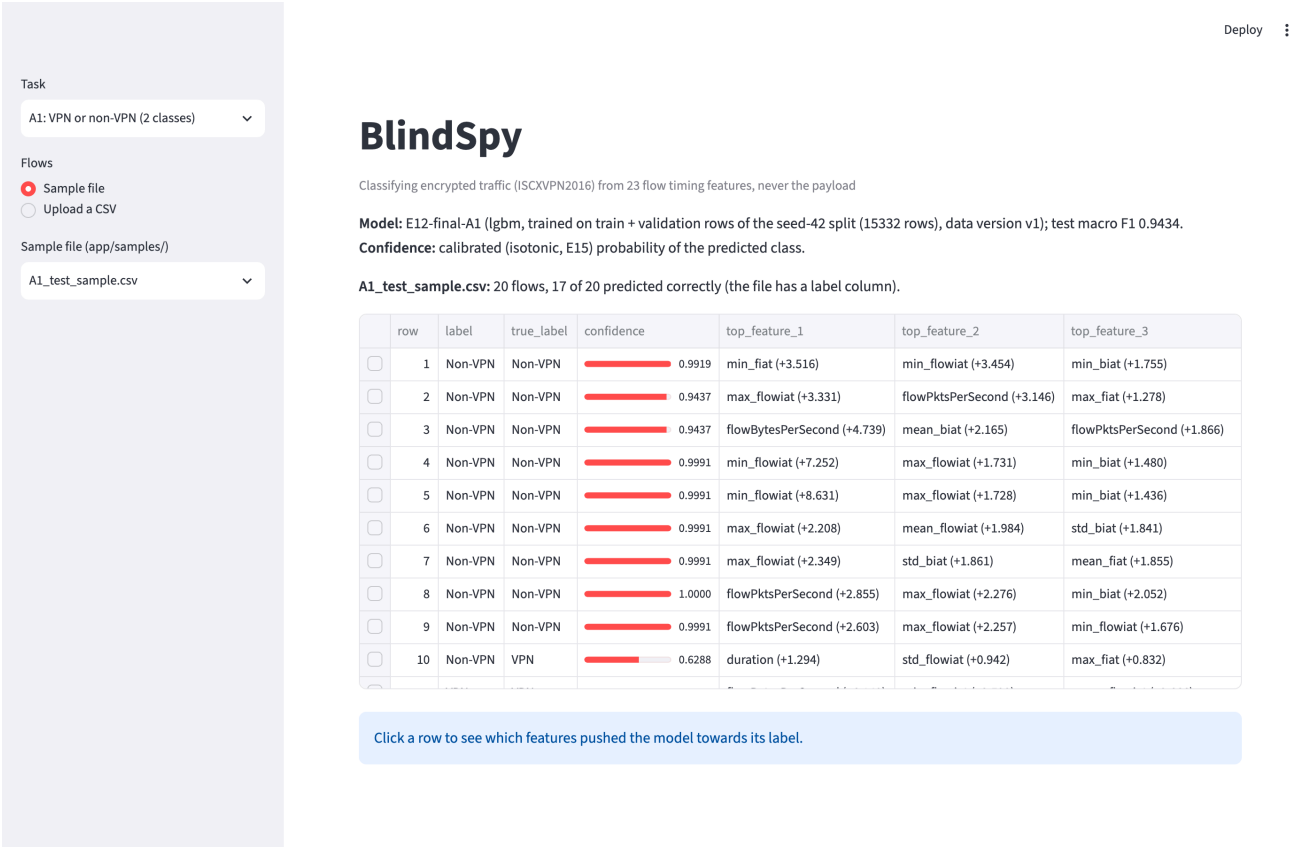


Figure 30. The demo: predictions for the A1 sample file

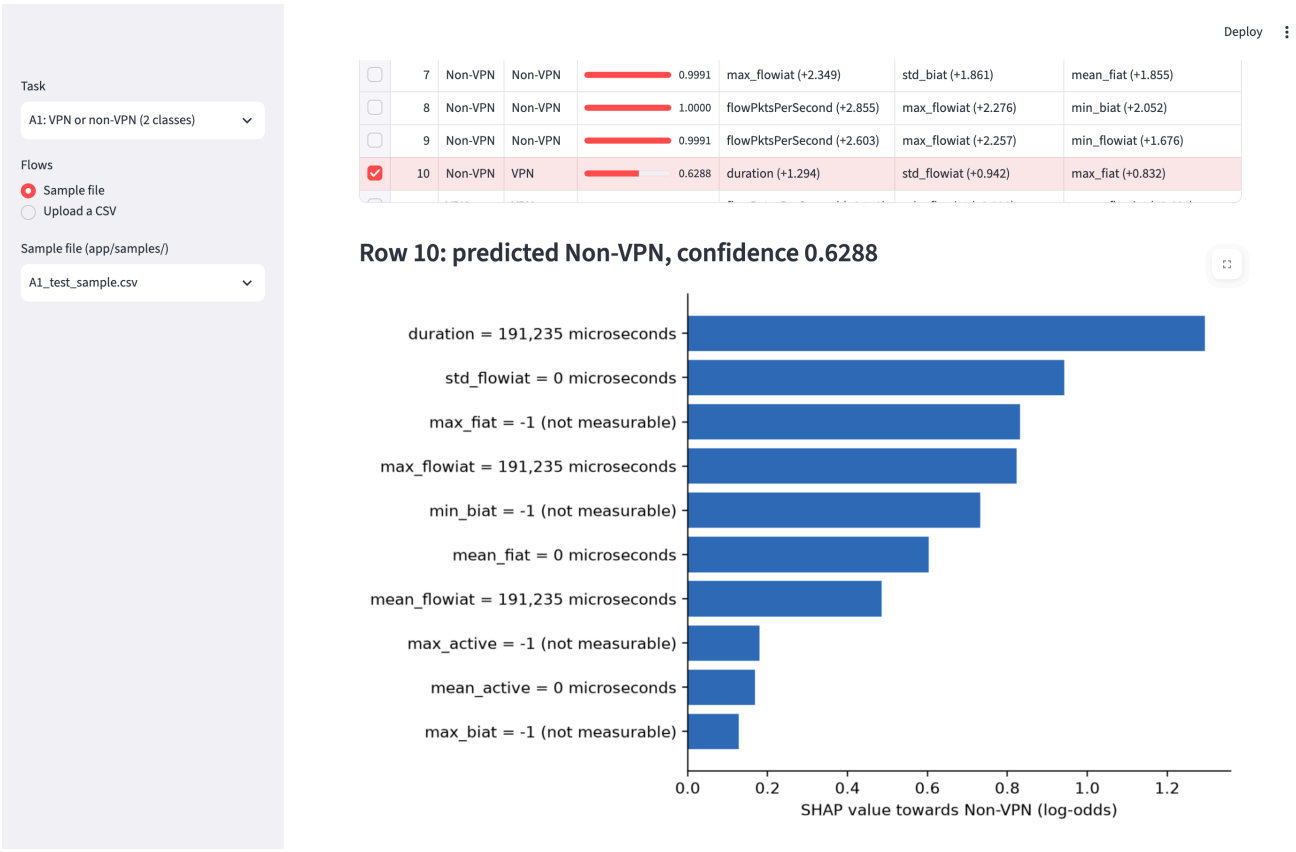


Figure 31. The demo: SHAP chart of row 10, a VPN flow predicted as Non-VPN with confidence 0.6288

Row 10 is one of the three wrong predictions in the A1 sample (17 of 20 correct). It is a two-packet flow (10.46 packets per second over 191,235 microseconds): its whole duration is one gap, and its forward and backward gaps are not measurable (`−1`: fewer than two packets in each direction). Its confidence, 0.6288, is the second lowest in the file; the lowest, 0.6267 (row 15), is also a wrong prediction. Section 4.3 showed that near-empty

flows like this often share their features with flows of another label, so timing features alone cannot always separate them.

6 Discussion

6.1 What worked

Reproducing the paper first. Before trying to improve anything, the paper's own protocol was rerun (10-fold cross-validation on every published row, Weka defaults). Our C4.5 approximation matched all 16 A1 values of the paper's Figure 2 within 0.05 and 1-NN matched 15 of 16 (E02 (decision-tree baseline), E03 (KNN, logistic regression and SVM baselines), Section 4.3). Every later number therefore starts from a baseline that agrees with the published one, and the changes we made (deduplication, a held-out test set, other models) can be judged against it.

Trees combined into ensembles, then boosted. On validation at 15 s, the Phase 2 decision tree (E02) scored 0.8901 (A1), 0.8247 (A2-VPN), 0.8294 (A2-NonVPN) and 0.7585 (B). The configurations chosen as final models scored 0.9308, 0.8830, 0.9075 and 0.8428 on the same rows (trained, like the tree, on the training rows only; Section 4.13), so most of the gain over the paper's algorithms came from three steps:

- a 300-tree Random Forest (+0.0302 to +0.0647 over the tree, E05);
- default gradient boosting (XGBoost +0.0067 to +0.0156 over the forest, E06 (gradient boosting));
- the same 50-trial tuning budget for every method type (E07, E17 (equal tuning budget)): tuning added at most +0.0151 to an ensemble, but it lifted the SVM by +0.3243 to +0.5277 and turned it from one of the weakest models into one 0.0017-0.0186 below each task's best.

The progress chart (Section 5.1) shows each step per task. The order of the seven method types was stable: their order by test score closely followed their order by validation score (Spearman 0.8214-0.9643, E12 (c)), so model selection on validation carried over to new rows.

The signed log for distance-based models. One preprocessing step, sign(x)·log1p(|x|), lifted KNN by 0.080-0.104 in every task (E04, KNN with a signed log) and was chosen by every logistic-regression and SVM search when it was offered (8 of 8, E17). Raw timing values span several orders of magnitude, so without it a few very large gaps decide every distance.

Choosing by the best validation score. The rule fixed before Phase 3 would have picked a simpler model whenever the gap was below 0.03. The measured spread of cross-validation was much smaller (standard deviation 0.0028-0.0058 for the chosen models, Section 3.8), so we chose by the highest validation score instead and kept the old rule's picks for comparison (Section 3.8). On the test rows the chosen model scored higher in all four tasks, clearly so in A1 (+0.0322) and A2-NonVPN (+0.0242) (E12 (b), Section 5.3).

Reproducing and correcting a recent result. Isnan et al.'s 88.01% was reproduced to within 0.0005 (0.8796 on the same split). Keeping the timeout files apart lowered the same method to a mean of 0.8430, and a size control showed that this drop comes from training-set size, not from mixing the timeouts: at equal size the mixed setup scored 0.8395 (E16 (a), (b), (d)). Without the control we would have blamed the mixing, which the first version of our E16 text did before the recheck (Section 4.18). Our tuned task-B XGBoost beat their forest at every timeout (mean 0.8647, E16 (c)).

Calibration where it was needed. All four headline models were overconfident on validation. Isotonic calibration, adopted by a rule fixed in advance for A1 and A2-VPN, lowered the test ECE from 0.0442 to 0.0090 and from 0.0709 to 0.0169 without changing a single prediction (E15 (calibration), E12, Section 5.4). The confidence that `predict.py` and the demo show is therefore close to the share of correct predictions at that confidence, for those two tasks.

6.2 What did not work

- **Class balancing** (E08) moved macro F1 by at most +-0.0064: a little up in A2-VPN (class weights +0.0062) and A2-NonVPN (oversampling +0.0047), down in A1 and B. The pre-stated gain of 0.01 was not reached (E08 (a) not supported). The weak classes (VPN-CHAT, VPN-MAIL, STREAMING) seem to

be weak not because they are small but because their flows look like those of other classes; E08 does not test this directly.

- **Two steps for task B** (E10, one model or two steps for task B): routing each flow through a VPN / non-VPN model first scored 0.0110 below the direct 14-class model and made more VPN / non-VPN errors (191 against 180). The direct model makes 55.05% of its errors across the VPN line as well, so separating VPN from non-VPN is the hard part of B however the model is built.
- **Tuning the ensembles** gave little: the validation score rose in 9 of 12 E07 searches although cross-validation rose in all 12, because the best of 50 cross-validation scores is optimistic. Only B improved by more than 0.01 (E07 (b) not supported).
- **Removing duplicates did not lower the paper-protocol scores**, contrary to the hypothesis (E02 (d), E03 (e)). The removed rows are near-empty flows (median 0-2 packets), and 39-92% of them have a twin with the same features and another label, so keeping them adds errors no model can avoid. The duplicate rule stays, because it guarantees that no flow is scored after being trained on.
- **Validation was not always within 0.03 of test** (E12 (a) not supported): A2-NonVPN scored 0.0328 higher on test. E14 (split-seed stability) had shown before the test set was opened that its official validation rows are harder than any of 10 resplits (0.9075 against 0.9257-0.9468), mostly for MAIL and P2P.

6.3 What the models learned: SHAP and the ablation

SHAP (on the test rows, no score computed) and the feature-group ablation (E09, on validation) answer two different questions: which features a model uses, and which it cannot do without. They agree on most points (Section 5.6):

- **The byte rate matters.** flowBytesPerSecond is the most used feature in A2-VPN, A2-NonVPN and B. Rates and duration together hold 21.4-26.9% of mean |SHAP|, and dropping them costs the most validation macro F1 in A2-VPN (0.0395), A2-NonVPN (0.0301) and B (0.0549). A timing-only model without the byte rate loses 0.015-0.044 (E09 (c) not supported), so the "timing only" claim needs the byte rate, which depends on packet sizes, not only on time.
- **In A1 the backward gaps matter more.** Dropping the backward inter-arrival times costs 0.0155, more than rates and duration (0.0122); for the binary VPN task they are the group the model can least do without. Why the replies' spacing matters most there was not tested.
- **Active and idle times matter least.** They hold 3.1-5.7% of mean |SHAP|, and dropping either group moves validation macro F1 by -0.0036 to +0.0051. A model on either group alone loses 0.38-0.63.
- **They differ on the flow gaps.** The flow inter-arrival times hold the largest SHAP share in A1, A2-VPN and A2-NonVPN (25.5-29.6%), yet dropping them costs at most 0.0166 in any task, probably because the strongly correlated forward and backward gaps carry much the same timing. With correlated features a model can swap one group for another; SHAP shows the one it happened to use.

The worked examples show the limit of a confidence score: the most confident errors are flows whose byte rate is typical of another class (a VPN-P2P flow at 365.1 bytes per second predicted as VPN-FT with confidence 0.9986, Section 5.6). Neither SHAP nor the ablation explains what causes a traffic pattern; both describe these models on these files.

6.4 Reproduced, improved, and what limits both

- **Reproduced:** the paper's A1 results under its own protocol (C4.5 16 of 16, KNN 15 of 16 within 0.05; E02, E03) and Isnan et al.'s 88.01% on their split (0.8796; E16 (a)).
- **Improved:** on the leakage-safe protocol, test macro F1 of A1 0.9434, A2-VPN 0.8652, A2-NonVPN 0.9403 and B 0.8480, each with a 95% interval (E12). The paper's two algorithms, even tuned (decision tree, KNN), score below the headline model in every task, clearly so after Holm in A1, A2-NonVPN and B (Section 5.3).

- **Limits:** the data are from a 2016 lab capture with known quality problems, flows from one capture may sit on both sides of every random split, and several of the comparisons above are close to the noise. Section 7 lists each limitation with its evidence.

7 Limitations

7.1 The data

- **2016 lab data.** ISCXVPN2016 was captured in 2016, in a lab, with a fixed set of applications and one VPN setup [1, 2]. Applications, VPN protocols and networks have changed since, so the scores in Section 5 describe traffic like these captures, not current traffic in general.
- **Documented capture problems.** Jorgensen et al. [14] found readable HTML in a capture that is supposed to be encrypted and VPN captures that hold several connections; they also cite a finding that 65% of the biflows come from the BlueStacks Android emulator, and they point out ambiguous labels such as skype_video, which could be VoIP or browsing. We use the published feature files and cannot correct any of this; a label error in the source is an error in our training and test rows alike.
- **The 60 s files hold different VOIP and FT flows.** At 60 s, non-VPN and VPN VoIP flows almost swap their packet rates and gaps, and non-VPN FT drops from 1,237-3,739 to 56.5 packets per second (E00 (data check) addendum, Section 2.8). It is not a label swap, and the cause cannot be found from the ARFF files. Results at 60 s are read with care.
- **Column names had to be inferred.** 9 of the 12 A1 and B files carry a header whose fiat / biat names are shifted by one pair, and the A2 files hold different features (`total_fiat`, `total_biat`) from A1 and B (`std_fiat`, `std_biat`). The mapping was decided from value checks that hold on 100% of rows (E00, Sections 2.3-2.4), not from documentation of the export.
- **Only the published features.** Of the dataset's packet captures, the VPN ones were downloaded in full and the non-VPN ones only in part, so flows could not be rebuilt from packets for every class. Rebuilding them was a stretch goal and was not attempted: every result uses the published 23 features, with their capture problems.

7.2 The protocol

- **No session-aware split.** The files have no flow IDs, addresses, timestamps or capture names, so flows of one recording session cannot be kept on one side of a split; E11 (session-aware split) could not run (E00). The direction of the effect was measured once, as a side result of the E15 (calibration) correction: holding out contiguous blocks of the file-ordered training rows, instead of shuffled rows, lowers the out-of-fold accuracy of the four headline configurations by 0.07-0.26 (A1 0.9321 to 0.6701, A2-VPN 0.9048 to 0.7897, A2-NonVPN 0.9566 to 0.8841, B 0.8738 to 0.7486). Neighbouring flows in a file, probably from the same capture, resemble each other more than random flows do, so scores on a random split may be optimistic for traffic from captures the model has never seen. This is an inference from the file order, not a session-aware test, and the size of the optimism is not known.
- **Timeouts are confounded.** The class mix changes with the timeout (one class's share by up to 24.6 percentage points in A2-NonVPN), and the 60 s files differ in content (7.1). Score differences between timeouts are therefore not read as a timeout effect (Sections 2.8, 5.8).
- **C4.5 approximation.** scikit-learn has no C4.5 pruning, so the paper's J48 is approximated by an unpruned entropy tree with at least 2 rows per leaf [11, 16]. The A1 reproduction is close (16 of 16 values within 0.05, Section 4.3), but the two algorithms are not identical.
- **The paper's A2 and B charts cannot be read per algorithm.** Its Figure 3 draws unlabelled lines, so for A2 and B only values stated in its text were compared (Section 4.3).
- **One deviation from a pre-set rule.** The final model per task was chosen by the highest validation score, not by the tie rule fixed before Phase 3 (Section 3.8). The change was made after E10 (one model or two steps for task B) and before the test set was opened, and the old rule's picks were scored too, so its cost is shown (Section 5.3), but it is still a change of plan after some validation results were known.

7.3 The results

- **Validation did not predict every test score within 0.03.** A2-NonVPN scored 0.0328 above its validation score (E12 (a) not supported). E14 (split-seed stability) shows its official validation rows are harder than any of 10 resplits (0.9075 against 0.9257-0.9468), so its validation score understates it.
- **Test scores outside the resplit range.** The A1 test score (0.9434) is above all 10 E14 resplits (at most 0.9397), and the A2-VPN test score (0.8652) is below them (at least 0.8689). The resplit models are trained on less data than the final models and a single 15% test set can land outside a 10-resplit range by chance, but one test set is one sample.
- **Intervals are conditional.** Every interval comes from stratified resampling of the test rows, so it keeps the test set's class counts fixed; it does not include the variation from drawing a different test set, or from training on a different split (E14 measures that separately).
- **Two of the 26 tests are borderline.** A1 against the SVM and A2-VPN against the decision tree have Holm p-values within the Monte Carlo error of 0.05 (bands of about 0.03-0.07 from the 10,000 resamples, Section 5.3). Under the rule fixed in advance both count as not clear; another draw of resamples could change them. No verdict depends on them.
- **Close comparisons.** The headline model is not clearly different from the SVM in any task, and the A2-VPN SVM (0.8670) and the B LightGBM (0.8567) scored slightly above the headline models on test. Nothing was swapped after the test set was seen.
- **Logistic regression did not converge in one search.** The tuned A2-NonVPN logistic regression (C = 985) stops at the fixed limit of 1,000 iterations (E17, equal tuning budget). Its score may be slightly below what a converged model would reach; it was not rerun after the result was seen.
- **Calibration is checked on one test set.** The test ECE (Section 5.4) is a single measurement with 15 bins and no interval.
- **"Timing only" needs the byte rate.** Without flowBytesPerSecond, which depends on packet sizes, validation macro F1 drops by 0.015-0.044 (E09 (c)). The models read no payload, but they do use the amount of data per second.

7.4 Related work

- **Bagui et al. 2017 [12]** is cited from its public abstract only (about 90%); the full paper was not available to us, so no detail beyond the abstract is claimed or compared.
- **Isnan et al. 2026 [13].** Their split stacks all four timeout files, so it mixes timeouts and is not comparable with a single-timeout score. E16 (reproducing Isnan et al.) shows the measured gap to a per-timeout score comes from training-set size (Section 4.18); we do not claim that their score is inflated by the mixing.

8 Conclusion

BlindSpy classifies ISCXVPN2016 flows from 23 timing statistics, without reading any payload, in the three tasks of the original paper: VPN or not (A1), the traffic type within VPN and within non-VPN traffic (A2, two separate models), and all 14 classes at once (B).

We first checked the data and the paper. The ARFF headers do not always describe their columns: most A1 and B files hold the paper's standard deviations under shifted names, and the A2 files hold different features, which we mapped from value checks before building any model (E00, data check). Under the paper's own protocol our C4.5 approximation and 1-NN reproduce its A1 results (16 of 16 and 15 of 16 values within 0.05; E02 (decision-tree baseline), E03 (KNN, logistic regression and SVM baselines)). A recent result on a stacked copy of the dataset (Isnan et al., 88.01%) was reproduced as well (0.8796), and its gap to a per-timeout score was traced to training-set size, not to mixing timeouts (E16, reproducing Isnan et al.).

On a leakage-safe protocol (deduplicated data, a stratified 70/15/15 split, every choice made on training and validation rows, the test set scored once), gradient-boosted trees were the best models on validation and scored test macro F1 0.9434 for A1, 0.8652 for the VPN model and 0.9403 for the non-VPN model of A2, and 0.8480 for B, each with a 95% bootstrap interval (E12, final test). Every method type had the same tuning budget (E07, E17 (equal tuning budget)), and the order of the seven types by validation score largely held on the test rows (Spearman 0.82-0.96). Class balancing and a two-step model for task B did not help (E08 (class balancing), E10 (one model or two steps for task B)). The byte rate and the shortest gaps between packets are among the features the models use most (SHAP), dropping the rates and duration costs the most in three of the four tasks (E09, feature-group ablation), and both methods find that active and idle times add little (Section 5.6). Confidence is calibrated where the validation data showed it was needed (E15, calibration), and the command-line predictor and the Streamlit demo [23] use the four final models to classify new flow files and explain each prediction.

These results hold for this dataset only. The data were captured in a lab in 2016, with specific applications and one VPN setup, and the captures have documented quality problems [14]. Flows from one capture can sit on both sides of a random split, and holding out contiguous blocks of the files lowers out-of-fold accuracy by 0.07-0.26 (Section 7.2), so performance on traffic from new captures, new applications or today's VPN services may well be lower than the scores reported here; it has not been measured. A session-aware split on data with capture identifiers, and a test on traffic captured independently of ISCXVPN2016, are the next steps before any of these models is used on a real network.

All runs, data versions and decisions are kept: 2,013 MLflow runs in one exported history table, one log entry per experiment, a changelog per data version, and a saved copy of the exact code at each milestone (Appendix).

References

1. G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of Encrypted and VPN Traffic using Time-related Features," in *Proc. 2nd Int. Conf. on Information Systems Security and Privacy (ICISSP)*, 2016, pp. 407–414.
2. Canadian Institute for Cybersecurity, University of New Brunswick, "VPN-nonVPN dataset (ISCXVPN2016)." [Online]. Available: https://www.unb.ca/cic/datasets/vpn.html
3. F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
4. T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
5. G. Ke et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," in *Advances in Neural Information Processing Systems 30*, 2017.
6. S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in Neural Information Processing Systems 30*, 2017.
7. T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A Next-generation Hyperparameter Optimization Framework," in *Proc. 25th ACM SIGKDD*, 2019, pp. 2623–2631.
8. M. Zaharia et al., "Accelerating the Machine Learning Lifecycle with MLflow," *IEEE Data Engineering Bulletin*, vol. 41, no. 4, pp. 39–45, 2018.
9. G. Lemaître, F. Nogueira, and C. K. Aridas, "Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning," *JMLR*, vol. 18, no. 17, pp. 1–5, 2017.
10. N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
11. J. R. Quinlan, *C4.5: Programs for Machine Learning*. Morgan Kaufmann, 1993.
12. S. Bagui, X. Fang, E. Kalaimannan, S. C. Bagui, and J. Sheehan, "Comparison of machine-learning algorithms for classification of VPN network traffic flow using time-related features," *Journal of Cyber Security Technology*, vol. 1, no. 2, pp. 108–126, 2017, doi: 10.1080/23742917.2017.1321891.
13. W. Isnan, Y. Fatmi, R. Darni, and V. Aulia, "Flow-Based Encrypted Network Traffic Classification Using Random Forest for Network Access Control," *Intechno Journal: Information Technology Journal*, vol. 8, no. 1, 2026, doi: 10.24076/intechnojournal.2026v8i1.2922.
14. S. Jorgensen et al., "Extensible Machine Learning for Encrypted Network Traffic Application Labeling via Uncertainty Quantification," *IEEE Transactions on Artificial Intelligence*, 2023, doi: 10.1109/TAI.2023.3244168 (arXiv:2205.05628).
15. G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of Encrypted and VPN Traffic Using Time-Related Features" (poster), UNB Research Expo, 2016. [Online]. Available: https://www.cs.unb.ca/research-expo/expos/2016/
16. University of Waikato, "Weka 3 API documentation (stable)": classes `weka.classifiers.trees.J48`, `weka.classifiers.lazy.IBk`, `weka.core.EuclideanDistance` and `weka.core.NormalizableDistance`. [Online]. Available: https://weka.sourceforge.io/doc.stable/ (accessed 28 Oct 2026).
17. S. Holm, "A Simple Sequentially Rejective Multiple Test Procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.
18. B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York: Chapman & Hall, 1993.
19. B. Zadrozny and C. Elkan, "Transforming Classifier Scores into Accurate Multiclass Probability Estimates," in *Proc. 8th ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining (KDD)*, 2002.
20. M. P. Naeini, G. F. Cooper, and M. Hauskrecht, "Obtaining Well Calibrated Probabilities Using Bayesian Binning," in *Proc. 29th AAAI Conf. on Artificial Intelligence*, 2015, pp. 2901–2907.
21. C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On Calibration of Modern Neural Networks," in *Proc. 34th Int. Conf. on Machine Learning (ICML)*, PMLR 70, 2017, pp. 1321–1330.

22. C. Spearman, "The Proof and Measurement of Association between Two Things," *American Journal of Psychology*, vol. 15, no. 1, pp. 72–101, 1904.
23. Snowflake Inc., "Streamlit: a faster way to build and share data apps" (open-source Python library, Apache 2.0 licence). [Online]. Available: https://streamlit.io

Appendix

A.1 Repository

The code, configurations, experiment notes and results are at https://github.com/Varun-U-Pratap/BlindSpy.

A.2 MLflow screenshots

The official MLflow store holds all 2,013 runs (A.5). Below: the run list of the `blindspy` experiment (newest first: the 30 E12 runs), and one final run, E12-final-B, with its test metrics and the tags every run carries (runner, machine, code version, lock-file hash, package versions, data version and data hashes). The demo is shown in Section 5.11.

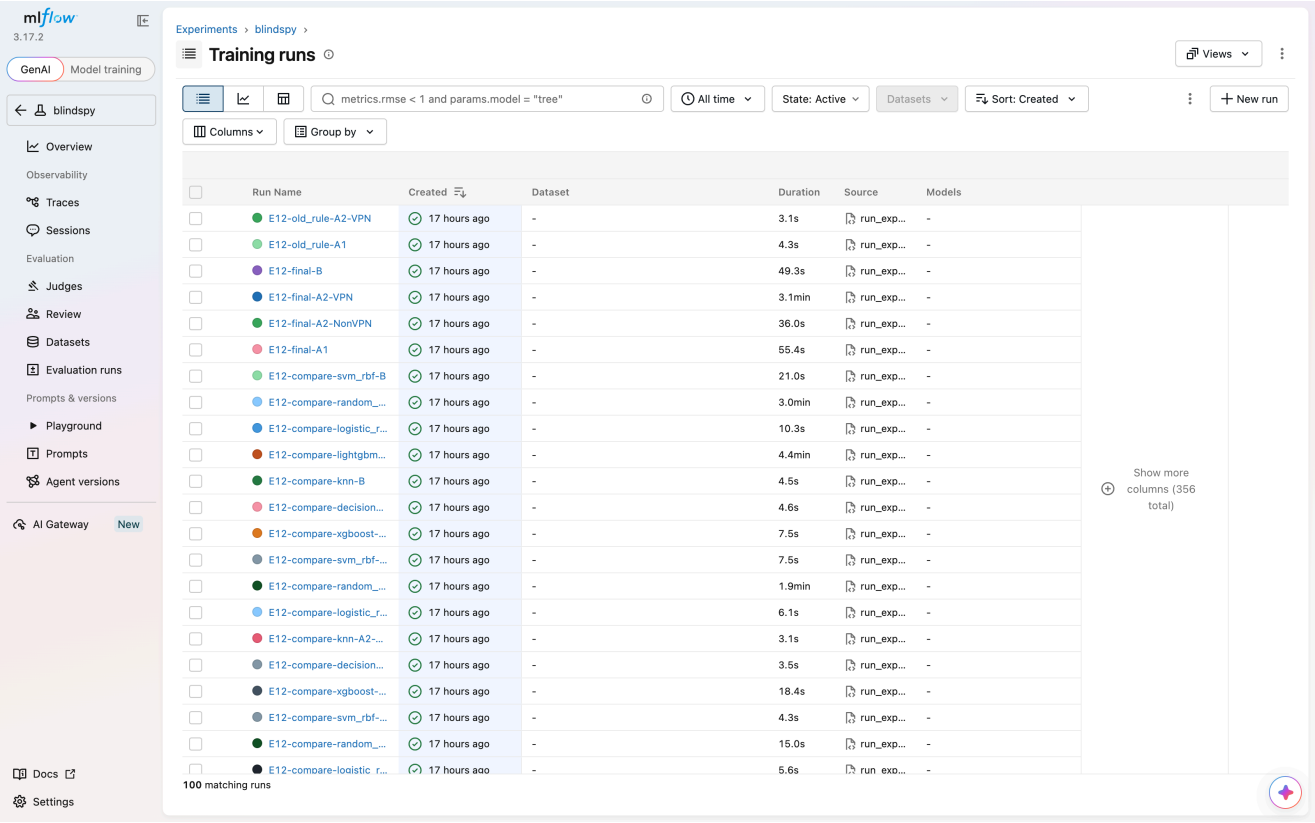


Figure 32. MLflow: the run list of the blindspy experiment

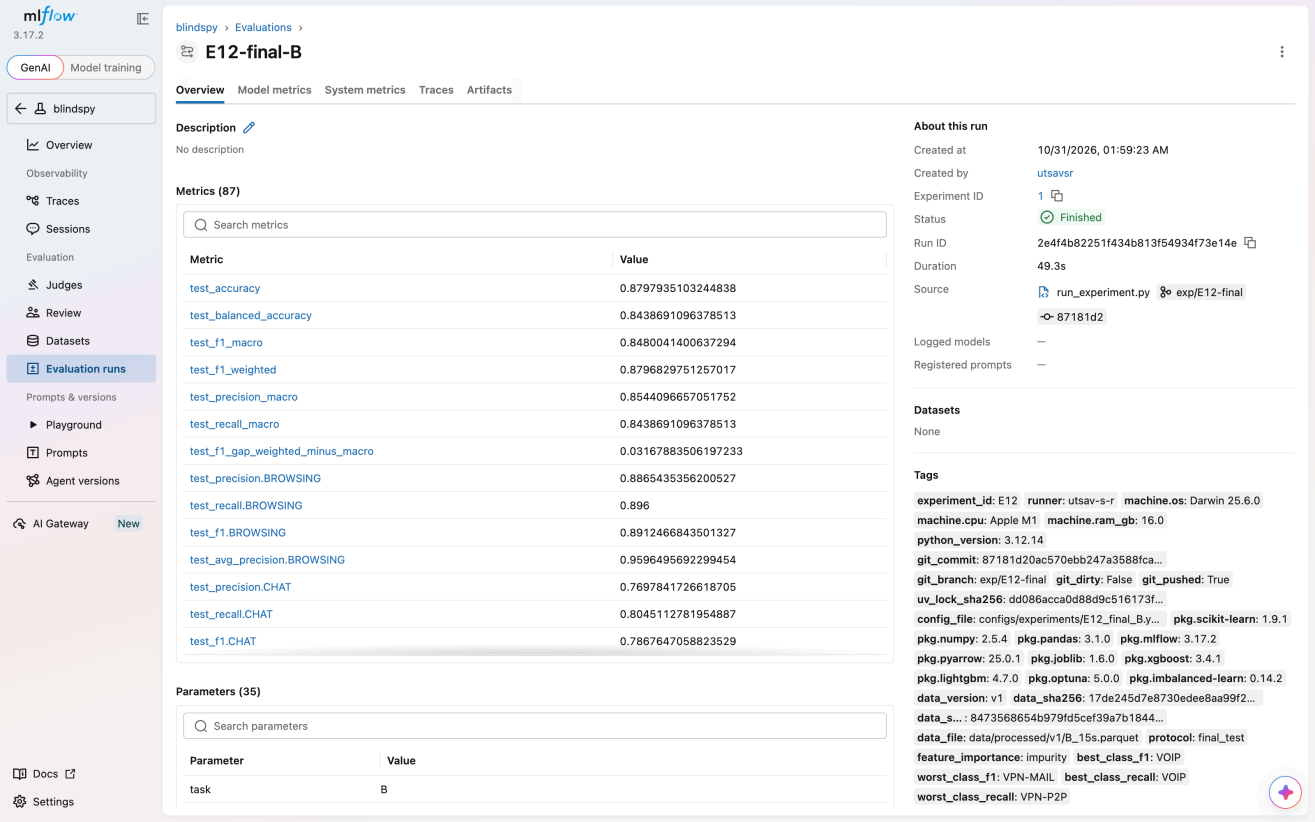


Figure 33. MLflow: run E12-final-B, its metrics and tags

A.3 Data changelog

One entry per cleaned-data version, newest first. A version is built with `uv run python scripts/prepare_data.py --version vN` into `data/processed/vN/` (never overwritten). Every number below comes from that version's `manifest.json` and `cleaning_log.csv`.

Version	Date	Author	What changed	Files	Rows (raw lines -> final)
kaggle-k1	2026-10-29	utsav-s-r	Kaggle CSV of Isnan et al. (2026) as its own version, for E16 only; data v1 unchanged	1 CSV (= the 8 A2 files stacked)	59,706 -> 40,987 (their dedup) / 56,030 kept per timeout
v1	2026-10-27	utsav-s-r	First cleaned version	16 core files (A1, A2-VPN, A2-NonVPN, B x 15/30/60/120 s)	179,120 -> 167,909

kaggle-k1 (2026-10-29, utsav-s-r)

What. The Kaggle CSV used by Isnan et al. (2026), `datasets/kaggle/consolidated_traffic_data.csv` (SHA-256 in `data/kaggle_sha256.txt`), as its own data version for E16. Built with `uv run python scripts/prepare_kaggle.py --version k1` into `data/processed/kaggle/k1/` (never overwritten); manifest copied to `data/kaggle_k1_manifest.json`. Every number below comes from that manifest.

- The CSV is the eight A2 files (VPN and NonVPN x 4 timeouts) stacked in their original row order: A2-NonVPN 120 s (5,151 rows), A2-VPN 120 s (5,631), A2-NonVPN 15 s (8,965), A2-VPN 15 s (9,793), A2-NonVPN 30 s (6,917), A2-VPN 30 s (7,734), A2-NonVPN 60 s (8,580), A2-VPN 60 s (6,935). Each block was checked against its ARFF file (23 features at 12 significant digits and the label), so every row's source file, timeout and row number are known (`rows.parquet`).
- Labels: the 14 task-B classes. Features: the 23 A2 features (total_fiat / total_biat layout).
- Split "isnan" (Isnan et al.'s setup): exact duplicates (23 features + label) dropped over the whole CSV, first copy kept: 59,706 -> 40,987 rows; stratified 80/20, seed 42: 32,789 / 8,198.
- Splits "per_timeout_<t>s" (corrected setup): each timeout on its own, duplicates dropped within it, then stratified 80/20, seed 42: 15 s 18,758 -> 18,074 (14,459 / 3,615); 30 s 14,651 -> 13,638 (10,910 / 2,728); 60 s 15,515 -> 14,729 (11,783 / 2,946); 120 s 10,782 -> 9,589 (7,671 / 1,918). 56,030 rows kept over the four timeouts.

- Distinct rows (23 features + label) by the number of timeout files they appear in: 1: 33,894; 2: 2,475; 3: 1,286; 4: 3,332 (7,093 in two or more, as E00 found).
- Split files keep the row order `train_test_split` returns (a seeded forest and an unshuffled 5-fold CV depend on it).
- Not done, on purpose: no cleaning beyond the duplicate rule (no odd-value rules, no column changes); the data is used as Isnan et al. used it.

Why. E16 reproduces Isnan et al. and then repeats their protocol with the timeouts kept apart; their data mixes timeouts, so it needs its own version, separate from v1.

Checks. Two builds into separate folders and the build in the main folder gave identical SHA-256 hashes for all 6 files; a second build into an existing folder is refused (`tests/test_kaggle.py`).

v1 (2026-10-27, utsav-s-r)

What. First cleaned version of the 16 core task files. Rules in `configs/base.yaml` (`cleaning:`), evidence in `reports/experiments/utsav-s-r/E00.md`.

- Parser: trailing commas stripped from every header line; 2 comma-only data lines skipped (one each in A1 60 s and B 60 s; B-AllinOne 60 s has one too but is not a core task).
- Column names: `COLUMN_MAP` renames the 9 A1/B files whose header shifts the fiat/biat names by one pair. A1 and B hold the paper's Table 2 features (fiat/biat as min, max, mean, std); A2 holds `total_fiat`/`total_biat` instead of `std_fiat`/`std_biat`.
- A1 120 s: `flowPktsPerSecond` restored from B 120 s in 10,288 of 10,782 rows (the column was corrupt; B 120 s is identical in the other 22 columns).
- Exact duplicates (23 features + label) dropped within each file, first copy kept: 11,209 rows (A1 3,857; A2-VPN 2,598; A2-NonVPN 1,078; B 3,676). Conflicting duplicates (same features, different label) kept: 1,102 rows over the 16 files.
- Odd values kept as-is and logged with counts: -1 "not measurable" codes, duration-0 single-packet flows, negative `min_flowiat`, 3-significant-figure rounding of large means/stds/rates, bytes per packet outside 40-1,500.
- Not done, on purpose: no imputation (0 missing values), no normalization, no feature selection (all 23 kept), no outlier removal, no class balancing at data level.
- Splits: stratified 70/15/15, seed 42, per file, saved in `splits/`. 0 exact duplicates cross train and test in any file.

Why. The raw ARFF files can't be used as published: wrong column names in 9 files, one corrupt column, parse glitches, and duplicate rows that would leak between train and test.

Size. Parquet 18.7 MB for the 16 files (20 MB folder with splits and log). The build is deterministic: three builds gave byte-identical files.

Per file (from the manifest):

File	Raw lines	Duplicates removed	Final	Train / val / test
A1 15 s	18,758	720	18,038	12,626 / 2,706 / 2,706
A1 30 s	14,651	1,041	13,610	9,526 / 2,042 / 2,042
A1 60 s	15,516 (1 comma-only)	838	14,677	10,273 / 2,202 / 2,202
A1 120 s	10,782	1,258	9,524	6,666 / 1,429 / 1,429
A2-VPN 15 s	9,793	413	9,380	6,566 / 1,407 / 1,407
A2-VPN 30 s	7,734	790	6,944	4,860 / 1,042 / 1,042
A2-VPN 60 s	6,935	412	6,523	4,565 / 979 / 979
A2-VPN 120 s	5,631	983	4,648	3,252 / 698 / 698

File	Raw lines	Duplicates removed	Final	Train / val / test
A2-NonVPN 15 s	8,965	271	8,694	6,085 / 1,304 / 1,305
A2-NonVPN 30 s	6,917	223	6,694	4,685 / 1,004 / 1,005
A2-NonVPN 60 s	8,580	374	8,206	5,744 / 1,231 / 1,231
A2-NonVPN 120 s	5,151	210	4,941	3,458 / 741 / 742
B 15 s	18,758	684	18,074	12,651 / 2,711 / 2,712
B 30 s	14,651	1,013	13,638	9,546 / 2,046 / 2,046
B 60 s	15,516 (1 comma-only)	786	14,729	10,309 / 2,210 / 2,210
B 120 s	10,782	1,193	9,589	6,711 / 1,439 / 1,439

A.4 Experiment log

The full experiment log, one entry per experiment (newest experiment first; failures, corrections and rechecks included as they were written):

Generated by scripts/export_history.py from reports/experiments/utsav-s-r/. Do not edit by hand.

E17 — Equal tuning budget for every method type (2026-10-28, utsav-s-r)

Why this experiment exists (decided 28 Oct after E10, user OK; plan item "Decided 28 Oct, after E10"): Phase 4 scores a comparison set, the best validation variant of each of the 7 method types per task. Up to now only some types were tuned: E07 searched XGBoost and KNN in all tasks, LightGBM in A2-VPN, A2-NonVPN and B, and Random Forest in A1. The decision tree, logistic regression and SVM ran with defaults only. A comparison of tuned against untuned models would favour the tuned types for the effort they got, not for what they can do (the weakness we cite for Isnan et al.). E17 gives every remaining type the same search E07 used, so each type enters the comparison with the same budget.

Hypothesis: tuning helps every type, most of all logistic regression and SVM. Their E03 defaults scored only 0.30-0.60 macro F1 because the raw timing features span six or more orders of magnitude (E04 showed that a signed log fixes this for KNN). Even tuned, they should stay below the gradient-boosted models, so the overall best per task does not change.

Threshold (set before running, never moved afterwards), validation macro F1 at 15 s; reference values from `reports/phase3_tables.md` and `reports/baseline_tables.md`:

- (a) in at least 12 of the 16 searches the tuned model scores above its untuned version (E02

tree, E03 logistic regression, E03 SVM, E05 Random Forest, E06 LightGBM);

- (b) after E17, the best run per task (highest 15 s validation macro F1 of all selection

experiments) is still an XGBoost or LightGBM model in at least 3 of the 4 tasks;

- (c) the searches choose the signed log for logistic regression and SVM in at least 6 of the 8

searches. Each part gets its own verdict.

Winner rule (decided 28 Oct, written here before running): a model whose 15 s validation macro F1 beats that task's current best by at least 0.03 is a winner and is rerun at 30, 60 and 120 s with its 15 s settings (nothing is re-tuned per timeout: that would be a different, 4 times costlier experiment). Why 0.03: it is above the largest Phase 2 CV standard deviation (0.025), so a smaller gain could be noise from the split; a fixed number set before running keeps the choice honest and repeatable. The current best is the highest 15 s validation macro F1 whatever the margin; the 0.03 only decides the reruns. (The final choice per task no longer uses the 0.03: it is the highest validation macro F1, decided 28 Oct after E10.)

Change vs the untuned runs (one change: the hyperparameters, chosen by search), 16 searches:

- decision tree, logistic regression, SVM-RBF: all 4 tasks;
- Random Forest: A2-VPN, A2-NonVPN, B (A1 was searched in E07);
- LightGBM: A1 (the other tasks were searched in E07).

Each search as E07: Optuna `TPESampler(seed=42)`, 50 trials run one after another, each trial scored by the mean macro F1 of 5-fold stratified CV on the training rows (folds in parallel, each model on one thread). Validation and test rows are never seen by the search. The best trial's settings are retrained on all training rows and scored on validation like any other run and saved; trials keep settings and scores only (nested runs tagged `trial=True`, protocol `optuna_trial`, kept out of the result tables). The retrained run's CV mean must equal the best trial's exactly.

Search ranges (user OK 28 Oct; written into `scripts/make_configs.py` and every `configs/experiments/E17_<model>_<task>_15s.yaml` before running; Random Forest and LightGBM use exactly the E07 ranges, so their searches match E07's):

- Decision tree: criterion gini or entropy; max_depth 3-40 or no limit; min_samples_leaf 1-50;

ccp_alpha (cost-complexity pruning, the nearest scikit-learn has to C4.5 pruning) 1e-6-1e-2 (log scale). A monotone transform such as the signed log cannot change a tree's splits, so it is not searched.

- Logistic regression: signed log before StandardScaler yes or no; C 1e-3-1e3 (log scale);

max_iter fixed at 1000 as in E03; L2 penalty, lbfgs solver (scikit-learn defaults).

- SVM-RBF: signed log before StandardScaler yes or no; C 1e-2-1e3 (log scale); gamma

1e-4-1e1 (log scale); cache_size fixed at 1000 MB (speed only, results unchanged).

- Random Forest (as E07): n_estimators 100-1000; max_depth 5-40 or no limit;

min_samples_leaf 1-10; max_features sqrt, log2 or a fraction 0.3-1.0.

- LightGBM (as E07): n_estimators 100-1000; learning_rate 0.01-0.3 (log scale); num_leaves

15-255; subsample 0.5-1.0 with subsample_freq fixed at 1; colsample_bytree 0.5-1.0; min_child_samples 1-50. The signed log is offered to logistic regression and SVM as a choice inside the search (not as a separate experiment like E04) because both, like KNN, depend on distances or linear scales that the raw feature ranges distort; the search decides per task. Balancing (E08) is not part of E17: it stays only on the models E08 ran on, and the comparison table shows each model's balancing.

Cost check before running (throwaway fits, nothing logged): one SVM fit on the 12,651 B training rows takes 2.2-3.9 s at the edges of the range (C 1000 with gamma 10 or 1e-4, C 0.01 with gamma 1e-4, signed log with C 1000), so no search is expected to stall.

Protocol as E07: data v1, 15 s, test set untouched. New registry option: `signed_log` for logistic regression and SVM (`src/blindspy/models.py`); without it the pipelines are exactly the E03 ones.

Result: 16 searches (hypothesis and configs recorded 22 s before the first run): 16 retrained runs and 800 trial runs, all FINISHED, data v1; 49.1 minutes of search in all. Numbers below come from `reports/phase3_tables.md` (tables "E17 searches", "E17 at 15 s against the best run before it", "E17 run details", "Current best at 15 s after E17"), generated by `scripts/results_tables.py --phase 3` from `reports/history.csv`. For all 16 searches the retrained run's CV macro F1 equals the best trial's exactly.

Validation macro F1 at 15 s, untuned -> tuned (chosen settings in table "E17 searches"):

Task	Tree	Logistic regression	SVM-RBF	Random Forest	LightGBM
A1	0.8901 -> 0.8905	0.5847 -> 0.6517	0.6016 -> 0.9259	(tuned in E07)	0.9181 -> 0.9308
A2-VPN	0.8247 -> 0.8292	0.4324 -> 0.7263	0.4682 -> 0.8707	0.8679 -> 0.8704	(tuned in E07)

Task	Tree	Logistic regression	SVM-RBF	Random Forest	LightGBM
A2-NonVPN	0.8294 -> 0.8294	0.4487 -> 0.8002	0.4870 -> 0.9058	0.8941 -> 0.8829	(tuned in E07)
B	0.7585 -> 0.7585	0.3072 -> 0.5862	0.2965 -> 0.8242	0.8147 -> 0.8167	(tuned in E07)

- SVM gains most: +0.32 to +0.53. Every SVM and logistic-regression search chose the signed log

(8 of 8); SVM chose C 5.6-879 and gamma 0.18-2.77, logistic regression C 22.3-985.2. Tuned SVM is now close to the boosters: A1 0.9259 (best 0.9308), A2-NonVPN 0.9058 (best 0.9075), A2-VPN 0.8707 (0.8830), B 0.8242 (0.8428). Its E03 score (0.30-0.60) came from the raw feature scales, not from the method.

- Logistic regression gains +0.07 to +0.35 but stays below every other type in every task

(0.59-0.80): even after the signed log, a linear model separates these classes much worse than the non-linear ones (why exactly is not tested here).

- Decision tree: the searches settle on (almost) the E02 tree, an unpruned entropy tree (largest

ccp_alpha chosen 0.00066). Change +0.0000 to +0.0044; in A2-NonVPN and B the tuned tree scores exactly the same as E02 (equal to all digits). More pruning did not help on these data.

- Random Forest (A2-VPN, A2-NonVPN, B): +0.0025, -0.0111 and +0.0020. In A2-NonVPN the tuned forest

(132 trees) has a higher CV score (0.9063 vs 0.9048) but a lower validation score: the best of 50 CV estimates is optimistic, as seen in E07.

- LightGBM A1: +0.0127, to 0.9308, now the best A1 model (above E06 XGBoost 0.9271 by +0.0037,

no winner).

- No winner (largest gain over the best before E17: +0.0037), so nothing is rerun at 30, 60 or

120 s. Current best at 15 s after E17: A1 E17 LightGBM 0.9308, A2-VPN E08 LightGBM + class weights 0.8830, A2-NonVPN E08 LightGBM + oversampling 0.9075, B E07 XGBoost 0.8428.

- Convergence: logistic regression keeps E03's max_iter = 1000 (fixed before running). In the

search, lbfgs stopped at that limit in some trials (216 ConvergenceWarnings in the batch log). Refitting the 4 chosen settings: A1, A2-VPN and B converge (260, 800 and 716 iterations); A2-NonVPN (C = 985) stops at 1,000, so its score may be slightly below what more iterations would give. All 4 chosen SVMs converge. Not changed after the fact; noted as a limitation.

- Cost (table "E17 run details"): tuned SVM predicts slowly (47.6-302.8 ms per 1,000 flows; 1,061-

5,711 support vectors) but is small (0.17-1.31 MiB as saved); tuned LightGBM A1 7.1 MiB and 191 ms per 1,000 flows; tuned Random Forest B 340.4 MiB uncompressed / 45.8 MiB as saved (708 trees, 2.03 million nodes), the second-largest Phase 3 model file (largest: E07 LightGBM B, 62.5 MiB).

- Threads: Random Forest and LightGBM run with n_jobs = 1. SVM has no n_jobs setting (libsvm fits

on one thread), the decision tree neither, and logistic regression logs n_jobs = None, scikit-learn's default, which means one job; so every model's own setting is one job, as the 28 Oct thread decision requires (CV folds in parallel). Numerical libraries underneath (BLAS, used by logistic regression) keep their default threading, as in every earlier run.

- 5-fold CV agrees with validation within 0.033 (CV - val -0.016 to +0.033; largest: tree

A2-NonVPN); CV std 0.0028-0.0132.

Storage: the 16 retrained E17 runs take 79.4 MiB (forest 68.0, LightGBM 7.2, SVM 2.9, tree 0.7, logistic regression 0.5); trials save no model. Whole store now: artifacts 664.1 MiB, mlflow.db 26.1 MiB (1,756 runs). Far below the plan's 20 GB line. Flag for Phase 4: the E17 Random Forest for B (45.8 MiB as saved) is the best B forest, so it is the forest in the B comparison set; it is under the 50 MB size limit for final model files, but close.

Verdict: (a) supported (tuned above untuned in 13 of 16 searches; 12 needed; not above: Random Forest A2-NonVPN -0.0111, tree A2-NonVPN and B 0.0000). (b) supported (the best run per task is XGBoost or LightGBM in 4 of 4 tasks). (c) supported (signed log chosen in 8 of 8 logistic- regression and SVM searches).

Decision: every method type now has the same 50-trial budget per task, so the comparison set can be built fairly. The final choice (highest validation macro F1, decided 28 Oct after E10) and the 28-model comparison set are written to configs/final/ next.

MLflow runs: E17-<model>-<task>-15s (retrained best); trials E17-<model>-<task>-15s-trial-NNN

E16 — Reproduce Isnan et al. (2026) on the Kaggle CSV, then correct it (2026-10-29, utsav-s-r)

Purpose (plan): Isnan et al. report 88.01% accuracy and 87.97% weighted F1 for a 300-tree Random Forest on 14 ISCXVPN2016 classes. E00 found that their data, the Kaggle CSV, is the eight A2 files from all four timeouts stacked, so their duplicate removal and their 80/20 split mix timeouts. E16 (a) reproduces their setup exactly, (b) repeats it with the timeouts kept apart, and (c) trains our best task-B model type and settings under (b). No E16 result feeds model selection (protocol tags are not "holdout"; E16 is not a model-selection experiment).

Data: own data version kaggle-k1 (`scripts/prepare_kaggle.py --version k1` -> `data/processed/kaggle/k1/`, manifest copied to `data/kaggle_k1_manifest.json`; DATA_CHANGELOG entry; data v1 untouched). Found while building it: the CSV keeps the A2 files' row order, in the order 120, 15, 30, 60 s (NonVPN then VPN within each), so every CSV row's source file, timeout and row number are known exactly; each block was checked against its ARFF file (all 23 features at 12 significant digits and the label). The split files keep the rows in the order `train_test_split` returns them (not sorted), because a seeded forest's bootstrap samples and an unshuffled 5-fold CV depend on row order, and Isnan et al. trained on that order (found in the recheck before E16; an earlier draft sorted them). Two builds into separate folders gave identical hashes for all 6 files. Counts (manifest): 59,706 rows, 14 labels (the B classes); 40,987 after dropping exact duplicates over the whole CSV (Isnan et al.: 40,987), split 32,789 / 8,198 (theirs: 32,789 / 8,198); 7,093 distinct rows appear in 2-4 timeout files (2,475 in 2, 1,286 in 3, 3,332 in 4), as E00 found.

Their setup, read from their paper (about/related/intechno-2026_rf-flow-classification.pdf, read again 29 Oct): exact duplicates dropped; labels encoded with LabelEncoder (alphabetical, the same class order as our string labels, so the stratified split is the same); stratified 80/20 split with random_state 42; GridSearchCV with stratified 5-fold cross-validation (scikit-learn's default, no shuffling) and accuracy as the score over n_estimators {100, 200, 300}, max_depth {None, 20, 30}, min_samples_split {2, 5}, min_samples_leaf {1, 2} (36 configurations, 180 fits), Gini, max_features sqrt, bootstrap, random_state 42; their pick 300 trees, max_depth None, min_samples_split 2, min_samples_leaf 1; test accuracy 88.01%, weighted F1 87.97%.

Protocols (all with seed 42; Random Forest with n_jobs = 1 per model and the grid's CV fits in parallel, which does not change a seeded forest's result):

- (a) Isnan et al.: their dedup over the whole CSV, their split, their grid search, then the

chosen model refitted on the 32,789 training rows and scored once on the 8,198 test rows.

- (b) Timeouts kept apart: for each timeout t, only the rows from the two A2 files of t,

exact duplicates dropped within t, a stratified 80/20 split of those rows (seed 42), the same grid search, the chosen model scored on t's test rows. A flow and its copies from other timeouts can then never be on both

sides. 4 runs (15, 30, 60, 120 s); after dedup 18,074 / 13,638 / 14,729 / 9,589 rows.

- (c) Our model under (b): the current best task-B model type and settings (E07 XGBoost, tuned:

`configs/final/B.yaml`, copied from E07-xgb-B-15s), retrained on each timeout's training rows of (b) and scored on the same test rows; no grid search (our settings are fixed). The Kaggle rows use the A2 column layout (total_fiat / total_biat), so the model is retrained, not reused. 4 runs. Reported for every run: accuracy and weighted F1 (Isnan et al.'s scores) and macro F1 (ours), per-class precision / recall / F1 and the confusion matrix; for (a) and (b) also the grid's best settings and best CV accuracy.

Hypothesis: (a) reproduces their numbers closely, since their setup is fully specified and seeded. (b) scores lower than (a): in (a) the same flow can sit in the training set at one timeout and in the test set at another (as a slightly different row, so dedup does not remove it), which helps the model; (b) removes that. (c) scores at least as high as the Random Forest under (b), as in our own experiments where tuned XGBoost beats the forest on task B (E05-E07).

Threshold (set before running, never moved afterwards):

- (a) test accuracy within 0.01 of 88.01% and weighted F1 within 0.01 of 87.97%;
- (b) the mean test accuracy over the 4 timeouts under (b) is at least 0.01 below (a)'s;
- (c) the mean test accuracy over the 4 timeouts under (c) is at least the mean under (b).

Each part gets its own verdict. The (a) - (b) gap is reported in any case.

Winner rule (decided 28 Oct): not applicable; E16 never feeds model selection, and its scores are on the Kaggle rows, not on our v1 validation rows.

Result: 9 runs (hypothesis recorded before the run code and the first run), all FINISHED, data version kaggle-k1. The first run (E16-per_timeout-xgb-120s, the new run kind) was checked by query before the others: the 120 s split's 7,671 / 1,918 rows, only the 23 features, data and split hashes equal to the kaggle-k1 manifest, the B headline model's settings, one thread, compression 3. Numbers below come from `reports/phase3_tables.md` (table "E16 Isnan et al. on the Kaggle CSV"), generated by `scripts/results_tables.py --phase 3` from `reports/history.csv`; the grid ranking from the run's `grid/cv_results.csv`. Test = the 80/20 split's Kaggle test rows (not our v1 test set).

Run	Train / test rows	Grid pick	Test accuracy	Weighted F1	Macro F1
(a) Isnan setup, timeouts mixed	32,789 / 8,198	200 trees, no depth limit, split 2, leaf 1	0.8796	0.8792	0.8407
(b) Random Forest, 15 s	14,459 / 3,615	300 trees, depth 30, split 2, leaf 1	0.8658	0.8652	0.8323
(b) Random Forest, 30 s	10,910 / 2,728	300 trees, depth 30, split 2, leaf 1	0.8537	0.8527	0.8150
(b) Random Forest, 60 s	11,783 / 2,946	300 trees, depth 30, split 2, leaf 1	0.8316	0.8308	0.7873
(b) Random Forest, 120 s	7,671 / 1,918	300 trees, no depth limit, split 5, leaf 1	0.8206	0.8170	0.7724
(c) our XGBoost, 15 s	14,459 / 3,615	fixed (configs/final/B.yaml)	0.8924	0.8919	0.8637
(c) our XGBoost, 30 s	10,910 / 2,728	fixed	0.8728	0.8720	0.8327
(c) our XGBoost, 60 s	11,783 / 2,946	fixed	0.8551	0.8544	0.8076
(c) our XGBoost, 120 s	7,671 / 1,918	fixed	0.8384	0.8346	0.7881

- (a) reproduces Isnan et al. almost exactly: accuracy 0.8796 against their 0.8801, weighted F1

0.8792 against 0.8797 (both 0.0005 lower), on the same 32,789 / 8,198 split. The grid picked 200 trees where they report 300, all other settings the same; in our grid their pick ranks 4th with a CV accuracy 0.00012 below ours (0.874043 vs 0.874165), and the top 6 settings lie within 0.0002 of each other, so the pick is a near-tie that small numerical differences (library versions) can reorder. Their accuracy hides the small classes: macro F1 is 0.8407, 0.039 below accuracy.

- (b) keeping the timeouts apart lowers the same method's accuracy to 0.8206-0.8658, mean 0.8430:

0.0366 below (a). Every single timeout is below (a), the shortest timeout best (15 s 0.8658). In (a) the same flow can sit in training at one timeout and in test at another, as a slightly different row that duplicate removal does not catch; (b) rules that out. So part of their 88.01% comes from mixing timeouts, not from the model. [Corrected 29 Oct, see addendum (d): this cause was not measured; at equal training size, mixing timeouts adds nothing, and the gap is the training-set size.]

- (c) our task-B model type and settings, trained on the same per-timeout rows, beats their tuned

forest at every timeout (+0.0266 at 15 s, +0.0191, +0.0235, +0.0178), mean accuracy 0.8647 against 0.8430, and macro F1 0.8637 at 15 s against the forest's 0.8323. At 15 s, under the corrected protocol, it scores 0.8924 accuracy: above their mixed-timeout 0.8801, without the timeout mixing.

- Grid search time: 5.6 min for (a), 1.1-2.2 min per timeout for (b) (180 fits each, CV fits in

parallel, one thread per forest).

- Not comparable with our own validation scores: these are Kaggle rows in the A2 column layout

(total_fiat / total_biat), with the Kaggle 80/20 split, not our v1 B data (std_fiat / std_biat) and 70/15/15 split. E16 feeds no model selection.

Storage (table "Storage of the Phase 3 runs"): the 5 forest runs take 120.0 MiB (largest model file 34.9 MiB, (a)), the 4 XGBoost runs 17.8 MiB; whole artifact store now 2,311.7 MiB, mlflow.db 34.8 MiB, far below the plan's 20 GB line.

Verdict: (a) supported (accuracy 0.8796 and weighted F1 0.8792, both within 0.01 of 0.8801 and 0.8797). (b) supported (per-timeout mean 0.8430, 0.0366 below (a); at least 0.01 needed). (c) supported (our XGBoost's per-timeout mean 0.8647 is above the forest's 0.8430).

Decision: for the report (related work and limitations): the published 88.01% is reproducible on the Kaggle CSV, but about 0.037 of accuracy depends on mixing the four timeout files; with the timeouts kept apart their forest scores 0.82-0.87, and our tuned XGBoost 0.84-0.89. Nothing in the final configs changes. [Corrected 29 Oct, see addendum (d): the 0.037 depends on the larger training set that stacking the timeouts gives, not on the mixing itself.]

MLflow runs: E16-isnan-rf, E16-per_timeout-rf-<t>s, E16-per_timeout-xgb-<t>s

Addendum (29 Oct, recheck): (d) training-size control, written before its runs

Why: the result above explains the (a) - (b) gap by timeout mixing ("the same flow can sit in training at one timeout and in test at another"). That was not measured. (a) also trains on far more rows than (b): 32,789 against 7,671-14,459 per timeout (2.3-4.3 times as many), and E13 shows our models gain from more training rows. So the gap may come mainly from training-set size. Disclosure: before this addendum, one unlogged check in a scratch folder (subsample seed 42 only, nothing kept) fitted (a)'s forest on 7,671-14,459 timeouts-mixed training rows and scored 0.82-0.85 on (a)'s test rows; that check is why (d) exists, and (d) repeats it as logged runs with 5 seeds.

Change vs (a) (one change: the number of training rows): (a)'s forest with (a)'s grid pick (200 trees, no depth limit, split 2, leaf 1, Gini, sqrt features, random_state 42, read from `reports/history.csv` by `scripts/make_configs.py`), trained on a stratified sample of (a)'s 32,789 timeouts-mixed training rows (`kaggle.subsample`) of the size of each per-timeout training set in (b) (14,459 / 10,910 / 11,783 / 7,671, from the kaggle-k1 manifest), subsample seeds 42-46, and scored once on (a)'s 8,198 test rows: overall and per timeout (new metrics `kaggle_test_accuracy_<t>s`). No grid search (settings fixed). Plus one run on all 32,789 rows (`E16-size-full`), which must reproduce (a)'s test accuracy exactly and adds (a)'s per-timeout breakdown. 4 x 5 + 1 = 21 runs; configs `configs/experiments/E16_size_*.yaml`.

Hypothesis (d): at equal training size, mixing timeouts adds little: the size-matched timeouts-mixed accuracy is close to (b)'s, so most of the 0.0366 gap is the training-set size.

Threshold (set before running, never moved afterwards): (d) supported if the mean over the 4 sizes (each size: the mean of its 5 seeds) of the timeouts-mixed test accuracy is less than 0.01 above (b)'s mean (0.8430). Reported in any case: the size part ((a) minus that mean) and the rest (that mean minus (b)'s), the per-timeout accuracies, and (b) at the same row count. Caveat stated with the result: (d) is scored on (a)'s test rows (all timeouts mixed), (b) on each timeout's own test rows, so the per-timeout comparison is between different test rows.

Winner rule (decided 28 Oct): not applicable (Kaggle rows; no model selection).

Result: 21 runs (hypothesis, configs and code recorded before the first run), all FINISHED, data version kaggle-k1, split hash = the manifest's isnan split. The full-size run was checked first: test accuracy 0.879604781654062, identical to (a)'s to all digits. Every run's training class counts equal a fresh `kaggle.subsample` with its size and seed (20 of 20), and all 21 saved models re-score to their logged accuracy. Numbers below come from `reports/phase3_tables.md` (table "E16 (d) training-size control"), generated by `scripts/results_tables.py --phase 3` from `reports/history.csv`.

Test accuracy on (a)'s 8,198 test rows (mean of 5 subsample seeds; the full-size row is one run), overall and on the test rows of each timeout, next to (b)'s forest trained on the same number of rows from one timeout (scored on that timeout's own test rows):

Training rows (timeouts mixed)	Accuracy mean (std)	15 s rows	30 s rows	60 s rows	120 s rows	(b), same row count
32,789 (all; = (a))	0.8796	0.8913	0.9038	0.8982	0.8222	
14,459	0.8504 (0.0021)	0.8643	0.8798	0.8655	0.7887	15 s: 0.8658
11,783	0.8435 (0.0028)	0.8592	0.8742	0.8558	0.7809	60 s: 0.8316
10,910	0.8396 (0.0024)	0.8559	0.8708	0.8517	0.7755	30 s: 0.8537
7,671	0.8243 (0.0038)	0.8402	0.8576	0.8372	0.7583	120 s: 0.8206

- Cut to the per-timeout training sizes, the timeouts-mixed forest scores 0.8243-0.8504, mean

0.8395: 0.0035 BELOW (b)'s 0.8430, not above it. Of the 0.0366 gap between (a) and (b), the training-set size accounts for 0.0402 ((a) minus the size-matched mean); the rest is -0.0035. Both parts are computed from unrounded accuracies (0.040150 and -0.003510, summing to the gap 0.036640), so the rounded figures differ from differences of the rounded accuracies by 0.0001. At equal size, mixing timeouts does not raise accuracy.

- So the reason (a) scores higher is that stacking the four timeout files gives 2.3-4.3 times as

many training rows, not that a flow seen in training at one timeout is recognised in test at another. Whether such near-copies exist was not measured; (d) shows they do not raise the score.

- Per timeout (different test rows, so a description, not a test): on the 30 s and 60 s test rows

the mixed forest at the matching size scores above (b) (0.8708 vs 0.8537, 0.8558 vs 0.8316), on the 15 s rows about the same (0.8643 vs 0.8658), and on the 120 s rows clearly below (0.7583 vs 0.8206). The 120 s rows are the hardest in every mixed run (0.8222 even with all rows): a model trained mostly on rows of other timeouts does worse on them.

- Accuracy rises steadily with training rows in the mixed setup too (0.8243 at 7,671 to 0.8796

at 32,789), as in E13.

Storage: the 21 runs take about 334 MiB (Random Forest of 200 trees; E16 forest runs together 453.6 MiB, table "Storage of the Phase 3 runs"); whole artifact store 2,645.3 MiB, mlflow.db 35.8 MiB, far below the plan's 20 GB line.

Verdict (d): supported (size-matched timeouts-mixed mean 0.8395 is 0.0035 below (b)'s 0.8430, i.e. less than 0.01 above it; the full-size run reproduces (a) exactly). The verdicts of (a), (b) and (c) are unchanged (their thresholds concern the scores, not the cause).

Correction (supersedes the cause given in the result and decision above): the published 88.01% is reproducible, and with the timeouts kept apart the same method scores 0.82-0.87 (mean 0.8430, 0.0366 lower). That drop is explained by training-set size: stacking the four timeout files gives each model 2.3-4.3 times as many training rows. Mixing timeouts in itself does not raise the score. What remains true: their protocol mixes timeouts, so its test rows mix four flow-building settings and their duplicates are mostly cross-timeout copies (E00); a score from it is not comparable with a single-timeout score such as ours. Our tuned XGBoost still beats their forest at every timeout under (b) (part (c)).

MLflow runs: E16-size-full, E16-size-<training rows>-s<seed>

E15 — Confidence calibration of the headline models, on validation (2026-10-31, utsav-s-r)

Purpose (plan, Phase 4; rules fixed in the E12 addendum item 3, 31 Oct): `predict.py` and the demo show a confidence for every prediction, so check on the validation rows, before the test set is opened, whether that confidence can be trusted, and decide per task whether an isotonic calibrator is used for the reported confidence. E15 never changes the predicted class, so it cannot change any test score of E12 except the test ECE.

What is measured (per task, the headline config `configs/final/<task>.yaml`, fitted on the training rows only, which reproduces its source run):

- ECE: confidence = the probability the model gives its predicted class; 15 equal-width bins on

[0, 1]; ECE = sum over bins of (rows in bin / all rows) x |accuracy in bin - mean confidence in bin| (`src/blindspy/calibration.py`); plus a reliability diagram (uncalibrated and isotonic).

- Calibrator: `CalibratedClassifierCV(method="isotonic", cv=5, ensemble=False)` fitted on the

training rows only (base model fitted once on all training rows; isotonic mapping from 5-fold out-of-fold predictions; no validation label is used to fit it). No other method is tried.

- Calibrated confidence = the calibrated probability of the UNCALIBRATED model's predicted class

(`calibration.predict_with_confidence`); the calibrated argmax is never used as the prediction. Logged as a check: how often the base model inside the calibrator predicts the same class as the plain fit (must be every row), and how often the calibrated argmax would differ (information only).

- Adoption rule (fixed 31 Oct): calibration is adopted for a task if the uncalibrated validation

ECE is above 0.05 AND the isotonic validation ECE is lower. The decision per task is written below and copied into the E12 configs before the test set is opened.

Hypothesis: gradient-boosted trees with hundreds of trees fitted to the training rows are somewhat overconfident on new rows, and the two A2 headline models were trained with class weights (A2-VPN) or random oversampling (A2-NonVPN), which moves probability mass toward small classes, so they are expected to be the least calibrated. Isotonic calibration, learned from out-of-fold predictions, should lower the ECE wherever it is clearly above zero.

Threshold (set before running, never moved afterwards):

- (a) the uncalibrated validation ECE is above 0.05 in at least 2 of the 4 tasks;
- (b) the isotonic validation ECE is lower than the uncalibrated one in all 4 tasks;
- (c) calibration never changes the predicted class: in all 4 tasks the base model inside the

calibrator predicts the same class as the plain fit on 100% of validation rows, and the plain fit's validation macro F1 equals the source run's (`source_val_f1_macro` in `configs/final/<task>.yaml`) to all digits. Each part gets its own verdict. The adoption decision follows the fixed rule above, not (a)/(b).

Winner rule (decided 28 Oct): not applicable; E15 changes no model, setting or prediction.

Change vs the headline model: none to the model; one isotonic calibrator added on top for the reported confidence. Configs `configs/experiments/E15_<task>_15s.yaml` (kind "calibration", `scripts/make_configs.py`), data v1, 15 s. Every run saves the calibrated model (compression level 3), the reliability table and diagram, and the validation predictions with both confidences.

Validation ECE is evidence for the decision only. The final model's calibration is measured once, on the test set, in E12 (with the probabilities actually used: calibrated where adopted, else the original; the uncalibrated test ECE beside it).

Correction before the runs (31 Oct 2026; user OK): shuffled calibration folds

The calibrator bullet above says `cv=5`. In scikit-learn an integer `cv` means 5-fold `StratifiedKFold` WITHOUT shuffling, and the training rows are stored in file order (`train.index` is increasing in all 4 tasks), so each fold holds out whole blocks of the file. Found in the first run (pilot) `E15-A1-15s` (run a98da1299a5c44e19ba6fac8c4ed5813, kept in MLflow and superseded): A1 validation ECE 0.0552 uncalibrated but 0.1918 "isotonic", mean isotonic confidence 0.7391 against a validation accuracy of 0.9309. Those two A1 ECE values are the only E15 results seen before this correction.

Root cause, measured on the TRAINING rows only (`scripts/fold_order_check.py` -> `reports/fold_order_check.json`; no validation or test row enters any model): the out-of-fold predictions the isotonic map learns from are much worse with unshuffled folds than with the project's shuffled folds, so the map learns that a high confidence is wrong far more often than it is on random-split rows:

Task	OOF accuracy, unshuffled (cv=5)	OOF accuracy, shuffled (seed 42)	OOF ECE, unshuffled	OOF ECE, shuffled
A1	0.6701	0.9321	0.2935	0.0509
A2-VPN	0.7897	0.9048	0.1522	0.0616
A2-NonVPN	0.8841	0.9566	0.0956	0.0350
B	0.7486	0.8738	0.1273	0.0420

(The shuffled OOF accuracies equal the headline source runs' CV mean accuracy exactly, as they use the same folds, and are within 0.009 of their validation accuracy.)

Correction: the calibrator uses `cv=StratifiedKFold(5, shuffle=True, random_state=42)`, the same folds as every other cross-validation in the project (`train.py`), via `calibration.calibration_folds`; the logged param `calibration.method` names it. Everything else above is unchanged: isotonic only, `ensemble=False`, fitted on training rows only, the predicted class is the uncalibrated one, the adoption rule (uncalibrated validation ECE > 0.05 and isotonic ECE lower) and thresholds (a)-(c) are not moved. All 4 E15 runs, A1 included, run again from the corrected code; only those runs count.

Side finding (inference, not proof): holding out contiguous blocks of the file costs 0.07-0.26 of out-of-fold accuracy, so neighbouring rows in the files, probably flows of the same capture, resemble each other more than random rows do. A random split puts such neighbours on both sides, so random-split scores may be optimistic for traffic from new captures. This is the session-leakage risk E11 could not test (no session IDs, E00); it goes to the report's Limitations. The file order is not a session ID, so this is not a session-aware evaluation and no score is changed by it.

Result: 4 runs (the hypothesis and the fold correction were recorded before them), all FINISHED, clean, data v1, data and split hashes equal to the manifest (checked by query). The superseded pilot (run a98da1299a5c44e19ba6fac8c4ed5813, unshuffled folds) stays in MLflow; tables use the newest run per name. Table "E15 calibration of the headline models on validation" in `reports/phase4_tables.md` (generated by `scripts/results_tables.py --phase 4`), copied:

task	run	headline	val accuracy	mean confidence, uncalibrated	mean confidence, isotonic	ECE uncalibrated	ECE isotonic	adopted	same predicted class (share of rows)	calibrated argmax would differ (share, unused)	val macro F1 = source run
A1	E15-A1-15s	E17-lgbm-A1-15s	0.9309	0.9861	0.9356	0.0552	0.014	yes	1.0	0.0007	yes

task	run	headline	val accuracy	mean confidence, uncalibrated	mean confidence, isotonic	ECE uncalibrated	ECE isotonic	adopted	same predicted class (share of rows)	calibrated argmax would differ (share, unused)	val macro F1 = source run
A2-VPN	E15-A2-VPN-15s	E08-lgbm_class_weight-A2-VPN-15s	0.8984	0.9692	0.9004	0.0714	0.0168	yes	1.0	0.0078	yes
A2-NonVPN	E15-A2-NonVPN-15s	E08-lgbm_oversample-A2-NonVPN-15s	0.9479	0.991	0.9469	0.0444	0.0152	no	1.0	0.0084	yes
B	E15-B-15s	E07-xgb-B-15s	0.8794	0.9144	0.8787	0.035	0.0099	no	1.0	0.0077	yes

Reliability diagrams: `reports/figures/e15_reliability.png` (from each run's `calibration/reliability_bins.csv`).

- All four headline models are overconfident on validation: mean confidence 0.9861 against

accuracy 0.9309 (A1), 0.9692 against 0.8984 (A2-VPN), 0.9910 against 0.9479 (A2-NonVPN), 0.9144 against 0.8794 (B). The mean isotonic confidence is within 0.005 of the accuracy in every task.

- The hypothesis that the two A2 models (class weights, oversampling) are the least calibrated holds

only for A2-VPN (ECE 0.0714, the highest); A2-NonVPN (0.0444) is below A1 (0.0552).

- A1's uncalibrated ECE (0.0552) is close to the 0.05 line; the rule is applied as written.
- The argmax of the calibrated probabilities would differ from the predicted class on 0.07-0.84%

of validation rows; it is never used, so these rows keep their uncalibrated class.

- The curves zig-zag in the low-confidence bins because few rows fall there; most rows sit in the

top bin (row counts per bin: each run's `calibration/reliability_bins.csv`).

Verdict:

- (a) supported: uncalibrated validation ECE above 0.05 in 2 of 4 tasks (A1 0.0552, A2-VPN 0.0714;

needed 2).

- (b) supported: the isotonic ECE is lower in all 4 tasks (0.0140, 0.0168, 0.0152, 0.0099).
- (c) supported: in all 4 tasks the calibrator's base model predicts the same class as the plain

fit on 100% of validation rows, and the validation macro F1 equals the source run's exactly. (Generated: `results.e15_verdicts`, judged on the unrounded logged values.)

Decision (fixed rule: uncalibrated validation ECE above 0.05 AND isotonic lower), recorded before the test set is opened:

- A1: calibration ADOPTED (0.0552 -> 0.0140).
- A2-VPN: calibration ADOPTED (0.0714 -> 0.0168).
- A2-NonVPN: NOT adopted (uncalibrated 0.0444 is not above 0.05), although isotonic would be lower.
- B: NOT adopted (uncalibrated 0.0350 is not above 0.05), although isotonic would be lower.

The rule is not changed after seeing these numbers. Consequences for E12 and predict.py: the A1 and A2-VPN headline configs get an isotonic calibrator refitted on train + validation (same shuffled folds, seed 42); their reported confidence is the calibrated probability of the uncalibrated predicted class. A2-NonVPN and B report the model's own probability. No predicted class or test macro F1 changes. The test ECE is reported once in E12 for all 4 headline models, with the probabilities actually used and the uncalibrated ECE beside it. The validation ECE above decided adoption, so it is not an independent measure of the calibrated models.

MLflow runs: E15-A1-15s, E15-A2-VPN-15s, E15-A2-NonVPN-15s, E15-B-15s (+ superseded pilot E15-A1-15s, run a98da1299a5c44e19ba6fac8c4ed5813)

E14 — Split-seed stability of the headline models (2026-10-28, utsav-s-r)

Purpose (plan, analysis only): how much the headline model's validation score depends on which flows happened to land in the training and validation sets. Isnan et al. call a model stable when the gap between cross-validation and held-out scores is under 1 percentage point with a standard deviation under 1 point.

Hypothesis: the scores barely move with the split, because each task has thousands of flows and the CV spread of the headline models is small (0.0028-0.0058, table "Current best at 15 s after E17" in `reports/phase3_tables.md`).

Threshold (set before running, never moved afterwards), validation macro F1 at 15 s over the 10 resplits:

- (a) in at least 3 of the 4 tasks the standard deviation is below 0.01;
- (b) in at least 3 of the 4 tasks the mean CV macro F1 minus the mean validation macro F1 is

within 0.01 (absolute);

- (c) in all 4 tasks the official split's validation macro F1 (the headline run) lies inside the

range of the 10 resplits. Each part gets its own verdict.

Winner rule (decided 28 Oct): E14 is analysis only and never changes the final configs, so it starts no reruns; a run above the current best by 0.03 or more (the winner margin, set above the largest Phase 2 CV standard deviation, 0.025) would be reported to the user, not acted on.

Change vs the headline model (one change: the split of train + validation). Each task keeps the headline model's type, every setting, its balancing and the model seed 42 (`configs/final/<task>.yaml`, named in `builds_on`). The training and validation rows of the official split are pooled and split again, stratified, with split seeds 42-51, keeping both sizes (`splits.resplit_train_val`). The test rows of the official seed-42 split are never pooled, resplit or looked at. Seed 42 here is a resplit of train + validation, not the official 70/15/15 split, so its score will not equal the official validation score. 10 seeds x 4 tasks = 40 runs, each with 5-fold CV on its own training rows. Reported: mean +- std of validation macro F1 and accuracy, min and max, mean CV and the mean CV - validation gap, and the validation confusion matrix averaged over the 10 runs (`reports/e14_mean_confusion/<task>.csv`). Logged param `split_seed`. Configs `configs/experiments/E14_seed<seed>_<task>_15s.yaml` (`scripts/make_configs.py`). Every run saves its model (compression level 3).

Result: 40 runs (hypothesis and configs recorded before the first run), all FINISHED, data v1. The first run (E14-seed42-A2-NonVPN-15s) was checked by query before the others: 6,085 training rows and the same class counts as the official split (stratified resplit), the test rows' class counts unchanged. Numbers below come from `reports/phase3_tables.md` (table "E14 split-seed stability"), generated by `scripts/results_tables.py --phase 3` from `reports/history.csv`; per-class F1 from `reports/history.csv`.

Validation scores at 15 s over 10 resplits of train + validation (seeds 42-51):

Task	macro F1 mean (std)	min - max	accuracy mean (std)	CV - val	official split
A1	0.9338 (0.0042)	0.9275 - 0.9397	0.9339 (0.0042)	-0.0032	0.9308
A2-VPN	0.8862 (0.0114)	0.8689 - 0.9020	0.9037 (0.0066)	-0.0057	0.8830
A2-NonVPN	0.9324 (0.0071)	0.9257 - 0.9468	0.9589 (0.0044)	-0.0041	0.9075
B	0.8476 (0.0084)	0.8344 - 0.8636	0.8819 (0.0074)	-0.0119	0.8428

- The split moves macro F1 by about 0.004-0.011 (std); accuracy moves less (0.004-0.007),

because the small classes, which macro F1 weights as much as the large ones, change most between splits.

- Cross-validation agrees with validation on average (mean CV - val -0.003 to -0.006; B

-0.012): by Isnan et al.'s rule (gap and std under 1 point) A1 and A2-NonVPN are stable on both counts, B on the std only (gap 0.0119), A2-VPN on the gap only (std 0.0114).

- The official split is not typical in A2-NonVPN: its validation macro F1 (0.9075) is below

all 10 resplits (0.9257-0.9468), 0.025 under their mean. The official validation rows are harder for MAIL (F1 0.827 against 0.853-0.959 on the resplits; 36 validation rows, so a few flows move it) and P2P (0.950 against 0.975-0.996). This also explains why A2-NonVPN showed the largest CV - validation gaps in E04-E17 (mean over its Phase 3 model-selection runs +0.0166, the other tasks -0.0002 to +0.0008, computed from `reports/history.csv`; headline: CV 0.9310, validation 0.9075). In the other three tasks the official score lies inside the resplit range (A1 and B in the lower half, A2-VPN near the middle).

- What this means for the choice: all A2-NonVPN candidates were compared on the same, harder

validation rows, so the comparison stays fair, but their validation scores understate typical performance by about 0.025; the test score in Phase 4 may come out higher than the validation score for that reason. Nothing is changed (analysis only).

- Mean confusion matrices over the 10 resplits: `reports/e14_mean_confusion/<task>.csv`.

Storage: the 40 E14 runs take 416.9 MiB (model files 412.6 MiB; largest run 25.9 MiB); whole store after E13 and E14 2,173.9 MiB + mlflow.db 34.3 MiB.

Verdict: (a) supported (std below 0.01 in 3 of 4 tasks; A2-VPN 0.0114 is the exception). (b) supported (|mean CV - mean val| within 0.01 in 3 of 4 tasks; B 0.0119 is the exception). (c) not supported (the official score lies inside the resplit range in 3 of 4 tasks; A2-NonVPN's 0.9075 is below the minimum, 0.9257).

Decision: no change to the final configs (analysis only, decided in the plan). Report: the validation scores are stable to about +-0.01 across splits; A2-NonVPN's official validation rows are unusually hard (by about 0.025, mostly MAIL and P2P), which the report states next to its validation score and its Phase 4 test score. E12 part 1 (written before this) is not changed.

MLflow runs: E14-seed<seed>-<task>-15s

E13 — Learning curve of the headline models (2026-10-28, utsav-s-r)

Purpose (plan, analysis only): whether more training data would still help, and how little data the headline model needs (as in Jorgensen et al., Fig. 2). It changes nothing about the final choice.

Hypothesis: macro F1 rises with the number of training rows and flattens: the last 20% of the training set adds little, because the 70% training split already holds thousands of flows per task (A1 12,626, A2-VPN 6,566, A2-NonVPN 6,085, B 12,651; `data/manifest.json`). Small subsamples should also vary more between seeds, since the smallest classes then have only a few dozen rows (MAIL: 166 training rows in A2-NonVPN and B, so about 17 at 10%).

Threshold (set before running, never moved afterwards), validation macro F1 at 15 s, mean over the 5 subsample seeds:

- (a) in all 4 tasks the full training set scores above 10% of it;
- (b) in at least 3 of the 4 tasks the gain from 80% to 100% is below 0.01;
- (c) in at least 3 of the 4 tasks the standard deviation over seeds is larger at 10% than at

80%. Each part gets its own verdict.

Winner rule (decided 28 Oct): E13 is analysis only and never changes the final configs, so it starts no reruns; a run above the current best by 0.03 or more (the winner margin, set above the largest Phase 2 CV standard deviation, 0.025) would be reported to the user, not acted on.

Change vs the headline model (one change: the training rows used). Each task keeps the headline model's type, every setting, its balancing and the model seed 42 (`configs/final/<task>.yaml`, named in `builds_on`). The training rows are a stratified sample (`splits.subsample_train`) of 10%, 20%, 40%, 60% or 80% of the training set, drawn with subsample seeds 42-46 (the plan's 5 repeats); validation rows are the same full validation set for every run, so all points are comparable; the test set is untouched. 100% is the headline run itself (not rerun; it has one value, no spread). 5 fractions x 5 seeds x 4 tasks = 100 runs; each also runs its 5-fold CV on its own sample. Logged params: `train_fraction`, `subsample_seed`, `n_train_rows`. Configs `configs/experiments/E13_f<percent>-s<seed>_<task>_15s.yaml` (`scripts/make_configs.py`). Every run saves its model (compression level 3), as for every official run.

Result: 100 runs (hypothesis and configs recorded before the first run), all FINISHED, data v1. The first run (E13-f010-s42-A2-NonVPN-15s) was checked by query before the others: 608 training rows (10% of 6,085), the headline model's settings and oversampling, validation unchanged. Numbers below come from `reports/phase3_tables.md` (table "E13 learning curve"), generated by `scripts/results_tables.py --phase 3` from `reports/history.csv`.

Validation macro F1 at 15 s, mean (std) over 5 subsample seeds; 100% is the headline run:

Training rows used	A1	A2-VPN	A2-NonVPN	B
10%	0.8724 (0.0062)	0.7793 (0.0052)	0.8176 (0.0093)	0.6969 (0.0097)
20%	0.8952 (0.0041)	0.8260 (0.0101)	0.8585 (0.0118)	0.7468 (0.0138)
40%	0.9106 (0.0066)	0.8582 (0.0041)	0.8806 (0.0057)	0.7975 (0.0036)
60%	0.9213 (0.0039)	0.8721 (0.0053)	0.8934 (0.0045)	0.8156 (0.0016)
80%	0.9270 (0.0029)	0.8804 (0.0036)	0.9005 (0.0033)	0.8297 (0.0029)
100%	0.9308	0.8830	0.9075	0.8428
training rows at 100%	12,626	6,566	6,085	12,651

- Every task gains steadily with more rows: from 10% to 100% +0.058 (A1), +0.104 (A2-VPN),

+0.090 (A2-NonVPN) and +0.146 (B). The 14-class task B needs the most data.

- The curves flatten: the last 20% adds +0.0038 (A1), +0.0026 (A2-VPN) and +0.0070

(A2-NonVPN), but still +0.0131 in B. B is the one task where more training flows would probably still help; the others are close to their limit with this model and these features.

- Small samples vary more between seeds: the std at 10% (0.0052-0.0097) is above the std at 80%

(0.0029-0.0036) in all 4 tasks; at 20% it is largest in three tasks (0.0101-0.0138). The 100% point is one run (the full training set, no subsample), so it has no spread.

- 40% of the training set (2,434-5,060 flows) already reaches within 0.020-0.045 of the full

score (A1 -0.0202, A2-VPN -0.0248, A2-NonVPN -0.0269, B -0.0453), so the models are usable with much less data than the dataset holds.

Storage: the 100 E13 runs take 548.2 MiB (model files 536.6 MiB; largest run 21.3 MiB); whole artifact store after E13 and E14 2,173.9 MiB, mlflow.db 34.3 MiB (measured from the store), far below the plan's 20 GB line.

Verdict: (a) supported (100% above 10% in 4 of 4 tasks). (b) supported (gain 80% -> 100% below 0.01 in 3 of 4 tasks; B +0.0131 is the exception). (c) supported (std at 10% above std at 80% in 4 of 4 tasks).

Decision: no change to the final configs (analysis only). For the report: more data would still help task B most; the binary and 7-class tasks are near their plateau. Next: E14.

MLflow runs: E13-f<percent>-s<seed>-<task>-15s

E12 — Final results on the test set (part 1 written 2026-10-28 in Phase 3, before the test set is opened; utsav-s-r)

Part 1 (this section) fixes, before any test score exists, which models Phase 4 scores, how, and what we expect. Part 2 (Phase 4) adds the results. Nothing in part 1 is changed after test scores are seen.

What is scored (decided 28 Oct after E10, plan item "Decided 28 Oct, after E10"; generated by `scripts/write_final_configs.py` from `reports/history.csv`; tables "Final choice per task", "Comparison set" and "What the superseded 28 Oct tie rule would pick" in `reports/phase3_tables.md`):

- Headline model per task (`configs/final/<task>.yaml`): the highest validation macro F1 of all

model-selection runs, any experiment and timeout; only fallback a saved file over 50 MB (none needed): - A1: E17 LightGBM, tuned (E17-lgbm-A1-15s), validation macro F1 0.9308; next best E06 XGBoost 0.9271; - A2-VPN: E08 LightGBM, tuned + class weights (E08-lgbm_class_weight-A2-VPN-15s), 0.8830; next best E08 LightGBM + oversampling 0.8830 (0.00001 lower); - A2-NonVPN: E08 LightGBM, tuned + random oversampling (E08-lgbm_oversample-A2-NonVPN-15s), 0.9075; next best E17 SVM 0.9058; - B: E07 XGBoost, tuned (E07-xgb-B-15s), 0.8428; next best E08 XGBoost + class weights 0.8364. All four are 15 s models (no 30, 60 or 120 s run is above them).

- Comparison set (`configs/final/compare/<task>_<type>.yaml`, 28 configs): per task the best 15 s

validation variant of each method type (decision tree, KNN, logistic regression, SVM-RBF, Random Forest, XGBoost, LightGBM). Every type had the same 50-trial tuning budget (E07, E17). Each headline model is also its type's comparison member (same source run), so it is trained and scored once and its score is used for both.

- Old-rule picks (`configs/final/old_rule/<task>.yaml`): what the superseded tie rule would pick,

computed by code (`results.old_rule_choice`): A1 and A2-VPN the E04 signed-log KNN (k = 5; 0.9048 and 0.8544), A2-NonVPN the E07 tuned KNN (0.8929, a comparison member), B the E17 tuned SVM (0.8242, a comparison member). Only the two not already in the comparison set get their own config. So Phase 4 trains and scores 30 distinct models: 28 comparison members + 2 old-rule picks.

Check before writing this (Phase 3, nothing logged): every one of the 34 configs, built exactly as written and fitted on the training rows, reproduces its source run's validation macro F1 to all digits (34 of 34).

How (Phase 4, plan): each model is retrained on train + validation with its config's settings and seed 42 and scored once on the seed-42 test set of its task at 15 s (A1 2,706 rows, A2-VPN 1,407, A2-NonVPN 1,305, B 2,712; smallest test class 36 rows; sizes from `data/manifest.json`). Macro F1 and accuracy with 1,000-resample bootstrap 95% intervals. A difference between two models of the same task is tested with a paired bootstrap on the same test rows, Holm-corrected for the number of comparisons; otherwise it is only described. Only the 4 headline models are saved to `models/final/` (kept with the project); the others stay as MLflow artifacts. The test scores never change the headline model (no choosing, swapping or re-tuning after test scores are seen).

Hypothesis: the validation scores were used for every choice, so they are slightly optimistic (the top of 29-34 candidate runs per task, all timeouts), but the cross-validation spread is small (0.003-0.006 for the headline models), so the test scores should be close to the validation scores and the order of the method types should mostly hold.

Threshold (set now, before the test set is opened, never moved afterwards):

- (a) every headline model's test macro F1 is within 0.03 of its validation macro F1;
- (b) the headline model scores above the old rule's pick on the test set in at least 3 of the 4

tasks (point estimates; the paired bootstrap reports in how many the difference is clear);

- (c) per task, the order of the 7 method types by test macro F1 agrees with their order by

validation macro F1 with Spearman correlation >= 0.8 in at least 3 of the 4 tasks. Each part gets its own verdict.

Winner rule (decided 28 Oct; a 15 s validation macro F1 at least 0.03 above the task's current best -> rerun at 30, 60 and 120 s): not applicable. E12 scores the fixed final and comparison models once on the test set; no model is chosen, rerun or changed because of it.

Result: (Phase 4)

Verdict: (Phase 4)

Decision: (Phase 4)

MLflow runs: E12-final-<task>, E12-compare-<type>-<task>, E12-old_rule-<task> (Phase 4)

Addendum to part 1 (31 Oct 2026, before the test set is opened): evaluation details

Part 1 said how the test set is used but left four details open. They are fixed here, before any test score exists; part 1 above is not edited. Where this addendum and part 1 differ (the number of bootstrap resamples), the addendum applies. No test-set score, prediction or metric existed when this was written: the MLflow store has no run with a test metric (checked 31 Oct by query: 0 runs). Only the test class counts were read, which `data/manifest.json` already records.

1. Significance tests (confirmatory). Exactly 26 paired tests, listed by run name below; everything else is described only, with intervals but no test.

- Metric: macro F1 only. Accuracy, weighted F1 and the other metrics get intervals and are

described, never tested, so no metric is picked after the fact.

- Statistic: d = macro F1 (headline) - macro F1 (other model) on the same test rows.
- Hypotheses: H0 d = 0, two-sided alternative d != 0.
- p-value from the paired bootstrap draws d*_1..d*_B (item 2):

p = min(1, 2 x min((1 + #{d* <= 0}) / (B + 1), (1 + #{d* >= 0}) / (B + 1))).

- Holm step-down over all 26 p-values at family-wise alpha = 0.05; Holm-adjusted p-values are

reported with the observed d and its 95% percentile interval.

- The 26 pairs (generated from `configs/final/` by code; neither old-rule pick is a comparison

member, so no pair is counted twice):

#	Task	Headline run	Other run (source run)
1	A1	E12-final-A1 (E17-lgbm-A1-15s)	E12-compare-decision_tree-A1 (E17-tree-A1-15s)
2	A1	E12-final-A1	E12-compare-knn-A1 (E07-knn_log_standard-A1-15s)
3	A1	E12-final-A1	E12-compare-logistic_regression-A1 (E17-logreg-A1-15s)
4	A1	E12-final-A1	E12-compare-random_forest-A1 (E05-rf-A1-15s)
5	A1	E12-final-A1	E12-compare-svm_rbf-A1 (E17-svm_rbf-A1-15s)
6	A1	E12-final-A1	E12-compare-xgboost-A1 (E06-xgb-A1-15s)
7	A1	E12-final-A1	E12-old_rule-A1 (E04-knn_log_standard-A1-15s)
8	A2-VPN	E12-final-A2-VPN (E08-lgbm_class_weight-A2-VPN-15s)	E12-compare-decision_tree-A2-VPN (E17-tree-A2-VPN-15s)
9	A2-VPN	E12-final-A2-VPN	E12-compare-knn-A2-VPN (E07-knn_log_standard-A2-VPN-15s)

#	Task	Headline run	Other run (source run)
10	A2-VPN	E12-final-A2-VPN	E12-compare-logistic_regression-A2-VPN (E17-logreg-A2-VPN-15s)
11	A2-VPN	E12-final-A2-VPN	E12-compare-random_forest-A2-VPN (E17-rf-A2-VPN-15s)
12	A2-VPN	E12-final-A2-VPN	E12-compare-svm_rbf-A2-VPN (E17-svm_rbf-A2-VPN-15s)
13	A2-VPN	E12-final-A2-VPN	E12-compare-xgboost-A2-VPN (E06-xgb-A2-VPN-15s)
14	A2-VPN	E12-final-A2-VPN	E12-old_rule-A2-VPN (E04-knn_log_standard-A2-VPN-15s)
15	A2-NonVPN	E12-final-A2-NonVPN (E08-lgbm_oversample-A2-NonVPN-15s)	E12-compare-decision_tree-A2-NonVPN (E02-tree-A2-NonVPN-15s)
16	A2-NonVPN	E12-final-A2-NonVPN	E12-compare-knn-A2-NonVPN (E07-knn_log_standard-A2-NonVPN-15s)
17	A2-NonVPN	E12-final-A2-NonVPN	E12-compare-logistic_regression-A2-NonVPN (E17-logreg-A2-NonVPN-15s)
18	A2-NonVPN	E12-final-A2-NonVPN	E12-compare-random_forest-A2-NonVPN (E05-rf-A2-NonVPN-15s)
19	A2-NonVPN	E12-final-A2-NonVPN	E12-compare-svm_rbf-A2-NonVPN (E17-svm_rbf-A2-NonVPN-15s)
20	A2-NonVPN	E12-final-A2-NonVPN	E12-compare-xgboost-A2-NonVPN (E07-xgb-A2-NonVPN-15s)
21	B	E12-final-B (E07-xgb-B-15s)	E12-compare-decision_tree-B (E02-tree-B-15s)
22	B	E12-final-B	E12-compare-knn-B (E07-knn_log_standard-B-15s)
23	B	E12-final-B	E12-compare-lightgbm-B (E07-lgbm-B-15s)
24	B	E12-final-B	E12-compare-logistic_regression-B (E17-logreg-B-15s)
25	B	E12-final-B	E12-compare-random_forest-B (E17-rf-B-15s)
26	B	E12-final-B	E12-compare-svm_rbf-B (E17-svm_rbf-B-15s)

2. Bootstrap: 10,000 stratified paired resamples per task, seed 42 (replaces part 1's 1,000).

- Stratified: each resample draws, within each true class, as many test rows as that class has,

with replacement, so every resample keeps the test set's class counts (the test split is itself stratified; smallest test class 36 rows, MAIL in A2-NonVPN and B). The intervals are therefore conditional on the observed class counts; this is stated with the results.

- Paired: one set of resample indices per task (numpy `default_rng(42)`), used for every model of

that task, for the intervals and for the tests.

- Intervals: 95% percentile intervals (2.5th and 97.5th percentiles) for macro F1, accuracy and

each difference d.

- Why 10,000 and not 1,000: with B resamples the smallest possible two-sided p is 2 / (B + 1).

For B = 1,000 that is 0.0020, above Holm's first threshold 0.05 / 26 = 0.0019, so no test could ever reject. For B = 10,000 it is 0.0002.

- All intervals and tests are computed afterwards from each run's saved test predictions

(item 4), not by refitting models.

3. Calibration (E15) is settled on validation before the test set is opened.

- Data: the 4 headline configs fitted on the training rows only (they reproduce the saved source

runs exactly); ECE measured on the validation rows.

- ECE: confidence = the probability the model gives its predicted class; 15 equal-width bins on

[0, 1]; ECE = sum over bins of (rows in bin / all rows) x |accuracy in bin - mean confidence in bin|; plus a reliability diagram.

- Calibrator: only isotonic, `CalibratedClassifierCV(method="isotonic", cv=5, ensemble=False)`,

fitted on the training rows. `ensemble=False` fits the base model once on all the rows it is given and learns the isotonic mapping from 5-fold out-of-fold predictions, so the base model is the same single model as the uncalibrated one, and no validation label is used to fit the mapping. No other calibration method is tried, on validation or test.

- Adoption rule per task: calibration is adopted if the uncalibrated validation ECE is above 0.05

AND the calibrated validation ECE is lower than the uncalibrated one. Decided per task and recorded in E15.md before the test set is opened.

- Calibration changes only the reported confidence, never the predicted class: the predicted

class is always the uncalibrated model's argmax, and the confidence reported (predict.py, demo) is the calibrated probability of THAT class. Multiclass isotonic calibration can reorder class probabilities, so the argmax of the calibrated probabilities is never used as the prediction. Test macro F1, accuracy and all tests above are therefore unaffected by calibration.

- Final model: where adopted, the same calibrator is fitted on train + validation (base model =

the train + validation retrain, the mapping from 5-fold out-of-fold predictions). The validation ECE is evidence for the decision only, not an independent measure of the final model's calibration; the test ECE (computed with the probabilities actually used: calibrated where adopted, else the original) is the one independent measure, reported once, with the uncalibrated test ECE beside it for reference.

4. Rehearsal before the official test runs, and crash rules.

- Rehearsal: the new run kind for the final runs is first run in a scratch clone with its own

MLflow store (runner name other than utsav-s-r), with each model fitted on the training rows and scored on the validation rows in place of the test rows; a unit test shows that this mode never predicts on test rows. The rehearsal output must show: all 30 distinct models (the 4 headline models among them), every logged field of the tracking table, each run's saved prediction file, the intervals for every model, all 26 tests with raw and Holm-adjusted p, data and split hashes equal to the manifest, and the code version and lock-file hash.

- Each official run saves its test predictions (row id, true label, predicted label, class

probabilities) as an MLflow artifact. The official runs go in one batch, from saved, shared code, after the rehearsal passes.

- Crash before a run's test predictions are written: rerun that run with the identical saved config and code (deterministic, seed 42).
- Crash after predictions or metrics were written: do not refit; compute what is missing from

the saved predictions.

- Any rerun after a test score was seen: allowed only as a technical rerun with identical code,

data and config, documented here in part 2 with the reason; never a changed model or setting.

- Every attempt, failed or not, stays in MLflow and is listed in part 2.

Correction to the addendum (31 Oct 2026, before the test set is opened; user OK)

Item 3 says `CalibratedClassifierCV(method="isotonic", cv=5, ensemble=False)`. In scikit-learn `cv=5` means unshuffled folds; the training rows are in file order, so the out-of-fold predictions the isotonic map is learned from were far worse than on random-split rows (A1 out-of-fold accuracy 0.6701 unshuffled vs 0.9321 shuffled; all 4 tasks in `reports/fold_order_check.json`, training rows only). Found in the E15 pilot run on validation; details in E15.md ("Correction before the runs"). The calibrator now uses `cv=StratifiedKFold(5, shuffle=True, random_state=42)`, the project's folds. Nothing else in the addendum changes (isotonic only, ensemble=False, training rows only, adoption rule, predicted class always the uncalibrated one). Still no test-set score, prediction or metric exists.

Part 2: results (Phase 4, 31 Oct 2026; the test set opened once)

Runs: 30 official runs (`configs/experiments/E12_*.yaml`, kind "final"), one batch, 01:42-02:00 IST on 31 Oct; clean, data v1, data and split hashes equal to the manifest, one lock-file hash. Attempts: exactly 30 E12 runs exist in the store; nothing crashed, nothing was rerun. Each model was fitted on train + validation (5-fold CV on the same rows for the CV column) and predicted the seed-42 test rows of its task at 15 s once; every prediction file holds exactly the task's test rows with their true labels (checked by query). Before the batch, the rehearsal (scratch copy of the code, runner "rehearsal", fit on train and scored on validation, no test row used) passed all 21 checks, including every model reproducing its source run's validation macro F1 exactly: `reports/e12_rehearsal_check.txt` (`scripts/check_rehearsal.py`). The training code and configs of the official runs equal those of the rehearsal.

All numbers below are copied from `reports/phase4_tables.md` (`scripts/results_tables.py --phase 4`, from each run's saved test predictions: 10,000 stratified paired resamples per task, seed 42; 95% percentile intervals, conditional on the test class counts). The table of all 30 models and the per-class tables are in that file.

Headline models on the test rows:

task	role	method type	run	source run	val macro F1	test macro F1	95% interval	test - val	test accuracy	accuracy 95% interval	test weighted F1	test balanced accuracy	CV macro F1 (fit rows)	CV - test	calibration
A1	headline	LightGBM	E12-final-A1	E17-lgbm-A1-15s	0.9308	0.9434	0.9348-0.9519	0.0126	0.9435	0.9350-0.9520	0.9435	0.9433	0.9322	-0.0111	isotonic
A2-VPN	headline	LightGBM	E12-final-A2-VPN	E08-lgbm_class_weight-A2-VPN-15s	0.883	0.8652	0.8431-0.8866	-0.0178	0.8955	0.8799-0.9112	0.896	0.8682	0.8827	0.0175	isotonic
A2-NonVPN	headline	LightGBM	E12-final-A2-NonVPN	E08-lgbm_oversample-A2-NonVPN-15s	0.9075	0.9403	0.9216-0.9573	0.0328	0.9609	0.9510-0.9709	0.9607	0.9343	0.9347	-0.0055	none
B	headline	XGBoost	E12-final-B	E07-xgb-B-15s	0.8428	0.848	0.8305-0.8651	0.0052	0.8798	0.8684-0.8916	0.8797	0.8439	0.8473	-0.0007	none

The 26 paired tests on macro F1 (fixed in the addendum; Holm over all 26 at 0.05):

task	headline	other	other role	other type	d (headline - other)	d 95% interval	p	Holm p	different (Holm, 0.05)
A1	E12-final-A1	E12-compare-knn-A1	comparison	KNN	0.0226	0.0140-0.0314	0.0002	0.0052	yes
A1	E12-final-A1	E12-compare-random_forest-A1	comparison	Random Forest	0.0145	0.0071-0.0219	0.0004	0.0064	yes
A1	E12-final-A1	E12-compare-svm_rbf-A1	comparison	SVM-RBF	0.0122	0.0038-0.0207	0.0042	0.0504	no
A1	E12-final-A1	E12-compare-xgboost-A1	comparison	XGBoost	0.013	0.0056-0.0204	0.0004	0.0064	yes
A1	E12-final-A1	E12-compare-decision_tree-A1	comparison	decision tree	0.0452	0.0352-0.0559	0.0002	0.0052	yes
A1	E12-final-A1	E12-compare-logistic_regression-A1	comparison	logistic regression	0.263	0.2446-0.2817	0.0002	0.0052	yes
A1	E12-final-A1	E12-old_rule-A1	old_rule	KNN	0.0322	0.0226-0.0422	0.0002	0.0052	yes
A2-VPN	E12-final-A2-VPN	E12-compare-knn-A2-VPN	comparison	KNN	0.0221	0.0052-0.0390	0.0132	0.1320	no

task	headline	other	other role	other type	d (headline - other)	d 95% interval	p	Holm p	different (Holm, 0.05)
A2-VPN	E12-final-A2-VPN	E12-compare-random_forest-A2-VPN	comparison	Random Forest	0.0058	-0.0117-0.0234	0.5123	1.0000	no
A2-VPN	E12-final-A2-VPN	E12-compare-svm_rbf-A2-VPN	comparison	SVM-RBF	-0.0018	-0.0224-0.0186	0.8525	1.0000	no
A2-VPN	E12-final-A2-VPN	E12-compare-xgboost-A2-VPN	comparison	XGBoost	0.0075	-0.0062-0.0214	0.2918	1.0000	no
A2-VPN	E12-final-A2-VPN	E12-compare-decision_tree-A2-VPN	comparison	decision tree	0.0345	0.0115-0.0577	0.0042	0.0504	no
A2-VPN	E12-final-A2-VPN	E12-compare-logistic_regression-A2-VPN	comparison	logistic regression	0.1473	0.1173-0.1777	0.0002	0.0052	yes
A2-VPN	E12-final-A2-VPN	E12-old_rule-A2-VPN	old_rule	KNN	0.0211	-0.0005-0.0426	0.0566	0.4752	no
A2-NonVPN	E12-final-A2-NonVPN	E12-compare-knn-A2-NonVPN	comparison	KNN	0.0242	0.0106-0.0387	0.0004	0.0064	yes
A2-NonVPN	E12-final-A2-NonVPN	E12-compare-random_forest-A2-NonVPN	comparison	Random Forest	0.0336	0.0181-0.0506	0.0002	0.0052	yes
A2-NonVPN	E12-final-A2-NonVPN	E12-compare-svm_rbf-A2-NonVPN	comparison	SVM-RBF	0.0158	-0.0001-0.0327	0.0528	0.4752	no
A2-NonVPN	E12-final-A2-NonVPN	E12-compare-xgboost-A2-NonVPN	comparison	XGBoost	0.0057	-0.0059-0.0181	0.3406	1.0000	no
A2-NonVPN	E12-final-A2-NonVPN	E12-compare-decision_tree-A2-NonVPN	comparison	decision tree	0.0578	0.0402-0.0763	0.0002	0.0052	yes
A2-NonVPN	E12-final-A2-NonVPN	E12-compare-logistic_regression-A2-NonVPN	comparison	logistic regression	0.135	0.1033-0.1687	0.0002	0.0052	yes
B	E12-final-B	E12-compare-knn-B	comparison	KNN	0.0227	0.0090-0.0364	0.0020	0.0260	yes
B	E12-final-B	E12-compare-lightgbm-B	comparison	LightGBM	-0.0087	-0.0197-0.0028	0.1368	0.9575	no
B	E12-final-B	E12-compare-random_forest-B	comparison	Random Forest	0.0077	-0.0040-0.0196	0.1952	1.0000	no
B	E12-final-B	E12-compare-svm_rbf-B	comparison	SVM-RBF	0.0069	-0.0093-0.0235	0.4078	1.0000	no
B	E12-final-B	E12-compare-decision_tree-B	comparison	decision tree	0.0594	0.0435-0.0762	0.0002	0.0052	yes
B	E12-final-B	E12-compare-logistic_regression-B	comparison	logistic regression	0.2391	0.2147-0.2648	0.0002	0.0052	yes

Headline models, described only (calibration measured once on the test rows, as fixed in E15):

task	run	calibration	test ECE (probabilities used)	test ECE uncalibrated	mean confidence	test accuracy	weighted - macro F1 gap	best class F1	worst class F1	best class recall	worst class recall	top-5 confusion pairs (true -> predicted: flows)	model file MiB	train time s	predict ms per 1,000 flows
A1	E12-final-A1	isotonic	0.009	0.0442	0.9376	0.9435	0.0001	VPN 0.9456	Non-VPN 0.9411	VPN 0.9487	Non-VPN 0.9378	Non-VPN -> VPN: 81; VPN -> Non-VPN: 72	7.63	4.31	192.9
A2-VPN	E12-final-A2-VPN	isotonic	0.0169	0.0709	0.889	0.8955	0.0308	VPN-VOIP 0.9941	VPN-MAIL 0.7132	VPN-VOIP 0.9882	VPN-MAIL 0.7077	VPN-FT -> VPN-CHAT: 19; VPN-BROWSING -> VPN-CHAT: 16; VPN-CHAT -> VPN-FT: 15; VPN-CHAT -> VPN-BROWSING: 11; VPN-FT -> VPN-BROWSING: 10	28.14	14.79	1372.9
A2-NonVPN	E12-final-A2-NonVPN	none	0.0339	0.0339	0.9942	0.9609	0.0204	VOIP 0.9964	STREAMING 0.8955	VOIP 0.9929	STREAMING 0.8824	CHAT -> BROWSING: 14; STREAMING -> BROWSING: 7; BROWSING -> CHAT: 4; FT -> BROWSING: 4; FT -> STREAMING: 4	4.92	6.24	325.6
B	E12-final-B	none	0.042	0.042	0.9213	0.8798	0.0317	VOIP 0.9917	VPN-MAIL 0.7259	VOIP 0.9905	VPN-P2P 0.7000	BROWSING -> VPN-BROWSING: 28; VPN-BROWSING -> BROWSING: 17; VPN-CHAT -> VPN-FT: 16; VPN-BROWSING -> VPN-CHAT: 13; VPN-CHAT -> VPN-BROWSING: 13	4.6	10.56	58.8

- Headline test macro F1: A1 0.9434, A2-VPN 0.8652, A2-NonVPN 0.9403, B 0.8480. Three of four are above their validation scores; A2-VPN is 0.0178 below.
- A2-NonVPN is 0.0328 above its validation score, just over the 0.03 line of (a). E14 had already

found its official validation rows unusually hard: the official validation macro F1 (0.9075) lay below all 10 resplits of train + validation (0.9257-0.9468); the test score lies inside that range.

- 14 of the 26 differences are clear after Holm (A1 6 of 7, A2-VPN 1 of 7, A2-NonVPN 4 of 6, B 3 of

6). The headline model is clearly better than logistic regression in every task and than the decision tree in A1, A2-NonVPN and B (A2-VPN: Holm p 0.0504). It is not clearly different from the SVM in any task, nor from XGBoost or LightGBM except A1 against XGBoost.

- Two comparison members scored higher than the headline model on the test rows, neither clearly:

the A2-VPN SVM (0.8670 vs 0.8652, d -0.0018, Holm p 1.0) and the B LightGBM (0.8567 vs 0.8480, d -0.0087, Holm p 0.9575). Nothing was swapped (part 1: the headline model is fixed).

- Old-rule picks: the headline model scores higher in all 4 tasks; clearly so (Holm) in A1

(d 0.0322, against the E04 KNN) and A2-NonVPN (d 0.0242, against the E07 KNN), not in A2-VPN (d 0.0211, Holm p 0.4752) or B (d 0.0069 against the E17 SVM, Holm p 1.0).

- Calibration on the test rows: the isotonic calibrators adopted in E15 lower the test ECE from

0.0442 to 0.0090 (A1) and from 0.0709 to 0.0169 (A2-VPN); the uncalibrated A2-NonVPN and B models have 0.0339 and 0.0420.

- Weakest classes: VPN-MAIL in A2-VPN and B (F1 0.7132 and 0.7259), STREAMING in A2-NonVPN

(0.8955); VPN-P2P has B's lowest recall (0.7000). The largest confusion in B is between BROWSING and VPN-BROWSING (28 + 17 flows).

Verdict:

- (a) NOT supported: test minus validation macro F1 is 0.0126 (A1), -0.0178 (A2-VPN), 0.0328

(A2-NonVPN) and 0.0052 (B); A2-NonVPN is outside 0.03 (the test score is higher).

- (b) supported: the headline model scores above the old rule's pick in 4 of 4 tasks (needed 3);

the paired test finds the difference clear in 2 (A1, A2-NonVPN).

- (c) supported: Spearman correlation between the validation and test order of the 7 method types

0.9643 (A1), 0.8214 (A2-VPN), 0.9286 (A2-NonVPN), 0.9643 (B); at least 0.8 in 4 of 4 (needed 3). (Generated: `results.e12_verdicts`, judged on unrounded values.)

Decision: the 4 headline models stay as fixed before the test set was opened; their train + validation models are saved in `models/final/<task>.joblib` with `<task>.json` (`scripts/save_final_models.py`; each reproduces its run's test predictions exactly; files 4.60-28.14 MiB, all under 50 MB). The failed part (a) is reported as it is: the test scores are not changed, and no model, setting or threshold is changed because of them. The comparison results are comparison only. No further test-set scoring happens (team rule).

MLflow runs: E12-final-<task> (4), E12-compare-<type>-<task> (24), E12-old_rule-<task> (2): 30 runs.

Recheck note (31 Oct 2026, after part 2; nothing changed)

An independent recomputation (plain scikit-learn on the saved prediction files, own bootstrap and Holm code) reproduced every test score, interval, p-value, Holm p and verdict above exactly when it draws the resamples in the same order as `results.stratified_indices` (0 mismatches in 56 values). With a different draw order it called 16 of the 26 tests clear instead of 14. The cause is Monte Carlo error, not a coding fault: two tests sit on the Holm line, and 10,000 resamples cannot settle them. The generated table "E12 Monte Carlo error of the 26

p-values" in `reports/phase4_tables.md` (`results.e12_mc_error`, from the same draws; band = 2 x tail share +- 2 standard errors, times the Holm rank multiplier) marks exactly these two as not settled:

task	other	smaller tail count	p	Holm p	Holm p band (+-2 SE)	different (Holm, 0.05)	settled by B resamples
A1	E12-compare-svm_rbf-A1	20	0.0042	0.0504	0.0284-0.0724	no	no
A2-VPN	E12-compare-decision_tree-A2-VPN	20	0.0042	0.0504	0.0261-0.0663	no	no

- The fixed result stands: under the rule fixed in the addendum both are not clear (Holm p 0.0504),

so 14 of 26 remain clear. Their bands (about 0.03-0.07) show that this yes/no is not settled by the resampling: the honest reading is "borderline", and the report says so.

- No verdict depends on them: (a)-(c) use point estimates, and the clear old-rule differences in (b)

(A1, A2-NonVPN) have Holm bands below 0.016. The other 24 tests are settled either way.

- No other seed or resample count was used for any reported number.

E11 — Session-aware split (does not run) (2026-10-28, utsav-s-r)

Planned test (plan, Phase 3): rerun the best models with a group split (GroupKFold) that keeps all flows of one capture session on the same side, and report the drop in score. Flows of one session share timing patterns, so a random split can put near-identical flows in train and test and make scores look better than they would on a new session.

Why it does not run: E00 found no session information in the data. The ARFF files hold only the 23 timing features and the label: no flow IDs, IP addresses, ports, timestamps or capture-file names (`data/manifest.json`: session_information false, reason "no IDs, IPs, ports, timestamps or capture-file names"). Without them no session can be rebuilt, so a group split is not possible from these files. Rebuilding sessions would need the original PCAP captures (a stretch goal in the plan; only part of the non-VPN PCAPs is available).

What is done instead: exact duplicates were removed before the split, and none crosses train and test (E00; data v1); E14 measures how much the scores move with the split seed; E16(b) keeps the four timeout files apart, since the same flows appear in several of them (E00). The report lists the missing session split as a limitation (section 7): the scores are for flows from the same captures as the training flows, not for new sessions.

Hypothesis, threshold, winner rule: not applicable (no run).

MLflow runs: none.

E10 — Task B: one 14-class model vs two steps (VPN / non-VPN, then 7 classes) (2026-10-28, utsav-s-r)

Hypothesis: task B mixes two questions, "is this flow VPN?" and "which application type?". A two-step model asks them in turn: a VPN / non-VPN model routes each flow to one of two 7-class models that each see only their own group. It could help if a model trained only on the VPN question gets the group right more often than the direct model does, and if each 7-class model separates types better without the other group's classes. It cannot recover from a wrong first step: such a flow lands in the wrong group and is always wrong. The direct model (current best B after E08, E07 XGBoost tuned) already gets the group right for most flows: computed before running from its validation confusion matrix (`results.vpn_group_errors`), 180 of its 327 errors on 2,711 validation rows cross the VPN / non-VPN line (group accuracy 0.9336; 55.05% of its errors), 147 are the wrong type within the right group. Expected: the two structures score about the same.

Threshold (set before running, never moved afterwards), on the same B validation rows at 15 s, against the direct model E07-xgb-B-15s (14-class validation macro F1 0.8428, from `reports/phase3_tables.md`, table

"Current best at 15 s after E08"):

- (a) the two-step 14-class validation macro F1 is within 0.01 of the direct model (|difference|

<= 0.01);

- (b) the two-step's first step makes fewer group errors than the direct model's 180;
- (c) more than half of the two-step's errors come from a wrong first step.

Each part gets its own verdict.

Winner rule (decided 28 Oct, written here before running): a model whose 15 s validation macro F1 beats that task's current best by at least 0.03 is a winner and is rerun at 30, 60 and 120 s with its 15 s settings (nothing is re-tuned per timeout: that would be a different, 4 times costlier experiment). Why 0.03: it is above the largest Phase 2 CV standard deviation (0.025), so a smaller gain could be noise from the split; a fixed number set before running keeps the choice honest and repeatable. The current best is the highest 15 s validation macro F1 whatever the margin; the 0.03 only decides the reruns. Tie rule for the final choice: direct is simpler than two-step.

Change vs the current best (one change: the structure). Every model in the two-step uses the current best B model type and settings, E07 XGBoost tuned (505 rounds, learning rate 0.101, depth 6, row sampling 0.915, column sampling 0.895, min_child_weight 2, seed 42, one thread), copied from the logged params of E07-xgb-B-15s (`results.run_settings`, `builds_on` in the config and as a param). The direct 14-class model is that run itself (not rerun). Model `xgb_two_step` (`src/blindspy/models.py`, `TwoStep`):

- step 1: the XGBoost model on the B training rows with label VPN / non-VPN, taken from the B

label (the 7 classes starting with "VPN-" are VPN). Not the A1 split (decided 28 Oct): A1 and B hold the same flows with different splits, so an A1-trained model has seen B validation flows;

- step 2: two XGBoost 7-class models, one on the B training rows whose true label is VPN, one on

the rest. Same column layout as B (std_fiat / std_biat, E00); the A2 files are not used (they hold total_fiat / total_biat);

- prediction: the step-1 group decides which 7-class model answers. Class probabilities are the

routed model's; the other group's classes get 0, so the most likely class is exactly the routed model's prediction. In 5-fold CV all three models are trained on each training fold only. The number of errors from a wrong first step = the validation errors that cross the VPN / non-VPN line (with hard routing these are the same thing), read from the confusion matrix of each run (`results.vpn_group_errors`).

Config: `configs/experiments/E10_xgb_two_step_B_15s.yaml` (`scripts/make_configs.py`). Protocol as E04-E08: 5-fold stratified CV on train, fit on train, score on validation (14-class macro F1); test set untouched. Model saved with joblib compression level 3. Data version v1.

New model type (two-step): its run's logged fields are checked by query.

Result: 1 run (config and hypothesis recorded before it), FINISHED, data v1. Numbers below come from `reports/phase3_tables.md` (tables "E10 task B: direct vs two-step", "E10 at 15 s against the best run before it", "E10 run details"), generated by `scripts/results_tables.py --phase 3` from `reports/history.csv` and the two runs' confusion matrices; per-class F1 from `reports/history.csv`.

Run check by query: `pipeline_steps` "TwoStep", `estimator_params` with base xgb, n_jobs 1 and the copied settings, `builds_on` E07-xgb-B-15s, `class_balancing` none, compression 3, both model sizes, child RSS, all evaluation files and the model saved. Trees: 7,575 = 505 (step 1, one tree per round) + 2 x 505 x 7 (two 7-class models, one tree per class per round): the three models are all there.

B validation, 15 s	direct (E07-xgb)	two-step (E10)
macro F1	0.8428	0.8318
CV macro F1	0.8405 +- 0.0056	0.8274 +- 0.0083
accuracy	0.8794	0.8727
errors (of 2,711 rows)	327	345
group wrong (VPN / non-VPN)	180	191
right group, wrong type	147	154
group accuracy	0.9336	0.9295
share of errors from the group	0.5505	0.5536

- The two-step is 0.0110 lower in macro F1 and worse on both kinds of error: its dedicated VPN /

non-VPN model gets the group wrong 11 times more than the direct model's implicit group (191 vs 180), and its two 7-class models make 7 more type errors (154 vs 147).

- Per class, 10 of 14 F1 scores fall (most: VPN-MAIL 0.6496 -> 0.6154, CHAT 0.8230 -> 0.7951, FT

0.8333 -> 0.8065, MAIL 0.7887 -> 0.7647), 3 rise a little (VPN-CHAT +0.0095, P2P +0.0074, VPN-FT +0.0013) and VPN-STREAMING is unchanged. The worst class stays VPN-MAIL.

- Cost: the two-step is cheaper despite three models: training 6.8 s vs 9.4 s, prediction 30.7 vs

56.8 ms per 1,000 flows, 3.7 vs 4.3 MiB as saved (12.1 vs 13.0 MiB uncompressed), child-process RSS 65.5 vs 102.9 MiB. Each 7-class model builds 7 trees per round instead of 14, and each sees only its group's rows.

- No winner (-0.0110), so nothing is rerun at 30, 60 or 120 s. CV agrees with validation (CV -

val -0.0044).

Verdict: (a) not supported (two-step minus direct = -0.0110, just outside the +-0.01 band). (b) not supported (191 group errors vs 180). (c) supported (55.36% of the two-step's errors come from a wrong first step).

Decision: keep the direct 14-class model for task B (higher macro F1, and simpler by the tie rule). Splitting the decision does not help here: the 14-class model separates VPN from non-VPN at least as well as the dedicated model (180 vs 191 group errors), and the 7-class models do not make fewer type errors (154 vs 147). Current best for B stays E07 XGBoost (tuned). Next: choose the final config per task (model and timeout, from the runs so far).

MLflow runs: E10-xgb_two_step-B-15s (direct: E07-xgb-B-15s)

E09 — Feature-group ablation on the headline models (2026-10-28, utsav-s-r)

Purpose (plan: analysis only): which kinds of timing information the headline model per task needs. It changes nothing about the final choice (E09 is not a model-selection experiment).

Hypothesis: the 23 features overlap a lot (E00: 19 of 253 feature pairs in A1 and B, 27 in A2-VPN and 20 in A2-NonVPN have Spearman |rho| >= 0.9, mostly within the active / idle group), so the model can make up for most missing groups with the others: dropping any one group should cost little, while no single group alone carries as much as all 23. The only size-based feature is flowBytesPerSecond; Jorgensen et al. found almost no loss when sizes were hidden by padding, so dropping it (the timing-only set) should cost very little.

Threshold (set before running, never moved afterwards), validation macro F1 at 15 s against the headline model with all 23 features (`configs/final/<task>.yaml`; A1 0.9308, A2-VPN 0.8830, A2-NonVPN 0.9075, B 0.8428, table "Final choice per task" in `reports/phase3_tables.md`):

- (a) in all 4 tasks, every group alone scores below all 23 features;

- (b) in at least 3 of the 4 tasks, dropping any one group lowers macro F1 by at most 0.02;
- (c) in at least 3 of the 4 tasks, the timing-only set is within 0.01 of all 23 features.

Each part gets its own verdict.

Stop rule (decided 28 Oct): E09 is analysis only. If any feature set scores at least 0.03 above all 23 features in a task, stop and ask the user before doing anything with it (it would mean a feature set beats the chosen model by more than the winner margin). The winner rule (a 15 s validation macro F1 at least 0.03 above the task's current best -> rerun at 30, 60 and 120 s; 0.03 because it is above the largest Phase 2 CV standard deviation, 0.025) therefore does not start reruns here: such a result goes to the user first.

Change vs the headline model (one change: the feature set). Each run keeps the headline model's type, every setting and its balancing (copied from `configs/final/<task>.yaml`, named in `builds_on`); only the columns change. Feature groups, found by name in the data's columns (`src/blindspy/features.py`), because A2 has total_fiat / total_biat where A1 and B have std_fiat / std_biat (E00):

- fiat (forward inter-arrival times): min, max, mean, and std (A1, B) or total (A2) — 4;
- biat (backward inter-arrival times): the same 4;
- flowiat (inter-arrival times in both directions): min, max, mean, std — 4;
- active: min, mean, max, std — 4; idle: min, mean, max, std — 4;
- rates_duration: duration, flowPktsPerSecond, flowBytesPerSecond — 3.

Feature sets (13 runs per task, 52 in all): each group alone (`only_<group>`), all minus each group (`no_<group>`), and `timing_only` (all but flowBytesPerSecond, 22 features). "all" is the headline run itself, not rerun. Configs `configs/experiments/E09_<set>_<task>_15s.yaml` (`scripts/make_configs.py`); the set is logged as param `feature_set`, the kept columns as `features` and `n_features`.

Protocol as E04-E17: 5-fold stratified CV on train, fit on train, score on validation (the features are dropped from all of them alike); test set untouched. Models saved with joblib compression level 3. Data version v1.

Result: 52 runs (hypothesis and configs recorded 16 s before the first run), all FINISHED, data v1. The first run (E09-only_fiat-A2-VPN-15s) was checked by query before the others: it kept exactly total_fiat, min_fiat, max_fiat and mean_fiat (the A2 layout) with the headline model's settings and class weights. Numbers below come from `reports/phase3_tables.md` (table "E09 feature-group ablation"), generated by `scripts/results_tables.py --phase 3` from `reports/history.csv`.

Validation macro F1 at 15 s, change against all 23 features:

Feature set	A1	A2-VPN	A2-NonVPN	B
all 23 (headline)	0.9308	0.8830	0.9075	0.8428
only fiat (4)	-0.1229	-0.1237	-0.1540	-0.2028
only biat (4)	-0.1064	-0.1631	-0.1804	-0.2330
only flowiat (4)	-0.0800	-0.0969	-0.0910	-0.1712
only active (4)	-0.3819	-0.5578	-0.5720	-0.6290
only idle (4)	-0.3803	-0.5763	-0.6037	-0.6345
only rates + duration (3)	-0.0915	-0.1040	-0.0604	-0.1385
no fiat (19)	-0.0052	-0.0179	-0.0051	-0.0184
no biat (19)	-0.0155	-0.0105	-0.0082	-0.0265
no flowiat (19)	+0.0014	-0.0078	-0.0045	-0.0166
no active (19)	-0.0011	+0.0030	+0.0018	+0.0051
no idle (19)	-0.0004	-0.0004	-0.0036	+0.0000
no rates + duration (20)	-0.0122	-0.0395	-0.0301	-0.0549

Feature set	A1	A2-VPN	A2-NonVPN	B
timing only, no flowBytesPerSecond (22)	-0.0148	-0.0340	-0.0331	-0.0441

- No group alone comes close to all 23 features: the best single group loses 0.060-0.139 (flow

inter-arrival times in A1 and A2-VPN, rates + duration in A2-NonVPN and B). Active or idle times alone are nearly useless (macro F1 0.21-0.55): many flows have no active / idle spread (E04: std_active and std_idle have interquartile range 0 on the training rows of every task, so at least half of the rows share one value).

- Dropping active or idle costs nothing (-0.0036 to +0.0051 in every task): the other groups

carry the same information. (B without idle scores 0.842816 against 0.842798, so the "+0.0000" is a real but tiny difference; the tuned XGBoost does split on the idle features, the other groups make up for them.)

- The most important group is rates + duration: dropping it costs 0.030-0.055 in A2-VPN,

A2-NonVPN and B (0.012 in A1, where the backward gaps matter most, -0.0155).

- Size information matters more than expected: dropping flowBytesPerSecond alone (timing only)

costs 0.015-0.044. This differs from Jorgensen et al., who found almost no loss when sizes were hidden by padding. Their setup differs (sizes hidden by padding rather than a feature dropped), so the results are not directly comparable; on these 23 features and these models the byte rate carries information the timing features do not replace.

- Stop rule: no feature set is 0.03 or more above all 23 features (largest gain +0.0051, B

without active), so nothing goes to the user and nothing changes in the final configs (E09 is analysis only).

Storage: the 52 E09 runs save 539.0 MiB of model files (largest 27.6 MiB, the A2-VPN LightGBM without active); the whole artifact store is now 1.2 GB on disk (`du`), far below the plan's 20 GB line.

Verdict: (a) supported (in all 4 tasks every group alone is below all 23 features; best single group -0.0604 to -0.1385). (b) not supported (dropping any one group stays within 0.02 only in A1, 1 of 4 tasks; worst drops A2-VPN -0.0395, A2-NonVPN -0.0301, B -0.0549, all from rates + duration). (c) not supported (timing-only within 0.01 in 0 of 4 tasks: -0.0148 to -0.0441).

Decision: no change to the final configs (analysis only, stop rule not triggered). For the report: the models rely most on the packet and byte rates with the duration and on the gap statistics; active / idle times add nothing given the rest; the one size-based feature is worth 0.015-0.044 macro F1, so "timing only" is not free on this data. Phase 4's SHAP explanation is compared with this ablation (plan). Next: E13 learning curve.

MLflow runs: E09-<set>-<task>-15s

E08 — Class balancing ladder on the current best model (2026-10-28, utsav-s-r)

Hypothesis: three of the four tasks are imbalanced (largest to smallest class at 15 s, from `data/manifest.json`: A1 1.08:1, A2-VPN 5.79:1, A2-NonVPN 11.85:1, B 11.85:1). A model trained without balancing favours the large classes, so the small ones lose recall, and macro F1 weights every class the same. Balancing the training rows (class weights, random oversampling or SMOTE) should raise the F1 of the weak small classes and with it macro F1 in A2-VPN, A2-NonVPN and B. A1 is almost balanced, so balancing should change little there.

Threshold (set before running, never moved afterwards), validation macro F1 at 15 s against the current best after E07 (the "none" step; reference values from `reports/phase3_tables.md`, table "Current best at 15 s after E07": A1 0.9271, A2-VPN 0.8768, A2-NonVPN 0.9028, B 0.8428):

- (a) the best balancing step beats "none" by at least 0.01 in at least 2 of the 3 imbalanced

tasks (A2-VPN, A2-NonVPN, B);

- (b) in A1, no step changes validation macro F1 by more than 0.01 either way;
- (c) in at least 2 of the 3 imbalanced tasks, the best balancing step raises the F1 of the

class that is weakest without balancing (A2-VPN VPN-CHAT 0.7343, A2-NonVPN STREAMING 0.8095, B VPN-MAIL 0.6496, from the runs' per-class F1). Each part gets its own verdict.

Ladder and stop rule (plan: "none -> class_weight -> random oversampling -> SMOTE; stop early if a step doesn't help"), per task, written here before running: step 0 "none" is the current best run itself (not rerun). A step helps if its 15 s validation macro F1 is higher than that of every earlier step in that task (any margin, the same rule as "current best"). The next step runs only if this one helped; otherwise that task's ladder stops. Steps therefore run in rounds: round 1 = class_weight in all 4 tasks; the next round only where the previous step helped. The order is the plan's "simpler first" order (no balancing < class_weight < oversampling < SMOTE).

Winner rule (decided 28 Oct, written here before running): a model whose 15 s validation macro F1 beats that task's current best by at least 0.03 is a winner and is rerun at 30, 60 and 120 s with its 15 s settings, balancing included (nothing is re-tuned per timeout: that would be a different, 4 times costlier experiment). Why 0.03: it is above the largest Phase 2 CV standard deviation (0.025), so a smaller gain could be noise from the split; a fixed number set before running keeps the choice honest and repeatable. The current best is the highest 15 s validation macro F1 whatever the margin; the 0.03 only decides the reruns.

Change vs the current best (one change: class balancing). Each task keeps the model type and every setting of its current best after E07, copied from the logged params of that run (not typed; `results.run_settings`, used by `scripts/make_configs.py`; each config names it in `builds_on`, also logged as a param):

- A1: E06 XGBoost, defaults (E06-xgb-A1-15s);
- A2-VPN: E07 LightGBM, tuned (E07-lgbm-A2-VPN-15s);
- A2-NonVPN: E07 LightGBM, tuned (E07-lgbm-A2-NonVPN-15s);
- B: E07 XGBoost, tuned (E07-xgb-B-15s).

Check before running: each base rebuilt from the copied settings, without balancing, fitted on the training rows reproduces its logged validation macro F1 exactly (4 of 4).

The steps (`balance` in the config, `src/blindspy/models.py` `build_model`; logged as param `class_balancing`):

- class_weight: `class_weight="balanced"` (weight of a class = rows / (classes x rows of that

class)). LightGBM has it natively; XGBoost has no class_weight, so the XGBoost wrapper turns it into per-row sample weights with `compute_sample_weight("balanced")` (decided 28 Oct).

- oversample: `RandomOverSampler(random_state=42)` repeats rows of the smaller classes until every

class has as many rows as the largest.

- smote: `SMOTE(k_neighbors=5, random_state=42)` creates new rows between a row and one of its 5

nearest neighbours of the same class until every class matches the largest. k = 5 is safe: the smallest training class is 53 rows (MAIL, A2-NonVPN 30 s), about 42 per CV training fold. The two resampling steps sit right before the model in an imbalanced-learn Pipeline, which resamples only the rows it is fitted on: each CV training fold and then the training set. CV test folds, validation and test rows are never resampled or seen. Known limit, kept on purpose (one change per experiment): SMOTE finds neighbours by Euclidean distance on the raw features, which the large-valued timing columns dominate; scaling first would be a second change.

Protocol as E04-E07: 5-fold stratified CV on train, fit on train, score on validation; test set untouched. Models saved with joblib compression level 3. Data version v1.

New model type (imbalanced-learn pipeline): the first resampling run's logged fields are checked by query before the rest of its round.

Result: 7 runs in 3 rounds, all FINISHED, data v1: round 1 (class_weight, 4 tasks), round 2 (oversample, A2-VPN and A2-NonVPN), round 3 (SMOTE, A2-NonVPN). Each round's configs and the previous round's results were recorded before it ran. Numbers below come from `reports/phase3_tables.md` (tables "E08 balancing ladder", "E08 at 15 s against the best run before it", "E08 run details"), generated by `scripts/results_tables.py --phase 3` from `reports/history.csv`; the ladder table also applies the stop rule.

Check before running: the 4 bases rebuilt from the copied settings reproduced their logged validation macro F1 exactly (A1 0.927067..., A2-VPN 0.876784..., A2-NonVPN 0.902764..., B 0.842798...). First resampling run (E08-lgbm_oversample-A2-NonVPN-15s) checked by query: `pipeline_steps` "RandomOverSampler -> LGBMClassifier", `preprocessing` "balance: RandomOverSampler(random_state=42)", `class_balancing` oversample, `builds_on`, n_jobs 1, both model sizes, compression 3, child RSS, all evaluation files and the model saved.

Validation macro F1 at 15 s per step ("none" = the current best after E07):

Task	none	class_weight	oversample	SMOTE	stopped after
A1	0.9271	0.9237	not run	not run	class_weight (did not help)
A2-VPN	0.8768	0.8830	0.8830	not run	oversample (did not help)
A2-NonVPN	0.9028	0.9049	0.9075	0.9046	SMOTE (did not help)
B	0.8428	0.8364	not run	not run	class_weight (did not help)

- A2-VPN: oversampling scored 0.882972 against 0.882982 for class weights (exact values from the

store), 0.00001 lower. By the rule set before running (any margin) it did not help and the ladder stopped; in practice the two are equal.

- Gains are small: the best step per task is +0.0062 (A2-VPN, class_weight), +0.0047 (A2-NonVPN,

oversample) and -0.0064 (B, class_weight); in A1 class weights cost 0.0034.

- The weak small classes gain a little where balancing helps: VPN-CHAT F1 0.7343 -> 0.7463

(A2-VPN), STREAMING 0.8095 -> 0.8372 (A2-NonVPN; its recall 0.761 -> 0.806). In B, class weights raise VPN-MAIL recall 0.585 -> 0.600 but lose more in precision, so its F1 falls 0.6496 -> 0.6290 and macro F1 falls too.

- No winner (largest gain over the best before E08: +0.0062), so nothing is rerun at 30, 60 or

120 s. New current best at 15 s (table "Current best at 15 s after E08"): A2-VPN E08-lgbm_class_weight 0.8830, A2-NonVPN E08-lgbm_oversample 0.9075; A1 (E06 XGBoost 0.9271) and B (E07 XGBoost 0.8428) unchanged. Both new bests are within 0.03 of the unbalanced model, so the final choice treats them as ties (tie rule: no balancing is simpler).

- Cost: class weights change training time little (A2-VPN 12.8 s, as without); resampling adds

rows, so it costs more: A2-NonVPN train 5.1 s (oversample) and 6.4 s (SMOTE) against 3.4 s (class weights). SMOTE also grows deeper trees (mean depth 8.6 against 6.3-6.6; 242,784 nodes) and predicts slowest (421 ms per 1,000 flows against 284-326). Model files 0.12-27.0 MiB as saved.

- 5-fold CV agrees with validation (|CV - val| <= 0.028; CV std 0.0034-0.0094).

Storage: the 7 E08 runs take 72.5 MiB (LightGBM 67.8, XGBoost 4.7; largest run 27.1 MiB, A2-VPN oversample). Whole store now: artifacts 580.9 MiB, mlflow.db 18.0 MiB. Far below the plan's 20 GB line.

Verdict: (a) not supported (the best step beats "none" by >= 0.01 in 0 of the 3 imbalanced tasks: A2-VPN +0.0062, A2-NonVPN +0.0047, B -0.0064). (b) supported (A1: the one step run changed macro F1 by -0.0034, within 0.01). (c) supported (the best step raises the F1 of the class that is weakest without balancing in 2 of 3 tasks: A2-VPN +0.012, A2-NonVPN +0.0277; B -0.0206).

Decision: balancing barely matters for these models (all changes within +-0.0064). That suggests the remaining errors of the weak classes (VPN-CHAT, STREAMING, VPN-MAIL) come from flows that look like those of other classes rather than from too few training rows; E08 does not test this. Current best after E08 per task: A1 E06 XGBoost (defaults), A2-VPN E08 LightGBM tuned + class weights, A2-NonVPN E08 LightGBM tuned + oversampling, B E07 XGBoost tuned. E10 (task B, direct vs two-step) uses the current best B model type and settings, E07 XGBoost tuned without balancing. Next: E10.

MLflow runs: E08-<model>_<step>-<task>-15s

E07 — Hyperparameter tuning with Optuna, same budget for every model (2026-10-28, utsav-s-r)

Hypothesis: E05 and E06 ran with default settings. Tuning the main settings (number of trees, depth, learning rate, row and column sampling, leaf size) to these data should improve each model a little, since defaults are general-purpose. The tuned KNN (number of neighbours, distance weighting, metric) gets the same budget so that no model wins only because it was tuned more (Isnan et al. list untuned baselines as a weakness). Tuning should not change the ranking: the tree ensembles stay ahead of KNN.

Threshold (set before running, never moved afterwards), validation macro F1 at 15 s; reference values from `reports/phase3_tables.md`:

- (a) in at least 8 of the 12 searches, the tuned model scores above its own untuned version

(E06 XGBoost, E05 Random Forest, E06 LightGBM, E04 signed-log KNN with k = 5);

- (b) the best E07 model per task beats the current best after E06, XGBoost with defaults

(A1 0.9271, A2-VPN 0.8758, A2-NonVPN 0.9015, B 0.8303), by at least 0.01 in at least 2 of the 4 tasks;

- (c) the tuned KNN stays below the best tuned tree ensemble in all 4 tasks.

Each part gets its own verdict.

Winner rule (decided 28 Oct, written here before running): a model whose 15 s validation macro F1 beats that task's current best by at least 0.03 is a winner and is rerun at 30, 60 and 120 s with its 15 s settings, tuned hyperparameters included (nothing is re-tuned per timeout: that would be a different, 4 times costlier experiment). Why 0.03: it is above the largest Phase 2 CV standard deviation (0.025), so a smaller gain could be noise from the split; a fixed number set before running keeps the choice honest and repeatable. The current best is the highest 15 s validation macro F1 whatever the margin; the 0.03 only decides the reruns.

Change vs the untuned runs (one change: the hyperparameters, chosen by search). Scope (decided 28 Oct; table "E07 scope" in `reports/phase3_tables.md`): per task the best 2 of Random Forest, XGBoost and LightGBM by 15 s validation macro F1, plus KNN with the better of the E03/E04 preprocessing, which is the signed log + StandardScaler in every task:

- A1: XGBoost, Random Forest, KNN;
- A2-VPN, A2-NonVPN, B: XGBoost, LightGBM, KNN.

That is 12 searches. Each search: Optuna `TPESampler(seed=42)`, 50 trials run one after another (parallel TPE trials are not reproducible), every trial scored by the mean macro F1 of 5-fold stratified CV on the training rows (the folds of one trial run in parallel, each model on one thread). Validation and test rows are never seen by the

search. The best trial's settings are then trained on all training rows and scored on validation exactly like any other run; that model is saved. Trials keep settings and scores only (no model file); they are nested MLflow runs tagged `trial=True`, protocol `optuna_trial`, and are kept out of the result tables.

Search ranges (fixed in the plan, written into `scripts/make_configs.py` and every `configs/experiments/E07_<model>_<task>_15s.yaml` before running):

- Random Forest: n_estimators 100-1000; max_depth 5-40 or no limit; min_samples_leaf 1-10;

max_features sqrt, log2 or a fraction 0.3-1.0.

- XGBoost: n_estimators 100-1000; learning_rate 0.01-0.3 (log scale); max_depth 3-12;

subsample 0.5-1.0; colsample_bytree 0.5-1.0; min_child_weight 1-50.

- LightGBM: n_estimators 100-1000; learning_rate 0.01-0.3 (log scale); num_leaves 15-255;

subsample 0.5-1.0; colsample_bytree 0.5-1.0; min_child_samples 1-50. LightGBM only uses `subsample` when `subsample_freq` > 0 (its default is 0), so its searches fix `subsample_freq=1` (sample rows anew for every tree); otherwise that range would do nothing.

- KNN (signed log + StandardScaler): n_neighbors 1-50; weights uniform or distance; metric

euclidean or manhattan. "No limit" and "fraction" are encoded as an extra categorical choice (`max_depth_limited`, `max_features_kind`), so TPE samples them like any other setting (`src/blindspy/tuning.py`).

Logged per search (on the retrained run): trials, total fits (trials x 5 folds), best trial, its CV macro F1 mean and standard deviation, search time, the search space and fixed settings, `search/trials.csv` and `search/history.png` (score per trial and best so far). Because the retrained run uses the same folds and seed, its CV mean must equal the best trial's exactly; this is checked.

New model type: the first study's runs (trials and the retrained run) are checked by query.

Storage (plan): mlruns/ is measured again after the searches with the tuned model sizes.

Result: 12 searches (hypothesis recorded 17 s before the first run): 12 retrained runs and 600 trial runs, all FINISHED, data v1. Numbers below come from `reports/phase3_tables.md` (tables "E07 searches", "E07 at 15 s against the best run before it", "E07 run details"), generated by `scripts/results_tables.py --phase 3` from `reports/history.csv`.

First study check (by query, before the other 11): E07-knn_log_standard-A1-15s logged all 50 trials as nested runs (tag `trial=True`, protocol `optuna_trial`, no model file), and the retrained run logged the search params, the `search_*` metrics, `search/trials.csv`, `search/history.png` and its model. For all 12 searches the retrained run's CV macro F1 equals the best trial's exactly (column "CV best == retrained CV": yes in 12 of 12), so the folds and seed carried through.

Search size and time: 50 trials and 250 fits per search, 3,000 fits in all. Search time 0.2-0.5 min for KNN, 1.6-7.4 min for XGBoost, 7.5-33.3 min for LightGBM and 11.6 min for the forest; 78.4 min in all (CV folds in parallel, each model on one thread).

Tuned vs untuned, validation macro F1 at 15 s:

Task	Model	untuned	tuned	change	chosen settings (short)
A1	KNN	0.9048	0.9163	+0.0115	k = 4, distance, manhattan
A1	Random Forest	0.9203	0.9181	-0.0022	181 trees, leaf 4, 43% of features
A1	XGBoost	0.9271	0.9259	-0.0011	694 rounds, lr 0.051, depth 12
A2-VPN	KNN	0.8544	0.8625	+0.0082	k = 2, distance, euclidean

Task	Model	untuned	tuned	change	chosen settings (short)
A2-VPN	LightGBM	0.8694	0.8768	+0.0074	809 rounds, lr 0.019, 125 leaves
A2-VPN	XGBoost	0.8758	0.8661	-0.0097	337 rounds, lr 0.112, depth 6
A2-NonVPN	KNN	0.8670	0.8929	+0.0260	k = 1, distance, euclidean
A2-NonVPN	LightGBM	0.8983	0.9028	+0.0045	912 rounds, lr 0.061, 35 leaves
A2-NonVPN	XGBoost	0.9015	0.9025	+0.0010	543 rounds, lr 0.092, depth 10
B	KNN	0.7644	0.8006	+0.0361	k = 2, distance, manhattan
B	LightGBM	0.8201	0.8352	+0.0151	725 rounds, lr 0.016, 153 leaves
B	XGBoost	0.8303	0.8428	+0.0125	505 rounds, lr 0.101, depth 6

(Full chosen settings, incl. row/column sampling and leaf sizes, in table "E07 searches".)

- Tuning raised the 5-fold CV score in all 12 searches (column "CV untuned -> tuned"), but the

validation score in only 9: the forest and XGBoost in A1 and XGBoost in A2-VPN dropped by 0.001-0.010. The best of 50 trials is picked by CV, so its CV score is optimistic (it is the maximum of 50 noisy estimates); validation is the honest check, and there the gains are small.

- KNN gains most (+0.008 to +0.036) and always picks distance weighting with k = 1-4: on these

data the nearest one to four flows decide (why is not tested here).

- Current best at 15 s after E07 (table "Current best at 15 s after E07"): A1 E06 XGBoost

(untuned) 0.9271, A2-VPN E07 LightGBM 0.8768, A2-NonVPN E07 LightGBM 0.9028, B E07 XGBoost 0.8428. The gains over the best before E07 are -0.0011, +0.0010, +0.0013 and +0.0125: no winner (all below 0.03), so nothing is rerun at 30, 60 or 120 s. A2-VPN and A2-NonVPN change current best by about 0.001, inside any CV spread; the final choice treats such gaps as ties (tie rule: gap < 0.03 -> simpler model, then faster predictor).

- Settings at the edge of a fixed range: A1 XGBoost max_depth = 12 (upper limit), A2-NonVPN KNN

k = 1 (lower limit), A2-VPN LightGBM min_child_samples = 49 (limit 50). The ranges were fixed in the plan before running and are not widened afterwards; a wider search is a limitation to note, not a change made after seeing results.

- 5-fold CV agrees with validation within 0.03 (column "CV - val", -0.005 to +0.030; the largest is

A2-NonVPN LightGBM, CV 0.9329 vs validation 0.9028).

- Cost: the tuned LightGBM models are large and slow to predict, because multi-class boosting

builds one tree per class per round and the search chose many rounds with many leaves. B: 10,150 trees, 2.68 million nodes, 143.4 MiB uncompressed / 62.5 MiB as saved, 3,272 ms per 1,000 flows; A2-VPN: 5,663 trees, 57.6 / 25.0 MiB, 1,209 ms per 1,000 flows. Tuned XGBoost stays small (1.0-4.3 MiB as saved, 16-57 ms per 1,000 flows), tuned KNN 0.68-1.44 MiB and 6.9-45.0 ms. Child-process RSS of a fit: up to 1,007 MiB (LightGBM, B), against 430 MiB seen by tracemalloc.

Storage (plan: measure again after the searches; table "Storage of the Phase 3 runs"): the 12 retrained E07 runs take 110.5 MiB (KNN 4.8, LightGBM 92.6, forest 3.8, XGBoost 9.2; largest run 62.7 MiB, LightGBM B). Trial runs save no model. Whole store now: artifacts 508.4 MiB, mlflow.db 17.7 MiB (932 runs). The E05 projection (about 2.9 GiB, below the plan's 20 GB line) still holds. One flag for Phase 4: the LightGBM B model file (62.5 MiB) is above the 50 MB size limit for final model files, so if it were ever chosen as a final model it could not be stored as an ordinary file; it is not the current best.

Verdict: (a) supported (tuned above untuned in 9 of 12 searches; 8 needed). (b) not supported (the best E07 model beats the best after E06 by >= 0.01 in 1 of 4 tasks, B +0.0125; A1 -0.0011, A2-VPN +0.0010, A2-NonVPN +0.0013). (c) supported (tuned KNN below the best tuned ensemble in 4 of 4 tasks, by 0.0096-0.0422).

Decision: tuning helps a little at most: the defaults of XGBoost were already close to the best these data allow, except in B. E08 builds on the current best per task after E07 (one change: class balancing): A1 E06 XGBoost (defaults), A2-VPN E07 LightGBM (tuned), A2-NonVPN E07 LightGBM (tuned), B E07 XGBoost (tuned). No winner reruns. Next: E08.

MLflow runs: E07-<model>-<task>-15s (retrained best); trials E07-<model>-<task>-15s-trial-NNN

E06 — Gradient boosting: XGBoost and LightGBM with default settings (2026-10-28, utsav-s-r)

Hypothesis: gradient-boosted trees are usually the strongest model family on tables of numbers. Each tree corrects the errors of the trees before it, which should help the classes the forest still mixes up (VPN-CHAT, STREAMING, the mail classes). With default settings they should at least match the 300-tree Random Forest (E05), and XGBoost and LightGBM, which use similar algorithms, should score close to each other.

Threshold (set before running, never moved afterwards), validation macro F1 at 15 s, against the current best after E05, the Random Forest (A1 0.9203, A2-VPN 0.8679, A2-NonVPN 0.8941, B 0.8147; `reports/phase3_tables.md`):

- (a) the better of XGBoost and LightGBM beats the forest in at least 2 of the 4 tasks (any

margin);

- (b) XGBoost and LightGBM are within 0.02 of each other in at least 3 of the 4 tasks.

Each part gets its own verdict.

Winner rule (decided 28 Oct, written here before running): a model whose 15 s validation macro F1 beats that task's current best by at least 0.03 is a winner and is rerun at 30, 60 and 120 s with its 15 s settings. Why 0.03: it is above the largest Phase 2 CV standard deviation (0.025), so a smaller gain could be noise from the split; a fixed number set before running keeps the choice honest and repeatable. The current best is the highest 15 s validation macro F1 whatever the margin; the 0.03 only decides the reruns.

What E07 takes from this: per task, the best 2 of Random Forest (E05), XGBoost and LightGBM by 15 s validation macro F1 are tuned with Optuna (decided 28 Oct), plus the signed-log KNN.

Change vs the current best (one change: the model family): model `xgb = XGBLabelled` (`src/blindspy/models.py`), XGBoost's `XGBClassifier` with every setting at its default (100 rounds, learning rate 0.3, max depth 6, histogram trees) except `n_jobs=1` and `random_state=42`; the wrapper only encodes class names to 0..k-1 for XGBoost and back. Model `lgbm = LGBMClassifier` with defaults (100 rounds, learning rate 0.1, 31 leaves) except `n_jobs=1`, `random_state=42` and `verbose=-1` (silences its log, no effect on the model). No scaling and no log (trees). One thread per model, the 5 CV folds in parallel. Configs: `configs/experiments/E06_{xgb,lgbm}_<task>_15s.yaml` (`scripts/make_configs.py`). Protocol as before; test set untouched. Models saved with compression level 3. Data version v1.

New model types: the first run of each is checked by query (all tracking-table fields, plus number of trees, mean and max depth and nodes read from the booster's own tree dump). The memory measure that matters here is the child-process RSS: XGBoost and LightGBM allocate in C++, which tracemalloc does not see.

Result: 8 runs at 15 s (all FINISHED, data v1). Numbers below come from `reports/phase3_tables.md` (generated by `scripts/results_tables.py --phase 3` from `reports/history.csv`).

New-model-type check (by query, first runs E06-xgb-A1-15s and E06-lgbm-A1-15s): every field of the tracking table was logged, with tree counts, depths and nodes. One gap: the `estimator` param is scikit-learn's repr, which is cut at 700 characters; for XGBoost it shows every default as None and the cut hid `random_state=42`. Fixed: a new param `estimator_params` holds every explicit setting as JSON (XGBoost: n_jobs 1,

random_state 42, objective), and XGBoost's resolved settings (defaults filled in, e.g. max_depth 6, eta 0.3) are saved as `model/xgboost_config.json`. All 8 runs were then made; the rerun of E06-xgb-A1-15s gave the same validation macro F1 (0.92707) as the first.

Validation macro F1 at 15 s:

Task	current best before (E05 forest)	E06 XGBoost	E06 LightGBM	gain of the better one	winner
A1	0.9203	0.9271	0.9181	+0.0067	no
A2-VPN	0.8679	0.8758	0.8694	+0.0079	no
A2-NonVPN	0.8941	0.9015	0.8983	+0.0075	no
B	0.8147	0.8303	0.8201	+0.0156	no

- XGBoost with default settings is the new current best in all 4 tasks. No model clears the

0.03 winner margin, so there are no reruns at other timeouts.

- XGBoost beats LightGBM in all 4 tasks, by 0.0032-0.0102. LightGBM is below the forest on A1

(-0.0022) and above it elsewhere.

- CV agrees with validation (|CV - validation| <= 0.024; CV std 0.0025-0.0117). On A2-NonVPN

both boosters score about 0.02 higher in CV than on validation, as the forest did (+0.011).

- Worst class by F1: VPN-CHAT in A2-VPN (0.72 both), STREAMING in A2-NonVPN (XGBoost 0.78,

LightGBM 0.82), VPN-MAIL in B (XGBoost 0.66, LightGBM 0.62). Best: VOIP / VPN-VOIP (0.99-1.00).

- Complexity: 100 boosting rounds, one tree per class per round in multi-class tasks (100 trees

for A1, 700 for A2, 1,400 for B). XGBoost trees are at most 6 deep (its default limit; mean 5.7-6.0), 7,500-66,808 nodes in all; LightGBM grows 31-leaf trees leaf-wise, mean depth 11.3-13.4 (max 30), 6,100-85,400 nodes.

- Cost: training 0.19-2.3 s (XGBoost 0.06 to 0.39 times the forest's time on the same task), prediction 1.1-11.8 ms

per 1,000 flows for XGBoost and 5.1-98.3 ms for LightGBM. Models are 0.32-4.60 MiB uncompressed and 0.12-2.01 MiB as saved, against 5-22 MiB for the forest. Memory: tracemalloc 0.4-13.9 MiB, child-process RSS 7.4-41.7 MiB.

Storage (plan: measure with compression on, per model type; table "Storage of the Phase 3 runs"): the 4 XGBoost runs take 2.5 MiB in all (at most 1.2 MiB per run, model file 1.1 MiB), the 4 LightGBM runs 4.6 MiB (at most 2.2 MiB per run, model file 2.0 MiB). Whole store after these runs: artifacts 397.9 MiB, mlflow.db 11.6 MiB (`reports/phase3_tables.md`). The largest booster file (2.0 MiB) is far below the largest forest file (22.4 MiB, E05), so the E05 projection (about 2.9 GiB, below the 20 GB line) still holds; E07 measures the tuned models again.

Verdict: (a) supported (the better booster beats the forest in 4 of 4 tasks; gains 0.0067, 0.0079, 0.0075, 0.0156). (b) supported (XGBoost and LightGBM within 0.02 in 4 of 4 tasks).

Decision: XGBoost (defaults) is the current best in all four tasks. E07 tunes, per task, the best 2 of Random Forest / XGBoost / LightGBM by 15 s validation macro F1 (table "E07 scope"): A1 XGBoost and Random Forest; A2-VPN, A2-NonVPN and B XGBoost and LightGBM. Plus signed-log KNN (standard scaling, the better E04 variant in every task). That is 12 searches of 50 trials.

MLflow runs: E06-xgb-*, E06-lgbm-*

E05 — Random Forest, 300 trees (2026-10-28, utsav-s-r)

Hypothesis: the E02 decision tree is unpruned (depth 16-35), so it fits noise in its training rows. A Random Forest averages 300 such trees, each grown on a bootstrap sample with a random subset of features at every split, which lowers that variance. It should clearly beat the single tree, and since the current best (signed-log KNN, E04) is only 0.006-0.038 above the tree, the forest should also pass it in at least half of the tasks.

Threshold (set before running, never moved afterwards), validation macro F1 at 15 s; reference values from `reports/baseline_tables.md` (E02) and `reports/phase3_tables.md` (current best after E04):

- (a) the forest beats the E02 tree (A1 0.8901, A2-VPN 0.8247, A2-NonVPN 0.8294, B 0.7585) by

at least 0.03 in at least 3 of the 4 tasks;

- (b) the forest beats the current best, E04-knn_log_standard (A1 0.9048, A2-VPN 0.8544,

A2-NonVPN 0.8670, B 0.7644), in at least 2 of the 4 tasks (any margin). Each part gets its own verdict.

Winner rule (decided 28 Oct, written here before running): a model whose 15 s validation macro F1 beats that task's current best by at least 0.03 is a winner and is rerun at 30, 60 and 120 s with its 15 s settings. Why 0.03: it is above the largest Phase 2 CV standard deviation (0.025), so a smaller gain could be noise from the split; a fixed number set before running keeps the choice honest and repeatable. The current best is the highest 15 s validation macro F1 whatever the margin; the 0.03 only decides the reruns.

Change vs the current best (one change: the model family): model `rf` = `RandomForestClassifier(n_estimators=300, random_state=42, n_jobs=1)`; every other setting is scikit-learn's default (Gini, no depth limit, `max_features="sqrt"`, bootstrap, no class weights). No scaling (trees don't need it) and no log. One thread per model, the 5 CV folds in parallel (logged as `model_n_jobs` and `cv_n_jobs`). Configs: `configs/experiments/E05_rf_<task>_15s.yaml` (`scripts/make_configs.py`). Protocol as before: 5-fold stratified CV on train, fit on train, score on validation; test set untouched. Models saved with joblib compression level 3. Data version v1.

New model type: the first run's logged fields are checked by query (plan: pilot check for every new model type), including the forest's complexity (number of trees, mean and max depth, nodes).

Storage (plan): after the runs, mlruns/ is measured per model type with compression on, and the project total is projected again.

Result: 4 runs at 15 s and 3 winner reruns (B at 30, 60, 120 s) (all FINISHED, data v1). Numbers below come from `reports/phase3_tables.md` (generated by `scripts/results_tables.py --phase 3` from `reports/history.csv`).

New-model-type check (by query, first run E05-rf-A1-15s): every field of the tracking table is logged, plus the forest's complexity (n_trees 300, mean / max depth, total and mean nodes), impurity feature importance, `model_n_jobs` 1 and `cv_n_jobs` -1. Nothing missing.

Validation macro F1 at 15 s:

Task	E02 tree	current best before (E04 signed-log KNN)	E05 Random Forest	gain over best	winner
A1	0.8901	0.9048	0.9203	+0.0156	no
A2-VPN	0.8247	0.8544	0.8679	+0.0135	no
A2-NonVPN	0.8294	0.8670	0.8941	+0.0271	no
B	0.7585	0.7644	0.8147	+0.0503	yes

- The forest is the new current best in all 4 tasks. Gains over the single E02 tree: +0.0302

(A1), +0.0432 (A2-VPN), +0.0647 (A2-NonVPN), +0.0561 (B).

- Only B clears the 0.03 winner margin. Winner reruns for B (same settings): 0.8114 at 30 s,

0.7771 at 60 s, 0.7835 at 120 s, against the best other run at each timeout (E02 tree 0.7476, 0.7058, 0.7158): +0.064, +0.071, +0.068. The gain holds at every timeout and is largest where the tree was weakest. B still scores lower at 60 s and 120 s than at 15 s and 30 s, as every model did in Phase 2; with the class-mix differences (E00 point 10) and the different 60 s VOIP/FT flows (E00 point 15) this is not read as a timeout effect.

- CV agrees with validation: |CV - validation| <= 0.027 in all 7 runs, CV std 0.0049-0.0136.
- Worst class by F1: VPN-CHAT in A2-VPN (0.71), STREAMING in A2-NonVPN (0.81), VPN-MAIL in B 15 s

(0.62). In B at 120 s STREAMING drops to 0.49. Best: VOIP / VPN-VOIP (0.93-1.00).

- Complexity: 300 trees of mean depth 20.3-26.5 (deepest tree 34), 249,282 nodes (A2-NonVPN 15 s) to

1,023,108 nodes (B 15 s).

- Cost: training 2.0-6.1 s on one thread (5-fold CV 3.2-7.1 s with the folds in parallel),

prediction 21.9-32.7 ms per 1,000 flows (2.0 to 3.3 times the signed-log KNN of E04 on the same task). Model 28.7-171.9 MiB uncompressed, 5.3-22.4 MiB as saved with compression level 3 (4.8 to 7.7 times smaller). Memory: tracemalloc sees only 1.4-2.5 MiB, but the child-process RSS of a fit is 33.1-187.9 MiB. The trees live in memory that tracemalloc doesn't track, which is why the setup change added the RSS measure.

Storage (plan: measure with compression on, per model type; table "Storage of the Phase 3 runs"): the 7 forest runs take 102.4 MiB in all, at most 22.6 MiB per run (model file 22.4 MiB, B 15 s). The signed-log KNN runs take at most 1.6 MiB each. Whole store after these runs: artifacts 390.4 MiB, mlflow.db 11.2 MiB. Projection: the 260 ensemble runs estimated in E03 at about 11 MiB each (the mean compressed forest file over the four 15 s tasks: 9.4, 8.4, 5.3 and 22.4 MiB) come to about 2.9 GiB. That is far below the plan's 20 GB warning line, so nothing needs to be raised with the user. E06 and E07 re-measure with boosting and tuned sizes.

Verdict: (a) supported (forest >= tree + 0.03 in 4 of 4 tasks; smallest gain 0.0302 on A1). (b) supported (forest above the E04 signed-log KNN in 4 of 4 tasks).

Decision: keep the forest as the current best in all four tasks; E06 (boosting) is compared against it. E07 tunes the best 2 of Random Forest / XGBoost / LightGBM per task (after E06) plus the signed-log KNN. Next: E06.

MLflow runs: E05-rf-*

E04 — KNN with a signed log transform: StandardScaler vs RobustScaler (2026-10-28, utsav-s-r)

Hypothesis: KNN classifies by distance, so on standard-scaled raw features a few very large timing values (microseconds up to about 1.2e8, E00) dominate every distance. A signed log sign(x) * log1p(|x|) before scaling compresses those tails, so the distance reflects the shape of the whole flow and KNN (k = 5, as in E03) scores higher. After the log the columns are much less heavy-tailed, so StandardScaler and RobustScaler should then give similar results.

Threshold (set before running, never moved afterwards), at 15 s, against E03 KNN (k = 5, StandardScaler, no log; validation macro F1 A1 0.813, A2-VPN 0.774, A2-NonVPN 0.773, B 0.660, from `reports/baseline_tables.md`):

- (a) the better of the two E04 variants beats E03 KNN validation macro F1 by at least 0.02 in

at least 3 of the 4 tasks;

- (b) the two E04 variants are within 0.02 of each other in at least 3 of the 4 tasks.

Each part gets its own verdict.

Winner rule (decided 28 Oct, written here before running): a model whose 15 s validation macro F1 beats that task's current best by at least 0.03 is a winner and is rerun at 30, 60 and 120 s with its 15 s settings. Why 0.03: it is above the largest Phase 2 CV standard deviation (0.025), so a smaller gain could be noise from the split; a fixed number set before running keeps the choice honest and repeatable. The current best at 15 s is the E02 tree (A1 0.890, A2-VPN 0.825, A2-NonVPN 0.829, B 0.759). The current best is the highest 15 s validation macro F1 whatever the margin; the 0.03 only decides the reruns.

What E07 takes from this: the KNN that E07 tunes uses, per task, whichever of the E03 and E04 preprocessings has the higher 15 s validation macro F1.

Change vs E03 (one change: the preprocessing): model `knn_log_standard` = Pipeline(signed log, `StandardScaler`, `KNeighborsClassifier(n_neighbors=5)`) and `knn_log_robust` = Pipeline(signed log, `RobustScaler`, same KNN). The signed log is `src/blindspy/features.py` (a fixed formula inside the pipeline; nothing learned from data; data v1 unchanged). Why the signed log and not plain log1p: log1p(-1) is minus infinity, and the features hold -1 "not measurable" codes and 586 negative min_flowiat values (E00). The signed log is defined everywhere, keeps the order of values and keeps -1 apart from 0. Not chosen: shifting each column so its minimum is 0 before the log (a shift learned from training data, different per column, and -1 merges into small values); logging only the non-negative columns (inconsistent, leaves large values uncompressed); replacing -1 by 0 plus flag columns (changes the feature set, so E04 would change two things). Configs: `configs/experiments/E04_knn_log_{standard,robust}_<task>_15s.yaml` (`scripts/make_configs.py`). Protocol as E03: 5-fold stratified CV on train, fit on train, score on validation; test set untouched. Models saved with joblib compression level 3. Data version v1.

Pilot: the first Phase 3 run (E04-knn_log_standard-A1-15s) is checked by hand in the MLflow UI against the plan's tracking table, including the new Phase 3 items (compression level, both model sizes, child-process RSS, n_jobs), before the other 7 runs.

Result: 8 runs at 15 s and 3 winner reruns (all FINISHED, data v1). Numbers below come from `reports/phase3_tables.md` (generated by `scripts/results_tables.py --phase 3` from `reports/history.csv`).

Pilot: the first run (E04-knn_log_standard-A1-15s) was checked by hand in the MLflow UI against the tracking table. Everything was logged (identity, lock hash and package versions incl. xgboost / lightgbm / optuna / imbalanced-learn, settings, data version and hashes, all scores incl. ROC-AUC, timing, both model sizes, compression level 3, child-process RSS, n_jobs, every file). The saved compressed model reloaded and reproduced validation macro F1 0.90477 exactly. One gap: the param `pipeline_steps` said "FunctionTransformer" without naming the function. Fixed by a new param `preprocessing` ("log: FunctionTransformer(signed_log); scale: StandardScaler()"); all 8 runs then came from the corrected code. The pilot's first run stays in MLflow; tables use the newest run per name.

Validation macro F1 at 15 s (E03 KNN and E02 tree from `reports/baseline_tables.md`):

Task	E03 KNN (no log)	signed log + StandardScaler	signed log + RobustScaler	E02 tree
A1	0.813	0.9048	0.8929	0.8901
A2-VPN	0.774	0.8544	0.8372	0.8247
A2-NonVPN	0.773	0.8670	0.8540	0.8294
B	0.660	0.7644	0.7420	0.7585

- The signed log lifts KNN by 0.080 to 0.104 macro F1 in every task (better variant minus

E03 KNN: 0.0921, 0.0801, 0.0935, 0.104).

- StandardScaler beats RobustScaler after the log in all 4 tasks, by 0.0118 to 0.0224.
- Signed-log KNN with StandardScaler is now the best model at 15 s in all 4 tasks, above the

E02 tree by +0.0146 (A1), +0.0296 (A2-VPN), +0.0376 (A2-NonVPN) and +0.0059 (B). So it becomes the current best everywhere. Only A2-NonVPN clears the 0.03 winner margin; A2-VPN misses it by 0.0004.

- Winner reruns, A2-NonVPN (same settings): 0.8523 at 30 s, 0.8590 at 60 s, 0.8491 at 120 s,

against the best other run at each timeout (E02 tree 0.8283, 0.8246, 0.8393). So the gain holds at every timeout (+0.024, +0.034, +0.010). At 120 s the CV spread is the widest seen so far (std 0.0252) and validation sits 0.025 above CV.

- 5-fold CV agrees with validation (|CV - validation| <= 0.025 in all 11 runs; CV std

0.0042-0.0252).

- Worst class by F1: STREAMING in A2-NonVPN (0.69-0.74; CHAT 0.69 at 120 s), VPN-CHAT in A2-VPN

(0.63-0.69), MAIL (0.56, standard) or VPN-CHAT (0.56, robust) in B. Best: VOIP / VPN-VOIP (0.97-0.99). In A1 both classes are within 0.02 of each other.

- Cost: training 0.006-0.033 s (KNN only stores the rows), prediction 7.7-11.5 ms per 1,000

flows (26 to 39 times the E02 tree's median of 0.30 ms, `reports/baseline_tables.md`), model 0.64-2.32 MiB uncompressed and 0.42-1.44 MiB as saved with compression level 3 (about 1.6 times smaller), peak memory 1.8-6.7 MiB (tracemalloc) and 1.9-6.5 MiB (child-process RSS). `model_n_jobs` is logged as None for KNN: scikit-learn's default, which means one job, so KNN also runs on one thread, as the 28 Oct thread decision requires.

Why it works: the timing features span six or more orders of magnitude. With plain standard scaling one long idle period or huge rate decides the whole distance, so neighbours are "flows with similar outliers". After the signed log, a factor of 10 is the same step anywhere on the scale, so all 23 features contribute. Why RobustScaler is worse: it divides by the interquartile range, which is 0 for std_active and std_idle on the training rows of every task (most flows have no active/idle spread). scikit-learn then leaves those two columns unscaled, so their non-zero values reach |x| = 18.3-18.9 after scaling, against at most 4.8-5.5 for any column with StandardScaler, and they dominate the distance (table "E04: columns RobustScaler leaves unscaled").

Verdict: (a) supported (better variant >= E03 KNN + 0.02 in 4 of 4 tasks; smallest gain 0.0801). (b) supported (variants within 0.02 in 3 of 4 tasks: A1 0.0118, A2-VPN 0.0172, A2-NonVPN 0.0129; B 0.0224 is outside).

Decision: keep signed log + StandardScaler (`knn_log_standard`) as the current best for all four tasks and as the KNN preprocessing for E07 (it beat E03 in every task). E05 and E06 are now compared against it. Next: E05 Random Forest.

MLflow runs: E04-knn_log_standard-*, E04-knn_log_robust-*

Addendum (Phase 3 recheck, 28 Oct): the setup check (plan Phase 3 item 1, done before E04) was until now recorded only in local notes, so it is written here. The setup change holds model_compress: 3 in configs/base.yaml, xgboost / lightgbm / optuna / imbalanced-learn in pyproject.toml and uv.lock, the new logging in train.py, and the test that a compressed model reloads and predicts identically. A fresh scratch copy of the code with its own MLflow store (never the official one) rebuilt data v1 byte-identical to the main folder's (34 of 34 files, SHA-256 compared again in this recheck) and reran E02-tree-A1-15s and E03-knn-B-15s: every non-timing metric equals the official Phase 2 run (28 and 72 metrics, 0 different; compared again in this recheck from both stores).

E03 — KNN, Logistic Regression, SVM-RBF and the paper benchmark for KNN (2026-10-28, utsav-s-r)

Hypothesis (holdout baselines): KNN (k = 5) on standard-scaled features comes close to the tree, since flows of one app sit near each other in timing space. Logistic Regression and an RBF SVM, with plain standard scaling, do worse than the tree: the features are heavy-tailed (microsecond times up to about 1.2e8 next to

many -1 and 0 codes, E00), so after standard scaling most flows are squeezed together, which hurts a linear boundary and a single RBF width. All three still beat the floor. (Log transforms are tested in E04, not here.)

Threshold (set before running, never moved afterwards):

- (a) each of KNN, Logistic Regression and SVM-RBF beats E01 validation macro F1 by at least

0.10 in all 16 configs;

- (b) KNN validation macro F1 is within 0.10 of the E02 tree in at least 12 of 16 configs;
- (c) Logistic Regression and SVM-RBF each score below the E02 tree in at least 12 of 16

configs. Each part gets its own verdict; the experiment is supported if all three hold.

Hypothesis (paper benchmark, part of this experiment by decision of 2026-10-28): with the paper's protocol and Weka's IBk defaults, 1-NN reproduces the paper's A1 KNN results, and duplicates inflate its 10-fold scores even more than the tree's, since a duplicate in the training folds is always the nearest neighbour.

Threshold (paper benchmark, set before running):

- (d) on the published rows, at least 12 of the 16 A1 chart values for KNN are matched within

+-0.05;

- (e) macro F1 on data v1 is lower than on the published rows in at least 12 of the 16 files.

A2 and B are compared with the values the paper states; no threshold (same reason as E02).

Note on what was seen before the thresholds were written: an unlogged dry run of the code on A1 15 s and B 15 s gave validation macro F1 0.813 / 0.661 (KNN), 0.585 / 0.307 (Logistic Regression) and 0.602 / 0.297 (SVM-RBF). The thresholds cover all 16 configs, of which those two had been seen.

Change vs E01: three models, each a Pipeline with `StandardScaler` fitted on training rows only, then `KNeighborsClassifier(n_neighbors=5)`, `LogisticRegression(max_iter=1000, random_state=42)` (lbfgs; max_iter raised from 100 so the solver converges; all else default) or `SVC(kernel="rbf", C=1.0, gamma="scale", random_state=42)`. SVM scores for ROC/PR curves come from `decision_function` (no Platt scaling, which would add an internal CV). Benchmark model `ibk = MinMaxScaler + KNeighborsClassifier(n_neighbors=1)`: Weka IBk defaults are `-K 1` with `LinearNNSearch` and `EuclideanDistance`, which normalises each attribute to its range unless `-D` is set (Weka Javadoc: `weka.classifiers.lazy.IBk`, `weka.core.EuclideanDistance`, `weka.core.NormalizableDistance`). Configs: `configs/experiments/E03_{knn,logreg,svm_rbf}_<task>_<timeout>s.yaml` and `E03_paper_ibk_<rows>_<task>_<timeout>s.yaml`. Data version v1.

Storage check (plan: after E01-E03, every model saved): see the end of Result.

Result: 80 baseline runs, all FINISHED. The 32 IBk benchmark runs come (see E02). Numbers below come from `reports/baseline_tables.md` (generated by `scripts/results_tables.py` from `reports/history.csv`).

Holdout (validation macro F1, data v1, 15 / 30 / 60 / 120 s; tree from E02 for reference):

Task	KNN (k = 5)	Logistic Regression	SVM-RBF	Tree (E02)
A1	0.813 / 0.825 / 0.813 / 0.807	0.585 / 0.598 / 0.541 / 0.576	0.602 / 0.610 / 0.545 / 0.621	0.890 / 0.864 / 0.857 / 0.857
A2-VPN	0.774 / 0.788 / 0.730 / 0.727	0.432 / 0.536 / 0.339 / 0.332	0.468 / 0.542 / 0.330 / 0.332	0.825 / 0.828 / 0.775 / 0.780
A2-NonVPN	0.773 / 0.783 / 0.750 / 0.769	0.449 / 0.486 / 0.447 / 0.482	0.487 / 0.468 / 0.441 / 0.471	0.829 / 0.828 / 0.825 / 0.839
B	0.660 / 0.647 / 0.617 / 0.637	0.307 / 0.320 / 0.271 / 0.270	0.296 / 0.311 / 0.278 / 0.274	0.759 / 0.748 / 0.706 / 0.716

- KNN is second everywhere, 0.039-0.101 below the tree. Its A1 ROC-AUC is 0.891-0.899, higher

than the tree's 0.858-0.890, because 5 neighbours give smoother scores.

- Logistic Regression and SVM-RBF fail on the small classes: MAIL / VPN-MAIL F1 is 0.00 in most

A2 and B configs, so macro F1 sits far below weighted F1 (gap up to 0.225, against 0.000-0.087 for KNN). In A1 they lean heavily to one class (e.g. SVM 60 s: Non-VPN recall 0.95, VPN recall 0.22).

- The SVM keeps 55-88% of the training rows as support vectors (e.g. B 15 s: 11,076 of 12,651).

So the RBF kernel on standard-scaled heavy-tailed features separates the classes poorly, as hypothesised. Logistic Regression converged in at most 244 iterations (limit 1,000; the default of 100 would have stopped early).

- Generalisation gap |CV - validation| <= 0.042 for all three; CV std <= 0.025.
- Cost (median over 16 configs): KNN trains in 0.01 s, but predicts 1,000 flows in 8.0 ms and

stores the training set (1.2 MB median, 2.3 MB max). Logistic Regression: 0.08 s, 0.38 ms, 3.8 KB. SVM: 0.63 s (2.9 s max), 210 ms (597 ms max), 1.1 MB (4.2 MB max). Peak fit memory is 3.0-3.6 MB (tracemalloc). For the SVM this is a lower bound, since libsvm's own memory isn't counted.

Paper benchmark, IBk (1-NN, min-max scaling):

- A1, published rows: 15 of 16 chart values within +-0.05. The largest miss is VPN recall at

120 s (paper 0.860, ours 0.803). Example: VPN precision 15 s, paper 0.848, ours 0.863.

- B best-timeout average precision (paper 0.711): ours 0.710 (macro mean, published rows).

Per-class B precision at 15 s is within 0.01 of the paper's text values. VPN-MAIL at 120 s is far off (paper 0.688; ours 0.820 on published rows, 0.521 on v1); the mail classes are small.

- Duplicates: v1 scores lower than the published rows in only 2 of 16 files (mean change +0.012,

up to +0.040 for A2-NonVPN 120 s). The cause is the same as in E02: the removed duplicates are mostly 0-2 packet flows with twins of other labels. 1-NN copies the label of an identical twin, so extra copies add errors.

Storage check (plan: every official model saved):

- Phase 2 store after these runs (147 runs incl. 1 smoke and 2 pilot runs): artifacts 137.8 MB,

mlflow.db 5.8 MB. Per model type (table "Storage"): majority 1.4 MB for 16 runs, Logistic Regression 1.6 MB, tree 5.5 MB, KNN 23.1 MB, SVM 27.4 MB; benchmark tree 11.7 MB and IBk 66.7 MB for 32 runs each. Largest single run: 4.3 MB (SVM).

- Phase 3 measurement (not logged; no model choice made from it): a 300-tree Random Forest saved

with joblib is 171.9 MB on B 15 s, 44.7 MB on A1 15 s and 48.4 MB on A2-VPN 15 s. With joblib compression level 3, the B file is 22.4 MB (load time 0.17 s instead of 0.08 s).

- Projection for the whole project, if tree ensembles win and every run's model is saved

uncompressed: about 260 ensemble runs in Phases 3-4 (E05 16, E07 retrains 8, E08 16, E09 52, E10 3, E13 120 at about half size, E14 40, E16 3, finals 4). At about 78 MB average for 300 trees that gives about 18 GB, and more if tuning picks up to 1,000 trees. This is at the plan's 20 GB warning line, so the user is told before Phase 3. Compressed (level 3), the same runs would take about 2-3 GB.

Verdict: (a) supported for all three models (each beats E01 by >= 0.10 in 16 of 16 configs; smallest gains 0.455 KNN, 0.183 Logistic Regression, 0.187 SVM-RBF). (b) supported (KNN within 0.10 of the tree in 15 of 16; miss: B 30 s, 0.101). (c) supported (Logistic Regression and SVM-RBF below the tree in 16 of 16 each). (d) supported (15 of 16 A1 values within +-0.05). (e) not supported (2 of 16 files lower on v1; needed 12).

Decision: keep KNN as the strongest non-tree baseline; it gets the same tuning budget as the best models in E07, as the plan says. Logistic Regression and SVM-RBF are kept as reported baselines only. Next: E04 tests whether a log transform or RobustScaler fixes KNN's trouble with heavy-tailed features. Storage: the user decides on joblib compression before Phase 3.

MLflow runs: E03-knn-*, E03-logreg-*, E03-svm_rbf-* (holdout), E03-paper-ibk-* (benchmark)

Addendum (2026-10-28): storage decision. The user chose to keep every official model and save it with joblib compression level 3 from Phase 3 on. Compression is lossless: a compressed model reloads and predicts identically. The 300-tree Random Forest above drops from 171.9 MB to 22.4 MB, so the projected total is about 2-3 GB instead of about 18 GB. Phase 2 models stay as saved. Metrics, plots, configs and code versions are kept for every run either way.

Corrections and additions (recheck, 2026-10-28).

- Model file sizes: KNN max 2.3 MB (was written 2.4), SVM median 1.1 MB and max 4.2 MB (was

1.2 / 4.3; 4.3 MB is the SVM's largest whole run folder, not its model file). The run count is 147 (80 + 64 + 2 pilot + 1 smoke), not 146.

- Training times in these runs were measured with tracemalloc switched on. In a recheck (B 15 s,

median of 5 fits) this slowed fitting by x1.00 for the SVM, x1.19 for Logistic Regression and x2.3 for KNN (0.012 s -> 0.027 s). The KNN and Logistic Regression training times above are overestimates by those factors; the ranking of models by cost doesn't change. From Phase 3 on, time and memory come from separate fits.

- `process_max_rss_mb` is the high-water mark of the whole batch process, not a per-run figure.

It is not used anywhere and is no longer logged.

- The paper comparison is now also one table per task, for both row sources, with our per-class

10-fold precision and recall and the paper's value where it gives one.

Addendum (2026-10-28): clean timing rerun. All 144 Phase 2 configs (80 baselines, 64 benchmark) were rerun from the fixed code, where training time comes from a fit without tracemalloc. Every one of the 6,516 non-timing metric values (scores, per-class results, model sizes, complexity) and every parameter is identical to the first runs. The first runs stay in MLflow and in history.csv; the tables now use the rerun. Cost numbers in this entry are from the rerun. Median training time, first runs -> rerun: KNN 0.021 -> 0.010 s, Logistic Regression 0.116 -> 0.084 s, SVM 0.636 -> 0.626 s. Prediction times differ only by machine noise (KNN 7.9 -> 8.0 ms, SVM 211 -> 210 ms per 1,000 flows). The overestimates noted in the corrections above are resolved. MLflow store after the rerun (291 runs): artifacts 275.2 MB, mlflow.db 10.4 MB.

E02 — Decision tree (C4.5 approximation) and the paper benchmark for C4.5 (2026-10-28, utsav-s-r)

Hypothesis (holdout baseline): an unpruned entropy decision tree, the closest scikit-learn model to C4.5, learns the timing patterns well above the E01 floor in every task, and on A1 comes close to the paper's C4.5 (per-class precision and recall 0.83-0.92 in the paper's Figure 2).

Threshold (set before running, never moved afterwards):

- (a) validation macro F1 >= E01 validation macro F1 + 0.30 in all 16 configs;
- (b) A1 validation macro F1 >= 0.80 at every timeout (4 configs).

Supported only if both hold.

Hypothesis (paper benchmark, part of this experiment by decision of 2026-10-28): with the paper's protocol (10-fold stratified CV on the whole file, predictions pooled over the folds) and Weka's J48 defaults approximated, our tree reproduces the paper's A1 results. Duplicate rows inflate 10-fold scores, because a flow's copy can sit in the training folds.

Threshold (paper benchmark, set before running):

- (c) on the rows as published, at least 12 of the 16 A1 chart values (2 classes x precision

and recall x 4 timeouts, `reports/paper_baseline.csv`) are matched within +-0.05;

- (d) macro F1 on data v1 (deduplicated) is lower than on the published rows in at least 12 of

the 16 files. A2 and B are compared with the values the paper states in its text; no threshold, because Figure 3 can't be assigned to C4.5 or KNN (see `reports/paper_baseline_NOTES.md`).

Note on what was seen before the thresholds were written: while testing the code, an unlogged dry run on A1 15 s and B 15 s gave validation macro F1 0.890 and 0.759 for this tree. The thresholds above cover all 16 configs and the benchmark, which had not been run.

Change vs E01: model `tree` = `DecisionTreeClassifier(criterion="entropy", random_state=42)`, no depth limit, no pruning, no scaling (trees don't need it). Benchmark model `j48` = `DecisionTreeClassifier(criterion="entropy", min_samples_leaf=2, random_state=42)`: Weka J48 defaults are `-C 0.25 -M 2` with pruning on (Weka Javadoc, `weka.classifiers.trees.J48`); scikit-learn has no C4.5 pessimistic pruning, so only the 2-rows-per-leaf rule is copied. Configs: `configs/experiments/E02_tree_<task>_<timeout>s.yaml` and `E02_paper_j48_<rows>_<task>_<timeout>s.yaml` (`scripts/make_configs.py`). Rows for the benchmark: `published` = every row of the ARFF file as published (duplicates kept, values untouched, names from COLUMN_MAP; A1 120 s keeps its published `flowPktsPerSecond`) and `v1` = all rows of data v1 (train + validation + test together, as the paper's protocol uses the whole file; these scores are never used for model choice). Data version v1.

Result: 80 baseline runs, all FINISHED. Numbers below come from `reports/baseline_tables.md` (generated by `scripts/results_tables.py` from `reports/history.csv`). The 64 paper-benchmark runs (32 in this experiment) come (the first attempt was refused by the shared-code guard because generated files were not saved yet; nothing was logged; the rerun came from the clean code).

Holdout (validation macro F1, data v1):

- A1 0.890 / 0.864 / 0.857 / 0.857 (15 / 30 / 60 / 120 s); A2-VPN 0.825 / 0.828 / 0.775 / 0.780;

A2-NonVPN 0.829 / 0.828 / 0.825 / 0.839; B 0.759 / 0.748 / 0.706 / 0.716. Best baseline in all 16 configs. Gain over E01: 0.499 to 0.765.

- 5-fold CV agrees with validation: |CV mean - validation| <= 0.054, CV std 0.005-0.023.
- A1 ROC-AUC 0.858-0.890 (a single tree gives few distinct scores, so the curve is coarse).
- Macro vs weighted F1 gap 0.000 (A1) to 0.069 (B 60 s). Best class by F1: VOIP or VPN-VOIP

(0.92-0.99) in every multi-class config; worst: VPN-CHAT, CHAT, STREAMING, MAIL or VPN-MAIL (0.42-0.71).

- Top confusion pair in B at every timeout: BROWSING vs VPN-BROWSING (29-45 validation flows each

way): the tree knows the flow is browsing but not whether it went through the VPN. In A2 the confusions are among CHAT, FT and BROWSING.

- Complexity: unpruned trees of depth 16-35, 591-3,437 nodes. Cost: train 0.11 s (median),

predict 0.30 ms per 1,000 flows, model file 134 KB median (593 KB max), peak fit memory 1.3 MB.

- B and A2-VPN score lower at 60 s and 120 s than at 15 s and 30 s. Class mix differs between

timeouts (E00, point 10) and the 60 s files hold differently shaped VOIP and FT flows (E00, point 15), so this is not read as a timeout effect.

Paper benchmark (10-fold CV, pooled predictions; `notebooks/02_baselines.ipynb`):

- A1, published rows: all 16 chart values matched within +-0.05; largest difference 0.021

(e.g. VPN precision 15 s: paper 0.890, ours 0.892; Non-VPN recall 120 s: 0.843 vs 0.853). On data v1 the values move by at most 0.053.

- A2 average precision at 15 s (paper 0.84 VPN, 0.89 non-VPN): ours 0.839 and 0.882 (macro mean)

on published rows. B best-timeout average precision (paper 0.783): ours 0.770 macro, 0.808 weighted, so the paper's "average" is probably the plain mean (not stated in the paper). B per-class precision at 15 s within 0.04 of the paper's text values; MAIL at 120 s is the largest miss (0.680 vs 0.790).

- Duplicates: on data v1 the 10-fold macro F1 is lower than on the published rows in only 6 of

16 files (mean change +0.0013; range -0.018 to +0.014).

Surprise, and why (d) failed: the removed duplicates are mostly degenerate flows. 39-92% of them have an identical feature twin with a different label (2-10% of all rows), and their median flow has 0-2 packets (all flows: 8-43). Keeping every copy multiplies errors no timing model can avoid, which offsets the leakage gain. Table "What the removed duplicates are".

Pilot (not part of the 80): the first end-to-end run (E_pilot-tree-A1-15s) was checked by hand in the MLflow UI against the plan's tracking table. It showed three gaps: no feature-importance file, class counts only as a JSON file, and a clipped confusion-matrix title. All three were fixed and the pilot rerun gave identical scores (validation macro F1 0.8901), which also showed the runs are deterministic.

Verdict: (a) supported (16 of 16 configs gain >= 0.30; smallest gain 0.499). (b) supported (A1 0.890 / 0.864 / 0.857 / 0.857, all >= 0.80). (c) supported (16 of 16 A1 values within +-0.05). (d) not supported (6 of 16 files lower on v1; needed 12).

Decision: keep the decision tree as the reference baseline (best of E01-E03 in every config). Keep the duplicate rule of data v1. The benchmark shows it does not inflate or deflate scores much (about +-0.02), and it prevents train/test copies in our own protocol. The report states that the hypothesis about inflation was wrong and why. Next: E04-E06 in Phase 3 must beat the tree's validation macro F1.

MLflow runs: E02-tree-* (holdout), E02-paper-j48-* (benchmark, tag protocol=paper_10fold)

Corrections and additions (recheck, 2026-10-28).

- The worst-class list above now includes VPN-MAIL (B 15 s, F1 0.55); it was left out.
- Training times in these runs were measured with tracemalloc switched on, which slows fitting

(tree x1.05 in a recheck on B 15 s, median of 5 fits). The tree's 0.12 s median is therefore a slight overestimate. From Phase 3 on, time and memory come from separate fits.

- `process_max_rss_mb` in these runs is the high-water mark of the whole batch process (it only

grows from run to run), not a per-run figure. It is not used anywhere and is no longer logged.

- The plan asks for the paper comparison as one table per task. Our per-class 10-fold precision

and recall are now tabled for every task and both row sources, with the paper's value beside ours where it gives one ("C" chart, "T" text). The 8 A1 values that the paper both draws and states are marked as such (`reports/baseline_tables.md`, `notebooks/02_baselines.ipynb`).

Addendum (2026-10-28): clean timing rerun. All 144 Phase 2 configs (80 baselines, 64 benchmark) were rerun from the fixed code, where training time comes from a fit without tracemalloc. Every one of the 6,516 non-timing metric values (scores, per-class results, model sizes, complexity) and every parameter is identical to the first runs. The first runs stay in MLflow and in history.csv; the tables now use the rerun. Cost numbers in this entry are from the rerun. Tree training time: 0.11 s median (0.12 s in the first runs). The "slight overestimate" noted in the corrections above is resolved.

E01 — Majority-class baseline, all tasks and timeouts (2026-10-28, utsav-s-r)

Hypothesis: a model that always predicts the largest training class sets the floor every model must beat. Its validation accuracy equals that class's share p of the validation set, and its macro F1 equals 2p / ((1 + p) k) for k classes (F1 of the predicted class is 2p / (1 + p); every other class scores 0). The stratified split keeps p the same in train and validation, so CV and validation should agree.

Threshold (set before running, never moved afterwards): supported if, in all 16 configs (4 tasks x 4 timeouts), |validation accuracy - p| <= 0.001 and

validation macro F1 - 2p / ((1 + p) k)	**<= 0.001. Any miss means a bug in the pipeline or**

the metrics, and the runs are not used until it is fixed.

Change: none (first baseline). Model `majority = DummyClassifier(strategy="most_frequent", random_state=42)`. Configs: `configs/experiments/E01_majority_<task>_<timeout>s.yaml` (generated by `scripts/make_configs.py`). Protocol: 5-fold stratified CV on train, fit on train, score on validation (test not used). Data version v1.

Result: 80 baseline runs, all FINISHED. Numbers below come from `reports/baseline_tables.md` (generated by `scripts/results_tables.py` from `reports/history.csv`).

- All 16 configs match the arithmetic to 4 decimals (table "E01 check"): e.g. A1 15 s, largest

class VPN, p = 0.5185, validation accuracy 0.5185, expected and measured macro F1 0.3414; B 15 s, largest class VOIP, p = 0.1560, macro F1 0.0193.

- The floor in validation macro F1: A1 0.337-0.358, A2-VPN 0.060-0.093, A2-NonVPN 0.067-0.096,

B 0.019-0.030. Balanced accuracy is exactly 1/k (0.500, 0.143, 0.071) and A1 ROC-AUC 0.5.

- The largest class changes with the timeout (A1: VPN at 15/30 s, Non-VPN at 60/120 s;

A2-NonVPN: VOIP at 15 s, BROWSING after; B: VOIP at 15 s, BROWSING after), as E00 found.

- CV on train equals validation (std 0.000): the stratified split keeps class shares equal.
- Cost: under 0.03 s to train, about 1.4 KB per model file.

Verdict: supported (16 of 16 configs within 0.001 on both checks).

Decision: keep as the floor. Every later model is compared with it. Next: E02, E03.

MLflow runs: E01-majority-*

Addendum (2026-10-28): clean timing rerun. All 144 Phase 2 configs (80 baselines, 64 benchmark) were rerun from the fixed code, where training time comes from a fit without tracemalloc. Every one of the 6,516 non-timing metric values (scores, per-class results, model sizes, complexity) and every parameter is identical to the first runs. The first runs stay in MLflow and in history.csv; the tables now use the rerun. Cost numbers in this entry are from the rerun.

E00 — Data findings and data v1 (2026-10-27, utsav-s-r)

Hypothesis (from the plan, written before the check): the ARFF `@ATTRIBUTE` names may not match the data order. In A1 15 s row 1, `total_fiat` = 16 and `total_biat` = 4 in a 9.37 s flow, which looks wrong; the paper's Table 2 and the authors' 2016 poster list fiat/biat as min, mean, max, std, with no totals. Candidates: `total_*` are packet counts, `std_*`, or something else.

Threshold: no numeric threshold was written before the check. Decision rule used: a name list is accepted for a file when every structural identity (min <= mean <= max per group, 0 <= std <= max - min, gaps x (packets - 1) = duration) holds on 100% of applicable rows, allowing for the source's rounding. A list is rejected when the identities fail on most rows.

Change: none to models. Built the parser (`src/blindspy/data.py`), the checks (`src/blindspy/checks.py`), `scripts/prepare_data.py` and `src/blindspy/splits.py`, then data version v1. Config: `configs/base.yaml` (`cleaning:`). Data version v1. Every number below is in `data/manifest.json` (keys named in brackets) or `data/processed/v1/cleaning_log.csv`.

Result:

1. **Two header layouts.** 17 files use header "T" (`total_fiat`, `total_biat`, no std); the

three 60 s files of A1 and B (incl. B-AllinOne) use header "S" (`std_fiat`, `std_biat`, no totals), the paper's list. [`files[].header`]

1. **Column-name check** [`column_name_check`]. Read under the S names, every A1 and B file

passes fiat/biat min <= mean <= max and 0 <= std <= max - min on 100% of rows. Read under the T names, the same files pass min <= mean <= max on 0.00-0.02% and the packet-count identity on 0.00-0.05%. A2 files are the reverse: under the T names min <= mean <= max and "total/mean + 1, forward + backward = packet count (+-2%)" pass on 100%. Under the S names std <= max - min passes on 0%. The plan's packet-count test (mean_fiat x (total_fiat - 1) = duration +-10%) passes on 0-2% of rows in every file, so `total_*` are not packet counts. Verdict: A1 and B hold the paper's features, and 9 of their 12 headers shift the fiat/biat names by one pair. A2 holds `total_fiat`/`total_biat` (sum of the gaps) exactly as named.

1. **Final COLUMN_MAP** (`src/blindspy/data.py`; header name -> real content):

- A1 and B files with header T: `total_fiat`->`min_fiat`, `total_biat`->`min_biat`, `min_fiat`->`max_fiat`, `min_biat`->`max_biat`, `max_fiat`->`mean_fiat`, `max_biat`->`mean_biat`, `mean_fiat`->`std_fiat`, `mean_biat`->`std_biat`. The other 15 names are unchanged. - A1 and B files with header S, and A2 files: names unchanged. - Checked on the cleaned data: every structural check passes on 100% of rows in all 16 files [`tasks.*.consistency_checks`]. Bytes per packet falls in 40-1,500 B for 96.1-99.3% of flows; the rest are header-only packets (down to 30 B) and segments merged at capture (up to about 7 kB).

1. **Same flows in all families** [`family_match`]. At every timeout, 100% of A1 rows match an

A2 row on 7 integer columns. A1 labels equal the A2 VPN/Non-VPN labels key by key, and B labels equal the A2 labels (0 disagreeing keys), and all 21 shared columns agree on rows with a unique key. B and A1 are identical cell for cell, except A1 120 s `flowPktsPerSecond` (10,288 cells). There A1 agrees with A2 on 0.03% of paired rows, while B 120 s agrees on 100%. So A1 120 s has one corrupt column.

1. **File-family decision** (user OK, 2026-10-27): each task keeps its own files. A1 and B use

the paper's 23 features; A2 uses its 23 (totals in place of stds). Nothing is rebuilt. A1 120 s `flowPktsPerSecond` is restored from B 120 s. Reason: these are the files the paper used per scenario, so the paper comparison stays matched. A rebuild from A2 would swap 2 of the paper's features, and a 25-feature merge is ambiguous where keys repeat. E10 trains its 7-class step on B rows split by VPN, so both steps share the A1/B layout.

1. **Odd values** (rules in `configs/base.yaml`, counts per file in the cleaning log and

`tasks.*.odd_values`). No missing values and no infinities in any file. `-1` means "not measurable": min = max = -1 with mean = 0 when a direction has fewer than 2 packets (forward: 11.2% of A1 15 s flows) or when there is no active/idle period (59.0% of A1 15 s flows). Duration 0 marks single-packet flows (rates 0). Negative `min_flowiat` (down to -1,247 us) comes from out-of-order capture timestamps. Every mean, std and rate of 1e7 or more is stored to 3 significant figures (100% of such values; integer columns are exact). Rule (user OK): keep all values as-is, log every case, no flag columns, nothing set to 0.

1. **Duplicates** [`tasks.*.summary`, `*_per_class`]. Exact duplicates within files: 11,209 of

179,120 raw lines (3.02% in A2-NonVPN 15 s to 17.46% in A2-VPN 120 s), uneven by class. VPN-MAIL loses 483 of 848 flows at 30 s; in A1 120 s, 1,042 VPN rows go against 216 Non-VPN. Conflicting duplicates (same features, different label) after de-duplication: 1,102 rows in 3-77 groups per file. Rule (user OK): drop exact duplicates before splitting, keep conflicting rows. Result: 0 exact duplicates cross train and test in any file; 1-25 test rows per file have a conflicting twin in train [`tasks.*.test_rows_with_conflicting_twin_in_train`].

1. **Leakage columns:** none. Each file holds only the 23 numeric timing features and the label.
2. **Session information: no.** No IDs, IPs, ports, timestamps or capture-file names, so E11

(session-aware split) does not run; the report lists this as a limitation.

1. **Timeouts** [`timeouts`, `reports/figures/timeout_class_mix.png`]. Same names, order and

units at all four timeouts. Class shares move by up to 24.6 percentage points (A2-NonVPN VOIP 32.5% at 15 s vs 7.9% at 120 s), so timeout effects on scores are confounded by class mix.

1. **Kaggle CSV** [`kaggle`]. As a multiset it equals the 8 A2 files stacked: 59,706 rows,

0 rows without an A2 partner at 12 significant digits, the same labels. It has 40,987 unique rows (Isnan et al.'s count). 7,093 distinct flows appear in 2-4 timeout files (2,475 / 1,286 / 3,332), but only 3,676 rows are within-file duplicates. So most of Isnan et al.'s 18,719 "duplicates" are the same flow at different timeouts, and their 80/20 split mixes timeouts. Its header is the A2 header, so it holds `total_*` features, not the paper's.

1. **Data v1 created**: 16 Parquet files, 167,909 rows, 18.7 MB; deterministic build

(three builds byte-identical); `reports/DATA_CHANGELOG.md` v1 entry.

Surprises: the column-name problem is a one-pair name shift in 9 headers, not a scrambled order. A1 and B hold a different feature set from A2. One A1 file has a corrupt column. Every large mean/std/rate is rounded to 3 significant figures in the published files. BROWSING and VPN-BROWSING are capped at exactly 2,500 flows per raw A2 file.

Verdict: supported. The header names are wrong for 9 of the 12 A1/B files (identities 100% vs 0%), and right for A2 and for the three 60 s A1/B files.

Decision: keep. COLUMN_MAP and data v1 are used by every later experiment. Next: Phase 2 baselines on v1, starting with A1 at 15 s.

MLflow runs: none (data only).

Addendum (recheck, 2026-10-27). A recheck against the plan found four Phase 1 items only partly done. They are completed below; data v1 itself is unchanged (same files, same hashes). New evidence is written by `scripts/data_checks.py v1` to `reports/data_checks_v1.json`.

1. **Value ranges vs the paper's Table 2** (plan: column-name check, step 1). Under the corrected

names the A1/B feature list is exactly Table 2's 23 features; the A2 list differs only in `total_fiat`/`total_biat`. Table 2 gives no units. The identity mean_flowiat x (packets - 1) = duration, with packets = flowPktsPerSecond

x duration / 1e6, holds on 100% of rows in all 16 files, which fixes the units: times in microseconds, rates per second. Every measured gap, active and idle value is at most the flow's duration (100% of values), as Table 2's definitions require. Per-feature min / median / p99 / max are in the evidence file [`feature_ranges`].

1. **The pre-start numbers, now recomputed by code** [`published_header_matches_A1_vs_A2`]. As

published (names ignored), 0 A1 rows equal an A2 row in all 23 values at any timeout. 2,364 / 1,501 / 2,190 / 494 A1 rows (15 / 30 / 60 / 120 s) hold the same set of values in another order. A1 15 s has 18,038 unique rows counting the label and 18,015 unique feature rows. All three match the numbers noted before the project started. They follow from point 2 (two values differ in meaning; the names are shifted).

1. **New finding: the 60 s files differ** [`voip_ft_profile_per_timeout`,

`cross_timeout_label_consistency_A2`]. At 15/30/120 s, non-VPN VOIP has a median 100 packets/s and a 20 ms forward gap, and VPN-VOIP 2.1-2.2 packets/s and about 960 ms. At 60 s they are nearly reversed (VOIP 2.1 packets/s, 962 ms; VPN-VOIP 16.5 packets/s, 18 ms). Non-VPN FT falls from 1,237-3,739 to 56.5 packets/s. It is not a label swap: of 7,041 flows found identically in more than one A2 timeout file, 210 change label between timeouts. They are mostly 2-packet flows (median 2.0), and no VoIP pair is among them. The 60 s files are also the only ones with the paper-style header, so they may come from a separate export. Cause unknown from the ARFF files. Decision: no data change; recorded as a limitation, and 60 s results are read with this in mind.

1. **Notebook and report completed.** `01_eda.ipynb` now draws feature distributions at all four

timeouts (16 figures), correlation heatmaps for all four tasks (|rho| >= 0.9: 19 pairs in A1 and B, 27 in A2-VPN, 20 in A2-NonVPN, of 253), and the data funnel by class (`data_funnel_by_class_v1.png`), as the plan's Metrics section asks. Report section 2 gained the total funnel row, the per-class raw -> final -> train / val / test table, the value-range table and the 60 s finding.

1. **Note on the parser test:** the 3 rows per file in `tests/fixtures/hand_rows.json` were

extracted by script from the raw text (first, middle and last data line), not typed by hand. The test splits each raw line itself, independently of liac-arff, and compares every value and the label.

A.5 Every run

Generated by code from the run history, which holds every parameter, metric and tag of each run. One line per run, in start order. 2,013 runs: optuna_trial 1,400, holdout 419, paper_10fold 128, final_test 30, kaggle_isnan 22, kaggle_per_timeout 8, calibration 5, no protocol (setup smoke run) 1. Scores are macro F1 rounded to 4 decimals; a test score exists only for the 30 E12 runs (E16 scores on the Kaggle split are in its own tables).

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E_smoke	E_smoke						
E_pilot-tree-A1-15s	E_pilot	A1	tree	holdout	0.8884	0.8901	
E_pilot-tree-A1-15s	E_pilot	A1	tree	holdout	0.8884	0.8901	
E01-majority-A1-120s	E01	A1	majority	holdout	0.3413	0.3415	
E01-majority-A1-15s	E01	A1	majority	holdout	0.3414	0.3414	
E01-majority-A1-30s	E01	A1	majority	holdout	0.3371	0.337	
E01-majority-A1-60s	E01	A1	majority	holdout	0.3581	0.358	
E01-majority-A2-NonVPN-120s	E01	A2-NonVPN	majority	holdout	0.096	0.096	
E01-majority-A2-NonVPN-15s	E01	A2-NonVPN	majority	holdout	0.07	0.07	
E01-majority-A2-NonVPN-30s	E01	A2-NonVPN	majority	holdout	0.0777	0.0777	

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E01-majority-A2-NonVPN-60s	E01	A2-NonVPN	majority	holdout	0.0667	0.0667	
E01-majority-A2-VPN-120s	E01	A2-VPN	majority	holdout	0.093	0.093	
E01-majority-A2-VPN-15s	E01	A2-VPN	majority	holdout	0.0601	0.0601	
E01-majority-A2-VPN-30s	E01	A2-VPN	majority	holdout	0.0756	0.0756	
E01-majority-A2-VPN-60s	E01	A2-VPN	majority	holdout	0.0783	0.0782	
E01-majority-B-120s	E01	B	majority	holdout	0.0295	0.0295	
E01-majority-B-15s	E01	B	majority	holdout	0.0193	0.0193	
E01-majority-B-30s	E01	B	majority	holdout	0.0221	0.0221	
E01-majority-B-60s	E01	B	majority	holdout	0.0207	0.0207	
E02-tree-A1-120s	E02	A1	tree	holdout	0.8572	0.8571	
E02-tree-A1-15s	E02	A1	tree	holdout	0.8884	0.8901	
E02-tree-A1-30s	E02	A1	tree	holdout	0.8729	0.8643	
E02-tree-A1-60s	E02	A1	tree	holdout	0.8524	0.857	
E02-tree-A2-NonVPN-120s	E02	A2-NonVPN	tree	holdout	0.8065	0.8393	
E02-tree-A2-NonVPN-15s	E02	A2-NonVPN	tree	holdout	0.8619	0.8294	
E02-tree-A2-NonVPN-30s	E02	A2-NonVPN	tree	holdout	0.8171	0.8283	
E02-tree-A2-NonVPN-60s	E02	A2-NonVPN	tree	holdout	0.829	0.8246	
E02-tree-A2-VPN-120s	E02	A2-VPN	tree	holdout	0.7253	0.7795	
E02-tree-A2-VPN-15s	E02	A2-VPN	tree	holdout	0.8103	0.8247	
E02-tree-A2-VPN-30s	E02	A2-VPN	tree	holdout	0.7964	0.8276	
E02-tree-A2-VPN-60s	E02	A2-VPN	tree	holdout	0.7603	0.7745	
E02-tree-B-120s	E02	B	tree	holdout	0.6844	0.7158	
E02-tree-B-15s	E02	B	tree	holdout	0.7575	0.7585	
E02-tree-B-30s	E02	B	tree	holdout	0.7333	0.7476	
E02-tree-B-60s	E02	B	tree	holdout	0.7035	0.7058	
E03-knn-A1-120s	E03	A1	knn	holdout	0.8028	0.807	
E03-knn-A1-15s	E03	A1	knn	holdout	0.8211	0.8126	
E03-knn-A1-30s	E03	A1	knn	holdout	0.8138	0.8255	
E03-knn-A1-60s	E03	A1	knn	holdout	0.8082	0.8126	
E03-knn-A2-NonVPN-120s	E03	A2-NonVPN	knn	holdout	0.7312	0.7689	
E03-knn-A2-NonVPN-15s	E03	A2-NonVPN	knn	holdout	0.7921	0.7735	
E03-knn-A2-NonVPN-30s	E03	A2-NonVPN	knn	holdout	0.7832	0.7829	
E03-knn-A2-NonVPN-60s	E03	A2-NonVPN	knn	holdout	0.74	0.7498	
E03-knn-A2-VPN-120s	E03	A2-VPN	knn	holdout	0.6916	0.7266	
E03-knn-A2-VPN-15s	E03	A2-VPN	knn	holdout	0.7671	0.7743	
E03-knn-A2-VPN-30s	E03	A2-VPN	knn	holdout	0.7686	0.7881	
E03-knn-A2-VPN-60s	E03	A2-VPN	knn	holdout	0.7083	0.7298	
E03-knn-B-120s	E03	B	knn	holdout	0.6072	0.6372	
E03-knn-B-15s	E03	B	knn	holdout	0.6631	0.6605	
E03-knn-B-30s	E03	B	knn	holdout	0.6535	0.6466	
E03-knn-B-60s	E03	B	knn	holdout	0.6066	0.6171	

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E03-logreg-A1-120s	E03	A1	logreg	holdout	0.5692	0.5764	
E03-logreg-A1-15s	E03	A1	logreg	holdout	0.5996	0.5847	
E03-logreg-A1-30s	E03	A1	logreg	holdout	0.6003	0.5978	
E03-logreg-A1-60s	E03	A1	logreg	holdout	0.5509	0.5413	
E03-logreg-A2-NonVPN-120s	E03	A2-NonVPN	logreg	holdout	0.4724	0.4816	
E03-logreg-A2-NonVPN-15s	E03	A2-NonVPN	logreg	holdout	0.4593	0.4487	
E03-logreg-A2-NonVPN-30s	E03	A2-NonVPN	logreg	holdout	0.4586	0.4859	
E03-logreg-A2-NonVPN-60s	E03	A2-NonVPN	logreg	holdout	0.4539	0.4474	
E03-logreg-A2-VPN-120s	E03	A2-VPN	logreg	holdout	0.3434	0.3322	
E03-logreg-A2-VPN-15s	E03	A2-VPN	logreg	holdout	0.4105	0.4324	
E03-logreg-A2-VPN-30s	E03	A2-VPN	logreg	holdout	0.4946	0.5364	
E03-logreg-A2-VPN-60s	E03	A2-VPN	logreg	holdout	0.3339	0.3389	
E03-logreg-B-120s	E03	B	logreg	holdout	0.25	0.2696	
E03-logreg-B-15s	E03	B	logreg	holdout	0.2993	0.3072	
E03-logreg-B-30s	E03	B	logreg	holdout	0.3233	0.3196	
E03-logreg-B-60s	E03	B	logreg	holdout	0.275	0.271	
E03-svm_rbf-A1-120s	E03	A1	svm_rbf	holdout	0.608	0.6209	
E03-svm_rbf-A1-15s	E03	A1	svm_rbf	holdout	0.6081	0.6016	
E03-svm_rbf-A1-30s	E03	A1	svm_rbf	holdout	0.6192	0.6099	
E03-svm_rbf-A1-60s	E03	A1	svm_rbf	holdout	0.5519	0.5447	
E03-svm_rbf-A2-NonVPN-120s	E03	A2-NonVPN	svm_rbf	holdout	0.4408	0.4709	
E03-svm_rbf-A2-NonVPN-15s	E03	A2-NonVPN	svm_rbf	holdout	0.4787	0.487	
E03-svm_rbf-A2-NonVPN-30s	E03	A2-NonVPN	svm_rbf	holdout	0.426	0.4681	
E03-svm_rbf-A2-NonVPN-60s	E03	A2-NonVPN	svm_rbf	holdout	0.4317	0.4411	
E03-svm_rbf-A2-VPN-120s	E03	A2-VPN	svm_rbf	holdout	0.3175	0.332	
E03-svm_rbf-A2-VPN-15s	E03	A2-VPN	svm_rbf	holdout	0.4484	0.4682	
E03-svm_rbf-A2-VPN-30s	E03	A2-VPN	svm_rbf	holdout	0.5072	0.5421	
E03-svm_rbf-A2-VPN-60s	E03	A2-VPN	svm_rbf	holdout	0.3313	0.3296	
E03-svm_rbf-B-120s	E03	B	svm_rbf	holdout	0.2509	0.2735	
E03-svm_rbf-B-15s	E03	B	svm_rbf	holdout	0.2777	0.2965	
E03-svm_rbf-B-30s	E03	B	svm_rbf	holdout	0.2923	0.3106	
E03-svm_rbf-B-60s	E03	B	svm_rbf	holdout	0.2615	0.2782	
E02-paper-j48-published-A1-120s	E02	A1	tree	paper_10fold	0.8688		
E02-paper-j48-published-A1-15s	E02	A1	tree	paper_10fold	0.8888		
E02-paper-j48-published-A1-30s	E02	A1	tree	paper_10fold	0.8696		
E02-paper-j48-published-A1-60s	E02	A1	tree	paper_10fold	0.8562		
E02-paper-j48-published-A2-NonVPN-120s	E02	A2-NonVPN	tree	paper_10fold	0.8294		
E02-paper-j48-published-A2-NonVPN-15s	E02	A2-NonVPN	tree	paper_10fold	0.8731		
E02-paper-j48-published-A2-NonVPN-30s	E02	A2-NonVPN	tree	paper_10fold	0.8303		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E02-paper-j48-published-A2-NonVPN-60s	E02	A2-NonVPN	tree	paper_10fold	0.8359		
E02-paper-j48-published-A2-VPN-120s	E02	A2-VPN	tree	paper_10fold	0.779		
E02-paper-j48-published-A2-VPN-15s	E02	A2-VPN	tree	paper_10fold	0.836		
E02-paper-j48-published-A2-VPN-30s	E02	A2-VPN	tree	paper_10fold	0.8389		
E02-paper-j48-published-A2-VPN-60s	E02	A2-VPN	tree	paper_10fold	0.7741		
E02-paper-j48-published-B-120s	E02	B	tree	paper_10fold	0.7183		
E02-paper-j48-published-B-15s	E02	B	tree	paper_10fold	0.765		
E02-paper-j48-published-B-30s	E02	B	tree	paper_10fold	0.7535		
E02-paper-j48-published-B-60s	E02	B	tree	paper_10fold	0.7185		
E02-paper-j48-v1-A1-120s	E02	A1	tree	paper_10fold	0.8694		
E02-paper-j48-v1-A1-15s	E02	A1	tree	paper_10fold	0.8954		
E02-paper-j48-v1-A1-30s	E02	A1	tree	paper_10fold	0.8766		
E02-paper-j48-v1-A1-60s	E02	A1	tree	paper_10fold	0.8702		
E02-paper-j48-v1-A2-NonVPN-120s	E02	A2-NonVPN	tree	paper_10fold	0.8346		
E02-paper-j48-v1-A2-NonVPN-15s	E02	A2-NonVPN	tree	paper_10fold	0.8768		
E02-paper-j48-v1-A2-NonVPN-30s	E02	A2-NonVPN	tree	paper_10fold	0.8359		
E02-paper-j48-v1-A2-NonVPN-60s	E02	A2-NonVPN	tree	paper_10fold	0.8483		
E02-paper-j48-v1-A2-VPN-120s	E02	A2-VPN	tree	paper_10fold	0.7719		
E02-paper-j48-v1-A2-VPN-15s	E02	A2-VPN	tree	paper_10fold	0.8197		
E02-paper-j48-v1-A2-VPN-30s	E02	A2-VPN	tree	paper_10fold	0.821		
E02-paper-j48-v1-A2-VPN-60s	E02	A2-VPN	tree	paper_10fold	0.7702		
E02-paper-j48-v1-B-120s	E02	B	tree	paper_10fold	0.718		
E02-paper-j48-v1-B-15s	E02	B	tree	paper_10fold	0.7707		
E02-paper-j48-v1-B-30s	E02	B	tree	paper_10fold	0.7488		
E02-paper-j48-v1-B-60s	E02	B	tree	paper_10fold	0.7293		
E03-paper-ibk-published-A1-120s	E03	A1	knn_minmax	paper_10fold	0.8146		
E03-paper-ibk-published-A1-15s	E03	A1	knn_minmax	paper_10fold	0.8424		
E03-paper-ibk-published-A1-30s	E03	A1	knn_minmax	paper_10fold	0.8276		
E03-paper-ibk-published-A1-60s	E03	A1	knn_minmax	paper_10fold	0.8217		
E03-paper-ibk-published-A2-NonVPN-120s	E03	A2-NonVPN	knn_minmax	paper_10fold	0.7349		
E03-paper-ibk-published-A2-NonVPN-15s	E03	A2-NonVPN	knn_minmax	paper_10fold	0.7949		
E03-paper-ibk-published-A2-NonVPN-30s	E03	A2-NonVPN	knn_minmax	paper_10fold	0.7769		
E03-paper-ibk-published-A2-NonVPN-60s	E03	A2-NonVPN	knn_minmax	paper_10fold	0.7687		
E03-paper-ibk-published-A2-VPN-120s	E03	A2-VPN	knn_minmax	paper_10fold	0.7611		
E03-paper-ibk-published-A2-VPN-15s	E03	A2-VPN	knn_minmax	paper_10fold	0.7766		
E03-paper-ibk-published-A2-VPN-30s	E03	A2-VPN	knn_minmax	paper_10fold	0.8047		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E03-paper-ibk-published-A2-VPN-60s	E03	A2-VPN	knn_minmax	paper_10fold	0.7377		
E03-paper-ibk-published-B-120s	E03	B	knn_minmax	paper_10fold	0.6531		
E03-paper-ibk-published-B-15s	E03	B	knn_minmax	paper_10fold	0.69		
E03-paper-ibk-published-B-30s	E03	B	knn_minmax	paper_10fold	0.6958		
E03-paper-ibk-published-B-60s	E03	B	knn_minmax	paper_10fold	0.6561		
E03-paper-ibk-v1-A1-120s	E03	A1	knn_minmax	paper_10fold	0.8218		
E03-paper-ibk-v1-A1-15s	E03	A1	knn_minmax	paper_10fold	0.8524		
E03-paper-ibk-v1-A1-30s	E03	A1	knn_minmax	paper_10fold	0.8331		
E03-paper-ibk-v1-A1-60s	E03	A1	knn_minmax	paper_10fold	0.8326		
E03-paper-ibk-v1-A2-NonVPN-120s	E03	A2-NonVPN	knn_minmax	paper_10fold	0.7751		
E03-paper-ibk-v1-A2-NonVPN-15s	E03	A2-NonVPN	knn_minmax	paper_10fold	0.8118		
E03-paper-ibk-v1-A2-NonVPN-30s	E03	A2-NonVPN	knn_minmax	paper_10fold	0.7989		
E03-paper-ibk-v1-A2-NonVPN-60s	E03	A2-NonVPN	knn_minmax	paper_10fold	0.7899		
E03-paper-ibk-v1-A2-VPN-120s	E03	A2-VPN	knn_minmax	paper_10fold	0.7441		
E03-paper-ibk-v1-A2-VPN-15s	E03	A2-VPN	knn_minmax	paper_10fold	0.793		
E03-paper-ibk-v1-A2-VPN-30s	E03	A2-VPN	knn_minmax	paper_10fold	0.81		
E03-paper-ibk-v1-A2-VPN-60s	E03	A2-VPN	knn_minmax	paper_10fold	0.7552		
E03-paper-ibk-v1-B-120s	E03	B	knn_minmax	paper_10fold	0.661		
E03-paper-ibk-v1-B-15s	E03	B	knn_minmax	paper_10fold	0.7071		
E03-paper-ibk-v1-B-30s	E03	B	knn_minmax	paper_10fold	0.6931		
E03-paper-ibk-v1-B-60s	E03	B	knn_minmax	paper_10fold	0.6697		
E01-majority-A1-120s	E01	A1	majority	holdout	0.3413	0.3415	
E01-majority-A1-15s	E01	A1	majority	holdout	0.3414	0.3414	
E01-majority-A1-30s	E01	A1	majority	holdout	0.3371	0.337	
E01-majority-A1-60s	E01	A1	majority	holdout	0.3581	0.358	
E01-majority-A2-NonVPN-120s	E01	A2-NonVPN	majority	holdout	0.096	0.096	
E01-majority-A2-NonVPN-15s	E01	A2-NonVPN	majority	holdout	0.07	0.07	
E01-majority-A2-NonVPN-30s	E01	A2-NonVPN	majority	holdout	0.0777	0.0777	
E01-majority-A2-NonVPN-60s	E01	A2-NonVPN	majority	holdout	0.0667	0.0667	
E01-majority-A2-VPN-120s	E01	A2-VPN	majority	holdout	0.093	0.093	
E01-majority-A2-VPN-15s	E01	A2-VPN	majority	holdout	0.0601	0.0601	
E01-majority-A2-VPN-30s	E01	A2-VPN	majority	holdout	0.0756	0.0756	
E01-majority-A2-VPN-60s	E01	A2-VPN	majority	holdout	0.0783	0.0782	
E01-majority-B-120s	E01	B	majority	holdout	0.0295	0.0295	
E01-majority-B-15s	E01	B	majority	holdout	0.0193	0.0193	
E01-majority-B-30s	E01	B	majority	holdout	0.0221	0.0221	
E01-majority-B-60s	E01	B	majority	holdout	0.0207	0.0207	
E02-tree-A1-120s	E02	A1	tree	holdout	0.8572	0.8571	
E02-tree-A1-15s	E02	A1	tree	holdout	0.8884	0.8901	
E02-tree-A1-30s	E02	A1	tree	holdout	0.8729	0.8643	
E02-tree-A1-60s	E02	A1	tree	holdout	0.8524	0.857	

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E02-tree-A2-NonVPN-120s	E02	A2-NonVPN	tree	holdout	0.8065	0.8393	
E02-tree-A2-NonVPN-15s	E02	A2-NonVPN	tree	holdout	0.8619	0.8294	
E02-tree-A2-NonVPN-30s	E02	A2-NonVPN	tree	holdout	0.8171	0.8283	
E02-tree-A2-NonVPN-60s	E02	A2-NonVPN	tree	holdout	0.829	0.8246	
E02-tree-A2-VPN-120s	E02	A2-VPN	tree	holdout	0.7253	0.7795	
E02-tree-A2-VPN-15s	E02	A2-VPN	tree	holdout	0.8103	0.8247	
E02-tree-A2-VPN-30s	E02	A2-VPN	tree	holdout	0.7964	0.8276	
E02-tree-A2-VPN-60s	E02	A2-VPN	tree	holdout	0.7603	0.7745	
E02-tree-B-120s	E02	B	tree	holdout	0.6844	0.7158	
E02-tree-B-15s	E02	B	tree	holdout	0.7575	0.7585	
E02-tree-B-30s	E02	B	tree	holdout	0.7333	0.7476	
E02-tree-B-60s	E02	B	tree	holdout	0.7035	0.7058	
E03-knn-A1-120s	E03	A1	knn	holdout	0.8028	0.807	
E03-knn-A1-15s	E03	A1	knn	holdout	0.8211	0.8126	
E03-knn-A1-30s	E03	A1	knn	holdout	0.8138	0.8255	
E03-knn-A1-60s	E03	A1	knn	holdout	0.8082	0.8126	
E03-knn-A2-NonVPN-120s	E03	A2-NonVPN	knn	holdout	0.7312	0.7689	
E03-knn-A2-NonVPN-15s	E03	A2-NonVPN	knn	holdout	0.7921	0.7735	
E03-knn-A2-NonVPN-30s	E03	A2-NonVPN	knn	holdout	0.7832	0.7829	
E03-knn-A2-NonVPN-60s	E03	A2-NonVPN	knn	holdout	0.74	0.7498	
E03-knn-A2-VPN-120s	E03	A2-VPN	knn	holdout	0.6916	0.7266	
E03-knn-A2-VPN-15s	E03	A2-VPN	knn	holdout	0.7671	0.7743	
E03-knn-A2-VPN-30s	E03	A2-VPN	knn	holdout	0.7686	0.7881	
E03-knn-A2-VPN-60s	E03	A2-VPN	knn	holdout	0.7083	0.7298	
E03-knn-B-120s	E03	B	knn	holdout	0.6072	0.6372	
E03-knn-B-15s	E03	B	knn	holdout	0.6631	0.6605	
E03-knn-B-30s	E03	B	knn	holdout	0.6535	0.6466	
E03-knn-B-60s	E03	B	knn	holdout	0.6066	0.6171	
E03-logreg-A1-120s	E03	A1	logreg	holdout	0.5692	0.5764	
E03-logreg-A1-15s	E03	A1	logreg	holdout	0.5996	0.5847	
E03-logreg-A1-30s	E03	A1	logreg	holdout	0.6003	0.5978	
E03-logreg-A1-60s	E03	A1	logreg	holdout	0.5509	0.5413	
E03-logreg-A2-NonVPN-120s	E03	A2-NonVPN	logreg	holdout	0.4724	0.4816	
E03-logreg-A2-NonVPN-15s	E03	A2-NonVPN	logreg	holdout	0.4593	0.4487	
E03-logreg-A2-NonVPN-30s	E03	A2-NonVPN	logreg	holdout	0.4586	0.4859	
E03-logreg-A2-NonVPN-60s	E03	A2-NonVPN	logreg	holdout	0.4539	0.4474	
E03-logreg-A2-VPN-120s	E03	A2-VPN	logreg	holdout	0.3434	0.3322	
E03-logreg-A2-VPN-15s	E03	A2-VPN	logreg	holdout	0.4105	0.4324	
E03-logreg-A2-VPN-30s	E03	A2-VPN	logreg	holdout	0.4946	0.5364	

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E03-logreg-A2-VPN-60s	E03	A2-VPN	logreg	holdout	0.3339	0.3389	
E03-logreg-B-120s	E03	B	logreg	holdout	0.25	0.2696	
E03-logreg-B-15s	E03	B	logreg	holdout	0.2993	0.3072	
E03-logreg-B-30s	E03	B	logreg	holdout	0.3233	0.3196	
E03-logreg-B-60s	E03	B	logreg	holdout	0.275	0.271	
E03-svm_rbf-A1-120s	E03	A1	svm_rbf	holdout	0.608	0.6209	
E03-svm_rbf-A1-15s	E03	A1	svm_rbf	holdout	0.6081	0.6016	
E03-svm_rbf-A1-30s	E03	A1	svm_rbf	holdout	0.6192	0.6099	
E03-svm_rbf-A1-60s	E03	A1	svm_rbf	holdout	0.5519	0.5447	
E03-svm_rbf-A2-NonVPN-120s	E03	A2-NonVPN	svm_rbf	holdout	0.4408	0.4709	
E03-svm_rbf-A2-NonVPN-15s	E03	A2-NonVPN	svm_rbf	holdout	0.4787	0.487	
E03-svm_rbf-A2-NonVPN-30s	E03	A2-NonVPN	svm_rbf	holdout	0.426	0.4681	
E03-svm_rbf-A2-NonVPN-60s	E03	A2-NonVPN	svm_rbf	holdout	0.4317	0.4411	
E03-svm_rbf-A2-VPN-120s	E03	A2-VPN	svm_rbf	holdout	0.3175	0.332	
E03-svm_rbf-A2-VPN-15s	E03	A2-VPN	svm_rbf	holdout	0.4484	0.4682	
E03-svm_rbf-A2-VPN-30s	E03	A2-VPN	svm_rbf	holdout	0.5072	0.5421	
E03-svm_rbf-A2-VPN-60s	E03	A2-VPN	svm_rbf	holdout	0.3313	0.3296	
E03-svm_rbf-B-120s	E03	B	svm_rbf	holdout	0.2509	0.2735	
E03-svm_rbf-B-15s	E03	B	svm_rbf	holdout	0.2777	0.2965	
E03-svm_rbf-B-30s	E03	B	svm_rbf	holdout	0.2923	0.3106	
E03-svm_rbf-B-60s	E03	B	svm_rbf	holdout	0.2615	0.2782	
E02-paper-j48-published-A1-120s	E02	A1	tree	paper_10fold	0.8688		
E02-paper-j48-published-A1-15s	E02	A1	tree	paper_10fold	0.8888		
E02-paper-j48-published-A1-30s	E02	A1	tree	paper_10fold	0.8696		
E02-paper-j48-published-A1-60s	E02	A1	tree	paper_10fold	0.8562		
E02-paper-j48-published-A2-NonVPN-120s	E02	A2-NonVPN	tree	paper_10fold	0.8294		
E02-paper-j48-published-A2-NonVPN-15s	E02	A2-NonVPN	tree	paper_10fold	0.8731		
E02-paper-j48-published-A2-NonVPN-30s	E02	A2-NonVPN	tree	paper_10fold	0.8303		
E02-paper-j48-published-A2-NonVPN-60s	E02	A2-NonVPN	tree	paper_10fold	0.8359		
E02-paper-j48-published-A2-VPN-120s	E02	A2-VPN	tree	paper_10fold	0.779		
E02-paper-j48-published-A2-VPN-15s	E02	A2-VPN	tree	paper_10fold	0.836		
E02-paper-j48-published-A2-VPN-30s	E02	A2-VPN	tree	paper_10fold	0.8389		
E02-paper-j48-published-A2-VPN-60s	E02	A2-VPN	tree	paper_10fold	0.7741		
E02-paper-j48-published-B-120s	E02	B	tree	paper_10fold	0.7183		
E02-paper-j48-published-B-15s	E02	B	tree	paper_10fold	0.765		
E02-paper-j48-published-B-30s	E02	B	tree	paper_10fold	0.7535		
E02-paper-j48-published-B-60s	E02	B	tree	paper_10fold	0.7185		
E02-paper-j48-v1-A1-120s	E02	A1	tree	paper_10fold	0.8694		
E02-paper-j48-v1-A1-15s	E02	A1	tree	paper_10fold	0.8954		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E02-paper-j48-v1-A1-30s	E02	A1	tree	paper_10fold	0.8766		
E02-paper-j48-v1-A1-60s	E02	A1	tree	paper_10fold	0.8702		
E02-paper-j48-v1-A2-NonVPN-120s	E02	A2-NonVPN	tree	paper_10fold	0.8346		
E02-paper-j48-v1-A2-NonVPN-15s	E02	A2-NonVPN	tree	paper_10fold	0.8768		
E02-paper-j48-v1-A2-NonVPN-30s	E02	A2-NonVPN	tree	paper_10fold	0.8359		
E02-paper-j48-v1-A2-NonVPN-60s	E02	A2-NonVPN	tree	paper_10fold	0.8483		
E02-paper-j48-v1-A2-VPN-120s	E02	A2-VPN	tree	paper_10fold	0.7719		
E02-paper-j48-v1-A2-VPN-15s	E02	A2-VPN	tree	paper_10fold	0.8197		
E02-paper-j48-v1-A2-VPN-30s	E02	A2-VPN	tree	paper_10fold	0.821		
E02-paper-j48-v1-A2-VPN-60s	E02	A2-VPN	tree	paper_10fold	0.7702		
E02-paper-j48-v1-B-120s	E02	B	tree	paper_10fold	0.718		
E02-paper-j48-v1-B-15s	E02	B	tree	paper_10fold	0.7707		
E02-paper-j48-v1-B-30s	E02	B	tree	paper_10fold	0.7488		
E02-paper-j48-v1-B-60s	E02	B	tree	paper_10fold	0.7293		
E03-paper-ibk-published-A1-120s	E03	A1	knn_minmax	paper_10fold	0.8146		
E03-paper-ibk-published-A1-15s	E03	A1	knn_minmax	paper_10fold	0.8424		
E03-paper-ibk-published-A1-30s	E03	A1	knn_minmax	paper_10fold	0.8276		
E03-paper-ibk-published-A1-60s	E03	A1	knn_minmax	paper_10fold	0.8217		
E03-paper-ibk-published-A2-NonVPN-120s	E03	A2-NonVPN	knn_minmax	paper_10fold	0.7349		
E03-paper-ibk-published-A2-NonVPN-15s	E03	A2-NonVPN	knn_minmax	paper_10fold	0.7949		
E03-paper-ibk-published-A2-NonVPN-30s	E03	A2-NonVPN	knn_minmax	paper_10fold	0.7769		
E03-paper-ibk-published-A2-NonVPN-60s	E03	A2-NonVPN	knn_minmax	paper_10fold	0.7687		
E03-paper-ibk-published-A2-VPN-120s	E03	A2-VPN	knn_minmax	paper_10fold	0.7611		
E03-paper-ibk-published-A2-VPN-15s	E03	A2-VPN	knn_minmax	paper_10fold	0.7766		
E03-paper-ibk-published-A2-VPN-30s	E03	A2-VPN	knn_minmax	paper_10fold	0.8047		
E03-paper-ibk-published-A2-VPN-60s	E03	A2-VPN	knn_minmax	paper_10fold	0.7377		
E03-paper-ibk-published-B-120s	E03	B	knn_minmax	paper_10fold	0.6531		
E03-paper-ibk-published-B-15s	E03	B	knn_minmax	paper_10fold	0.69		
E03-paper-ibk-published-B-30s	E03	B	knn_minmax	paper_10fold	0.6958		
E03-paper-ibk-published-B-60s	E03	B	knn_minmax	paper_10fold	0.6561		
E03-paper-ibk-v1-A1-120s	E03	A1	knn_minmax	paper_10fold	0.8218		
E03-paper-ibk-v1-A1-15s	E03	A1	knn_minmax	paper_10fold	0.8524		
E03-paper-ibk-v1-A1-30s	E03	A1	knn_minmax	paper_10fold	0.8331		
E03-paper-ibk-v1-A1-60s	E03	A1	knn_minmax	paper_10fold	0.8326		
E03-paper-ibk-v1-A2-NonVPN-120s	E03	A2-NonVPN	knn_minmax	paper_10fold	0.7751		
E03-paper-ibk-v1-A2-NonVPN-15s	E03	A2-NonVPN	knn_minmax	paper_10fold	0.8118		
E03-paper-ibk-v1-A2-NonVPN-30s	E03	A2-NonVPN	knn_minmax	paper_10fold	0.7989		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E03-paper-ibk-v1-A2-NonVPN-60s	E03	A2-NonVPN	knn_minmax	paper_10fold	0.7899		
E03-paper-ibk-v1-A2-VPN-120s	E03	A2-VPN	knn_minmax	paper_10fold	0.7441		
E03-paper-ibk-v1-A2-VPN-15s	E03	A2-VPN	knn_minmax	paper_10fold	0.793		
E03-paper-ibk-v1-A2-VPN-30s	E03	A2-VPN	knn_minmax	paper_10fold	0.81		
E03-paper-ibk-v1-A2-VPN-60s	E03	A2-VPN	knn_minmax	paper_10fold	0.7552		
E03-paper-ibk-v1-B-120s	E03	B	knn_minmax	paper_10fold	0.661		
E03-paper-ibk-v1-B-15s	E03	B	knn_minmax	paper_10fold	0.7071		
E03-paper-ibk-v1-B-30s	E03	B	knn_minmax	paper_10fold	0.6931		
E03-paper-ibk-v1-B-60s	E03	B	knn_minmax	paper_10fold	0.6697		
E04-knn_log_standard-A1-15s	E04	A1	knn_log_standard	holdout	0.9033	0.9048	
E04-knn_log_robust-A1-15s	E04	A1	knn_log_robust	holdout	0.8874	0.8929	
E04-knn_log_robust-A2-NonVPN-15s	E04	A2-NonVPN	knn_log_robust	holdout	0.8688	0.854	
E04-knn_log_robust-A2-VPN-15s	E04	A2-VPN	knn_log_robust	holdout	0.8185	0.8372	
E04-knn_log_robust-B-15s	E04	B	knn_log_robust	holdout	0.7555	0.742	
E04-knn_log_standard-A1-15s	E04	A1	knn_log_standard	holdout	0.9033	0.9048	
E04-knn_log_standard-A2-NonVPN-15s	E04	A2-NonVPN	knn_log_standard	holdout	0.8861	0.867	
E04-knn_log_standard-A2-VPN-15s	E04	A2-VPN	knn_log_standard	holdout	0.8336	0.8544	
E04-knn_log_standard-B-15s	E04	B	knn_log_standard	holdout	0.7798	0.7644	
E04-knn_log_standard-A2-NonVPN-30s	E04	A2-NonVPN	knn_log_standard	holdout	0.8487	0.8523	
E04-knn_log_standard-A2-NonVPN-60s	E04	A2-NonVPN	knn_log_standard	holdout	0.8638	0.859	
E04-knn_log_standard-A2-NonVPN-120s	E04	A2-NonVPN	knn_log_standard	holdout	0.8243	0.8491	
E05-rf-A1-15s	E05	A1	rf	holdout	0.9172	0.9203	
E05-rf-A2-VPN-15s	E05	A2-VPN	rf	holdout	0.8637	0.8679	
E05-rf-A2-NonVPN-15s	E05	A2-NonVPN	rf	holdout	0.9048	0.8941	
E05-rf-B-15s	E05	B	rf	holdout	0.8102	0.8147	
E05-rf-B-30s	E05	B	rf	holdout	0.7979	0.8114	
E05-rf-B-60s	E05	B	rf	holdout	0.7656	0.7771	
E05-rf-B-120s	E05	B	rf	holdout	0.7562	0.7835	
E06-xgb-A1-15s	E06	A1	xgb	holdout	0.9251	0.9271	
E06-lgbm-A1-15s	E06	A1	lgbm	holdout	0.9175	0.9181	
E06-lgbm-A1-15s	E06	A1	lgbm	holdout	0.9175	0.9181	
E06-lgbm-A2-NonVPN-15s	E06	A2-NonVPN	lgbm	holdout	0.9224	0.8983	
E06-lgbm-A2-VPN-15s	E06	A2-VPN	lgbm	holdout	0.8827	0.8694	
E06-lgbm-B-15s	E06	B	lgbm	holdout	0.8305	0.8201	
E06-xgb-A1-15s	E06	A1	xgb	holdout	0.9251	0.9271	
E06-xgb-A2-NonVPN-15s	E06	A2-NonVPN	xgb	holdout	0.9237	0.9015	
E06-xgb-A2-VPN-15s	E06	A2-VPN	xgb	holdout	0.8799	0.8758	
E06-xgb-B-15s	E06	B	xgb	holdout	0.8334	0.8303	
E07-knn_log_standard-A1-15s	E07	A1	knn_log_standard	holdout	0.9122	0.9163	
E07-knn_log_standard-A1-15s-trial-000	E07	A1	knn_log_standard	optuna_trial	0.88		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E07-knn_log_standard-A1-15s-trial-001	E07	A1	knn_log_standard	optuna_trial	0.9075		
E07-knn_log_standard-A1-15s-trial-002	E07	A1	knn_log_standard	optuna_trial	0.8959		
E07-knn_log_standard-A1-15s-trial-003	E07	A1	knn_log_standard	optuna_trial	0.9034		
E07-knn_log_standard-A1-15s-trial-004	E07	A1	knn_log_standard	optuna_trial	0.8934		
E07-knn_log_standard-A1-15s-trial-005	E07	A1	knn_log_standard	optuna_trial	0.8789		
E07-knn_log_standard-A1-15s-trial-006	E07	A1	knn_log_standard	optuna_trial	0.8738		
E07-knn_log_standard-A1-15s-trial-007	E07	A1	knn_log_standard	optuna_trial	0.8475		
E07-knn_log_standard-A1-15s-trial-008	E07	A1	knn_log_standard	optuna_trial	0.8992		
E07-knn_log_standard-A1-15s-trial-009	E07	A1	knn_log_standard	optuna_trial	0.8842		
E07-knn_log_standard-A1-15s-trial-010	E07	A1	knn_log_standard	optuna_trial	0.9114		
E07-knn_log_standard-A1-15s-trial-011	E07	A1	knn_log_standard	optuna_trial	0.9114		
E07-knn_log_standard-A1-15s-trial-012	E07	A1	knn_log_standard	optuna_trial	0.9002		
E07-knn_log_standard-A1-15s-trial-013	E07	A1	knn_log_standard	optuna_trial	0.9114		
E07-knn_log_standard-A1-15s-trial-014	E07	A1	knn_log_standard	optuna_trial	0.8937		
E07-knn_log_standard-A1-15s-trial-015	E07	A1	knn_log_standard	optuna_trial	0.9084		
E07-knn_log_standard-A1-15s-trial-016	E07	A1	knn_log_standard	optuna_trial	0.9032		
E07-knn_log_standard-A1-15s-trial-017	E07	A1	knn_log_standard	optuna_trial	0.8824		
E07-knn_log_standard-A1-15s-trial-018	E07	A1	knn_log_standard	optuna_trial	0.9114		
E07-knn_log_standard-A1-15s-trial-019	E07	A1	knn_log_standard	optuna_trial	0.8956		
E07-knn_log_standard-A1-15s-trial-020	E07	A1	knn_log_standard	optuna_trial	0.8851		
E07-knn_log_standard-A1-15s-trial-021	E07	A1	knn_log_standard	optuna_trial	0.9114		
E07-knn_log_standard-A1-15s-trial-022	E07	A1	knn_log_standard	optuna_trial	0.9122		
E07-knn_log_standard-A1-15s-trial-023	E07	A1	knn_log_standard	optuna_trial	0.9109		
E07-knn_log_standard-A1-15s-trial-024	E07	A1	knn_log_standard	optuna_trial	0.8993		
E07-knn_log_standard-A1-15s-trial-025	E07	A1	knn_log_standard	optuna_trial	0.9108		
E07-knn_log_standard-A1-15s-trial-026	E07	A1	knn_log_standard	optuna_trial	0.8551		
E07-knn_log_standard-A1-15s-trial-027	E07	A1	knn_log_standard	optuna_trial	0.896		
E07-knn_log_standard-A1-15s-trial-028	E07	A1	knn_log_standard	optuna_trial	0.9035		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E07-knn_log_standard-A1-15s-trial-029	E07	A1	knn_log_standard	optuna_trial	0.8836		
E07-knn_log_standard-A1-15s-trial-030	E07	A1	knn_log_standard	optuna_trial	0.8712		
E07-knn_log_standard-A1-15s-trial-031	E07	A1	knn_log_standard	optuna_trial	0.9108		
E07-knn_log_standard-A1-15s-trial-032	E07	A1	knn_log_standard	optuna_trial	0.9075		
E07-knn_log_standard-A1-15s-trial-033	E07	A1	knn_log_standard	optuna_trial	0.9029		
E07-knn_log_standard-A1-15s-trial-034	E07	A1	knn_log_standard	optuna_trial	0.8993		
E07-knn_log_standard-A1-15s-trial-035	E07	A1	knn_log_standard	optuna_trial	0.9122		
E07-knn_log_standard-A1-15s-trial-036	E07	A1	knn_log_standard	optuna_trial	0.9059		
E07-knn_log_standard-A1-15s-trial-037	E07	A1	knn_log_standard	optuna_trial	0.9109		
E07-knn_log_standard-A1-15s-trial-038	E07	A1	knn_log_standard	optuna_trial	0.9016		
E07-knn_log_standard-A1-15s-trial-039	E07	A1	knn_log_standard	optuna_trial	0.8917		
E07-knn_log_standard-A1-15s-trial-040	E07	A1	knn_log_standard	optuna_trial	0.9054		
E07-knn_log_standard-A1-15s-trial-041	E07	A1	knn_log_standard	optuna_trial	0.9114		
E07-knn_log_standard-A1-15s-trial-042	E07	A1	knn_log_standard	optuna_trial	0.9122		
E07-knn_log_standard-A1-15s-trial-043	E07	A1	knn_log_standard	optuna_trial	0.9122		
E07-knn_log_standard-A1-15s-trial-044	E07	A1	knn_log_standard	optuna_trial	0.9122		
E07-knn_log_standard-A1-15s-trial-045	E07	A1	knn_log_standard	optuna_trial	0.9029		
E07-knn_log_standard-A1-15s-trial-046	E07	A1	knn_log_standard	optuna_trial	0.9049		
E07-knn_log_standard-A1-15s-trial-047	E07	A1	knn_log_standard	optuna_trial	0.9091		
E07-knn_log_standard-A1-15s-trial-048	E07	A1	knn_log_standard	optuna_trial	0.8356		
E07-knn_log_standard-A1-15s-trial-049	E07	A1	knn_log_standard	optuna_trial	0.9122		
E07-knn_log_standard-A2-VPN-15s	E07	A2-VPN	knn_log_standard	holdout	0.8575	0.8625	
E07-knn_log_standard-A2-VPN-15s-trial-000	E07	A2-VPN	knn_log_standard	optuna_trial	0.7916		
E07-knn_log_standard-A2-VPN-15s-trial-001	E07	A2-VPN	knn_log_standard	optuna_trial	0.853		
E07-knn_log_standard-A2-VPN-15s-trial-002	E07	A2-VPN	knn_log_standard	optuna_trial	0.8407		
E07-knn_log_standard-A2-VPN-15s-trial-003	E07	A2-VPN	knn_log_standard	optuna_trial	0.8449		
E07-knn_log_standard-A2-VPN-15s-trial-004	E07	A2-VPN	knn_log_standard	optuna_trial	0.8244		
E07-knn_log_standard-A2-VPN-15s-trial-005	E07	A2-VPN	knn_log_standard	optuna_trial	0.8053		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E07-knn_log_standard-A2-VPN-15s-trial-006	E07	A2-VPN	knn_log_standard	optuna_trial	0.7723		
E07-knn_log_standard-A2-VPN-15s-trial-007	E07	A2-VPN	knn_log_standard	optuna_trial	0.7538		
E07-knn_log_standard-A2-VPN-15s-trial-008	E07	A2-VPN	knn_log_standard	optuna_trial	0.8306		
E07-knn_log_standard-A2-VPN-15s-trial-009	E07	A2-VPN	knn_log_standard	optuna_trial	0.8138		
E07-knn_log_standard-A2-VPN-15s-trial-010	E07	A2-VPN	knn_log_standard	optuna_trial	0.8565		
E07-knn_log_standard-A2-VPN-15s-trial-011	E07	A2-VPN	knn_log_standard	optuna_trial	0.8565		
E07-knn_log_standard-A2-VPN-15s-trial-012	E07	A2-VPN	knn_log_standard	optuna_trial	0.8398		
E07-knn_log_standard-A2-VPN-15s-trial-013	E07	A2-VPN	knn_log_standard	optuna_trial	0.8565		
E07-knn_log_standard-A2-VPN-15s-trial-014	E07	A2-VPN	knn_log_standard	optuna_trial	0.8218		
E07-knn_log_standard-A2-VPN-15s-trial-015	E07	A2-VPN	knn_log_standard	optuna_trial	0.8572		
E07-knn_log_standard-A2-VPN-15s-trial-016	E07	A2-VPN	knn_log_standard	optuna_trial	0.8575		
E07-knn_log_standard-A2-VPN-15s-trial-017	E07	A2-VPN	knn_log_standard	optuna_trial	0.829		
E07-knn_log_standard-A2-VPN-15s-trial-018	E07	A2-VPN	knn_log_standard	optuna_trial	0.8575		
E07-knn_log_standard-A2-VPN-15s-trial-019	E07	A2-VPN	knn_log_standard	optuna_trial	0.8575		
E07-knn_log_standard-A2-VPN-15s-trial-020	E07	A2-VPN	knn_log_standard	optuna_trial	0.8357		
E07-knn_log_standard-A2-VPN-15s-trial-021	E07	A2-VPN	knn_log_standard	optuna_trial	0.8575		
E07-knn_log_standard-A2-VPN-15s-trial-022	E07	A2-VPN	knn_log_standard	optuna_trial	0.8514		
E07-knn_log_standard-A2-VPN-15s-trial-023	E07	A2-VPN	knn_log_standard	optuna_trial	0.8051		
E07-knn_log_standard-A2-VPN-15s-trial-024	E07	A2-VPN	knn_log_standard	optuna_trial	0.8237		
E07-knn_log_standard-A2-VPN-15s-trial-025	E07	A2-VPN	knn_log_standard	optuna_trial	0.8514		
E07-knn_log_standard-A2-VPN-15s-trial-026	E07	A2-VPN	knn_log_standard	optuna_trial	0.8419		
E07-knn_log_standard-A2-VPN-15s-trial-027	E07	A2-VPN	knn_log_standard	optuna_trial	0.8557		
E07-knn_log_standard-A2-VPN-15s-trial-028	E07	A2-VPN	knn_log_standard	optuna_trial	0.8055		
E07-knn_log_standard-A2-VPN-15s-trial-029	E07	A2-VPN	knn_log_standard	optuna_trial	0.8464		
E07-knn_log_standard-A2-VPN-15s-trial-030	E07	A2-VPN	knn_log_standard	optuna_trial	0.7797		
E07-knn_log_standard-A2-VPN-15s-trial-031	E07	A2-VPN	knn_log_standard	optuna_trial	0.8575		
E07-knn_log_standard-A2-VPN-15s-trial-032	E07	A2-VPN	knn_log_standard	optuna_trial	0.8575		
E07-knn_log_standard-A2-VPN-15s-trial-033	E07	A2-VPN	knn_log_standard	optuna_trial	0.854		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E07-knn_log_standard-A2-VPN-15s-trial-034	E07	A2-VPN	knn_log_standard	optuna_trial	0.851		
E07-knn_log_standard-A2-VPN-15s-trial-035	E07	A2-VPN	knn_log_standard	optuna_trial	0.847		
E07-knn_log_standard-A2-VPN-15s-trial-036	E07	A2-VPN	knn_log_standard	optuna_trial	0.8003		
E07-knn_log_standard-A2-VPN-15s-trial-037	E07	A2-VPN	knn_log_standard	optuna_trial	0.8434		
E07-knn_log_standard-A2-VPN-15s-trial-038	E07	A2-VPN	knn_log_standard	optuna_trial	0.8557		
E07-knn_log_standard-A2-VPN-15s-trial-039	E07	A2-VPN	knn_log_standard	optuna_trial	0.8357		
E07-knn_log_standard-A2-VPN-15s-trial-040	E07	A2-VPN	knn_log_standard	optuna_trial	0.8204		
E07-knn_log_standard-A2-VPN-15s-trial-041	E07	A2-VPN	knn_log_standard	optuna_trial	0.8575		
E07-knn_log_standard-A2-VPN-15s-trial-042	E07	A2-VPN	knn_log_standard	optuna_trial	0.8557		
E07-knn_log_standard-A2-VPN-15s-trial-043	E07	A2-VPN	knn_log_standard	optuna_trial	0.8575		
E07-knn_log_standard-A2-VPN-15s-trial-044	E07	A2-VPN	knn_log_standard	optuna_trial	0.8507		
E07-knn_log_standard-A2-VPN-15s-trial-045	E07	A2-VPN	knn_log_standard	optuna_trial	0.8413		
E07-knn_log_standard-A2-VPN-15s-trial-046	E07	A2-VPN	knn_log_standard	optuna_trial	0.8407		
E07-knn_log_standard-A2-VPN-15s-trial-047	E07	A2-VPN	knn_log_standard	optuna_trial	0.8449		
E07-knn_log_standard-A2-VPN-15s-trial-048	E07	A2-VPN	knn_log_standard	optuna_trial	0.8575		
E07-knn_log_standard-A2-VPN-15s-trial-049	E07	A2-VPN	knn_log_standard	optuna_trial	0.7507		
E07-knn_log_standard-A2-NonVPN-15s	E07	A2-NonVPN	knn_log_standard	holdout	0.906	0.8929	
E07-knn_log_standard-A2-NonVPN-15s-trial-000	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.8271		
E07-knn_log_standard-A2-NonVPN-15s-trial-001	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.8946		
E07-knn_log_standard-A2-NonVPN-15s-trial-002	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.8986		
E07-knn_log_standard-A2-NonVPN-15s-trial-003	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.8912		
E07-knn_log_standard-A2-NonVPN-15s-trial-004	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.853		
E07-knn_log_standard-A2-NonVPN-15s-trial-005	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.8439		
E07-knn_log_standard-A2-NonVPN-15s-trial-006	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.7806		
E07-knn_log_standard-A2-NonVPN-15s-trial-007	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.7733		
E07-knn_log_standard-A2-NonVPN-15s-trial-008	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.8723		
E07-knn_log_standard-A2-NonVPN-15s-trial-009	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.8491		
E07-knn_log_standard-A2-NonVPN-15s-trial-010	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.8388		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E07-knn_log_standard-A2-NonVPN-15s-trial-011	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.9029		
E07-knn_log_standard-A2-NonVPN-15s-trial-012	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.906		
E07-knn_log_standard-A2-NonVPN-15s-trial-013	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.9053		
E07-knn_log_standard-A2-NonVPN-15s-trial-014	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.906		
E07-knn_log_standard-A2-NonVPN-15s-trial-015	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.866		
E07-knn_log_standard-A2-NonVPN-15s-trial-016	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.906		
E07-knn_log_standard-A2-NonVPN-15s-trial-017	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.8735		
E07-knn_log_standard-A2-NonVPN-15s-trial-018	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.7397		
E07-knn_log_standard-A2-NonVPN-15s-trial-019	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.8966		
E07-knn_log_standard-A2-NonVPN-15s-trial-020	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.8712		
E07-knn_log_standard-A2-NonVPN-15s-trial-021	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.906		
E07-knn_log_standard-A2-NonVPN-15s-trial-022	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.8995		
E07-knn_log_standard-A2-NonVPN-15s-trial-023	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.888		
E07-knn_log_standard-A2-NonVPN-15s-trial-024	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.9055		
E07-knn_log_standard-A2-NonVPN-15s-trial-025	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.84		
E07-knn_log_standard-A2-NonVPN-15s-trial-026	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.9053		
E07-knn_log_standard-A2-NonVPN-15s-trial-027	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.8593		
E07-knn_log_standard-A2-NonVPN-15s-trial-028	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.845		
E07-knn_log_standard-A2-NonVPN-15s-trial-029	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.9022		
E07-knn_log_standard-A2-NonVPN-15s-trial-030	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.8086		
E07-knn_log_standard-A2-NonVPN-15s-trial-031	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.906		
E07-knn_log_standard-A2-NonVPN-15s-trial-032	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.906		
E07-knn_log_standard-A2-NonVPN-15s-trial-033	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.906		
E07-knn_log_standard-A2-NonVPN-15s-trial-034	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.8919		
E07-knn_log_standard-A2-NonVPN-15s-trial-035	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.8805		
E07-knn_log_standard-A2-NonVPN-15s-trial-036	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.8927		
E07-knn_log_standard-A2-NonVPN-15s-trial-037	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.8905		
E07-knn_log_standard-A2-NonVPN-15s-trial-038	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.8847		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E07-knn_log_standard-A2-NonVPN-15s-trial-039	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.9051		
E07-knn_log_standard-A2-NonVPN-15s-trial-040	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.8597		
E07-knn_log_standard-A2-NonVPN-15s-trial-041	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.906		
E07-knn_log_standard-A2-NonVPN-15s-trial-042	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.9055		
E07-knn_log_standard-A2-NonVPN-15s-trial-043	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.8943		
E07-knn_log_standard-A2-NonVPN-15s-trial-044	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.888		
E07-knn_log_standard-A2-NonVPN-15s-trial-045	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.8837		
E07-knn_log_standard-A2-NonVPN-15s-trial-046	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.8573		
E07-knn_log_standard-A2-NonVPN-15s-trial-047	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.906		
E07-knn_log_standard-A2-NonVPN-15s-trial-048	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.9022		
E07-knn_log_standard-A2-NonVPN-15s-trial-049	E07	A2-NonVPN	knn_log_standard	optuna_trial	0.8999		
E07-knn_log_standard-B-15s	E07	B	knn_log_standard	holdout	0.8044	0.8006	
E07-knn_log_standard-B-15s-trial-000	E07	B	knn_log_standard	optuna_trial	0.7239		
E07-knn_log_standard-B-15s-trial-001	E07	B	knn_log_standard	optuna_trial	0.7931		
E07-knn_log_standard-B-15s-trial-002	E07	B	knn_log_standard	optuna_trial	0.785		
E07-knn_log_standard-B-15s-trial-003	E07	B	knn_log_standard	optuna_trial	0.7863		
E07-knn_log_standard-B-15s-trial-004	E07	B	knn_log_standard	optuna_trial	0.7558		
E07-knn_log_standard-B-15s-trial-005	E07	B	knn_log_standard	optuna_trial	0.7336		
E07-knn_log_standard-B-15s-trial-006	E07	B	knn_log_standard	optuna_trial	0.6964		
E07-knn_log_standard-B-15s-trial-007	E07	B	knn_log_standard	optuna_trial	0.6654		
E07-knn_log_standard-B-15s-trial-008	E07	B	knn_log_standard	optuna_trial	0.7691		
E07-knn_log_standard-B-15s-trial-009	E07	B	knn_log_standard	optuna_trial	0.7423		
E07-knn_log_standard-B-15s-trial-010	E07	B	knn_log_standard	optuna_trial	0.8043		
E07-knn_log_standard-B-15s-trial-011	E07	B	knn_log_standard	optuna_trial	0.8043		
E07-knn_log_standard-B-15s-trial-012	E07	B	knn_log_standard	optuna_trial	0.7726		
E07-knn_log_standard-B-15s-trial-013	E07	B	knn_log_standard	optuna_trial	0.8044		
E07-knn_log_standard-B-15s-trial-014	E07	B	knn_log_standard	optuna_trial	0.7579		
E07-knn_log_standard-B-15s-trial-015	E07	B	knn_log_standard	optuna_trial	0.8006		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E07-knn_log_standard-B-15s-trial-016	E07	B	knn_log_standard	optuna_trial	0.7848		
E07-knn_log_standard-B-15s-trial-017	E07	B	knn_log_standard	optuna_trial	0.7196		
E07-knn_log_standard-B-15s-trial-018	E07	B	knn_log_standard	optuna_trial	0.659		
E07-knn_log_standard-B-15s-trial-019	E07	B	knn_log_standard	optuna_trial	0.7616		
E07-knn_log_standard-B-15s-trial-020	E07	B	knn_log_standard	optuna_trial	0.7726		
E07-knn_log_standard-B-15s-trial-021	E07	B	knn_log_standard	optuna_trial	0.8043		
E07-knn_log_standard-B-15s-trial-022	E07	B	knn_log_standard	optuna_trial	0.803		
E07-knn_log_standard-B-15s-trial-023	E07	B	knn_log_standard	optuna_trial	0.7998		
E07-knn_log_standard-B-15s-trial-024	E07	B	knn_log_standard	optuna_trial	0.8043		
E07-knn_log_standard-B-15s-trial-025	E07	B	knn_log_standard	optuna_trial	0.7373		
E07-knn_log_standard-B-15s-trial-026	E07	B	knn_log_standard	optuna_trial	0.7841		
E07-knn_log_standard-B-15s-trial-027	E07	B	knn_log_standard	optuna_trial	0.8043		
E07-knn_log_standard-B-15s-trial-028	E07	B	knn_log_standard	optuna_trial	0.775		
E07-knn_log_standard-B-15s-trial-029	E07	B	knn_log_standard	optuna_trial	0.7347		
E07-knn_log_standard-B-15s-trial-030	E07	B	knn_log_standard	optuna_trial	0.7954		
E07-knn_log_standard-B-15s-trial-031	E07	B	knn_log_standard	optuna_trial	0.8044		
E07-knn_log_standard-B-15s-trial-032	E07	B	knn_log_standard	optuna_trial	0.793		
E07-knn_log_standard-B-15s-trial-033	E07	B	knn_log_standard	optuna_trial	0.7759		
E07-knn_log_standard-B-15s-trial-034	E07	B	knn_log_standard	optuna_trial	0.803		
E07-knn_log_standard-B-15s-trial-035	E07	B	knn_log_standard	optuna_trial	0.7919		
E07-knn_log_standard-B-15s-trial-036	E07	B	knn_log_standard	optuna_trial	0.7644		
E07-knn_log_standard-B-15s-trial-037	E07	B	knn_log_standard	optuna_trial	0.7954		
E07-knn_log_standard-B-15s-trial-038	E07	B	knn_log_standard	optuna_trial	0.7823		
E07-knn_log_standard-B-15s-trial-039	E07	B	knn_log_standard	optuna_trial	0.7095		
E07-knn_log_standard-B-15s-trial-040	E07	B	knn_log_standard	optuna_trial	0.8044		
E07-knn_log_standard-B-15s-trial-041	E07	B	knn_log_standard	optuna_trial	0.8035		
E07-knn_log_standard-B-15s-trial-042	E07	B	knn_log_standard	optuna_trial	0.8025		
E07-knn_log_standard-B-15s-trial-043	E07	B	knn_log_standard	optuna_trial	0.7966		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E07-knn_log_standard-B-15s-trial-044	E07	B	knn_log_standard	optuna_trial	0.793		
E07-knn_log_standard-B-15s-trial-045	E07	B	knn_log_standard	optuna_trial	0.8044		
E07-knn_log_standard-B-15s-trial-046	E07	B	knn_log_standard	optuna_trial	0.7886		
E07-knn_log_standard-B-15s-trial-047	E07	B	knn_log_standard	optuna_trial	0.7662		
E07-knn_log_standard-B-15s-trial-048	E07	B	knn_log_standard	optuna_trial	0.7863		
E07-knn_log_standard-B-15s-trial-049	E07	B	knn_log_standard	optuna_trial	0.7308		
E07-xgb-A1-15s	E07	A1	xgb	holdout	0.9293	0.9259	
E07-xgb-A1-15s-trial-000	E07	A1	xgb	optuna_trial	0.9237		
E07-xgb-A1-15s-trial-001	E07	A1	xgb	optuna_trial	0.9058		
E07-xgb-A1-15s-trial-002	E07	A1	xgb	optuna_trial	0.8944		
E07-xgb-A1-15s-trial-003	E07	A1	xgb	optuna_trial	0.9129		
E07-xgb-A1-15s-trial-004	E07	A1	xgb	optuna_trial	0.9221		
E07-xgb-A1-15s-trial-005	E07	A1	xgb	optuna_trial	0.8684		
E07-xgb-A1-15s-trial-006	E07	A1	xgb	optuna_trial	0.8959		
E07-xgb-A1-15s-trial-007	E07	A1	xgb	optuna_trial	0.9085		
E07-xgb-A1-15s-trial-008	E07	A1	xgb	optuna_trial	0.9039		
E07-xgb-A1-15s-trial-009	E07	A1	xgb	optuna_trial	0.9128		
E07-xgb-A1-15s-trial-010	E07	A1	xgb	optuna_trial	0.9266		
E07-xgb-A1-15s-trial-011	E07	A1	xgb	optuna_trial	0.923		
E07-xgb-A1-15s-trial-012	E07	A1	xgb	optuna_trial	0.9273		
E07-xgb-A1-15s-trial-013	E07	A1	xgb	optuna_trial	0.9272		
E07-xgb-A1-15s-trial-014	E07	A1	xgb	optuna_trial	0.9248		
E07-xgb-A1-15s-trial-015	E07	A1	xgb	optuna_trial	0.9236		
E07-xgb-A1-15s-trial-016	E07	A1	xgb	optuna_trial	0.9268		
E07-xgb-A1-15s-trial-017	E07	A1	xgb	optuna_trial	0.9287		
E07-xgb-A1-15s-trial-018	E07	A1	xgb	optuna_trial	0.9272		
E07-xgb-A1-15s-trial-019	E07	A1	xgb	optuna_trial	0.9229		
E07-xgb-A1-15s-trial-020	E07	A1	xgb	optuna_trial	0.9269		
E07-xgb-A1-15s-trial-021	E07	A1	xgb	optuna_trial	0.9263		
E07-xgb-A1-15s-trial-022	E07	A1	xgb	optuna_trial	0.928		
E07-xgb-A1-15s-trial-023	E07	A1	xgb	optuna_trial	0.928		
E07-xgb-A1-15s-trial-024	E07	A1	xgb	optuna_trial	0.9266		
E07-xgb-A1-15s-trial-025	E07	A1	xgb	optuna_trial	0.9275		
E07-xgb-A1-15s-trial-026	E07	A1	xgb	optuna_trial	0.9256		
E07-xgb-A1-15s-trial-027	E07	A1	xgb	optuna_trial	0.9258		
E07-xgb-A1-15s-trial-028	E07	A1	xgb	optuna_trial	0.9287		
E07-xgb-A1-15s-trial-029	E07	A1	xgb	optuna_trial	0.927		
E07-xgb-A1-15s-trial-030	E07	A1	xgb	optuna_trial	0.9288		
E07-xgb-A1-15s-trial-031	E07	A1	xgb	optuna_trial	0.9263		
E07-xgb-A1-15s-trial-032	E07	A1	xgb	optuna_trial	0.9268		
E07-xgb-A1-15s-trial-033	E07	A1	xgb	optuna_trial	0.9286		
E07-xgb-A1-15s-trial-034	E07	A1	xgb	optuna_trial	0.9275		
E07-xgb-A1-15s-trial-035	E07	A1	xgb	optuna_trial	0.9275		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E07-xgb-A1-15s-trial-036	E07	A1	xgb	optuna_trial	0.9269		
E07-xgb-A1-15s-trial-037	E07	A1	xgb	optuna_trial	0.9276		
E07-xgb-A1-15s-trial-038	E07	A1	xgb	optuna_trial	0.928		
E07-xgb-A1-15s-trial-039	E07	A1	xgb	optuna_trial	0.9278		
E07-xgb-A1-15s-trial-040	E07	A1	xgb	optuna_trial	0.9274		
E07-xgb-A1-15s-trial-041	E07	A1	xgb	optuna_trial	0.9284		
E07-xgb-A1-15s-trial-042	E07	A1	xgb	optuna_trial	0.929		
E07-xgb-A1-15s-trial-043	E07	A1	xgb	optuna_trial	0.9276		
E07-xgb-A1-15s-trial-044	E07	A1	xgb	optuna_trial	0.9287		
E07-xgb-A1-15s-trial-045	E07	A1	xgb	optuna_trial	0.9283		
E07-xgb-A1-15s-trial-046	E07	A1	xgb	optuna_trial	0.9283		
E07-xgb-A1-15s-trial-047	E07	A1	xgb	optuna_trial	0.9291		
E07-xgb-A1-15s-trial-048	E07	A1	xgb	optuna_trial	0.9293		
E07-xgb-A1-15s-trial-049	E07	A1	xgb	optuna_trial	0.9281		
E07-xgb-A2-VPN-15s	E07	A2-VPN	xgb	holdout	0.8838	0.8661	
E07-xgb-A2-VPN-15s-trial-000	E07	A2-VPN	xgb	optuna_trial	0.8783		
E07-xgb-A2-VPN-15s-trial-001	E07	A2-VPN	xgb	optuna_trial	0.842		
E07-xgb-A2-VPN-15s-trial-002	E07	A2-VPN	xgb	optuna_trial	0.841		
E07-xgb-A2-VPN-15s-trial-003	E07	A2-VPN	xgb	optuna_trial	0.8535		
E07-xgb-A2-VPN-15s-trial-004	E07	A2-VPN	xgb	optuna_trial	0.8834		
E07-xgb-A2-VPN-15s-trial-005	E07	A2-VPN	xgb	optuna_trial	0.8155		
E07-xgb-A2-VPN-15s-trial-006	E07	A2-VPN	xgb	optuna_trial	0.8256		
E07-xgb-A2-VPN-15s-trial-007	E07	A2-VPN	xgb	optuna_trial	0.8559		
E07-xgb-A2-VPN-15s-trial-008	E07	A2-VPN	xgb	optuna_trial	0.8325		
E07-xgb-A2-VPN-15s-trial-009	E07	A2-VPN	xgb	optuna_trial	0.8675		
E07-xgb-A2-VPN-15s-trial-010	E07	A2-VPN	xgb	optuna_trial	0.8796		
E07-xgb-A2-VPN-15s-trial-011	E07	A2-VPN	xgb	optuna_trial	0.8723		
E07-xgb-A2-VPN-15s-trial-012	E07	A2-VPN	xgb	optuna_trial	0.8831		
E07-xgb-A2-VPN-15s-trial-013	E07	A2-VPN	xgb	optuna_trial	0.8808		
E07-xgb-A2-VPN-15s-trial-014	E07	A2-VPN	xgb	optuna_trial	0.8746		
E07-xgb-A2-VPN-15s-trial-015	E07	A2-VPN	xgb	optuna_trial	0.8739		
E07-xgb-A2-VPN-15s-trial-016	E07	A2-VPN	xgb	optuna_trial	0.88		
E07-xgb-A2-VPN-15s-trial-017	E07	A2-VPN	xgb	optuna_trial	0.8794		
E07-xgb-A2-VPN-15s-trial-018	E07	A2-VPN	xgb	optuna_trial	0.791		
E07-xgb-A2-VPN-15s-trial-019	E07	A2-VPN	xgb	optuna_trial	0.8753		
E07-xgb-A2-VPN-15s-trial-020	E07	A2-VPN	xgb	optuna_trial	0.8649		
E07-xgb-A2-VPN-15s-trial-021	E07	A2-VPN	xgb	optuna_trial	0.8838		
E07-xgb-A2-VPN-15s-trial-022	E07	A2-VPN	xgb	optuna_trial	0.8726		
E07-xgb-A2-VPN-15s-trial-023	E07	A2-VPN	xgb	optuna_trial	0.8826		
E07-xgb-A2-VPN-15s-trial-024	E07	A2-VPN	xgb	optuna_trial	0.8786		
E07-xgb-A2-VPN-15s-trial-025	E07	A2-VPN	xgb	optuna_trial	0.8826		
E07-xgb-A2-VPN-15s-trial-026	E07	A2-VPN	xgb	optuna_trial	0.8687		
E07-xgb-A2-VPN-15s-trial-027	E07	A2-VPN	xgb	optuna_trial	0.8746		
E07-xgb-A2-VPN-15s-trial-028	E07	A2-VPN	xgb	optuna_trial	0.8812		
E07-xgb-A2-VPN-15s-trial-029	E07	A2-VPN	xgb	optuna_trial	0.8767		
E07-xgb-A2-VPN-15s-trial-030	E07	A2-VPN	xgb	optuna_trial	0.8736		
E07-xgb-A2-VPN-15s-trial-031	E07	A2-VPN	xgb	optuna_trial	0.8837		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E07-xgb-A2-VPN-15s-trial-032	E07	A2-VPN	xgb	optuna_trial	0.8813		
E07-xgb-A2-VPN-15s-trial-033	E07	A2-VPN	xgb	optuna_trial	0.8754		
E07-xgb-A2-VPN-15s-trial-034	E07	A2-VPN	xgb	optuna_trial	0.877		
E07-xgb-A2-VPN-15s-trial-035	E07	A2-VPN	xgb	optuna_trial	0.8804		
E07-xgb-A2-VPN-15s-trial-036	E07	A2-VPN	xgb	optuna_trial	0.876		
E07-xgb-A2-VPN-15s-trial-037	E07	A2-VPN	xgb	optuna_trial	0.8727		
E07-xgb-A2-VPN-15s-trial-038	E07	A2-VPN	xgb	optuna_trial	0.8834		
E07-xgb-A2-VPN-15s-trial-039	E07	A2-VPN	xgb	optuna_trial	0.8831		
E07-xgb-A2-VPN-15s-trial-040	E07	A2-VPN	xgb	optuna_trial	0.8826		
E07-xgb-A2-VPN-15s-trial-041	E07	A2-VPN	xgb	optuna_trial	0.878		
E07-xgb-A2-VPN-15s-trial-042	E07	A2-VPN	xgb	optuna_trial	0.8832		
E07-xgb-A2-VPN-15s-trial-043	E07	A2-VPN	xgb	optuna_trial	0.8815		
E07-xgb-A2-VPN-15s-trial-044	E07	A2-VPN	xgb	optuna_trial	0.883		
E07-xgb-A2-VPN-15s-trial-045	E07	A2-VPN	xgb	optuna_trial	0.8812		
E07-xgb-A2-VPN-15s-trial-046	E07	A2-VPN	xgb	optuna_trial	0.8798		
E07-xgb-A2-VPN-15s-trial-047	E07	A2-VPN	xgb	optuna_trial	0.8835		
E07-xgb-A2-VPN-15s-trial-048	E07	A2-VPN	xgb	optuna_trial	0.8811		
E07-xgb-A2-VPN-15s-trial-049	E07	A2-VPN	xgb	optuna_trial	0.8831		
E07-xgb-A2-NonVPN-15s	E07	A2-NonVPN	xgb	holdout	0.9294	0.9025	
E07-xgb-A2-NonVPN-15s-trial-000	E07	A2-NonVPN	xgb	optuna_trial	0.9268		
E07-xgb-A2-NonVPN-15s-trial-001	E07	A2-NonVPN	xgb	optuna_trial	0.8743		
E07-xgb-A2-NonVPN-15s-trial-002	E07	A2-NonVPN	xgb	optuna_trial	0.8813		
E07-xgb-A2-NonVPN-15s-trial-003	E07	A2-NonVPN	xgb	optuna_trial	0.8968		
E07-xgb-A2-NonVPN-15s-trial-004	E07	A2-NonVPN	xgb	optuna_trial	0.9266		
E07-xgb-A2-NonVPN-15s-trial-005	E07	A2-NonVPN	xgb	optuna_trial	0.8671		
E07-xgb-A2-NonVPN-15s-trial-006	E07	A2-NonVPN	xgb	optuna_trial	0.8576		
E07-xgb-A2-NonVPN-15s-trial-007	E07	A2-NonVPN	xgb	optuna_trial	0.9072		
E07-xgb-A2-NonVPN-15s-trial-008	E07	A2-NonVPN	xgb	optuna_trial	0.8669		
E07-xgb-A2-NonVPN-15s-trial-009	E07	A2-NonVPN	xgb	optuna_trial	0.9112		
E07-xgb-A2-NonVPN-15s-trial-010	E07	A2-NonVPN	xgb	optuna_trial	0.9267		
E07-xgb-A2-NonVPN-15s-trial-011	E07	A2-NonVPN	xgb	optuna_trial	0.9194		
E07-xgb-A2-NonVPN-15s-trial-012	E07	A2-NonVPN	xgb	optuna_trial	0.926		
E07-xgb-A2-NonVPN-15s-trial-013	E07	A2-NonVPN	xgb	optuna_trial	0.9255		
E07-xgb-A2-NonVPN-15s-trial-014	E07	A2-NonVPN	xgb	optuna_trial	0.9211		
E07-xgb-A2-NonVPN-15s-trial-015	E07	A2-NonVPN	xgb	optuna_trial	0.9267		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E07-xgb-A2-NonVPN-15s-trial-016	E07	A2-NonVPN	xgb	optuna_trial	0.9213		
E07-xgb-A2-NonVPN-15s-trial-017	E07	A2-NonVPN	xgb	optuna_trial	0.9257		
E07-xgb-A2-NonVPN-15s-trial-018	E07	A2-NonVPN	xgb	optuna_trial	0.9155		
E07-xgb-A2-NonVPN-15s-trial-019	E07	A2-NonVPN	xgb	optuna_trial	0.9143		
E07-xgb-A2-NonVPN-15s-trial-020	E07	A2-NonVPN	xgb	optuna_trial	0.9188		
E07-xgb-A2-NonVPN-15s-trial-021	E07	A2-NonVPN	xgb	optuna_trial	0.9192		
E07-xgb-A2-NonVPN-15s-trial-022	E07	A2-NonVPN	xgb	optuna_trial	0.9248		
E07-xgb-A2-NonVPN-15s-trial-023	E07	A2-NonVPN	xgb	optuna_trial	0.9294		
E07-xgb-A2-NonVPN-15s-trial-024	E07	A2-NonVPN	xgb	optuna_trial	0.9281		
E07-xgb-A2-NonVPN-15s-trial-025	E07	A2-NonVPN	xgb	optuna_trial	0.9241		
E07-xgb-A2-NonVPN-15s-trial-026	E07	A2-NonVPN	xgb	optuna_trial	0.9252		
E07-xgb-A2-NonVPN-15s-trial-027	E07	A2-NonVPN	xgb	optuna_trial	0.9276		
E07-xgb-A2-NonVPN-15s-trial-028	E07	A2-NonVPN	xgb	optuna_trial	0.9207		
E07-xgb-A2-NonVPN-15s-trial-029	E07	A2-NonVPN	xgb	optuna_trial	0.9272		
E07-xgb-A2-NonVPN-15s-trial-030	E07	A2-NonVPN	xgb	optuna_trial	0.9225		
E07-xgb-A2-NonVPN-15s-trial-031	E07	A2-NonVPN	xgb	optuna_trial	0.9262		
E07-xgb-A2-NonVPN-15s-trial-032	E07	A2-NonVPN	xgb	optuna_trial	0.9269		
E07-xgb-A2-NonVPN-15s-trial-033	E07	A2-NonVPN	xgb	optuna_trial	0.9249		
E07-xgb-A2-NonVPN-15s-trial-034	E07	A2-NonVPN	xgb	optuna_trial	0.9262		
E07-xgb-A2-NonVPN-15s-trial-035	E07	A2-NonVPN	xgb	optuna_trial	0.9242		
E07-xgb-A2-NonVPN-15s-trial-036	E07	A2-NonVPN	xgb	optuna_trial	0.9278		
E07-xgb-A2-NonVPN-15s-trial-037	E07	A2-NonVPN	xgb	optuna_trial	0.9276		
E07-xgb-A2-NonVPN-15s-trial-038	E07	A2-NonVPN	xgb	optuna_trial	0.928		
E07-xgb-A2-NonVPN-15s-trial-039	E07	A2-NonVPN	xgb	optuna_trial	0.9263		
E07-xgb-A2-NonVPN-15s-trial-040	E07	A2-NonVPN	xgb	optuna_trial	0.9268		
E07-xgb-A2-NonVPN-15s-trial-041	E07	A2-NonVPN	xgb	optuna_trial	0.9266		
E07-xgb-A2-NonVPN-15s-trial-042	E07	A2-NonVPN	xgb	optuna_trial	0.9264		
E07-xgb-A2-NonVPN-15s-trial-043	E07	A2-NonVPN	xgb	optuna_trial	0.9282		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E07-xgb-A2-NonVPN-15s-trial-044	E07	A2-NonVPN	xgb	optuna_trial	0.9225		
E07-xgb-A2-NonVPN-15s-trial-045	E07	A2-NonVPN	xgb	optuna_trial	0.9268		
E07-xgb-A2-NonVPN-15s-trial-046	E07	A2-NonVPN	xgb	optuna_trial	0.9261		
E07-xgb-A2-NonVPN-15s-trial-047	E07	A2-NonVPN	xgb	optuna_trial	0.9256		
E07-xgb-A2-NonVPN-15s-trial-048	E07	A2-NonVPN	xgb	optuna_trial	0.9265		
E07-xgb-A2-NonVPN-15s-trial-049	E07	A2-NonVPN	xgb	optuna_trial	0.9284		
E07-xgb-B-15s	E07	B	xgb	holdout	0.8405	0.8428	
E07-xgb-B-15s-trial-000	E07	B	xgb	optuna_trial	0.8302		
E07-xgb-B-15s-trial-001	E07	B	xgb	optuna_trial	0.7827		
E07-xgb-B-15s-trial-002	E07	B	xgb	optuna_trial	0.7834		
E07-xgb-B-15s-trial-003	E07	B	xgb	optuna_trial	0.8001		
E07-xgb-B-15s-trial-004	E07	B	xgb	optuna_trial	0.8358		
E07-xgb-B-15s-trial-005	E07	B	xgb	optuna_trial	0.749		
E07-xgb-B-15s-trial-006	E07	B	xgb	optuna_trial	0.7599		
E07-xgb-B-15s-trial-007	E07	B	xgb	optuna_trial	0.8091		
E07-xgb-B-15s-trial-008	E07	B	xgb	optuna_trial	0.7706		
E07-xgb-B-15s-trial-009	E07	B	xgb	optuna_trial	0.8157		
E07-xgb-B-15s-trial-010	E07	B	xgb	optuna_trial	0.8311		
E07-xgb-B-15s-trial-011	E07	B	xgb	optuna_trial	0.8195		
E07-xgb-B-15s-trial-012	E07	B	xgb	optuna_trial	0.8379		
E07-xgb-B-15s-trial-013	E07	B	xgb	optuna_trial	0.8327		
E07-xgb-B-15s-trial-014	E07	B	xgb	optuna_trial	0.8285		
E07-xgb-B-15s-trial-015	E07	B	xgb	optuna_trial	0.8247		
E07-xgb-B-15s-trial-016	E07	B	xgb	optuna_trial	0.8352		
E07-xgb-B-15s-trial-017	E07	B	xgb	optuna_trial	0.8311		
E07-xgb-B-15s-trial-018	E07	B	xgb	optuna_trial	0.7094		
E07-xgb-B-15s-trial-019	E07	B	xgb	optuna_trial	0.8204		
E07-xgb-B-15s-trial-020	E07	B	xgb	optuna_trial	0.8168		
E07-xgb-B-15s-trial-021	E07	B	xgb	optuna_trial	0.8249		
E07-xgb-B-15s-trial-022	E07	B	xgb	optuna_trial	0.8346		
E07-xgb-B-15s-trial-023	E07	B	xgb	optuna_trial	0.838		
E07-xgb-B-15s-trial-024	E07	B	xgb	optuna_trial	0.8386		
E07-xgb-B-15s-trial-025	E07	B	xgb	optuna_trial	0.8307		
E07-xgb-B-15s-trial-026	E07	B	xgb	optuna_trial	0.8303		
E07-xgb-B-15s-trial-027	E07	B	xgb	optuna_trial	0.8371		
E07-xgb-B-15s-trial-028	E07	B	xgb	optuna_trial	0.8374		
E07-xgb-B-15s-trial-029	E07	B	xgb	optuna_trial	0.8385		
E07-xgb-B-15s-trial-030	E07	B	xgb	optuna_trial	0.8195		
E07-xgb-B-15s-trial-031	E07	B	xgb	optuna_trial	0.8365		
E07-xgb-B-15s-trial-032	E07	B	xgb	optuna_trial	0.8397		
E07-xgb-B-15s-trial-033	E07	B	xgb	optuna_trial	0.8195		
E07-xgb-B-15s-trial-034	E07	B	xgb	optuna_trial	0.8352		
E07-xgb-B-15s-trial-035	E07	B	xgb	optuna_trial	0.8405		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E07-xgb-B-15s-trial-036	E07	B	xgb	optuna_trial	0.8368		
E07-xgb-B-15s-trial-037	E07	B	xgb	optuna_trial	0.8339		
E07-xgb-B-15s-trial-038	E07	B	xgb	optuna_trial	0.8383		
E07-xgb-B-15s-trial-039	E07	B	xgb	optuna_trial	0.8399		
E07-xgb-B-15s-trial-040	E07	B	xgb	optuna_trial	0.8355		
E07-xgb-B-15s-trial-041	E07	B	xgb	optuna_trial	0.8318		
E07-xgb-B-15s-trial-042	E07	B	xgb	optuna_trial	0.8399		
E07-xgb-B-15s-trial-043	E07	B	xgb	optuna_trial	0.8322		
E07-xgb-B-15s-trial-044	E07	B	xgb	optuna_trial	0.8365		
E07-xgb-B-15s-trial-045	E07	B	xgb	optuna_trial	0.8358		
E07-xgb-B-15s-trial-046	E07	B	xgb	optuna_trial	0.8403		
E07-xgb-B-15s-trial-047	E07	B	xgb	optuna_trial	0.8361		
E07-xgb-B-15s-trial-048	E07	B	xgb	optuna_trial	0.8376		
E07-xgb-B-15s-trial-049	E07	B	xgb	optuna_trial	0.8377		
E07-rf-A1-15s	E07	A1	rf	holdout	0.9209	0.9181	
E07-rf-A1-15s-trial-000	E07	A1	rf	optuna_trial	0.9204		
E07-rf-A1-15s-trial-001	E07	A1	rf	optuna_trial	0.9097		
E07-rf-A1-15s-trial-002	E07	A1	rf	optuna_trial	0.9161		
E07-rf-A1-15s-trial-003	E07	A1	rf	optuna_trial	0.9139		
E07-rf-A1-15s-trial-004	E07	A1	rf	optuna_trial	0.9142		
E07-rf-A1-15s-trial-005	E07	A1	rf	optuna_trial	0.9171		
E07-rf-A1-15s-trial-006	E07	A1	rf	optuna_trial	0.9103		
E07-rf-A1-15s-trial-007	E07	A1	rf	optuna_trial	0.9178		
E07-rf-A1-15s-trial-008	E07	A1	rf	optuna_trial	0.9196		
E07-rf-A1-15s-trial-009	E07	A1	rf	optuna_trial	0.9097		
E07-rf-A1-15s-trial-010	E07	A1	rf	optuna_trial	0.854		
E07-rf-A1-15s-trial-011	E07	A1	rf	optuna_trial	0.9188		
E07-rf-A1-15s-trial-012	E07	A1	rf	optuna_trial	0.9164		
E07-rf-A1-15s-trial-013	E07	A1	rf	optuna_trial	0.9172		
E07-rf-A1-15s-trial-014	E07	A1	rf	optuna_trial	0.92		
E07-rf-A1-15s-trial-015	E07	A1	rf	optuna_trial	0.9204		
E07-rf-A1-15s-trial-016	E07	A1	rf	optuna_trial	0.9197		
E07-rf-A1-15s-trial-017	E07	A1	rf	optuna_trial	0.9203		
E07-rf-A1-15s-trial-018	E07	A1	rf	optuna_trial	0.9187		
E07-rf-A1-15s-trial-019	E07	A1	rf	optuna_trial	0.9185		
E07-rf-A1-15s-trial-020	E07	A1	rf	optuna_trial	0.918		
E07-rf-A1-15s-trial-021	E07	A1	rf	optuna_trial	0.9192		
E07-rf-A1-15s-trial-022	E07	A1	rf	optuna_trial	0.9182		
E07-rf-A1-15s-trial-023	E07	A1	rf	optuna_trial	0.9204		
E07-rf-A1-15s-trial-024	E07	A1	rf	optuna_trial	0.9205		
E07-rf-A1-15s-trial-025	E07	A1	rf	optuna_trial	0.8369		
E07-rf-A1-15s-trial-026	E07	A1	rf	optuna_trial	0.9166		
E07-rf-A1-15s-trial-027	E07	A1	rf	optuna_trial	0.9167		
E07-rf-A1-15s-trial-028	E07	A1	rf	optuna_trial	0.9164		
E07-rf-A1-15s-trial-029	E07	A1	rf	optuna_trial	0.9187		
E07-rf-A1-15s-trial-030	E07	A1	rf	optuna_trial	0.9181		
E07-rf-A1-15s-trial-031	E07	A1	rf	optuna_trial	0.9171		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E07-rf-A1-15s-trial-032	E07	A1	rf	optuna_trial	0.92		
E07-rf-A1-15s-trial-033	E07	A1	rf	optuna_trial	0.92		
E07-rf-A1-15s-trial-034	E07	A1	rf	optuna_trial	0.9203		
E07-rf-A1-15s-trial-035	E07	A1	rf	optuna_trial	0.9175		
E07-rf-A1-15s-trial-036	E07	A1	rf	optuna_trial	0.914		
E07-rf-A1-15s-trial-037	E07	A1	rf	optuna_trial	0.9191		
E07-rf-A1-15s-trial-038	E07	A1	rf	optuna_trial	0.9196		
E07-rf-A1-15s-trial-039	E07	A1	rf	optuna_trial	0.914		
E07-rf-A1-15s-trial-040	E07	A1	rf	optuna_trial	0.9203		
E07-rf-A1-15s-trial-041	E07	A1	rf	optuna_trial	0.9183		
E07-rf-A1-15s-trial-042	E07	A1	rf	optuna_trial	0.9185		
E07-rf-A1-15s-trial-043	E07	A1	rf	optuna_trial	0.9197		
E07-rf-A1-15s-trial-044	E07	A1	rf	optuna_trial	0.9203		
E07-rf-A1-15s-trial-045	E07	A1	rf	optuna_trial	0.9019		
E07-rf-A1-15s-trial-046	E07	A1	rf	optuna_trial	0.9197		
E07-rf-A1-15s-trial-047	E07	A1	rf	optuna_trial	0.9203		
E07-rf-A1-15s-trial-048	E07	A1	rf	optuna_trial	0.919		
E07-rf-A1-15s-trial-049	E07	A1	rf	optuna_trial	0.9209		
E07-lgbm-A2-VPN-15s	E07	A2-VPN	lgbm	holdout	0.8902	0.8768	
E07-lgbm-A2-VPN-15s-trial-000	E07	A2-VPN	lgbm	optuna_trial	0.8789		
E07-lgbm-A2-VPN-15s-trial-001	E07	A2-VPN	lgbm	optuna_trial	0.8829		
E07-lgbm-A2-VPN-15s-trial-002	E07	A2-VPN	lgbm	optuna_trial	0.8845		
E07-lgbm-A2-VPN-15s-trial-003	E07	A2-VPN	lgbm	optuna_trial	0.8846		
E07-lgbm-A2-VPN-15s-trial-004	E07	A2-VPN	lgbm	optuna_trial	0.8839		
E07-lgbm-A2-VPN-15s-trial-005	E07	A2-VPN	lgbm	optuna_trial	0.8881		
E07-lgbm-A2-VPN-15s-trial-006	E07	A2-VPN	lgbm	optuna_trial	0.8848		
E07-lgbm-A2-VPN-15s-trial-007	E07	A2-VPN	lgbm	optuna_trial	0.8866		
E07-lgbm-A2-VPN-15s-trial-008	E07	A2-VPN	lgbm	optuna_trial	0.8876		
E07-lgbm-A2-VPN-15s-trial-009	E07	A2-VPN	lgbm	optuna_trial	0.3091		
E07-lgbm-A2-VPN-15s-trial-010	E07	A2-VPN	lgbm	optuna_trial	0.8828		
E07-lgbm-A2-VPN-15s-trial-011	E07	A2-VPN	lgbm	optuna_trial	0.8902		
E07-lgbm-A2-VPN-15s-trial-012	E07	A2-VPN	lgbm	optuna_trial	0.8897		
E07-lgbm-A2-VPN-15s-trial-013	E07	A2-VPN	lgbm	optuna_trial	0.8871		
E07-lgbm-A2-VPN-15s-trial-014	E07	A2-VPN	lgbm	optuna_trial	0.8885		
E07-lgbm-A2-VPN-15s-trial-015	E07	A2-VPN	lgbm	optuna_trial	0.8856		
E07-lgbm-A2-VPN-15s-trial-016	E07	A2-VPN	lgbm	optuna_trial	0.8871		
E07-lgbm-A2-VPN-15s-trial-017	E07	A2-VPN	lgbm	optuna_trial	0.8872		
E07-lgbm-A2-VPN-15s-trial-018	E07	A2-VPN	lgbm	optuna_trial	0.8869		
E07-lgbm-A2-VPN-15s-trial-019	E07	A2-VPN	lgbm	optuna_trial	0.8885		
E07-lgbm-A2-VPN-15s-trial-020	E07	A2-VPN	lgbm	optuna_trial	0.8879		
E07-lgbm-A2-VPN-15s-trial-021	E07	A2-VPN	lgbm	optuna_trial	0.8891		
E07-lgbm-A2-VPN-15s-trial-022	E07	A2-VPN	lgbm	optuna_trial	0.8878		
E07-lgbm-A2-VPN-15s-trial-023	E07	A2-VPN	lgbm	optuna_trial	0.8874		
E07-lgbm-A2-VPN-15s-trial-024	E07	A2-VPN	lgbm	optuna_trial	0.887		
E07-lgbm-A2-VPN-15s-trial-025	E07	A2-VPN	lgbm	optuna_trial	0.8885		
E07-lgbm-A2-VPN-15s-trial-026	E07	A2-VPN	lgbm	optuna_trial	0.8847		
E07-lgbm-A2-VPN-15s-trial-027	E07	A2-VPN	lgbm	optuna_trial	0.8888		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E07-lgbm-A2-VPN-15s-trial-028	E07	A2-VPN	lgbm	optuna_trial	0.8881		
E07-lgbm-A2-VPN-15s-trial-029	E07	A2-VPN	lgbm	optuna_trial	0.8884		
E07-lgbm-A2-VPN-15s-trial-030	E07	A2-VPN	lgbm	optuna_trial	0.8845		
E07-lgbm-A2-VPN-15s-trial-031	E07	A2-VPN	lgbm	optuna_trial	0.8897		
E07-lgbm-A2-VPN-15s-trial-032	E07	A2-VPN	lgbm	optuna_trial	0.8885		
E07-lgbm-A2-VPN-15s-trial-033	E07	A2-VPN	lgbm	optuna_trial	0.8866		
E07-lgbm-A2-VPN-15s-trial-034	E07	A2-VPN	lgbm	optuna_trial	0.8838		
E07-lgbm-A2-VPN-15s-trial-035	E07	A2-VPN	lgbm	optuna_trial	0.8876		
E07-lgbm-A2-VPN-15s-trial-036	E07	A2-VPN	lgbm	optuna_trial	0.8888		
E07-lgbm-A2-VPN-15s-trial-037	E07	A2-VPN	lgbm	optuna_trial	0.8876		
E07-lgbm-A2-VPN-15s-trial-038	E07	A2-VPN	lgbm	optuna_trial	0.8852		
E07-lgbm-A2-VPN-15s-trial-039	E07	A2-VPN	lgbm	optuna_trial	0.8889		
E07-lgbm-A2-VPN-15s-trial-040	E07	A2-VPN	lgbm	optuna_trial	0.8843		
E07-lgbm-A2-VPN-15s-trial-041	E07	A2-VPN	lgbm	optuna_trial	0.8891		
E07-lgbm-A2-VPN-15s-trial-042	E07	A2-VPN	lgbm	optuna_trial	0.8888		
E07-lgbm-A2-VPN-15s-trial-043	E07	A2-VPN	lgbm	optuna_trial	0.8877		
E07-lgbm-A2-VPN-15s-trial-044	E07	A2-VPN	lgbm	optuna_trial	0.8875		
E07-lgbm-A2-VPN-15s-trial-045	E07	A2-VPN	lgbm	optuna_trial	0.8875		
E07-lgbm-A2-VPN-15s-trial-046	E07	A2-VPN	lgbm	optuna_trial	0.8895		
E07-lgbm-A2-VPN-15s-trial-047	E07	A2-VPN	lgbm	optuna_trial	0.8867		
E07-lgbm-A2-VPN-15s-trial-048	E07	A2-VPN	lgbm	optuna_trial	0.8876		
E07-lgbm-A2-VPN-15s-trial-049	E07	A2-VPN	lgbm	optuna_trial	0.8873		
E07-lgbm-A2-NonVPN-15s	E07	A2-NonVPN	lgbm	holdout	0.9329	0.9028	
E07-lgbm-A2-NonVPN-15s-trial-000	E07	A2-NonVPN	lgbm	optuna_trial	0.7919		
E07-lgbm-A2-NonVPN-15s-trial-001	E07	A2-NonVPN	lgbm	optuna_trial	0.9235		
E07-lgbm-A2-NonVPN-15s-trial-002	E07	A2-NonVPN	lgbm	optuna_trial	0.9295		
E07-lgbm-A2-NonVPN-15s-trial-003	E07	A2-NonVPN	lgbm	optuna_trial	0.9285		
E07-lgbm-A2-NonVPN-15s-trial-004	E07	A2-NonVPN	lgbm	optuna_trial	0.9264		
E07-lgbm-A2-NonVPN-15s-trial-005	E07	A2-NonVPN	lgbm	optuna_trial	0.9233		
E07-lgbm-A2-NonVPN-15s-trial-006	E07	A2-NonVPN	lgbm	optuna_trial	0.9233		
E07-lgbm-A2-NonVPN-15s-trial-007	E07	A2-NonVPN	lgbm	optuna_trial	0.9263		
E07-lgbm-A2-NonVPN-15s-trial-008	E07	A2-NonVPN	lgbm	optuna_trial	0.9262		
E07-lgbm-A2-NonVPN-15s-trial-009	E07	A2-NonVPN	lgbm	optuna_trial	0.6219		
E07-lgbm-A2-NonVPN-15s-trial-010	E07	A2-NonVPN	lgbm	optuna_trial	0.9266		
E07-lgbm-A2-NonVPN-15s-trial-011	E07	A2-NonVPN	lgbm	optuna_trial	0.9286		
E07-lgbm-A2-NonVPN-15s-trial-012	E07	A2-NonVPN	lgbm	optuna_trial	0.9241		
E07-lgbm-A2-NonVPN-15s-trial-013	E07	A2-NonVPN	lgbm	optuna_trial	0.9302		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E07-lgbm-A2-NonVPN-15s-trial-014	E07	A2-NonVPN	lgbm	optuna_trial	0.9284		
E07-lgbm-A2-NonVPN-15s-trial-015	E07	A2-NonVPN	lgbm	optuna_trial	0.9295		
E07-lgbm-A2-NonVPN-15s-trial-016	E07	A2-NonVPN	lgbm	optuna_trial	0.9272		
E07-lgbm-A2-NonVPN-15s-trial-017	E07	A2-NonVPN	lgbm	optuna_trial	0.9265		
E07-lgbm-A2-NonVPN-15s-trial-018	E07	A2-NonVPN	lgbm	optuna_trial	0.9305		
E07-lgbm-A2-NonVPN-15s-trial-019	E07	A2-NonVPN	lgbm	optuna_trial	0.9295		
E07-lgbm-A2-NonVPN-15s-trial-020	E07	A2-NonVPN	lgbm	optuna_trial	0.9257		
E07-lgbm-A2-NonVPN-15s-trial-021	E07	A2-NonVPN	lgbm	optuna_trial	0.9315		
E07-lgbm-A2-NonVPN-15s-trial-022	E07	A2-NonVPN	lgbm	optuna_trial	0.9296		
E07-lgbm-A2-NonVPN-15s-trial-023	E07	A2-NonVPN	lgbm	optuna_trial	0.9234		
E07-lgbm-A2-NonVPN-15s-trial-024	E07	A2-NonVPN	lgbm	optuna_trial	0.9294		
E07-lgbm-A2-NonVPN-15s-trial-025	E07	A2-NonVPN	lgbm	optuna_trial	0.9283		
E07-lgbm-A2-NonVPN-15s-trial-026	E07	A2-NonVPN	lgbm	optuna_trial	0.9277		
E07-lgbm-A2-NonVPN-15s-trial-027	E07	A2-NonVPN	lgbm	optuna_trial	0.9258		
E07-lgbm-A2-NonVPN-15s-trial-028	E07	A2-NonVPN	lgbm	optuna_trial	0.9297		
E07-lgbm-A2-NonVPN-15s-trial-029	E07	A2-NonVPN	lgbm	optuna_trial	0.9297		
E07-lgbm-A2-NonVPN-15s-trial-030	E07	A2-NonVPN	lgbm	optuna_trial	0.9295		
E07-lgbm-A2-NonVPN-15s-trial-031	E07	A2-NonVPN	lgbm	optuna_trial	0.9287		
E07-lgbm-A2-NonVPN-15s-trial-032	E07	A2-NonVPN	lgbm	optuna_trial	0.9285		
E07-lgbm-A2-NonVPN-15s-trial-033	E07	A2-NonVPN	lgbm	optuna_trial	0.9305		
E07-lgbm-A2-NonVPN-15s-trial-034	E07	A2-NonVPN	lgbm	optuna_trial	0.9292		
E07-lgbm-A2-NonVPN-15s-trial-035	E07	A2-NonVPN	lgbm	optuna_trial	0.927		
E07-lgbm-A2-NonVPN-15s-trial-036	E07	A2-NonVPN	lgbm	optuna_trial	0.9298		
E07-lgbm-A2-NonVPN-15s-trial-037	E07	A2-NonVPN	lgbm	optuna_trial	0.9282		
E07-lgbm-A2-NonVPN-15s-trial-038	E07	A2-NonVPN	lgbm	optuna_trial	0.9287		
E07-lgbm-A2-NonVPN-15s-trial-039	E07	A2-NonVPN	lgbm	optuna_trial	0.9311		
E07-lgbm-A2-NonVPN-15s-trial-040	E07	A2-NonVPN	lgbm	optuna_trial	0.9299		
E07-lgbm-A2-NonVPN-15s-trial-041	E07	A2-NonVPN	lgbm	optuna_trial	0.9314		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E07-lgbm-A2-NonVPN-15s-trial-042	E07	A2-NonVPN	lgbm	optuna_trial	0.9291		
E07-lgbm-A2-NonVPN-15s-trial-043	E07	A2-NonVPN	lgbm	optuna_trial	0.9293		
E07-lgbm-A2-NonVPN-15s-trial-044	E07	A2-NonVPN	lgbm	optuna_trial	0.9307		
E07-lgbm-A2-NonVPN-15s-trial-045	E07	A2-NonVPN	lgbm	optuna_trial	0.9292		
E07-lgbm-A2-NonVPN-15s-trial-046	E07	A2-NonVPN	lgbm	optuna_trial	0.9287		
E07-lgbm-A2-NonVPN-15s-trial-047	E07	A2-NonVPN	lgbm	optuna_trial	0.9303		
E07-lgbm-A2-NonVPN-15s-trial-048	E07	A2-NonVPN	lgbm	optuna_trial	0.9329		
E07-lgbm-A2-NonVPN-15s-trial-049	E07	A2-NonVPN	lgbm	optuna_trial	0.9299		
E07-lgbm-B-15s	E07	B	lgbm	holdout	0.8447	0.8352	
E07-lgbm-B-15s-trial-000	E07	B	lgbm	optuna_trial	0.0944		
E07-lgbm-B-15s-trial-001	E07	B	lgbm	optuna_trial	0.8329		
E07-lgbm-B-15s-trial-002	E07	B	lgbm	optuna_trial	0.8389		
E07-lgbm-B-15s-trial-003	E07	B	lgbm	optuna_trial	0.8393		
E07-lgbm-B-15s-trial-004	E07	B	lgbm	optuna_trial	0.833		
E07-lgbm-B-15s-trial-005	E07	B	lgbm	optuna_trial	0.8362		
E07-lgbm-B-15s-trial-006	E07	B	lgbm	optuna_trial	0.8333		
E07-lgbm-B-15s-trial-007	E07	B	lgbm	optuna_trial	0.8303		
E07-lgbm-B-15s-trial-008	E07	B	lgbm	optuna_trial	0.8392		
E07-lgbm-B-15s-trial-009	E07	B	lgbm	optuna_trial	0.0709		
E07-lgbm-B-15s-trial-010	E07	B	lgbm	optuna_trial	0.8356		
E07-lgbm-B-15s-trial-011	E07	B	lgbm	optuna_trial	0.8362		
E07-lgbm-B-15s-trial-012	E07	B	lgbm	optuna_trial	0.8379		
E07-lgbm-B-15s-trial-013	E07	B	lgbm	optuna_trial	0.8408		
E07-lgbm-B-15s-trial-014	E07	B	lgbm	optuna_trial	0.8421		
E07-lgbm-B-15s-trial-015	E07	B	lgbm	optuna_trial	0.8392		
E07-lgbm-B-15s-trial-016	E07	B	lgbm	optuna_trial	0.8411		
E07-lgbm-B-15s-trial-017	E07	B	lgbm	optuna_trial	0.8408		
E07-lgbm-B-15s-trial-018	E07	B	lgbm	optuna_trial	0.8396		
E07-lgbm-B-15s-trial-019	E07	B	lgbm	optuna_trial	0.8424		
E07-lgbm-B-15s-trial-020	E07	B	lgbm	optuna_trial	0.8414		
E07-lgbm-B-15s-trial-021	E07	B	lgbm	optuna_trial	0.8395		
E07-lgbm-B-15s-trial-022	E07	B	lgbm	optuna_trial	0.8412		
E07-lgbm-B-15s-trial-023	E07	B	lgbm	optuna_trial	0.8418		
E07-lgbm-B-15s-trial-024	E07	B	lgbm	optuna_trial	0.8423		
E07-lgbm-B-15s-trial-025	E07	B	lgbm	optuna_trial	0.8419		
E07-lgbm-B-15s-trial-026	E07	B	lgbm	optuna_trial	0.8426		
E07-lgbm-B-15s-trial-027	E07	B	lgbm	optuna_trial	0.841		
E07-lgbm-B-15s-trial-028	E07	B	lgbm	optuna_trial	0.8425		
E07-lgbm-B-15s-trial-029	E07	B	lgbm	optuna_trial	0.8413		
E07-lgbm-B-15s-trial-030	E07	B	lgbm	optuna_trial	0.8402		
E07-lgbm-B-15s-trial-031	E07	B	lgbm	optuna_trial	0.8412		
E07-lgbm-B-15s-trial-032	E07	B	lgbm	optuna_trial	0.8404		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E07-lgbm-B-15s-trial-033	E07	B	lgbm	optuna_trial	0.8441		
E07-lgbm-B-15s-trial-034	E07	B	lgbm	optuna_trial	0.842		
E07-lgbm-B-15s-trial-035	E07	B	lgbm	optuna_trial	0.8407		
E07-lgbm-B-15s-trial-036	E07	B	lgbm	optuna_trial	0.842		
E07-lgbm-B-15s-trial-037	E07	B	lgbm	optuna_trial	0.8413		
E07-lgbm-B-15s-trial-038	E07	B	lgbm	optuna_trial	0.8429		
E07-lgbm-B-15s-trial-039	E07	B	lgbm	optuna_trial	0.8407		
E07-lgbm-B-15s-trial-040	E07	B	lgbm	optuna_trial	0.8446		
E07-lgbm-B-15s-trial-041	E07	B	lgbm	optuna_trial	0.8411		
E07-lgbm-B-15s-trial-042	E07	B	lgbm	optuna_trial	0.8447		
E07-lgbm-B-15s-trial-043	E07	B	lgbm	optuna_trial	0.8429		
E07-lgbm-B-15s-trial-044	E07	B	lgbm	optuna_trial	0.8433		
E07-lgbm-B-15s-trial-045	E07	B	lgbm	optuna_trial	0.8433		
E07-lgbm-B-15s-trial-046	E07	B	lgbm	optuna_trial	0.8417		
E07-lgbm-B-15s-trial-047	E07	B	lgbm	optuna_trial	0.8426		
E07-lgbm-B-15s-trial-048	E07	B	lgbm	optuna_trial	0.8397		
E07-lgbm-B-15s-trial-049	E07	B	lgbm	optuna_trial	0.844		
E08-lgbm_class_weight-A2-NonVPN-15s	E08	A2-NonVPN	lgbm	holdout	0.9333	0.9049	
E08-lgbm_class_weight-A2-VPN-15s	E08	A2-VPN	lgbm	holdout	0.8893	0.883	
E08-xgb_class_weight-A1-15s	E08	A1	xgb	holdout	0.9249	0.9237	
E08-xgb_class_weight-B-15s	E08	B	xgb	holdout	0.8358	0.8364	
E08-lgbm_oversample-A2-NonVPN-15s	E08	A2-NonVPN	lgbm	holdout	0.931	0.9075	
E08-lgbm_oversample-A2-VPN-15s	E08	A2-VPN	lgbm	holdout	0.8859	0.883	
E08-lgbm_smote-A2-NonVPN-15s	E08	A2-NonVPN	lgbm	holdout	0.9253	0.9046	
E10-xgb_two_step-B-15s	E10	B	xgb_two_step	holdout	0.8274	0.8318	
E17-tree-A1-15s	E17	A1	tree	holdout	0.8915	0.8905	
E17-tree-A1-15s-trial-000	E17	A1	tree	optuna_trial	0.8437		
E17-tree-A1-15s-trial-001	E17	A1	tree	optuna_trial	0.8542		
E17-tree-A1-15s-trial-002	E17	A1	tree	optuna_trial	0.8763		
E17-tree-A1-15s-trial-003	E17	A1	tree	optuna_trial	0.8543		
E17-tree-A1-15s-trial-004	E17	A1	tree	optuna_trial	0.8856		
E17-tree-A1-15s-trial-005	E17	A1	tree	optuna_trial	0.807		
E17-tree-A1-15s-trial-006	E17	A1	tree	optuna_trial	0.883		
E17-tree-A1-15s-trial-007	E17	A1	tree	optuna_trial	0.8164		
E17-tree-A1-15s-trial-008	E17	A1	tree	optuna_trial	0.8768		
E17-tree-A1-15s-trial-009	E17	A1	tree	optuna_trial	0.8654		
E17-tree-A1-15s-trial-010	E17	A1	tree	optuna_trial	0.8868		
E17-tree-A1-15s-trial-011	E17	A1	tree	optuna_trial	0.8112		
E17-tree-A1-15s-trial-012	E17	A1	tree	optuna_trial	0.8781		
E17-tree-A1-15s-trial-013	E17	A1	tree	optuna_trial	0.887		
E17-tree-A1-15s-trial-014	E17	A1	tree	optuna_trial	0.8797		
E17-tree-A1-15s-trial-015	E17	A1	tree	optuna_trial	0.8908		
E17-tree-A1-15s-trial-016	E17	A1	tree	optuna_trial	0.8871		
E17-tree-A1-15s-trial-017	E17	A1	tree	optuna_trial	0.8845		
E17-tree-A1-15s-trial-018	E17	A1	tree	optuna_trial	0.8902		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E17-tree-A1-15s-trial-019	E17	A1	tree	optuna_trial	0.8908		
E17-tree-A1-15s-trial-020	E17	A1	tree	optuna_trial	0.8345		
E17-tree-A1-15s-trial-021	E17	A1	tree	optuna_trial	0.8907		
E17-tree-A1-15s-trial-022	E17	A1	tree	optuna_trial	0.8276		
E17-tree-A1-15s-trial-023	E17	A1	tree	optuna_trial	0.8838		
E17-tree-A1-15s-trial-024	E17	A1	tree	optuna_trial	0.8817		
E17-tree-A1-15s-trial-025	E17	A1	tree	optuna_trial	0.7215		
E17-tree-A1-15s-trial-026	E17	A1	tree	optuna_trial	0.8863		
E17-tree-A1-15s-trial-027	E17	A1	tree	optuna_trial	0.8788		
E17-tree-A1-15s-trial-028	E17	A1	tree	optuna_trial	0.8781		
E17-tree-A1-15s-trial-029	E17	A1	tree	optuna_trial	0.8733		
E17-tree-A1-15s-trial-030	E17	A1	tree	optuna_trial	0.8831		
E17-tree-A1-15s-trial-031	E17	A1	tree	optuna_trial	0.8915		
E17-tree-A1-15s-trial-032	E17	A1	tree	optuna_trial	0.8884		
E17-tree-A1-15s-trial-033	E17	A1	tree	optuna_trial	0.853		
E17-tree-A1-15s-trial-034	E17	A1	tree	optuna_trial	0.8856		
E17-tree-A1-15s-trial-035	E17	A1	tree	optuna_trial	0.8882		
E17-tree-A1-15s-trial-036	E17	A1	tree	optuna_trial	0.8814		
E17-tree-A1-15s-trial-037	E17	A1	tree	optuna_trial	0.8913		
E17-tree-A1-15s-trial-038	E17	A1	tree	optuna_trial	0.8896		
E17-tree-A1-15s-trial-039	E17	A1	tree	optuna_trial	0.8886		
E17-tree-A1-15s-trial-040	E17	A1	tree	optuna_trial	0.8636		
E17-tree-A1-15s-trial-041	E17	A1	tree	optuna_trial	0.8882		
E17-tree-A1-15s-trial-042	E17	A1	tree	optuna_trial	0.8796		
E17-tree-A1-15s-trial-043	E17	A1	tree	optuna_trial	0.8803		
E17-tree-A1-15s-trial-044	E17	A1	tree	optuna_trial	0.8865		
E17-tree-A1-15s-trial-045	E17	A1	tree	optuna_trial	0.8871		
E17-tree-A1-15s-trial-046	E17	A1	tree	optuna_trial	0.8632		
E17-tree-A1-15s-trial-047	E17	A1	tree	optuna_trial	0.832		
E17-tree-A1-15s-trial-048	E17	A1	tree	optuna_trial	0.8173		
E17-tree-A1-15s-trial-049	E17	A1	tree	optuna_trial	0.8869		
E17-tree-A2-VPN-15s	E17	A2-VPN	tree	holdout	0.8129	0.8292	
E17-tree-A2-VPN-15s-trial-000	E17	A2-VPN	tree	optuna_trial	0.7394		
E17-tree-A2-VPN-15s-trial-001	E17	A2-VPN	tree	optuna_trial	0.7104		
E17-tree-A2-VPN-15s-trial-002	E17	A2-VPN	tree	optuna_trial	0.7548		
E17-tree-A2-VPN-15s-trial-003	E17	A2-VPN	tree	optuna_trial	0.7489		
E17-tree-A2-VPN-15s-trial-004	E17	A2-VPN	tree	optuna_trial	0.7994		
E17-tree-A2-VPN-15s-trial-005	E17	A2-VPN	tree	optuna_trial	0.6141		
E17-tree-A2-VPN-15s-trial-006	E17	A2-VPN	tree	optuna_trial	0.7875		
E17-tree-A2-VPN-15s-trial-007	E17	A2-VPN	tree	optuna_trial	0.7293		
E17-tree-A2-VPN-15s-trial-008	E17	A2-VPN	tree	optuna_trial	0.7769		
E17-tree-A2-VPN-15s-trial-009	E17	A2-VPN	tree	optuna_trial	0.7338		
E17-tree-A2-VPN-15s-trial-010	E17	A2-VPN	tree	optuna_trial	0.8005		
E17-tree-A2-VPN-15s-trial-011	E17	A2-VPN	tree	optuna_trial	0.7396		
E17-tree-A2-VPN-15s-trial-012	E17	A2-VPN	tree	optuna_trial	0.7856		
E17-tree-A2-VPN-15s-trial-013	E17	A2-VPN	tree	optuna_trial	0.8009		
E17-tree-A2-VPN-15s-trial-014	E17	A2-VPN	tree	optuna_trial	0.7804		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E17-tree-A2-VPN-15s-trial-015	E17	A2-VPN	tree	optuna_trial	0.8107		
E17-tree-A2-VPN-15s-trial-016	E17	A2-VPN	tree	optuna_trial	0.8096		
E17-tree-A2-VPN-15s-trial-017	E17	A2-VPN	tree	optuna_trial	0.8113		
E17-tree-A2-VPN-15s-trial-018	E17	A2-VPN	tree	optuna_trial	0.7199		
E17-tree-A2-VPN-15s-trial-019	E17	A2-VPN	tree	optuna_trial	0.7899		
E17-tree-A2-VPN-15s-trial-020	E17	A2-VPN	tree	optuna_trial	0.7639		
E17-tree-A2-VPN-15s-trial-021	E17	A2-VPN	tree	optuna_trial	0.7596		
E17-tree-A2-VPN-15s-trial-022	E17	A2-VPN	tree	optuna_trial	0.6506		
E17-tree-A2-VPN-15s-trial-023	E17	A2-VPN	tree	optuna_trial	0.8129		
E17-tree-A2-VPN-15s-trial-024	E17	A2-VPN	tree	optuna_trial	0.7511		
E17-tree-A2-VPN-15s-trial-025	E17	A2-VPN	tree	optuna_trial	0.7721		
E17-tree-A2-VPN-15s-trial-026	E17	A2-VPN	tree	optuna_trial	0.7747		
E17-tree-A2-VPN-15s-trial-027	E17	A2-VPN	tree	optuna_trial	0.8084		
E17-tree-A2-VPN-15s-trial-028	E17	A2-VPN	tree	optuna_trial	0.8069		
E17-tree-A2-VPN-15s-trial-029	E17	A2-VPN	tree	optuna_trial	0.8126		
E17-tree-A2-VPN-15s-trial-030	E17	A2-VPN	tree	optuna_trial	0.8072		
E17-tree-A2-VPN-15s-trial-031	E17	A2-VPN	tree	optuna_trial	0.8101		
E17-tree-A2-VPN-15s-trial-032	E17	A2-VPN	tree	optuna_trial	0.7875		
E17-tree-A2-VPN-15s-trial-033	E17	A2-VPN	tree	optuna_trial	0.8102		
E17-tree-A2-VPN-15s-trial-034	E17	A2-VPN	tree	optuna_trial	0.8069		
E17-tree-A2-VPN-15s-trial-035	E17	A2-VPN	tree	optuna_trial	0.7721		
E17-tree-A2-VPN-15s-trial-036	E17	A2-VPN	tree	optuna_trial	0.8096		
E17-tree-A2-VPN-15s-trial-037	E17	A2-VPN	tree	optuna_trial	0.799		
E17-tree-A2-VPN-15s-trial-038	E17	A2-VPN	tree	optuna_trial	0.7209		
E17-tree-A2-VPN-15s-trial-039	E17	A2-VPN	tree	optuna_trial	0.7969		
E17-tree-A2-VPN-15s-trial-040	E17	A2-VPN	tree	optuna_trial	0.7856		
E17-tree-A2-VPN-15s-trial-041	E17	A2-VPN	tree	optuna_trial	0.7996		
E17-tree-A2-VPN-15s-trial-042	E17	A2-VPN	tree	optuna_trial	0.7931		
E17-tree-A2-VPN-15s-trial-043	E17	A2-VPN	tree	optuna_trial	0.8075		
E17-tree-A2-VPN-15s-trial-044	E17	A2-VPN	tree	optuna_trial	0.806		
E17-tree-A2-VPN-15s-trial-045	E17	A2-VPN	tree	optuna_trial	0.7949		
E17-tree-A2-VPN-15s-trial-046	E17	A2-VPN	tree	optuna_trial	0.8103		
E17-tree-A2-VPN-15s-trial-047	E17	A2-VPN	tree	optuna_trial	0.292		
E17-tree-A2-VPN-15s-trial-048	E17	A2-VPN	tree	optuna_trial	0.8108		
E17-tree-A2-VPN-15s-trial-049	E17	A2-VPN	tree	optuna_trial	0.7787		
E17-tree-A2-NonVPN-15s	E17	A2-NonVPN	tree	holdout	0.8619	0.8294	
E17-tree-A2-NonVPN-15s-trial-000	E17	A2-NonVPN	tree	optuna_trial	0.8089		
E17-tree-A2-NonVPN-15s-trial-001	E17	A2-NonVPN	tree	optuna_trial	0.7294		
E17-tree-A2-NonVPN-15s-trial-002	E17	A2-NonVPN	tree	optuna_trial	0.7882		
E17-tree-A2-NonVPN-15s-trial-003	E17	A2-NonVPN	tree	optuna_trial	0.7819		
E17-tree-A2-NonVPN-15s-trial-004	E17	A2-NonVPN	tree	optuna_trial	0.8401		
E17-tree-A2-NonVPN-15s-trial-005	E17	A2-NonVPN	tree	optuna_trial	0.7194		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E17-tree-A2-NonVPN-15s-trial-006	E17	A2-NonVPN	tree	optuna_trial	0.8249		
E17-tree-A2-NonVPN-15s-trial-007	E17	A2-NonVPN	tree	optuna_trial	0.7402		
E17-tree-A2-NonVPN-15s-trial-008	E17	A2-NonVPN	tree	optuna_trial	0.8412		
E17-tree-A2-NonVPN-15s-trial-009	E17	A2-NonVPN	tree	optuna_trial	0.7731		
E17-tree-A2-NonVPN-15s-trial-010	E17	A2-NonVPN	tree	optuna_trial	0.8393		
E17-tree-A2-NonVPN-15s-trial-011	E17	A2-NonVPN	tree	optuna_trial	0.8582		
E17-tree-A2-NonVPN-15s-trial-012	E17	A2-NonVPN	tree	optuna_trial	0.8582		
E17-tree-A2-NonVPN-15s-trial-013	E17	A2-NonVPN	tree	optuna_trial	0.8591		
E17-tree-A2-NonVPN-15s-trial-014	E17	A2-NonVPN	tree	optuna_trial	0.7981		
E17-tree-A2-NonVPN-15s-trial-015	E17	A2-NonVPN	tree	optuna_trial	0.8372		
E17-tree-A2-NonVPN-15s-trial-016	E17	A2-NonVPN	tree	optuna_trial	0.819		
E17-tree-A2-NonVPN-15s-trial-017	E17	A2-NonVPN	tree	optuna_trial	0.8293		
E17-tree-A2-NonVPN-15s-trial-018	E17	A2-NonVPN	tree	optuna_trial	0.819		
E17-tree-A2-NonVPN-15s-trial-019	E17	A2-NonVPN	tree	optuna_trial	0.7971		
E17-tree-A2-NonVPN-15s-trial-020	E17	A2-NonVPN	tree	optuna_trial	0.7262		
E17-tree-A2-NonVPN-15s-trial-021	E17	A2-NonVPN	tree	optuna_trial	0.8582		
E17-tree-A2-NonVPN-15s-trial-022	E17	A2-NonVPN	tree	optuna_trial	0.8591		
E17-tree-A2-NonVPN-15s-trial-023	E17	A2-NonVPN	tree	optuna_trial	0.8591		
E17-tree-A2-NonVPN-15s-trial-024	E17	A2-NonVPN	tree	optuna_trial	0.8495		
E17-tree-A2-NonVPN-15s-trial-025	E17	A2-NonVPN	tree	optuna_trial	0.8541		
E17-tree-A2-NonVPN-15s-trial-026	E17	A2-NonVPN	tree	optuna_trial	0.8591		
E17-tree-A2-NonVPN-15s-trial-027	E17	A2-NonVPN	tree	optuna_trial	0.8516		
E17-tree-A2-NonVPN-15s-trial-028	E17	A2-NonVPN	tree	optuna_trial	0.8619		
E17-tree-A2-NonVPN-15s-trial-029	E17	A2-NonVPN	tree	optuna_trial	0.5061		
E17-tree-A2-NonVPN-15s-trial-030	E17	A2-NonVPN	tree	optuna_trial	0.8275		
E17-tree-A2-NonVPN-15s-trial-031	E17	A2-NonVPN	tree	optuna_trial	0.8431		
E17-tree-A2-NonVPN-15s-trial-032	E17	A2-NonVPN	tree	optuna_trial	0.8401		
E17-tree-A2-NonVPN-15s-trial-033	E17	A2-NonVPN	tree	optuna_trial	0.8619		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E17-tree-A2-NonVPN-15s-trial-034	E17	A2-NonVPN	tree	optuna_trial	0.8619		
E17-tree-A2-NonVPN-15s-trial-035	E17	A2-NonVPN	tree	optuna_trial	0.8516		
E17-tree-A2-NonVPN-15s-trial-036	E17	A2-NonVPN	tree	optuna_trial	0.8595		
E17-tree-A2-NonVPN-15s-trial-037	E17	A2-NonVPN	tree	optuna_trial	0.8373		
E17-tree-A2-NonVPN-15s-trial-038	E17	A2-NonVPN	tree	optuna_trial	0.8139		
E17-tree-A2-NonVPN-15s-trial-039	E17	A2-NonVPN	tree	optuna_trial	0.8516		
E17-tree-A2-NonVPN-15s-trial-040	E17	A2-NonVPN	tree	optuna_trial	0.8595		
E17-tree-A2-NonVPN-15s-trial-041	E17	A2-NonVPN	tree	optuna_trial	0.8619		
E17-tree-A2-NonVPN-15s-trial-042	E17	A2-NonVPN	tree	optuna_trial	0.8591		
E17-tree-A2-NonVPN-15s-trial-043	E17	A2-NonVPN	tree	optuna_trial	0.8373		
E17-tree-A2-NonVPN-15s-trial-044	E17	A2-NonVPN	tree	optuna_trial	0.8401		
E17-tree-A2-NonVPN-15s-trial-045	E17	A2-NonVPN	tree	optuna_trial	0.8595		
E17-tree-A2-NonVPN-15s-trial-046	E17	A2-NonVPN	tree	optuna_trial	0.8275		
E17-tree-A2-NonVPN-15s-trial-047	E17	A2-NonVPN	tree	optuna_trial	0.8519		
E17-tree-A2-NonVPN-15s-trial-048	E17	A2-NonVPN	tree	optuna_trial	0.8619		
E17-tree-A2-NonVPN-15s-trial-049	E17	A2-NonVPN	tree	optuna_trial	0.8619		
E17-tree-B-15s	E17	B	tree	holdout	0.7575	0.7585	
E17-tree-B-15s-trial-000	E17	B	tree	optuna_trial	0.6589		
E17-tree-B-15s-trial-001	E17	B	tree	optuna_trial	0.6376		
E17-tree-B-15s-trial-002	E17	B	tree	optuna_trial	0.6857		
E17-tree-B-15s-trial-003	E17	B	tree	optuna_trial	0.627		
E17-tree-B-15s-trial-004	E17	B	tree	optuna_trial	0.7337		
E17-tree-B-15s-trial-005	E17	B	tree	optuna_trial	0.5319		
E17-tree-B-15s-trial-006	E17	B	tree	optuna_trial	0.7214		
E17-tree-B-15s-trial-007	E17	B	tree	optuna_trial	0.6222		
E17-tree-B-15s-trial-008	E17	B	tree	optuna_trial	0.7065		
E17-tree-B-15s-trial-009	E17	B	tree	optuna_trial	0.6586		
E17-tree-B-15s-trial-010	E17	B	tree	optuna_trial	0.7332		
E17-tree-B-15s-trial-011	E17	B	tree	optuna_trial	0.5941		
E17-tree-B-15s-trial-012	E17	B	tree	optuna_trial	0.7235		
E17-tree-B-15s-trial-013	E17	B	tree	optuna_trial	0.736		
E17-tree-B-15s-trial-014	E17	B	tree	optuna_trial	0.6971		
E17-tree-B-15s-trial-015	E17	B	tree	optuna_trial	0.573		
E17-tree-B-15s-trial-016	E17	B	tree	optuna_trial	0.7416		
E17-tree-B-15s-trial-017	E17	B	tree	optuna_trial	0.7411		
E17-tree-B-15s-trial-018	E17	B	tree	optuna_trial	0.7575		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E17-tree-B-15s-trial-019	E17	B	tree	optuna_trial	0.7215		
E17-tree-B-15s-trial-020	E17	B	tree	optuna_trial	0.7448		
E17-tree-B-15s-trial-021	E17	B	tree	optuna_trial	0.7569		
E17-tree-B-15s-trial-022	E17	B	tree	optuna_trial	0.7571		
E17-tree-B-15s-trial-023	E17	B	tree	optuna_trial	0.727		
E17-tree-B-15s-trial-024	E17	B	tree	optuna_trial	0.7476		
E17-tree-B-15s-trial-025	E17	B	tree	optuna_trial	0.7457		
E17-tree-B-15s-trial-026	E17	B	tree	optuna_trial	0.6891		
E17-tree-B-15s-trial-027	E17	B	tree	optuna_trial	0.7079		
E17-tree-B-15s-trial-028	E17	B	tree	optuna_trial	0.741		
E17-tree-B-15s-trial-029	E17	B	tree	optuna_trial	0.725		
E17-tree-B-15s-trial-030	E17	B	tree	optuna_trial	0.7336		
E17-tree-B-15s-trial-031	E17	B	tree	optuna_trial	0.7447		
E17-tree-B-15s-trial-032	E17	B	tree	optuna_trial	0.2316		
E17-tree-B-15s-trial-033	E17	B	tree	optuna_trial	0.7358		
E17-tree-B-15s-trial-034	E17	B	tree	optuna_trial	0.6623		
E17-tree-B-15s-trial-035	E17	B	tree	optuna_trial	0.7444		
E17-tree-B-15s-trial-036	E17	B	tree	optuna_trial	0.748		
E17-tree-B-15s-trial-037	E17	B	tree	optuna_trial	0.7355		
E17-tree-B-15s-trial-038	E17	B	tree	optuna_trial	0.7503		
E17-tree-B-15s-trial-039	E17	B	tree	optuna_trial	0.7132		
E17-tree-B-15s-trial-040	E17	B	tree	optuna_trial	0.7359		
E17-tree-B-15s-trial-041	E17	B	tree	optuna_trial	0.7235		
E17-tree-B-15s-trial-042	E17	B	tree	optuna_trial	0.7503		
E17-tree-B-15s-trial-043	E17	B	tree	optuna_trial	0.7353		
E17-tree-B-15s-trial-044	E17	B	tree	optuna_trial	0.7364		
E17-tree-B-15s-trial-045	E17	B	tree	optuna_trial	0.7575		
E17-tree-B-15s-trial-046	E17	B	tree	optuna_trial	0.736		
E17-tree-B-15s-trial-047	E17	B	tree	optuna_trial	0.7444		
E17-tree-B-15s-trial-048	E17	B	tree	optuna_trial	0.7575		
E17-tree-B-15s-trial-049	E17	B	tree	optuna_trial	0.725		
E17-logreg-A1-15s	E17	A1	logreg	holdout	0.6762	0.6517	
E17-logreg-A1-15s-trial-000	E17	A1	logreg	optuna_trial	0.6747		
E17-logreg-A1-15s-trial-001	E17	A1	logreg	optuna_trial	0.5728		
E17-logreg-A1-15s-trial-002	E17	A1	logreg	optuna_trial	0.6745		
E17-logreg-A1-15s-trial-003	E17	A1	logreg	optuna_trial	0.6304		
E17-logreg-A1-15s-trial-004	E17	A1	logreg	optuna_trial	0.574		
E17-logreg-A1-15s-trial-005	E17	A1	logreg	optuna_trial	0.6744		
E17-logreg-A1-15s-trial-006	E17	A1	logreg	optuna_trial	0.6178		
E17-logreg-A1-15s-trial-007	E17	A1	logreg	optuna_trial	0.6671		
E17-logreg-A1-15s-trial-008	E17	A1	logreg	optuna_trial	0.6524		
E17-logreg-A1-15s-trial-009	E17	A1	logreg	optuna_trial	0.6351		
E17-logreg-A1-15s-trial-010	E17	A1	logreg	optuna_trial	0.6752		
E17-logreg-A1-15s-trial-011	E17	A1	logreg	optuna_trial	0.6758		
E17-logreg-A1-15s-trial-012	E17	A1	logreg	optuna_trial	0.675		
E17-logreg-A1-15s-trial-013	E17	A1	logreg	optuna_trial	0.6753		
E17-logreg-A1-15s-trial-014	E17	A1	logreg	optuna_trial	0.6752		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E17-logreg-A1-15s-trial-015	E17	A1	logreg	optuna_trial	0.6296		
E17-logreg-A1-15s-trial-016	E17	A1	logreg	optuna_trial	0.6749		
E17-logreg-A1-15s-trial-017	E17	A1	logreg	optuna_trial	0.586		
E17-logreg-A1-15s-trial-018	E17	A1	logreg	optuna_trial	0.6748		
E17-logreg-A1-15s-trial-019	E17	A1	logreg	optuna_trial	0.675		
E17-logreg-A1-15s-trial-020	E17	A1	logreg	optuna_trial	0.6318		
E17-logreg-A1-15s-trial-021	E17	A1	logreg	optuna_trial	0.6751		
E17-logreg-A1-15s-trial-022	E17	A1	logreg	optuna_trial	0.6755		
E17-logreg-A1-15s-trial-023	E17	A1	logreg	optuna_trial	0.6762		
E17-logreg-A1-15s-trial-024	E17	A1	logreg	optuna_trial	0.6747		
E17-logreg-A1-15s-trial-025	E17	A1	logreg	optuna_trial	0.6753		
E17-logreg-A1-15s-trial-026	E17	A1	logreg	optuna_trial	0.6256		
E17-logreg-A1-15s-trial-027	E17	A1	logreg	optuna_trial	0.6747		
E17-logreg-A1-15s-trial-028	E17	A1	logreg	optuna_trial	0.6747		
E17-logreg-A1-15s-trial-029	E17	A1	logreg	optuna_trial	0.6757		
E17-logreg-A1-15s-trial-030	E17	A1	logreg	optuna_trial	0.6749		
E17-logreg-A1-15s-trial-031	E17	A1	logreg	optuna_trial	0.6757		
E17-logreg-A1-15s-trial-032	E17	A1	logreg	optuna_trial	0.6749		
E17-logreg-A1-15s-trial-033	E17	A1	logreg	optuna_trial	0.6743		
E17-logreg-A1-15s-trial-034	E17	A1	logreg	optuna_trial	0.5711		
E17-logreg-A1-15s-trial-035	E17	A1	logreg	optuna_trial	0.6303		
E17-logreg-A1-15s-trial-036	E17	A1	logreg	optuna_trial	0.6743		
E17-logreg-A1-15s-trial-037	E17	A1	logreg	optuna_trial	0.6748		
E17-logreg-A1-15s-trial-038	E17	A1	logreg	optuna_trial	0.6743		
E17-logreg-A1-15s-trial-039	E17	A1	logreg	optuna_trial	0.6757		
E17-logreg-A1-15s-trial-040	E17	A1	logreg	optuna_trial	0.6745		
E17-logreg-A1-15s-trial-041	E17	A1	logreg	optuna_trial	0.6743		
E17-logreg-A1-15s-trial-042	E17	A1	logreg	optuna_trial	0.6752		
E17-logreg-A1-15s-trial-043	E17	A1	logreg	optuna_trial	0.6316		
E17-logreg-A1-15s-trial-044	E17	A1	logreg	optuna_trial	0.6753		
E17-logreg-A1-15s-trial-045	E17	A1	logreg	optuna_trial	0.6745		
E17-logreg-A1-15s-trial-046	E17	A1	logreg	optuna_trial	0.6743		
E17-logreg-A1-15s-trial-047	E17	A1	logreg	optuna_trial	0.6214		
E17-logreg-A1-15s-trial-048	E17	A1	logreg	optuna_trial	0.6753		
E17-logreg-A1-15s-trial-049	E17	A1	logreg	optuna_trial	0.6759		
E17-logreg-A2-VPN-15s	E17	A2-VPN	logreg	holdout	0.7344	0.7263	
E17-logreg-A2-VPN-15s-trial-000	E17	A2-VPN	logreg	optuna_trial	0.7335		
E17-logreg-A2-VPN-15s-trial-001	E17	A2-VPN	logreg	optuna_trial	0.2499		
E17-logreg-A2-VPN-15s-trial-002	E17	A2-VPN	logreg	optuna_trial	0.7295		
E17-logreg-A2-VPN-15s-trial-003	E17	A2-VPN	logreg	optuna_trial	0.5975		
E17-logreg-A2-VPN-15s-trial-004	E17	A2-VPN	logreg	optuna_trial	0.2706		
E17-logreg-A2-VPN-15s-trial-005	E17	A2-VPN	logreg	optuna_trial	0.7184		
E17-logreg-A2-VPN-15s-trial-006	E17	A2-VPN	logreg	optuna_trial	0.4521		
E17-logreg-A2-VPN-15s-trial-007	E17	A2-VPN	logreg	optuna_trial	0.6643		
E17-logreg-A2-VPN-15s-trial-008	E17	A2-VPN	logreg	optuna_trial	0.5883		
E17-logreg-A2-VPN-15s-trial-009	E17	A2-VPN	logreg	optuna_trial	0.424		
E17-logreg-A2-VPN-15s-trial-010	E17	A2-VPN	logreg	optuna_trial	0.7334		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E17-logreg-A2-VPN-15s-trial-011	E17	A2-VPN	logreg	optuna_trial	0.7325		
E17-logreg-A2-VPN-15s-trial-012	E17	A2-VPN	logreg	optuna_trial	0.7337		
E17-logreg-A2-VPN-15s-trial-013	E17	A2-VPN	logreg	optuna_trial	0.7329		
E17-logreg-A2-VPN-15s-trial-014	E17	A2-VPN	logreg	optuna_trial	0.734		
E17-logreg-A2-VPN-15s-trial-015	E17	A2-VPN	logreg	optuna_trial	0.5583		
E17-logreg-A2-VPN-15s-trial-016	E17	A2-VPN	logreg	optuna_trial	0.7332		
E17-logreg-A2-VPN-15s-trial-017	E17	A2-VPN	logreg	optuna_trial	0.3782		
E17-logreg-A2-VPN-15s-trial-018	E17	A2-VPN	logreg	optuna_trial	0.7333		
E17-logreg-A2-VPN-15s-trial-019	E17	A2-VPN	logreg	optuna_trial	0.7318		
E17-logreg-A2-VPN-15s-trial-020	E17	A2-VPN	logreg	optuna_trial	0.5832		
E17-logreg-A2-VPN-15s-trial-021	E17	A2-VPN	logreg	optuna_trial	0.732		
E17-logreg-A2-VPN-15s-trial-022	E17	A2-VPN	logreg	optuna_trial	0.7344		
E17-logreg-A2-VPN-15s-trial-023	E17	A2-VPN	logreg	optuna_trial	0.7331		
E17-logreg-A2-VPN-15s-trial-024	E17	A2-VPN	logreg	optuna_trial	0.7322		
E17-logreg-A2-VPN-15s-trial-025	E17	A2-VPN	logreg	optuna_trial	0.7233		
E17-logreg-A2-VPN-15s-trial-026	E17	A2-VPN	logreg	optuna_trial	0.7321		
E17-logreg-A2-VPN-15s-trial-027	E17	A2-VPN	logreg	optuna_trial	0.7324		
E17-logreg-A2-VPN-15s-trial-028	E17	A2-VPN	logreg	optuna_trial	0.7325		
E17-logreg-A2-VPN-15s-trial-029	E17	A2-VPN	logreg	optuna_trial	0.6957		
E17-logreg-A2-VPN-15s-trial-030	E17	A2-VPN	logreg	optuna_trial	0.7343		
E17-logreg-A2-VPN-15s-trial-031	E17	A2-VPN	logreg	optuna_trial	0.7344		
E17-logreg-A2-VPN-15s-trial-032	E17	A2-VPN	logreg	optuna_trial	0.7338		
E17-logreg-A2-VPN-15s-trial-033	E17	A2-VPN	logreg	optuna_trial	0.4938		
E17-logreg-A2-VPN-15s-trial-034	E17	A2-VPN	logreg	optuna_trial	0.7303		
E17-logreg-A2-VPN-15s-trial-035	E17	A2-VPN	logreg	optuna_trial	0.4282		
E17-logreg-A2-VPN-15s-trial-036	E17	A2-VPN	logreg	optuna_trial	0.2012		
E17-logreg-A2-VPN-15s-trial-037	E17	A2-VPN	logreg	optuna_trial	0.7333		
E17-logreg-A2-VPN-15s-trial-038	E17	A2-VPN	logreg	optuna_trial	0.7298		
E17-logreg-A2-VPN-15s-trial-039	E17	A2-VPN	logreg	optuna_trial	0.3015		
E17-logreg-A2-VPN-15s-trial-040	E17	A2-VPN	logreg	optuna_trial	0.7323		
E17-logreg-A2-VPN-15s-trial-041	E17	A2-VPN	logreg	optuna_trial	0.7337		
E17-logreg-A2-VPN-15s-trial-042	E17	A2-VPN	logreg	optuna_trial	0.7325		
E17-logreg-A2-VPN-15s-trial-043	E17	A2-VPN	logreg	optuna_trial	0.7312		
E17-logreg-A2-VPN-15s-trial-044	E17	A2-VPN	logreg	optuna_trial	0.7327		
E17-logreg-A2-VPN-15s-trial-045	E17	A2-VPN	logreg	optuna_trial	0.7319		
E17-logreg-A2-VPN-15s-trial-046	E17	A2-VPN	logreg	optuna_trial	0.7234		
E17-logreg-A2-VPN-15s-trial-047	E17	A2-VPN	logreg	optuna_trial	0.71		
E17-logreg-A2-VPN-15s-trial-048	E17	A2-VPN	logreg	optuna_trial	0.5949		
E17-logreg-A2-VPN-15s-trial-049	E17	A2-VPN	logreg	optuna_trial	0.734		
E17-logreg-A2-NonVPN-15s	E17	A2-NonVPN	logreg	holdout	0.8138	0.8002	
E17-logreg-A2-NonVPN-15s-trial-000	E17	A2-NonVPN	logreg	optuna_trial	0.7964		
E17-logreg-A2-NonVPN-15s-trial-001	E17	A2-NonVPN	logreg	optuna_trial	0.344		
E17-logreg-A2-NonVPN-15s-trial-002	E17	A2-NonVPN	logreg	optuna_trial	0.7761		
E17-logreg-A2-NonVPN-15s-trial-003	E17	A2-NonVPN	logreg	optuna_trial	0.5869		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E17-logreg-A2-NonVPN-15s-trial-004	E17	A2-NonVPN	logreg	optuna_trial	0.3562		
E17-logreg-A2-NonVPN-15s-trial-005	E17	A2-NonVPN	logreg	optuna_trial	0.749		
E17-logreg-A2-NonVPN-15s-trial-006	E17	A2-NonVPN	logreg	optuna_trial	0.4762		
E17-logreg-A2-NonVPN-15s-trial-007	E17	A2-NonVPN	logreg	optuna_trial	0.6849		
E17-logreg-A2-NonVPN-15s-trial-008	E17	A2-NonVPN	logreg	optuna_trial	0.5186		
E17-logreg-A2-NonVPN-15s-trial-009	E17	A2-NonVPN	logreg	optuna_trial	0.4561		
E17-logreg-A2-NonVPN-15s-trial-010	E17	A2-NonVPN	logreg	optuna_trial	0.8093		
E17-logreg-A2-NonVPN-15s-trial-011	E17	A2-NonVPN	logreg	optuna_trial	0.8107		
E17-logreg-A2-NonVPN-15s-trial-012	E17	A2-NonVPN	logreg	optuna_trial	0.8095		
E17-logreg-A2-NonVPN-15s-trial-013	E17	A2-NonVPN	logreg	optuna_trial	0.7982		
E17-logreg-A2-NonVPN-15s-trial-014	E17	A2-NonVPN	logreg	optuna_trial	0.8036		
E17-logreg-A2-NonVPN-15s-trial-015	E17	A2-NonVPN	logreg	optuna_trial	0.5311		
E17-logreg-A2-NonVPN-15s-trial-016	E17	A2-NonVPN	logreg	optuna_trial	0.8085		
E17-logreg-A2-NonVPN-15s-trial-017	E17	A2-NonVPN	logreg	optuna_trial	0.7914		
E17-logreg-A2-NonVPN-15s-trial-018	E17	A2-NonVPN	logreg	optuna_trial	0.4399		
E17-logreg-A2-NonVPN-15s-trial-019	E17	A2-NonVPN	logreg	optuna_trial	0.8095		
E17-logreg-A2-NonVPN-15s-trial-020	E17	A2-NonVPN	logreg	optuna_trial	0.8091		
E17-logreg-A2-NonVPN-15s-trial-021	E17	A2-NonVPN	logreg	optuna_trial	0.8105		
E17-logreg-A2-NonVPN-15s-trial-022	E17	A2-NonVPN	logreg	optuna_trial	0.8007		
E17-logreg-A2-NonVPN-15s-trial-023	E17	A2-NonVPN	logreg	optuna_trial	0.8081		
E17-logreg-A2-NonVPN-15s-trial-024	E17	A2-NonVPN	logreg	optuna_trial	0.5553		
E17-logreg-A2-NonVPN-15s-trial-025	E17	A2-NonVPN	logreg	optuna_trial	0.8008		
E17-logreg-A2-NonVPN-15s-trial-026	E17	A2-NonVPN	logreg	optuna_trial	0.8124		
E17-logreg-A2-NonVPN-15s-trial-027	E17	A2-NonVPN	logreg	optuna_trial	0.8023		
E17-logreg-A2-NonVPN-15s-trial-028	E17	A2-NonVPN	logreg	optuna_trial	0.5092		
E17-logreg-A2-NonVPN-15s-trial-029	E17	A2-NonVPN	logreg	optuna_trial	0.7882		
E17-logreg-A2-NonVPN-15s-trial-030	E17	A2-NonVPN	logreg	optuna_trial	0.81		
E17-logreg-A2-NonVPN-15s-trial-031	E17	A2-NonVPN	logreg	optuna_trial	0.8096		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E17-logreg-A2-NonVPN-15s-trial-032	E17	A2-NonVPN	logreg	optuna_trial	0.8093		
E17-logreg-A2-NonVPN-15s-trial-033	E17	A2-NonVPN	logreg	optuna_trial	0.8049		
E17-logreg-A2-NonVPN-15s-trial-034	E17	A2-NonVPN	logreg	optuna_trial	0.5751		
E17-logreg-A2-NonVPN-15s-trial-035	E17	A2-NonVPN	logreg	optuna_trial	0.7992		
E17-logreg-A2-NonVPN-15s-trial-036	E17	A2-NonVPN	logreg	optuna_trial	0.2479		
E17-logreg-A2-NonVPN-15s-trial-037	E17	A2-NonVPN	logreg	optuna_trial	0.8116		
E17-logreg-A2-NonVPN-15s-trial-038	E17	A2-NonVPN	logreg	optuna_trial	0.5868		
E17-logreg-A2-NonVPN-15s-trial-039	E17	A2-NonVPN	logreg	optuna_trial	0.7986		
E17-logreg-A2-NonVPN-15s-trial-040	E17	A2-NonVPN	logreg	optuna_trial	0.7764		
E17-logreg-A2-NonVPN-15s-trial-041	E17	A2-NonVPN	logreg	optuna_trial	0.8101		
E17-logreg-A2-NonVPN-15s-trial-042	E17	A2-NonVPN	logreg	optuna_trial	0.809		
E17-logreg-A2-NonVPN-15s-trial-043	E17	A2-NonVPN	logreg	optuna_trial	0.8043		
E17-logreg-A2-NonVPN-15s-trial-044	E17	A2-NonVPN	logreg	optuna_trial	0.8105		
E17-logreg-A2-NonVPN-15s-trial-045	E17	A2-NonVPN	logreg	optuna_trial	0.8119		
E17-logreg-A2-NonVPN-15s-trial-046	E17	A2-NonVPN	logreg	optuna_trial	0.5401		
E17-logreg-A2-NonVPN-15s-trial-047	E17	A2-NonVPN	logreg	optuna_trial	0.801		
E17-logreg-A2-NonVPN-15s-trial-048	E17	A2-NonVPN	logreg	optuna_trial	0.8138		
E17-logreg-A2-NonVPN-15s-trial-049	E17	A2-NonVPN	logreg	optuna_trial	0.8101		
E17-logreg-B-15s	E17	B	logreg	holdout	0.5909	0.5862	
E17-logreg-B-15s-trial-000	E17	B	logreg	optuna_trial	0.59		
E17-logreg-B-15s-trial-001	E17	B	logreg	optuna_trial	0.1635		
E17-logreg-B-15s-trial-002	E17	B	logreg	optuna_trial	0.5851		
E17-logreg-B-15s-trial-003	E17	B	logreg	optuna_trial	0.368		
E17-logreg-B-15s-trial-004	E17	B	logreg	optuna_trial	0.1684		
E17-logreg-B-15s-trial-005	E17	B	logreg	optuna_trial	0.5704		
E17-logreg-B-15s-trial-006	E17	B	logreg	optuna_trial	0.3125		
E17-logreg-B-15s-trial-007	E17	B	logreg	optuna_trial	0.5152		
E17-logreg-B-15s-trial-008	E17	B	logreg	optuna_trial	0.3907		
E17-logreg-B-15s-trial-009	E17	B	logreg	optuna_trial	0.2975		
E17-logreg-B-15s-trial-010	E17	B	logreg	optuna_trial	0.5903		
E17-logreg-B-15s-trial-011	E17	B	logreg	optuna_trial	0.5896		
E17-logreg-B-15s-trial-012	E17	B	logreg	optuna_trial	0.5895		
E17-logreg-B-15s-trial-013	E17	B	logreg	optuna_trial	0.5901		
E17-logreg-B-15s-trial-014	E17	B	logreg	optuna_trial	0.5906		
E17-logreg-B-15s-trial-015	E17	B	logreg	optuna_trial	0.3309		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E17-logreg-B-15s-trial-016	E17	B	logreg	optuna_trial	0.5892		
E17-logreg-B-15s-trial-017	E17	B	logreg	optuna_trial	0.27		
E17-logreg-B-15s-trial-018	E17	B	logreg	optuna_trial	0.5897		
E17-logreg-B-15s-trial-019	E17	B	logreg	optuna_trial	0.5906		
E17-logreg-B-15s-trial-020	E17	B	logreg	optuna_trial	0.356		
E17-logreg-B-15s-trial-021	E17	B	logreg	optuna_trial	0.5886		
E17-logreg-B-15s-trial-022	E17	B	logreg	optuna_trial	0.5906		
E17-logreg-B-15s-trial-023	E17	B	logreg	optuna_trial	0.5886		
E17-logreg-B-15s-trial-024	E17	B	logreg	optuna_trial	0.5894		
E17-logreg-B-15s-trial-025	E17	B	logreg	optuna_trial	0.3202		
E17-logreg-B-15s-trial-026	E17	B	logreg	optuna_trial	0.5891		
E17-logreg-B-15s-trial-027	E17	B	logreg	optuna_trial	0.5788		
E17-logreg-B-15s-trial-028	E17	B	logreg	optuna_trial	0.5907		
E17-logreg-B-15s-trial-029	E17	B	logreg	optuna_trial	0.5891		
E17-logreg-B-15s-trial-030	E17	B	logreg	optuna_trial	0.5904		
E17-logreg-B-15s-trial-031	E17	B	logreg	optuna_trial	0.5906		
E17-logreg-B-15s-trial-032	E17	B	logreg	optuna_trial	0.5909		
E17-logreg-B-15s-trial-033	E17	B	logreg	optuna_trial	0.5904		
E17-logreg-B-15s-trial-034	E17	B	logreg	optuna_trial	0.1388		
E17-logreg-B-15s-trial-035	E17	B	logreg	optuna_trial	0.5902		
E17-logreg-B-15s-trial-036	E17	B	logreg	optuna_trial	0.5901		
E17-logreg-B-15s-trial-037	E17	B	logreg	optuna_trial	0.589		
E17-logreg-B-15s-trial-038	E17	B	logreg	optuna_trial	0.3663		
E17-logreg-B-15s-trial-039	E17	B	logreg	optuna_trial	0.5617		
E17-logreg-B-15s-trial-040	E17	B	logreg	optuna_trial	0.5901		
E17-logreg-B-15s-trial-041	E17	B	logreg	optuna_trial	0.5894		
E17-logreg-B-15s-trial-042	E17	B	logreg	optuna_trial	0.5898		
E17-logreg-B-15s-trial-043	E17	B	logreg	optuna_trial	0.5896		
E17-logreg-B-15s-trial-044	E17	B	logreg	optuna_trial	0.3451		
E17-logreg-B-15s-trial-045	E17	B	logreg	optuna_trial	0.3015		
E17-logreg-B-15s-trial-046	E17	B	logreg	optuna_trial	0.5844		
E17-logreg-B-15s-trial-047	E17	B	logreg	optuna_trial	0.589		
E17-logreg-B-15s-trial-048	E17	B	logreg	optuna_trial	0.5896		
E17-logreg-B-15s-trial-049	E17	B	logreg	optuna_trial	0.5889		
E17-svm_rbf-A1-15s	E17	A1	svm_rbf	holdout	0.9187	0.9259	
E17-svm_rbf-A1-15s-trial-000	E17	A1	svm_rbf	optuna_trial	0.8839		
E17-svm_rbf-A1-15s-trial-001	E17	A1	svm_rbf	optuna_trial	0.5911		
E17-svm_rbf-A1-15s-trial-002	E17	A1	svm_rbf	optuna_trial	0.6434		
E17-svm_rbf-A1-15s-trial-003	E17	A1	svm_rbf	optuna_trial	0.3585		
E17-svm_rbf-A1-15s-trial-004	E17	A1	svm_rbf	optuna_trial	0.6256		
E17-svm_rbf-A1-15s-trial-005	E17	A1	svm_rbf	optuna_trial	0.5649		
E17-svm_rbf-A1-15s-trial-006	E17	A1	svm_rbf	optuna_trial	0.703		
E17-svm_rbf-A1-15s-trial-007	E17	A1	svm_rbf	optuna_trial	0.5757		
E17-svm_rbf-A1-15s-trial-008	E17	A1	svm_rbf	optuna_trial	0.9113		
E17-svm_rbf-A1-15s-trial-009	E17	A1	svm_rbf	optuna_trial	0.6298		
E17-svm_rbf-A1-15s-trial-010	E17	A1	svm_rbf	optuna_trial	0.9126		
E17-svm_rbf-A1-15s-trial-011	E17	A1	svm_rbf	optuna_trial	0.9145		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E17-svm_rbf-A1-15s-trial-012	E17	A1	svm_rbf	optuna_trial	0.7773		
E17-svm_rbf-A1-15s-trial-013	E17	A1	svm_rbf	optuna_trial	0.9154		
E17-svm_rbf-A1-15s-trial-014	E17	A1	svm_rbf	optuna_trial	0.9122		
E17-svm_rbf-A1-15s-trial-015	E17	A1	svm_rbf	optuna_trial	0.9162		
E17-svm_rbf-A1-15s-trial-016	E17	A1	svm_rbf	optuna_trial	0.9175		
E17-svm_rbf-A1-15s-trial-017	E17	A1	svm_rbf	optuna_trial	0.9046		
E17-svm_rbf-A1-15s-trial-018	E17	A1	svm_rbf	optuna_trial	0.9021		
E17-svm_rbf-A1-15s-trial-019	E17	A1	svm_rbf	optuna_trial	0.9178		
E17-svm_rbf-A1-15s-trial-020	E17	A1	svm_rbf	optuna_trial	0.9166		
E17-svm_rbf-A1-15s-trial-021	E17	A1	svm_rbf	optuna_trial	0.9155		
E17-svm_rbf-A1-15s-trial-022	E17	A1	svm_rbf	optuna_trial	0.7369		
E17-svm_rbf-A1-15s-trial-023	E17	A1	svm_rbf	optuna_trial	0.8666		
E17-svm_rbf-A1-15s-trial-024	E17	A1	svm_rbf	optuna_trial	0.8914		
E17-svm_rbf-A1-15s-trial-025	E17	A1	svm_rbf	optuna_trial	0.9161		
E17-svm_rbf-A1-15s-trial-026	E17	A1	svm_rbf	optuna_trial	0.9151		
E17-svm_rbf-A1-15s-trial-027	E17	A1	svm_rbf	optuna_trial	0.7444		
E17-svm_rbf-A1-15s-trial-028	E17	A1	svm_rbf	optuna_trial	0.7772		
E17-svm_rbf-A1-15s-trial-029	E17	A1	svm_rbf	optuna_trial	0.9173		
E17-svm_rbf-A1-15s-trial-030	E17	A1	svm_rbf	optuna_trial	0.8409		
E17-svm_rbf-A1-15s-trial-031	E17	A1	svm_rbf	optuna_trial	0.8323		
E17-svm_rbf-A1-15s-trial-032	E17	A1	svm_rbf	optuna_trial	0.8959		
E17-svm_rbf-A1-15s-trial-033	E17	A1	svm_rbf	optuna_trial	0.9166		
E17-svm_rbf-A1-15s-trial-034	E17	A1	svm_rbf	optuna_trial	0.9153		
E17-svm_rbf-A1-15s-trial-035	E17	A1	svm_rbf	optuna_trial	0.7443		
E17-svm_rbf-A1-15s-trial-036	E17	A1	svm_rbf	optuna_trial	0.7438		
E17-svm_rbf-A1-15s-trial-037	E17	A1	svm_rbf	optuna_trial	0.7631		
E17-svm_rbf-A1-15s-trial-038	E17	A1	svm_rbf	optuna_trial	0.7054		
E17-svm_rbf-A1-15s-trial-039	E17	A1	svm_rbf	optuna_trial	0.7949		
E17-svm_rbf-A1-15s-trial-040	E17	A1	svm_rbf	optuna_trial	0.9044		
E17-svm_rbf-A1-15s-trial-041	E17	A1	svm_rbf	optuna_trial	0.917		
E17-svm_rbf-A1-15s-trial-042	E17	A1	svm_rbf	optuna_trial	0.7597		
E17-svm_rbf-A1-15s-trial-043	E17	A1	svm_rbf	optuna_trial	0.914		
E17-svm_rbf-A1-15s-trial-044	E17	A1	svm_rbf	optuna_trial	0.9173		
E17-svm_rbf-A1-15s-trial-045	E17	A1	svm_rbf	optuna_trial	0.9104		
E17-svm_rbf-A1-15s-trial-046	E17	A1	svm_rbf	optuna_trial	0.9166		
E17-svm_rbf-A1-15s-trial-047	E17	A1	svm_rbf	optuna_trial	0.9187		
E17-svm_rbf-A1-15s-trial-048	E17	A1	svm_rbf	optuna_trial	0.9175		
E17-svm_rbf-A1-15s-trial-049	E17	A1	svm_rbf	optuna_trial	0.9173		
E17-svm_rbf-A2-VPN-15s	E17	A2-VPN	svm_rbf	holdout	0.8668	0.8707	
E17-svm_rbf-A2-VPN-15s-trial-000	E17	A2-VPN	svm_rbf	optuna_trial	0.8349		
E17-svm_rbf-A2-VPN-15s-trial-001	E17	A2-VPN	svm_rbf	optuna_trial	0.2825		
E17-svm_rbf-A2-VPN-15s-trial-002	E17	A2-VPN	svm_rbf	optuna_trial	0.2331		
E17-svm_rbf-A2-VPN-15s-trial-003	E17	A2-VPN	svm_rbf	optuna_trial	0.1303		
E17-svm_rbf-A2-VPN-15s-trial-004	E17	A2-VPN	svm_rbf	optuna_trial	0.5336		
E17-svm_rbf-A2-VPN-15s-trial-005	E17	A2-VPN	svm_rbf	optuna_trial	0.2371		
E17-svm_rbf-A2-VPN-15s-trial-006	E17	A2-VPN	svm_rbf	optuna_trial	0.53		
E17-svm_rbf-A2-VPN-15s-trial-007	E17	A2-VPN	svm_rbf	optuna_trial	0.2818		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E17-svm_rbf-A2-VPN-15s-trial-008	E17	A2-VPN	svm_rbf	optuna_trial	0.8628		
E17-svm_rbf-A2-VPN-15s-trial-009	E17	A2-VPN	svm_rbf	optuna_trial	0.584		
E17-svm_rbf-A2-VPN-15s-trial-010	E17	A2-VPN	svm_rbf	optuna_trial	0.8607		
E17-svm_rbf-A2-VPN-15s-trial-011	E17	A2-VPN	svm_rbf	optuna_trial	0.8648		
E17-svm_rbf-A2-VPN-15s-trial-012	E17	A2-VPN	svm_rbf	optuna_trial	0.778		
E17-svm_rbf-A2-VPN-15s-trial-013	E17	A2-VPN	svm_rbf	optuna_trial	0.8563		
E17-svm_rbf-A2-VPN-15s-trial-014	E17	A2-VPN	svm_rbf	optuna_trial	0.8283		
E17-svm_rbf-A2-VPN-15s-trial-015	E17	A2-VPN	svm_rbf	optuna_trial	0.8659		
E17-svm_rbf-A2-VPN-15s-trial-016	E17	A2-VPN	svm_rbf	optuna_trial	0.864		
E17-svm_rbf-A2-VPN-15s-trial-017	E17	A2-VPN	svm_rbf	optuna_trial	0.8607		
E17-svm_rbf-A2-VPN-15s-trial-018	E17	A2-VPN	svm_rbf	optuna_trial	0.7837		
E17-svm_rbf-A2-VPN-15s-trial-019	E17	A2-VPN	svm_rbf	optuna_trial	0.8123		
E17-svm_rbf-A2-VPN-15s-trial-020	E17	A2-VPN	svm_rbf	optuna_trial	0.8159		
E17-svm_rbf-A2-VPN-15s-trial-021	E17	A2-VPN	svm_rbf	optuna_trial	0.8651		
E17-svm_rbf-A2-VPN-15s-trial-022	E17	A2-VPN	svm_rbf	optuna_trial	0.8619		
E17-svm_rbf-A2-VPN-15s-trial-023	E17	A2-VPN	svm_rbf	optuna_trial	0.7294		
E17-svm_rbf-A2-VPN-15s-trial-024	E17	A2-VPN	svm_rbf	optuna_trial	0.8633		
E17-svm_rbf-A2-VPN-15s-trial-025	E17	A2-VPN	svm_rbf	optuna_trial	0.7455		
E17-svm_rbf-A2-VPN-15s-trial-026	E17	A2-VPN	svm_rbf	optuna_trial	0.8594		
E17-svm_rbf-A2-VPN-15s-trial-027	E17	A2-VPN	svm_rbf	optuna_trial	0.8004		
E17-svm_rbf-A2-VPN-15s-trial-028	E17	A2-VPN	svm_rbf	optuna_trial	0.7703		
E17-svm_rbf-A2-VPN-15s-trial-029	E17	A2-VPN	svm_rbf	optuna_trial	0.7071		
E17-svm_rbf-A2-VPN-15s-trial-030	E17	A2-VPN	svm_rbf	optuna_trial	0.8352		
E17-svm_rbf-A2-VPN-15s-trial-031	E17	A2-VPN	svm_rbf	optuna_trial	0.8592		
E17-svm_rbf-A2-VPN-15s-trial-032	E17	A2-VPN	svm_rbf	optuna_trial	0.8597		
E17-svm_rbf-A2-VPN-15s-trial-033	E17	A2-VPN	svm_rbf	optuna_trial	0.8653		
E17-svm_rbf-A2-VPN-15s-trial-034	E17	A2-VPN	svm_rbf	optuna_trial	0.8614		
E17-svm_rbf-A2-VPN-15s-trial-035	E17	A2-VPN	svm_rbf	optuna_trial	0.8619		
E17-svm_rbf-A2-VPN-15s-trial-036	E17	A2-VPN	svm_rbf	optuna_trial	0.8668		
E17-svm_rbf-A2-VPN-15s-trial-037	E17	A2-VPN	svm_rbf	optuna_trial	0.863		
E17-svm_rbf-A2-VPN-15s-trial-038	E17	A2-VPN	svm_rbf	optuna_trial	0.8366		
E17-svm_rbf-A2-VPN-15s-trial-039	E17	A2-VPN	svm_rbf	optuna_trial	0.7671		
E17-svm_rbf-A2-VPN-15s-trial-040	E17	A2-VPN	svm_rbf	optuna_trial	0.8442		
E17-svm_rbf-A2-VPN-15s-trial-041	E17	A2-VPN	svm_rbf	optuna_trial	0.8622		
E17-svm_rbf-A2-VPN-15s-trial-042	E17	A2-VPN	svm_rbf	optuna_trial	0.866		
E17-svm_rbf-A2-VPN-15s-trial-043	E17	A2-VPN	svm_rbf	optuna_trial	0.8628		
E17-svm_rbf-A2-VPN-15s-trial-044	E17	A2-VPN	svm_rbf	optuna_trial	0.8647		
E17-svm_rbf-A2-VPN-15s-trial-045	E17	A2-VPN	svm_rbf	optuna_trial	0.7722		
E17-svm_rbf-A2-VPN-15s-trial-046	E17	A2-VPN	svm_rbf	optuna_trial	0.8509		
E17-svm_rbf-A2-VPN-15s-trial-047	E17	A2-VPN	svm_rbf	optuna_trial	0.8651		
E17-svm_rbf-A2-VPN-15s-trial-048	E17	A2-VPN	svm_rbf	optuna_trial	0.856		
E17-svm_rbf-A2-VPN-15s-trial-049	E17	A2-VPN	svm_rbf	optuna_trial	0.8659		
E17-svm_rbf-A2-NonVPN-15s	E17	A2-NonVPN	svm_rbf	holdout	0.9256	0.9058	
E17-svm_rbf-A2-NonVPN-15s-trial-000	E17	A2-NonVPN	svm_rbf	optuna_trial	0.9165		
E17-svm_rbf-A2-NonVPN-15s-trial-001	E17	A2-NonVPN	svm_rbf	optuna_trial	0.3356		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E17-svm_rbf-A2-NonVPN-15s-trial-002	E17	A2-NonVPN	svm_rbf	optuna_trial	0.296		
E17-svm_rbf-A2-NonVPN-15s-trial-003	E17	A2-NonVPN	svm_rbf	optuna_trial	0.1015		
E17-svm_rbf-A2-NonVPN-15s-trial-004	E17	A2-NonVPN	svm_rbf	optuna_trial	0.5193		
E17-svm_rbf-A2-NonVPN-15s-trial-005	E17	A2-NonVPN	svm_rbf	optuna_trial	0.3332		
E17-svm_rbf-A2-NonVPN-15s-trial-006	E17	A2-NonVPN	svm_rbf	optuna_trial	0.5718		
E17-svm_rbf-A2-NonVPN-15s-trial-007	E17	A2-NonVPN	svm_rbf	optuna_trial	0.3743		
E17-svm_rbf-A2-NonVPN-15s-trial-008	E17	A2-NonVPN	svm_rbf	optuna_trial	0.9189		
E17-svm_rbf-A2-NonVPN-15s-trial-009	E17	A2-NonVPN	svm_rbf	optuna_trial	0.5202		
E17-svm_rbf-A2-NonVPN-15s-trial-010	E17	A2-NonVPN	svm_rbf	optuna_trial	0.923		
E17-svm_rbf-A2-NonVPN-15s-trial-011	E17	A2-NonVPN	svm_rbf	optuna_trial	0.9079		
E17-svm_rbf-A2-NonVPN-15s-trial-012	E17	A2-NonVPN	svm_rbf	optuna_trial	0.9256		
E17-svm_rbf-A2-NonVPN-15s-trial-013	E17	A2-NonVPN	svm_rbf	optuna_trial	0.6778		
E17-svm_rbf-A2-NonVPN-15s-trial-014	E17	A2-NonVPN	svm_rbf	optuna_trial	0.7714		
E17-svm_rbf-A2-NonVPN-15s-trial-015	E17	A2-NonVPN	svm_rbf	optuna_trial	0.8797		
E17-svm_rbf-A2-NonVPN-15s-trial-016	E17	A2-NonVPN	svm_rbf	optuna_trial	0.8733		
E17-svm_rbf-A2-NonVPN-15s-trial-017	E17	A2-NonVPN	svm_rbf	optuna_trial	0.9154		
E17-svm_rbf-A2-NonVPN-15s-trial-018	E17	A2-NonVPN	svm_rbf	optuna_trial	0.7177		
E17-svm_rbf-A2-NonVPN-15s-trial-019	E17	A2-NonVPN	svm_rbf	optuna_trial	0.923		
E17-svm_rbf-A2-NonVPN-15s-trial-020	E17	A2-NonVPN	svm_rbf	optuna_trial	0.8467		
E17-svm_rbf-A2-NonVPN-15s-trial-021	E17	A2-NonVPN	svm_rbf	optuna_trial	0.9234		
E17-svm_rbf-A2-NonVPN-15s-trial-022	E17	A2-NonVPN	svm_rbf	optuna_trial	0.7568		
E17-svm_rbf-A2-NonVPN-15s-trial-023	E17	A2-NonVPN	svm_rbf	optuna_trial	0.8914		
E17-svm_rbf-A2-NonVPN-15s-trial-024	E17	A2-NonVPN	svm_rbf	optuna_trial	0.8399		
E17-svm_rbf-A2-NonVPN-15s-trial-025	E17	A2-NonVPN	svm_rbf	optuna_trial	0.9061		
E17-svm_rbf-A2-NonVPN-15s-trial-026	E17	A2-NonVPN	svm_rbf	optuna_trial	0.923		
E17-svm_rbf-A2-NonVPN-15s-trial-027	E17	A2-NonVPN	svm_rbf	optuna_trial	0.9198		
E17-svm_rbf-A2-NonVPN-15s-trial-028	E17	A2-NonVPN	svm_rbf	optuna_trial	0.9186		
E17-svm_rbf-A2-NonVPN-15s-trial-029	E17	A2-NonVPN	svm_rbf	optuna_trial	0.8667		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E17-svm_rbf-A2-NonVPN-15s-trial-030	E17	A2-NonVPN	svm_rbf	optuna_trial	0.9196		
E17-svm_rbf-A2-NonVPN-15s-trial-031	E17	A2-NonVPN	svm_rbf	optuna_trial	0.9226		
E17-svm_rbf-A2-NonVPN-15s-trial-032	E17	A2-NonVPN	svm_rbf	optuna_trial	0.9209		
E17-svm_rbf-A2-NonVPN-15s-trial-033	E17	A2-NonVPN	svm_rbf	optuna_trial	0.9001		
E17-svm_rbf-A2-NonVPN-15s-trial-034	E17	A2-NonVPN	svm_rbf	optuna_trial	0.7396		
E17-svm_rbf-A2-NonVPN-15s-trial-035	E17	A2-NonVPN	svm_rbf	optuna_trial	0.9222		
E17-svm_rbf-A2-NonVPN-15s-trial-036	E17	A2-NonVPN	svm_rbf	optuna_trial	0.9192		
E17-svm_rbf-A2-NonVPN-15s-trial-037	E17	A2-NonVPN	svm_rbf	optuna_trial	0.1841		
E17-svm_rbf-A2-NonVPN-15s-trial-038	E17	A2-NonVPN	svm_rbf	optuna_trial	0.7071		
E17-svm_rbf-A2-NonVPN-15s-trial-039	E17	A2-NonVPN	svm_rbf	optuna_trial	0.5055		
E17-svm_rbf-A2-NonVPN-15s-trial-040	E17	A2-NonVPN	svm_rbf	optuna_trial	0.9182		
E17-svm_rbf-A2-NonVPN-15s-trial-041	E17	A2-NonVPN	svm_rbf	optuna_trial	0.9251		
E17-svm_rbf-A2-NonVPN-15s-trial-042	E17	A2-NonVPN	svm_rbf	optuna_trial	0.9226		
E17-svm_rbf-A2-NonVPN-15s-trial-043	E17	A2-NonVPN	svm_rbf	optuna_trial	0.6402		
E17-svm_rbf-A2-NonVPN-15s-trial-044	E17	A2-NonVPN	svm_rbf	optuna_trial	0.3853		
E17-svm_rbf-A2-NonVPN-15s-trial-045	E17	A2-NonVPN	svm_rbf	optuna_trial	0.9118		
E17-svm_rbf-A2-NonVPN-15s-trial-046	E17	A2-NonVPN	svm_rbf	optuna_trial	0.9173		
E17-svm_rbf-A2-NonVPN-15s-trial-047	E17	A2-NonVPN	svm_rbf	optuna_trial	0.9123		
E17-svm_rbf-A2-NonVPN-15s-trial-048	E17	A2-NonVPN	svm_rbf	optuna_trial	0.9225		
E17-svm_rbf-A2-NonVPN-15s-trial-049	E17	A2-NonVPN	svm_rbf	optuna_trial	0.6303		
E17-svm_rbf-B-15s	E17	B	svm_rbf	holdout	0.8228	0.8242	
E17-svm_rbf-B-15s-trial-000	E17	B	svm_rbf	optuna_trial	0.7868		
E17-svm_rbf-B-15s-trial-001	E17	B	svm_rbf	optuna_trial	0.2026		
E17-svm_rbf-B-15s-trial-002	E17	B	svm_rbf	optuna_trial	0.2505		
E17-svm_rbf-B-15s-trial-003	E17	B	svm_rbf	optuna_trial	0.0619		
E17-svm_rbf-B-15s-trial-004	E17	B	svm_rbf	optuna_trial	0.3729		
E17-svm_rbf-B-15s-trial-005	E17	B	svm_rbf	optuna_trial	0.1369		
E17-svm_rbf-B-15s-trial-006	E17	B	svm_rbf	optuna_trial	0.3824		
E17-svm_rbf-B-15s-trial-007	E17	B	svm_rbf	optuna_trial	0.1832		
E17-svm_rbf-B-15s-trial-008	E17	B	svm_rbf	optuna_trial	0.814		
E17-svm_rbf-B-15s-trial-009	E17	B	svm_rbf	optuna_trial	0.3583		
E17-svm_rbf-B-15s-trial-010	E17	B	svm_rbf	optuna_trial	0.8194		
E17-svm_rbf-B-15s-trial-011	E17	B	svm_rbf	optuna_trial	0.8181		
E17-svm_rbf-B-15s-trial-012	E17	B	svm_rbf	optuna_trial	0.6248		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E17-svm_rbf-B-15s-trial-013	E17	B	svm_rbf	optuna_trial	0.8106		
E17-svm_rbf-B-15s-trial-014	E17	B	svm_rbf	optuna_trial	0.4338		
E17-svm_rbf-B-15s-trial-015	E17	B	svm_rbf	optuna_trial	0.7935		
E17-svm_rbf-B-15s-trial-016	E17	B	svm_rbf	optuna_trial	0.5738		
E17-svm_rbf-B-15s-trial-017	E17	B	svm_rbf	optuna_trial	0.6755		
E17-svm_rbf-B-15s-trial-018	E17	B	svm_rbf	optuna_trial	0.8092		
E17-svm_rbf-B-15s-trial-019	E17	B	svm_rbf	optuna_trial	0.8141		
E17-svm_rbf-B-15s-trial-020	E17	B	svm_rbf	optuna_trial	0.6346		
E17-svm_rbf-B-15s-trial-021	E17	B	svm_rbf	optuna_trial	0.5712		
E17-svm_rbf-B-15s-trial-022	E17	B	svm_rbf	optuna_trial	0.673		
E17-svm_rbf-B-15s-trial-023	E17	B	svm_rbf	optuna_trial	0.8093		
E17-svm_rbf-B-15s-trial-024	E17	B	svm_rbf	optuna_trial	0.8132		
E17-svm_rbf-B-15s-trial-025	E17	B	svm_rbf	optuna_trial	0.7877		
E17-svm_rbf-B-15s-trial-026	E17	B	svm_rbf	optuna_trial	0.8152		
E17-svm_rbf-B-15s-trial-027	E17	B	svm_rbf	optuna_trial	0.7421		
E17-svm_rbf-B-15s-trial-028	E17	B	svm_rbf	optuna_trial	0.6		
E17-svm_rbf-B-15s-trial-029	E17	B	svm_rbf	optuna_trial	0.801		
E17-svm_rbf-B-15s-trial-030	E17	B	svm_rbf	optuna_trial	0.8163		
E17-svm_rbf-B-15s-trial-031	E17	B	svm_rbf	optuna_trial	0.8173		
E17-svm_rbf-B-15s-trial-032	E17	B	svm_rbf	optuna_trial	0.8067		
E17-svm_rbf-B-15s-trial-033	E17	B	svm_rbf	optuna_trial	0.8222		
E17-svm_rbf-B-15s-trial-034	E17	B	svm_rbf	optuna_trial	0.8187		
E17-svm_rbf-B-15s-trial-035	E17	B	svm_rbf	optuna_trial	0.8228		
E17-svm_rbf-B-15s-trial-036	E17	B	svm_rbf	optuna_trial	0.8171		
E17-svm_rbf-B-15s-trial-037	E17	B	svm_rbf	optuna_trial	0.8212		
E17-svm_rbf-B-15s-trial-038	E17	B	svm_rbf	optuna_trial	0.761		
E17-svm_rbf-B-15s-trial-039	E17	B	svm_rbf	optuna_trial	0.7408		
E17-svm_rbf-B-15s-trial-040	E17	B	svm_rbf	optuna_trial	0.6046		
E17-svm_rbf-B-15s-trial-041	E17	B	svm_rbf	optuna_trial	0.8188		
E17-svm_rbf-B-15s-trial-042	E17	B	svm_rbf	optuna_trial	0.813		
E17-svm_rbf-B-15s-trial-043	E17	B	svm_rbf	optuna_trial	0.8105		
E17-svm_rbf-B-15s-trial-044	E17	B	svm_rbf	optuna_trial	0.8141		
E17-svm_rbf-B-15s-trial-045	E17	B	svm_rbf	optuna_trial	0.8154		
E17-svm_rbf-B-15s-trial-046	E17	B	svm_rbf	optuna_trial	0.6198		
E17-svm_rbf-B-15s-trial-047	E17	B	svm_rbf	optuna_trial	0.7031		
E17-svm_rbf-B-15s-trial-048	E17	B	svm_rbf	optuna_trial	0.6737		
E17-svm_rbf-B-15s-trial-049	E17	B	svm_rbf	optuna_trial	0.8166		
E17-lgbm-A1-15s	E17	A1	lgbm	holdout	0.932	0.9308	
E17-lgbm-A1-15s-trial-000	E17	A1	lgbm	optuna_trial	0.9269		
E17-lgbm-A1-15s-trial-001	E17	A1	lgbm	optuna_trial	0.9256		
E17-lgbm-A1-15s-trial-002	E17	A1	lgbm	optuna_trial	0.9281		
E17-lgbm-A1-15s-trial-003	E17	A1	lgbm	optuna_trial	0.9295		
E17-lgbm-A1-15s-trial-004	E17	A1	lgbm	optuna_trial	0.9277		
E17-lgbm-A1-15s-trial-005	E17	A1	lgbm	optuna_trial	0.9206		
E17-lgbm-A1-15s-trial-006	E17	A1	lgbm	optuna_trial	0.9272		
E17-lgbm-A1-15s-trial-007	E17	A1	lgbm	optuna_trial	0.9236		
E17-lgbm-A1-15s-trial-008	E17	A1	lgbm	optuna_trial	0.9291		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E17-lgbm-A1-15s-trial-009	E17	A1	lgbm	optuna_trial	0.9223		
E17-lgbm-A1-15s-trial-010	E17	A1	lgbm	optuna_trial	0.9304		
E17-lgbm-A1-15s-trial-011	E17	A1	lgbm	optuna_trial	0.9285		
E17-lgbm-A1-15s-trial-012	E17	A1	lgbm	optuna_trial	0.9291		
E17-lgbm-A1-15s-trial-013	E17	A1	lgbm	optuna_trial	0.9279		
E17-lgbm-A1-15s-trial-014	E17	A1	lgbm	optuna_trial	0.9291		
E17-lgbm-A1-15s-trial-015	E17	A1	lgbm	optuna_trial	0.9296		
E17-lgbm-A1-15s-trial-016	E17	A1	lgbm	optuna_trial	0.9303		
E17-lgbm-A1-15s-trial-017	E17	A1	lgbm	optuna_trial	0.9293		
E17-lgbm-A1-15s-trial-018	E17	A1	lgbm	optuna_trial	0.929		
E17-lgbm-A1-15s-trial-019	E17	A1	lgbm	optuna_trial	0.9306		
E17-lgbm-A1-15s-trial-020	E17	A1	lgbm	optuna_trial	0.93		
E17-lgbm-A1-15s-trial-021	E17	A1	lgbm	optuna_trial	0.9287		
E17-lgbm-A1-15s-trial-022	E17	A1	lgbm	optuna_trial	0.931		
E17-lgbm-A1-15s-trial-023	E17	A1	lgbm	optuna_trial	0.9283		
E17-lgbm-A1-15s-trial-024	E17	A1	lgbm	optuna_trial	0.9309		
E17-lgbm-A1-15s-trial-025	E17	A1	lgbm	optuna_trial	0.9299		
E17-lgbm-A1-15s-trial-026	E17	A1	lgbm	optuna_trial	0.9295		
E17-lgbm-A1-15s-trial-027	E17	A1	lgbm	optuna_trial	0.9298		
E17-lgbm-A1-15s-trial-028	E17	A1	lgbm	optuna_trial	0.9302		
E17-lgbm-A1-15s-trial-029	E17	A1	lgbm	optuna_trial	0.9311		
E17-lgbm-A1-15s-trial-030	E17	A1	lgbm	optuna_trial	0.9307		
E17-lgbm-A1-15s-trial-031	E17	A1	lgbm	optuna_trial	0.9311		
E17-lgbm-A1-15s-trial-032	E17	A1	lgbm	optuna_trial	0.9308		
E17-lgbm-A1-15s-trial-033	E17	A1	lgbm	optuna_trial	0.9312		
E17-lgbm-A1-15s-trial-034	E17	A1	lgbm	optuna_trial	0.9299		
E17-lgbm-A1-15s-trial-035	E17	A1	lgbm	optuna_trial	0.9307		
E17-lgbm-A1-15s-trial-036	E17	A1	lgbm	optuna_trial	0.9302		
E17-lgbm-A1-15s-trial-037	E17	A1	lgbm	optuna_trial	0.9301		
E17-lgbm-A1-15s-trial-038	E17	A1	lgbm	optuna_trial	0.9296		
E17-lgbm-A1-15s-trial-039	E17	A1	lgbm	optuna_trial	0.931		
E17-lgbm-A1-15s-trial-040	E17	A1	lgbm	optuna_trial	0.9304		
E17-lgbm-A1-15s-trial-041	E17	A1	lgbm	optuna_trial	0.9311		
E17-lgbm-A1-15s-trial-042	E17	A1	lgbm	optuna_trial	0.9293		
E17-lgbm-A1-15s-trial-043	E17	A1	lgbm	optuna_trial	0.9301		
E17-lgbm-A1-15s-trial-044	E17	A1	lgbm	optuna_trial	0.93		
E17-lgbm-A1-15s-trial-045	E17	A1	lgbm	optuna_trial	0.9285		
E17-lgbm-A1-15s-trial-046	E17	A1	lgbm	optuna_trial	0.932		
E17-lgbm-A1-15s-trial-047	E17	A1	lgbm	optuna_trial	0.93		
E17-lgbm-A1-15s-trial-048	E17	A1	lgbm	optuna_trial	0.9307		
E17-lgbm-A1-15s-trial-049	E17	A1	lgbm	optuna_trial	0.9308		
E17-rf-A2-VPN-15s	E17	A2-VPN	rf	holdout	0.8707	0.8704	
E17-rf-A2-VPN-15s-trial-000	E17	A2-VPN	rf	optuna_trial	0.8667		
E17-rf-A2-VPN-15s-trial-001	E17	A2-VPN	rf	optuna_trial	0.8307		
E17-rf-A2-VPN-15s-trial-002	E17	A2-VPN	rf	optuna_trial	0.8483		
E17-rf-A2-VPN-15s-trial-003	E17	A2-VPN	rf	optuna_trial	0.8377		
E17-rf-A2-VPN-15s-trial-004	E17	A2-VPN	rf	optuna_trial	0.8407		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E17-rf-A2-VPN-15s-trial-005	E17	A2-VPN	rf	optuna_trial	0.8556		
E17-rf-A2-VPN-15s-trial-006	E17	A2-VPN	rf	optuna_trial	0.8318		
E17-rf-A2-VPN-15s-trial-007	E17	A2-VPN	rf	optuna_trial	0.861		
E17-rf-A2-VPN-15s-trial-008	E17	A2-VPN	rf	optuna_trial	0.8646		
E17-rf-A2-VPN-15s-trial-009	E17	A2-VPN	rf	optuna_trial	0.8314		
E17-rf-A2-VPN-15s-trial-010	E17	A2-VPN	rf	optuna_trial	0.6957		
E17-rf-A2-VPN-15s-trial-011	E17	A2-VPN	rf	optuna_trial	0.8604		
E17-rf-A2-VPN-15s-trial-012	E17	A2-VPN	rf	optuna_trial	0.8479		
E17-rf-A2-VPN-15s-trial-013	E17	A2-VPN	rf	optuna_trial	0.8497		
E17-rf-A2-VPN-15s-trial-014	E17	A2-VPN	rf	optuna_trial	0.8611		
E17-rf-A2-VPN-15s-trial-015	E17	A2-VPN	rf	optuna_trial	0.8639		
E17-rf-A2-VPN-15s-trial-016	E17	A2-VPN	rf	optuna_trial	0.856		
E17-rf-A2-VPN-15s-trial-017	E17	A2-VPN	rf	optuna_trial	0.8544		
E17-rf-A2-VPN-15s-trial-018	E17	A2-VPN	rf	optuna_trial	0.8478		
E17-rf-A2-VPN-15s-trial-019	E17	A2-VPN	rf	optuna_trial	0.8608		
E17-rf-A2-VPN-15s-trial-020	E17	A2-VPN	rf	optuna_trial	0.8655		
E17-rf-A2-VPN-15s-trial-021	E17	A2-VPN	rf	optuna_trial	0.8625		
E17-rf-A2-VPN-15s-trial-022	E17	A2-VPN	rf	optuna_trial	0.8616		
E17-rf-A2-VPN-15s-trial-023	E17	A2-VPN	rf	optuna_trial	0.8638		
E17-rf-A2-VPN-15s-trial-024	E17	A2-VPN	rf	optuna_trial	0.8618		
E17-rf-A2-VPN-15s-trial-025	E17	A2-VPN	rf	optuna_trial	0.8679		
E17-rf-A2-VPN-15s-trial-026	E17	A2-VPN	rf	optuna_trial	0.8695		
E17-rf-A2-VPN-15s-trial-027	E17	A2-VPN	rf	optuna_trial	0.8691		
E17-rf-A2-VPN-15s-trial-028	E17	A2-VPN	rf	optuna_trial	0.8648		
E17-rf-A2-VPN-15s-trial-029	E17	A2-VPN	rf	optuna_trial	0.865		
E17-rf-A2-VPN-15s-trial-030	E17	A2-VPN	rf	optuna_trial	0.8585		
E17-rf-A2-VPN-15s-trial-031	E17	A2-VPN	rf	optuna_trial	0.8655		
E17-rf-A2-VPN-15s-trial-032	E17	A2-VPN	rf	optuna_trial	0.869		
E17-rf-A2-VPN-15s-trial-033	E17	A2-VPN	rf	optuna_trial	0.8698		
E17-rf-A2-VPN-15s-trial-034	E17	A2-VPN	rf	optuna_trial	0.8698		
E17-rf-A2-VPN-15s-trial-035	E17	A2-VPN	rf	optuna_trial	0.8599		
E17-rf-A2-VPN-15s-trial-036	E17	A2-VPN	rf	optuna_trial	0.8615		
E17-rf-A2-VPN-15s-trial-037	E17	A2-VPN	rf	optuna_trial	0.8625		
E17-rf-A2-VPN-15s-trial-038	E17	A2-VPN	rf	optuna_trial	0.8586		
E17-rf-A2-VPN-15s-trial-039	E17	A2-VPN	rf	optuna_trial	0.8657		
E17-rf-A2-VPN-15s-trial-040	E17	A2-VPN	rf	optuna_trial	0.8538		
E17-rf-A2-VPN-15s-trial-041	E17	A2-VPN	rf	optuna_trial	0.8696		
E17-rf-A2-VPN-15s-trial-042	E17	A2-VPN	rf	optuna_trial	0.8707		
E17-rf-A2-VPN-15s-trial-043	E17	A2-VPN	rf	optuna_trial	0.8696		
E17-rf-A2-VPN-15s-trial-044	E17	A2-VPN	rf	optuna_trial	0.867		
E17-rf-A2-VPN-15s-trial-045	E17	A2-VPN	rf	optuna_trial	0.8704		
E17-rf-A2-VPN-15s-trial-046	E17	A2-VPN	rf	optuna_trial	0.8572		
E17-rf-A2-VPN-15s-trial-047	E17	A2-VPN	rf	optuna_trial	0.8647		
E17-rf-A2-VPN-15s-trial-048	E17	A2-VPN	rf	optuna_trial	0.8553		
E17-rf-A2-VPN-15s-trial-049	E17	A2-VPN	rf	optuna_trial	0.8611		
E17-rf-A2-NonVPN-15s	E17	A2-NonVPN	rf	holdout	0.9063	0.8829	

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E17-rf-A2-NonVPN-15s-trial-000	E17	A2-NonVPN	rf	optuna_trial	0.8989		
E17-rf-A2-NonVPN-15s-trial-001	E17	A2-NonVPN	rf	optuna_trial	0.8642		
E17-rf-A2-NonVPN-15s-trial-002	E17	A2-NonVPN	rf	optuna_trial	0.8755		
E17-rf-A2-NonVPN-15s-trial-003	E17	A2-NonVPN	rf	optuna_trial	0.8743		
E17-rf-A2-NonVPN-15s-trial-004	E17	A2-NonVPN	rf	optuna_trial	0.8703		
E17-rf-A2-NonVPN-15s-trial-005	E17	A2-NonVPN	rf	optuna_trial	0.8933		
E17-rf-A2-NonVPN-15s-trial-006	E17	A2-NonVPN	rf	optuna_trial	0.8682		
E17-rf-A2-NonVPN-15s-trial-007	E17	A2-NonVPN	rf	optuna_trial	0.9041		
E17-rf-A2-NonVPN-15s-trial-008	E17	A2-NonVPN	rf	optuna_trial	0.8986		
E17-rf-A2-NonVPN-15s-trial-009	E17	A2-NonVPN	rf	optuna_trial	0.8695		
E17-rf-A2-NonVPN-15s-trial-010	E17	A2-NonVPN	rf	optuna_trial	0.8856		
E17-rf-A2-NonVPN-15s-trial-011	E17	A2-NonVPN	rf	optuna_trial	0.7952		
E17-rf-A2-NonVPN-15s-trial-012	E17	A2-NonVPN	rf	optuna_trial	0.9052		
E17-rf-A2-NonVPN-15s-trial-013	E17	A2-NonVPN	rf	optuna_trial	0.9046		
E17-rf-A2-NonVPN-15s-trial-014	E17	A2-NonVPN	rf	optuna_trial	0.9053		
E17-rf-A2-NonVPN-15s-trial-015	E17	A2-NonVPN	rf	optuna_trial	0.9049		
E17-rf-A2-NonVPN-15s-trial-016	E17	A2-NonVPN	rf	optuna_trial	0.905		
E17-rf-A2-NonVPN-15s-trial-017	E17	A2-NonVPN	rf	optuna_trial	0.8954		
E17-rf-A2-NonVPN-15s-trial-018	E17	A2-NonVPN	rf	optuna_trial	0.8943		
E17-rf-A2-NonVPN-15s-trial-019	E17	A2-NonVPN	rf	optuna_trial	0.9054		
E17-rf-A2-NonVPN-15s-trial-020	E17	A2-NonVPN	rf	optuna_trial	0.8795		
E17-rf-A2-NonVPN-15s-trial-021	E17	A2-NonVPN	rf	optuna_trial	0.8618		
E17-rf-A2-NonVPN-15s-trial-022	E17	A2-NonVPN	rf	optuna_trial	0.8989		
E17-rf-A2-NonVPN-15s-trial-023	E17	A2-NonVPN	rf	optuna_trial	0.8949		
E17-rf-A2-NonVPN-15s-trial-024	E17	A2-NonVPN	rf	optuna_trial	0.8947		
E17-rf-A2-NonVPN-15s-trial-025	E17	A2-NonVPN	rf	optuna_trial	0.8944		
E17-rf-A2-NonVPN-15s-trial-026	E17	A2-NonVPN	rf	optuna_trial	0.8986		
E17-rf-A2-NonVPN-15s-trial-027	E17	A2-NonVPN	rf	optuna_trial	0.9046		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E17-rf-A2-NonVPN-15s-trial-028	E17	A2-NonVPN	rf	optuna_trial	0.9042		
E17-rf-A2-NonVPN-15s-trial-029	E17	A2-NonVPN	rf	optuna_trial	0.8996		
E17-rf-A2-NonVPN-15s-trial-030	E17	A2-NonVPN	rf	optuna_trial	0.8935		
E17-rf-A2-NonVPN-15s-trial-031	E17	A2-NonVPN	rf	optuna_trial	0.9058		
E17-rf-A2-NonVPN-15s-trial-032	E17	A2-NonVPN	rf	optuna_trial	0.8931		
E17-rf-A2-NonVPN-15s-trial-033	E17	A2-NonVPN	rf	optuna_trial	0.8904		
E17-rf-A2-NonVPN-15s-trial-034	E17	A2-NonVPN	rf	optuna_trial	0.8956		
E17-rf-A2-NonVPN-15s-trial-035	E17	A2-NonVPN	rf	optuna_trial	0.9048		
E17-rf-A2-NonVPN-15s-trial-036	E17	A2-NonVPN	rf	optuna_trial	0.9053		
E17-rf-A2-NonVPN-15s-trial-037	E17	A2-NonVPN	rf	optuna_trial	0.8965		
E17-rf-A2-NonVPN-15s-trial-038	E17	A2-NonVPN	rf	optuna_trial	0.9053		
E17-rf-A2-NonVPN-15s-trial-039	E17	A2-NonVPN	rf	optuna_trial	0.9047		
E17-rf-A2-NonVPN-15s-trial-040	E17	A2-NonVPN	rf	optuna_trial	0.895		
E17-rf-A2-NonVPN-15s-trial-041	E17	A2-NonVPN	rf	optuna_trial	0.899		
E17-rf-A2-NonVPN-15s-trial-042	E17	A2-NonVPN	rf	optuna_trial	0.9056		
E17-rf-A2-NonVPN-15s-trial-043	E17	A2-NonVPN	rf	optuna_trial	0.9063		
E17-rf-A2-NonVPN-15s-trial-044	E17	A2-NonVPN	rf	optuna_trial	0.9063		
E17-rf-A2-NonVPN-15s-trial-045	E17	A2-NonVPN	rf	optuna_trial	0.9057		
E17-rf-A2-NonVPN-15s-trial-046	E17	A2-NonVPN	rf	optuna_trial	0.9044		
E17-rf-A2-NonVPN-15s-trial-047	E17	A2-NonVPN	rf	optuna_trial	0.902		
E17-rf-A2-NonVPN-15s-trial-048	E17	A2-NonVPN	rf	optuna_trial	0.8989		
E17-rf-A2-NonVPN-15s-trial-049	E17	A2-NonVPN	rf	optuna_trial	0.8971		
E17-rf-B-15s	E17	B	rf	holdout	0.827	0.8167	
E17-rf-B-15s-trial-000	E17	B	rf	optuna_trial	0.8232		
E17-rf-B-15s-trial-001	E17	B	rf	optuna_trial	0.78		
E17-rf-B-15s-trial-002	E17	B	rf	optuna_trial	0.802		
E17-rf-B-15s-trial-003	E17	B	rf	optuna_trial	0.795		
E17-rf-B-15s-trial-004	E17	B	rf	optuna_trial	0.7941		
E17-rf-B-15s-trial-005	E17	B	rf	optuna_trial	0.807		
E17-rf-B-15s-trial-006	E17	B	rf	optuna_trial	0.7871		
E17-rf-B-15s-trial-007	E17	B	rf	optuna_trial	0.8128		
E17-rf-B-15s-trial-008	E17	B	rf	optuna_trial	0.8223		

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E17-rf-B-15s-trial-009	E17	B	rf	optuna_trial	0.7873		
E17-rf-B-15s-trial-010	E17	B	rf	optuna_trial	0.604		
E17-rf-B-15s-trial-011	E17	B	rf	optuna_trial	0.8151		
E17-rf-B-15s-trial-012	E17	B	rf	optuna_trial	0.8024		
E17-rf-B-15s-trial-013	E17	B	rf	optuna_trial	0.8005		
E17-rf-B-15s-trial-014	E17	B	rf	optuna_trial	0.8239		
E17-rf-B-15s-trial-015	E17	B	rf	optuna_trial	0.8179		
E17-rf-B-15s-trial-016	E17	B	rf	optuna_trial	0.8249		
E17-rf-B-15s-trial-017	E17	B	rf	optuna_trial	0.8204		
E17-rf-B-15s-trial-018	E17	B	rf	optuna_trial	0.8267		
E17-rf-B-15s-trial-019	E17	B	rf	optuna_trial	0.8152		
E17-rf-B-15s-trial-020	E17	B	rf	optuna_trial	0.5047		
E17-rf-B-15s-trial-021	E17	B	rf	optuna_trial	0.8166		
E17-rf-B-15s-trial-022	E17	B	rf	optuna_trial	0.8117		
E17-rf-B-15s-trial-023	E17	B	rf	optuna_trial	0.8251		
E17-rf-B-15s-trial-024	E17	B	rf	optuna_trial	0.8108		
E17-rf-B-15s-trial-025	E17	B	rf	optuna_trial	0.8251		
E17-rf-B-15s-trial-026	E17	B	rf	optuna_trial	0.8227		
E17-rf-B-15s-trial-027	E17	B	rf	optuna_trial	0.827		
E17-rf-B-15s-trial-028	E17	B	rf	optuna_trial	0.8221		
E17-rf-B-15s-trial-029	E17	B	rf	optuna_trial	0.8055		
E17-rf-B-15s-trial-030	E17	B	rf	optuna_trial	0.8137		
E17-rf-B-15s-trial-031	E17	B	rf	optuna_trial	0.8162		
E17-rf-B-15s-trial-032	E17	B	rf	optuna_trial	0.8128		
E17-rf-B-15s-trial-033	E17	B	rf	optuna_trial	0.8258		
E17-rf-B-15s-trial-034	E17	B	rf	optuna_trial	0.8263		
E17-rf-B-15s-trial-035	E17	B	rf	optuna_trial	0.8235		
E17-rf-B-15s-trial-036	E17	B	rf	optuna_trial	0.8145		
E17-rf-B-15s-trial-037	E17	B	rf	optuna_trial	0.8131		
E17-rf-B-15s-trial-038	E17	B	rf	optuna_trial	0.8265		
E17-rf-B-15s-trial-039	E17	B	rf	optuna_trial	0.811		
E17-rf-B-15s-trial-040	E17	B	rf	optuna_trial	0.8058		
E17-rf-B-15s-trial-041	E17	B	rf	optuna_trial	0.8265		
E17-rf-B-15s-trial-042	E17	B	rf	optuna_trial	0.8261		
E17-rf-B-15s-trial-043	E17	B	rf	optuna_trial	0.8175		
E17-rf-B-15s-trial-044	E17	B	rf	optuna_trial	0.8132		
E17-rf-B-15s-trial-045	E17	B	rf	optuna_trial	0.8248		
E17-rf-B-15s-trial-046	E17	B	rf	optuna_trial	0.8177		
E17-rf-B-15s-trial-047	E17	B	rf	optuna_trial	0.8229		
E17-rf-B-15s-trial-048	E17	B	rf	optuna_trial	0.8126		
E17-rf-B-15s-trial-049	E17	B	rf	optuna_trial	0.826		
E09-only_fiat-A2-VPN-15s	E09	A2-VPN	lgbm	holdout	0.7326	0.7593	
E09-no_active-A1-15s	E09	A1	lgbm	holdout	0.93	0.9296	
E09-no_active-A2-NonVPN-15s	E09	A2-NonVPN	lgbm	holdout	0.9283	0.9093	
E09-no_active-A2-VPN-15s	E09	A2-VPN	lgbm	holdout	0.8895	0.886	
E09-no_active-B-15s	E09	B	xgb	holdout	0.84	0.8479	
E09-no_biat-A1-15s	E09	A1	lgbm	holdout	0.9161	0.9152	

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E09-no_biat-A2-NonVPN-15s	E09	A2-NonVPN	lgbm	holdout	0.924	0.8993	
E09-no_biat-A2-VPN-15s	E09	A2-VPN	lgbm	holdout	0.876	0.8724	
E09-no_biat-B-15s	E09	B	xgb	holdout	0.8189	0.8163	
E09-no_fiat-A1-15s	E09	A1	lgbm	holdout	0.9187	0.9256	
E09-no_fiat-A2-NonVPN-15s	E09	A2-NonVPN	lgbm	holdout	0.926	0.9024	
E09-no_fiat-A2-VPN-15s	E09	A2-VPN	lgbm	holdout	0.8723	0.8651	
E09-no_fiat-B-15s	E09	B	xgb	holdout	0.8266	0.8244	
E09-no_flowiat-A1-15s	E09	A1	lgbm	holdout	0.9222	0.9322	
E09-no_flowiat-A2-NonVPN-15s	E09	A2-NonVPN	lgbm	holdout	0.9189	0.903	
E09-no_flowiat-A2-VPN-15s	E09	A2-VPN	lgbm	holdout	0.8833	0.8752	
E09-no_flowiat-B-15s	E09	B	xgb	holdout	0.8284	0.8262	
E09-no_idle-A1-15s	E09	A1	lgbm	holdout	0.9302	0.9304	
E09-no_idle-A2-NonVPN-15s	E09	A2-NonVPN	lgbm	holdout	0.93	0.9039	
E09-no_idle-A2-VPN-15s	E09	A2-VPN	lgbm	holdout	0.8893	0.8826	
E09-no_idle-B-15s	E09	B	xgb	holdout	0.8405	0.8428	
E09-no_rates_duration-A1-15s	E09	A1	lgbm	holdout	0.9133	0.9185	
E09-no_rates_duration-A2-NonVPN-15s	E09	A2-NonVPN	lgbm	holdout	0.898	0.8774	
E09-no_rates_duration-A2-VPN-15s	E09	A2-VPN	lgbm	holdout	0.8449	0.8435	
E09-no_rates_duration-B-15s	E09	B	xgb	holdout	0.7936	0.7879	
E09-only_active-A1-15s	E09	A1	lgbm	holdout	0.5618	0.5489	
E09-only_active-A2-NonVPN-15s	E09	A2-NonVPN	lgbm	holdout	0.3208	0.3355	
E09-only_active-A2-VPN-15s	E09	A2-VPN	lgbm	holdout	0.3222	0.3252	
E09-only_active-B-15s	E09	B	xgb	holdout	0.2262	0.2138	
E09-only_biat-A1-15s	E09	A1	lgbm	holdout	0.8136	0.8243	
E09-only_biat-A2-NonVPN-15s	E09	A2-NonVPN	lgbm	holdout	0.7407	0.7271	
E09-only_biat-A2-VPN-15s	E09	A2-VPN	lgbm	holdout	0.6958	0.7199	
E09-only_biat-B-15s	E09	B	xgb	holdout	0.6147	0.6098	
E09-only_fiat-A1-15s	E09	A1	lgbm	holdout	0.8111	0.8078	
E09-only_fiat-A2-NonVPN-15s	E09	A2-NonVPN	lgbm	holdout	0.772	0.7535	
E09-only_fiat-B-15s	E09	B	xgb	holdout	0.6399	0.64	
E09-only_flowiat-A1-15s	E09	A1	lgbm	holdout	0.8456	0.8507	
E09-only_flowiat-A2-NonVPN-15s	E09	A2-NonVPN	lgbm	holdout	0.8288	0.8165	
E09-only_flowiat-A2-VPN-15s	E09	A2-VPN	lgbm	holdout	0.769	0.7861	
E09-only_flowiat-B-15s	E09	B	xgb	holdout	0.6926	0.6716	
E09-only_idle-A1-15s	E09	A1	lgbm	holdout	0.5614	0.5505	
E09-only_idle-A2-NonVPN-15s	E09	A2-NonVPN	lgbm	holdout	0.3225	0.3038	
E09-only_idle-A2-VPN-15s	E09	A2-VPN	lgbm	holdout	0.3208	0.3067	
E09-only_idle-B-15s	E09	B	xgb	holdout	0.2118	0.2083	
E09-only_rates_duration-A1-15s	E09	A1	lgbm	holdout	0.8363	0.8392	

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E09-only_rates_duration-A2-NonVPN-15s	E09	A2-NonVPN	lgbm	holdout	0.8396	0.8471	
E09-only_rates_duration-A2-VPN-15s	E09	A2-VPN	lgbm	holdout	0.7661	0.779	
E09-only_rates_duration-B-15s	E09	B	xgb	holdout	0.6886	0.7043	
E09-timing_only-A1-15s	E09	A1	lgbm	holdout	0.914	0.9159	
E09-timing_only-A2-NonVPN-15s	E09	A2-NonVPN	lgbm	holdout	0.9	0.8744	
E09-timing_only-A2-VPN-15s	E09	A2-VPN	lgbm	holdout	0.841	0.849	
E09-timing_only-B-15s	E09	B	xgb	holdout	0.7949	0.7987	
E13-f010-s42-A2-NonVPN-15s	E13	A2-NonVPN	lgbm	holdout	0.7864	0.8137	
E14-seed42-A2-NonVPN-15s	E14	A2-NonVPN	lgbm	holdout	0.9291	0.9331	
E13-f010-s42-A1-15s	E13	A1	lgbm	holdout	0.869	0.8639	
E13-f010-s42-A2-VPN-15s	E13	A2-VPN	lgbm	holdout	0.7587	0.7759	
E13-f010-s42-B-15s	E13	B	xgb	holdout	0.6727	0.6953	
E13-f010-s43-A1-15s	E13	A1	lgbm	holdout	0.8768	0.8702	
E13-f010-s43-A2-NonVPN-15s	E13	A2-NonVPN	lgbm	holdout	0.8291	0.8163	
E13-f010-s43-A2-VPN-15s	E13	A2-VPN	lgbm	holdout	0.7571	0.7862	
E13-f010-s43-B-15s	E13	B	xgb	holdout	0.6695	0.7119	
E13-f010-s44-A1-15s	E13	A1	lgbm	holdout	0.8807	0.8757	
E13-f010-s44-A2-NonVPN-15s	E13	A2-NonVPN	lgbm	holdout	0.8217	0.805	
E13-f010-s44-A2-VPN-15s	E13	A2-VPN	lgbm	holdout	0.7613	0.7776	
E13-f010-s44-B-15s	E13	B	xgb	holdout	0.6817	0.6997	
E13-f010-s45-A1-15s	E13	A1	lgbm	holdout	0.8895	0.8804	
E13-f010-s45-A2-NonVPN-15s	E13	A2-NonVPN	lgbm	holdout	0.8103	0.8286	
E13-f010-s45-A2-VPN-15s	E13	A2-VPN	lgbm	holdout	0.7638	0.7832	
E13-f010-s45-B-15s	E13	B	xgb	holdout	0.6768	0.6906	
E13-f010-s46-A1-15s	E13	A1	lgbm	holdout	0.8637	0.8718	
E13-f010-s46-A2-NonVPN-15s	E13	A2-NonVPN	lgbm	holdout	0.7842	0.8244	
E13-f010-s46-A2-VPN-15s	E13	A2-VPN	lgbm	holdout	0.7717	0.7739	
E13-f010-s46-B-15s	E13	B	xgb	holdout	0.6612	0.6869	
E13-f020-s42-A1-15s	E13	A1	lgbm	holdout	0.8872	0.8897	
E13-f020-s42-A2-NonVPN-15s	E13	A2-NonVPN	lgbm	holdout	0.8761	0.8652	
E13-f020-s42-A2-VPN-15s	E13	A2-VPN	lgbm	holdout	0.8123	0.8324	
E13-f020-s42-B-15s	E13	B	xgb	holdout	0.7366	0.7347	
E13-f020-s43-A1-15s	E13	A1	lgbm	holdout	0.8943	0.8963	
E13-f020-s43-A2-NonVPN-15s	E13	A2-NonVPN	lgbm	holdout	0.8907	0.8443	
E13-f020-s43-A2-VPN-15s	E13	A2-VPN	lgbm	holdout	0.8054	0.8125	
E13-f020-s43-B-15s	E13	B	xgb	holdout	0.7436	0.7609	
E13-f020-s44-A1-15s	E13	A1	lgbm	holdout	0.8931	0.8966	
E13-f020-s44-A2-NonVPN-15s	E13	A2-NonVPN	lgbm	holdout	0.8765	0.8503	
E13-f020-s44-A2-VPN-15s	E13	A2-VPN	lgbm	holdout	0.7975	0.8304	

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E13-f020-s44-B-15s	E13	B	xgb	holdout	0.752	0.7545	
E13-f020-s45-A1-15s	E13	A1	lgbm	holdout	0.9134	0.8929	
E13-f020-s45-A2-NonVPN-15s	E13	A2-NonVPN	lgbm	holdout	0.8804	0.8587	
E13-f020-s45-A2-VPN-15s	E13	A2-VPN	lgbm	holdout	0.8153	0.8365	
E13-f020-s45-B-15s	E13	B	xgb	holdout	0.7607	0.7295	
E13-f020-s46-A1-15s	E13	A1	lgbm	holdout	0.8944	0.9005	
E13-f020-s46-A2-NonVPN-15s	E13	A2-NonVPN	lgbm	holdout	0.8631	0.8741	
E13-f020-s46-A2-VPN-15s	E13	A2-VPN	lgbm	holdout	0.8004	0.8183	
E13-f020-s46-B-15s	E13	B	xgb	holdout	0.7614	0.7543	
E13-f040-s42-A1-15s	E13	A1	lgbm	holdout	0.9046	0.9085	
E13-f040-s42-A2-NonVPN-15s	E13	A2-NonVPN	lgbm	holdout	0.9046	0.874	
E13-f040-s42-A2-VPN-15s	E13	A2-VPN	lgbm	holdout	0.8555	0.8552	
E13-f040-s42-B-15s	E13	B	xgb	holdout	0.7813	0.7969	
E13-f040-s43-A1-15s	E13	A1	lgbm	holdout	0.9161	0.9026	
E13-f040-s43-A2-NonVPN-15s	E13	A2-NonVPN	lgbm	holdout	0.8984	0.889	
E13-f040-s43-A2-VPN-15s	E13	A2-VPN	lgbm	holdout	0.845	0.8616	
E13-f040-s43-B-15s	E13	B	xgb	holdout	0.7898	0.7947	
E13-f040-s44-A1-15s	E13	A1	lgbm	holdout	0.9091	0.9207	
E13-f040-s44-A2-NonVPN-15s	E13	A2-NonVPN	lgbm	holdout	0.9097	0.8811	
E13-f040-s44-A2-VPN-15s	E13	A2-VPN	lgbm	holdout	0.855	0.8567	
E13-f040-s44-B-15s	E13	B	xgb	holdout	0.7967	0.8032	
E13-f040-s45-A1-15s	E13	A1	lgbm	holdout	0.9101	0.9096	
E13-f040-s45-A2-NonVPN-15s	E13	A2-NonVPN	lgbm	holdout	0.8874	0.877	
E13-f040-s45-A2-VPN-15s	E13	A2-VPN	lgbm	holdout	0.8366	0.8634	
E13-f040-s45-B-15s	E13	B	xgb	holdout	0.7987	0.7944	
E13-f040-s46-A1-15s	E13	A1	lgbm	holdout	0.9075	0.9118	
E13-f040-s46-A2-NonVPN-15s	E13	A2-NonVPN	lgbm	holdout	0.9088	0.8819	
E13-f040-s46-A2-VPN-15s	E13	A2-VPN	lgbm	holdout	0.8577	0.8541	
E13-f040-s46-B-15s	E13	B	xgb	holdout	0.796	0.7985	
E13-f060-s42-A1-15s	E13	A1	lgbm	holdout	0.9218	0.9188	
E13-f060-s42-A2-NonVPN-15s	E13	A2-NonVPN	lgbm	holdout	0.9218	0.8928	
E13-f060-s42-A2-VPN-15s	E13	A2-VPN	lgbm	holdout	0.8639	0.881	
E13-f060-s42-B-15s	E13	B	xgb	holdout	0.8039	0.8172	
E13-f060-s43-A1-15s	E13	A1	lgbm	holdout	0.9186	0.9204	
E13-f060-s43-A2-NonVPN-15s	E13	A2-NonVPN	lgbm	holdout	0.9189	0.8988	
E13-f060-s43-A2-VPN-15s	E13	A2-VPN	lgbm	holdout	0.8646	0.8704	
E13-f060-s43-B-15s	E13	B	xgb	holdout	0.8197	0.817	
E13-f060-s44-A1-15s	E13	A1	lgbm	holdout	0.9194	0.9218	
E13-f060-s44-A2-NonVPN-15s	E13	A2-NonVPN	lgbm	holdout	0.9241	0.8894	
E13-f060-s44-A2-VPN-15s	E13	A2-VPN	lgbm	holdout	0.86	0.8727	

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E13-f060-s44-B-15s	E13	B	xgb	holdout	0.8215	0.8137	
E13-f060-s45-A1-15s	E13	A1	lgbm	holdout	0.9227	0.9177	
E13-f060-s45-A2-NonVPN-15s	E13	A2-NonVPN	lgbm	holdout	0.9242	0.8887	
E13-f060-s45-A2-VPN-15s	E13	A2-VPN	lgbm	holdout	0.8589	0.8688	
E13-f060-s45-B-15s	E13	B	xgb	holdout	0.8141	0.816	
E13-f060-s46-A1-15s	E13	A1	lgbm	holdout	0.9143	0.9278	
E13-f060-s46-A2-NonVPN-15s	E13	A2-NonVPN	lgbm	holdout	0.9106	0.8971	
E13-f060-s46-A2-VPN-15s	E13	A2-VPN	lgbm	holdout	0.8695	0.8677	
E13-f060-s46-B-15s	E13	B	xgb	holdout	0.8173	0.8142	
E13-f080-s42-A1-15s	E13	A1	lgbm	holdout	0.9263	0.9244	
E13-f080-s42-A2-NonVPN-15s	E13	A2-NonVPN	lgbm	holdout	0.9295	0.8953	
E13-f080-s42-A2-VPN-15s	E13	A2-VPN	lgbm	holdout	0.8728	0.8806	
E13-f080-s42-B-15s	E13	B	xgb	holdout	0.8291	0.8342	
E13-f080-s43-A1-15s	E13	A1	lgbm	holdout	0.9236	0.9244	
E13-f080-s43-A2-NonVPN-15s	E13	A2-NonVPN	lgbm	holdout	0.9328	0.8994	
E13-f080-s43-A2-VPN-15s	E13	A2-VPN	lgbm	holdout	0.8794	0.8771	
E13-f080-s43-B-15s	E13	B	xgb	holdout	0.8287	0.8312	
E13-f080-s44-A1-15s	E13	A1	lgbm	holdout	0.928	0.9274	
E13-f080-s44-A2-NonVPN-15s	E13	A2-NonVPN	lgbm	holdout	0.9324	0.9031	
E13-f080-s44-A2-VPN-15s	E13	A2-VPN	lgbm	holdout	0.8713	0.8854	
E13-f080-s44-B-15s	E13	B	xgb	holdout	0.8303	0.8278	
E13-f080-s45-A1-15s	E13	A1	lgbm	holdout	0.9262	0.9315	
E13-f080-s45-A2-NonVPN-15s	E13	A2-NonVPN	lgbm	holdout	0.9296	0.9012	
E13-f080-s45-A2-VPN-15s	E13	A2-VPN	lgbm	holdout	0.8761	0.882	
E13-f080-s45-B-15s	E13	B	xgb	holdout	0.8234	0.8274	
E13-f080-s46-A1-15s	E13	A1	lgbm	holdout	0.9222	0.927	
E13-f080-s46-A2-NonVPN-15s	E13	A2-NonVPN	lgbm	holdout	0.9273	0.9033	
E13-f080-s46-A2-VPN-15s	E13	A2-VPN	lgbm	holdout	0.8791	0.8766	
E13-f080-s46-B-15s	E13	B	xgb	holdout	0.8327	0.828	
E14-seed42-A1-15s	E14	A1	lgbm	holdout	0.9313	0.9311	
E14-seed42-A2-VPN-15s	E14	A2-VPN	lgbm	holdout	0.8818	0.8803	
E14-seed42-B-15s	E14	B	xgb	holdout	0.8354	0.8543	
E14-seed43-A1-15s	E14	A1	lgbm	holdout	0.9315	0.9367	
E14-seed43-A2-NonVPN-15s	E14	A2-NonVPN	lgbm	holdout	0.9315	0.9257	
E14-seed43-A2-VPN-15s	E14	A2-VPN	lgbm	holdout	0.8774	0.8887	
E14-seed43-B-15s	E14	B	xgb	holdout	0.8372	0.8406	
E14-seed44-A1-15s	E14	A1	lgbm	holdout	0.9317	0.9345	
E14-seed44-A2-NonVPN-15s	E14	A2-NonVPN	lgbm	holdout	0.9301	0.9268	
E14-seed44-A2-VPN-15s	E14	A2-VPN	lgbm	holdout	0.8742	0.8924	
E14-seed44-B-15s	E14	B	xgb	holdout	0.8329	0.8471	
E14-seed45-A1-15s	E14	A1	lgbm	holdout	0.9317	0.9275	

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E14-seed45-A2-NonVPN-15s	E14	A2-NonVPN	lgbm	holdout	0.9307	0.9279	
E14-seed45-A2-VPN-15s	E14	A2-VPN	lgbm	holdout	0.8776	0.902	
E14-seed45-B-15s	E14	B	xgb	holdout	0.8351	0.8422	
E14-seed46-A1-15s	E14	A1	lgbm	holdout	0.9285	0.9289	
E14-seed46-A2-NonVPN-15s	E14	A2-NonVPN	lgbm	holdout	0.9282	0.9288	
E14-seed46-A2-VPN-15s	E14	A2-VPN	lgbm	holdout	0.888	0.8696	
E14-seed46-B-15s	E14	B	xgb	holdout	0.8325	0.8636	
E14-seed47-A1-15s	E14	A1	lgbm	holdout	0.9285	0.9397	
E14-seed47-A2-NonVPN-15s	E14	A2-NonVPN	lgbm	holdout	0.9259	0.9468	
E14-seed47-A2-VPN-15s	E14	A2-VPN	lgbm	holdout	0.8841	0.8912	
E14-seed47-B-15s	E14	B	xgb	holdout	0.8362	0.8437	
E14-seed48-A1-15s	E14	A1	lgbm	holdout	0.9328	0.9356	
E14-seed48-A2-NonVPN-15s	E14	A2-NonVPN	lgbm	holdout	0.9238	0.939	
E14-seed48-A2-VPN-15s	E14	A2-VPN	lgbm	holdout	0.8862	0.8689	
E14-seed48-B-15s	E14	B	xgb	holdout	0.8371	0.8552	
E14-seed49-A1-15s	E14	A1	lgbm	holdout	0.9291	0.9374	
E14-seed49-A2-NonVPN-15s	E14	A2-NonVPN	lgbm	holdout	0.926	0.9265	
E14-seed49-A2-VPN-15s	E14	A2-VPN	lgbm	holdout	0.8789	0.8782	
E14-seed49-B-15s	E14	B	xgb	holdout	0.8329	0.8447	
E14-seed50-A1-15s	E14	A1	lgbm	holdout	0.931	0.9296	
E14-seed50-A2-NonVPN-15s	E14	A2-NonVPN	lgbm	holdout	0.9276	0.9395	
E14-seed50-A2-VPN-15s	E14	A2-VPN	lgbm	holdout	0.8766	0.8941	
E14-seed50-B-15s	E14	B	xgb	holdout	0.8404	0.8344	
E14-seed51-A1-15s	E14	A1	lgbm	holdout	0.9298	0.9371	
E14-seed51-A2-NonVPN-15s	E14	A2-NonVPN	lgbm	holdout	0.9301	0.9305	
E14-seed51-A2-VPN-15s	E14	A2-VPN	lgbm	holdout	0.8802	0.8966	
E14-seed51-B-15s	E14	B	xgb	holdout	0.8378	0.8507	
E16-per_timeout-xgb-120s	E16	kaggle	xgb	kaggle_per_timeout			
E16-isnan-rf	E16	kaggle	rf	kaggle_isnan			
E16-per_timeout-rf-120s	E16	kaggle	rf	kaggle_per_timeout			
E16-per_timeout-rf-15s	E16	kaggle	rf	kaggle_per_timeout			
E16-per_timeout-rf-30s	E16	kaggle	rf	kaggle_per_timeout			
E16-per_timeout-rf-60s	E16	kaggle	rf	kaggle_per_timeout			
E16-per_timeout-xgb-15s	E16	kaggle	xgb	kaggle_per_timeout			
E16-per_timeout-xgb-30s	E16	kaggle	xgb	kaggle_per_timeout			
E16-per_timeout-xgb-60s	E16	kaggle	xgb	kaggle_per_timeout			
E16-size-full	E16	kaggle	rf	kaggle_isnan			
E16-size-10910-s42	E16	kaggle	rf	kaggle_isnan			
E16-size-10910-s43	E16	kaggle	rf	kaggle_isnan			
E16-size-10910-s44	E16	kaggle	rf	kaggle_isnan			
E16-size-10910-s45	E16	kaggle	rf	kaggle_isnan			
E16-size-10910-s46	E16	kaggle	rf	kaggle_isnan			

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E16-size-11783-s42	E16	kaggle	rf	kaggle_isnan			
E16-size-11783-s43	E16	kaggle	rf	kaggle_isnan			
E16-size-11783-s44	E16	kaggle	rf	kaggle_isnan			
E16-size-11783-s45	E16	kaggle	rf	kaggle_isnan			
E16-size-11783-s46	E16	kaggle	rf	kaggle_isnan			
E16-size-14459-s42	E16	kaggle	rf	kaggle_isnan			
E16-size-14459-s43	E16	kaggle	rf	kaggle_isnan			
E16-size-14459-s44	E16	kaggle	rf	kaggle_isnan			
E16-size-14459-s45	E16	kaggle	rf	kaggle_isnan			
E16-size-14459-s46	E16	kaggle	rf	kaggle_isnan			
E16-size-7671-s42	E16	kaggle	rf	kaggle_isnan			
E16-size-7671-s43	E16	kaggle	rf	kaggle_isnan			
E16-size-7671-s44	E16	kaggle	rf	kaggle_isnan			
E16-size-7671-s45	E16	kaggle	rf	kaggle_isnan			
E16-size-7671-s46	E16	kaggle	rf	kaggle_isnan			
E15-A1-15s	E15	A1	lgbm	calibration		0.9308	
E15-A1-15s	E15	A1	lgbm	calibration		0.9308	
E15-A2-VPN-15s	E15	A2-VPN	lgbm	calibration		0.883	
E15-A2-NonVPN-15s	E15	A2-NonVPN	lgbm	calibration		0.9075	
E15-B-15s	E15	B	xgb	calibration		0.8428	
E12-compare-decision_tree-A1	E12	A1	tree	final_test	0.8983		0.8982
E12-compare-knn-A1	E12	A1	knn_log_standard	final_test	0.9174		0.9208
E12-compare-logistic_regression-A1	E12	A1	logreg	final_test	0.673		0.6804
E12-compare-random_forest-A1	E12	A1	rf	final_test	0.9199		0.9289
E12-compare-svm_rbf-A1	E12	A1	svm_rbf	final_test	0.9222		0.9311
E12-compare-xgboost-A1	E12	A1	xgb	final_test	0.929		0.9304
E12-compare-decision_tree-A2-NonVPN	E12	A2-NonVPN	tree	final_test	0.8686		0.8825
E12-compare-knn-A2-NonVPN	E12	A2-NonVPN	knn_log_standard	final_test	0.9091		0.916
E12-compare-logistic_regression-A2-NonVPN	E12	A2-NonVPN	logreg	final_test	0.7992		0.8053
E12-compare-random_forest-A2-NonVPN	E12	A2-NonVPN	rf	final_test	0.908		0.9067
E12-compare-svm_rbf-A2-NonVPN	E12	A2-NonVPN	svm_rbf	final_test	0.923		0.9244
E12-compare-xgboost-A2-NonVPN	E12	A2-NonVPN	xgb	final_test	0.9293		0.9346
E12-compare-decision_tree-A2-VPN	E12	A2-VPN	tree	final_test	0.8219		0.8308
E12-compare-knn-A2-VPN	E12	A2-VPN	knn_log_standard	final_test	0.8577		0.8431
E12-compare-logistic_regression-A2-VPN	E12	A2-VPN	logreg	final_test	0.7275		0.7179
E12-compare-random_forest-A2-VPN	E12	A2-VPN	rf	final_test	0.8688		0.8594
E12-compare-svm_rbf-A2-VPN	E12	A2-VPN	svm_rbf	final_test	0.8672		0.867
E12-compare-xgboost-A2-VPN	E12	A2-VPN	xgb	final_test	0.882		0.8577
E12-compare-decision_tree-B	E12	B	tree	final_test	0.7574		0.7886
E12-compare-knn-B	E12	B	knn_log_standard	final_test	0.811		0.8253

run	experiment	task	model	protocol	CV or 10-fold macro F1	val macro F1	test macro F1
E12-compare-lightgbm-B	E12	B	lgbm	final_test	0.8501		0.8567
E12-compare-logistic_regression-B	E12	B	logreg	final_test	0.5885		0.6089
E12-compare-random_forest-B	E12	B	rf	final_test	0.8359		0.8403
E12-compare-svm_rbf-B	E12	B	svm_rbf	final_test	0.8248		0.8411
E12-final-A1	E12	A1	lgbm	final_test	0.9322		0.9434
E12-final-A2-NonVPN	E12	A2-NonVPN	lgbm	final_test	0.9347		0.9403
E12-final-A2-VPN	E12	A2-VPN	lgbm	final_test	0.8827		0.8652
E12-final-B	E12	B	xgb	final_test	0.8473		0.848
E12-old_rule-A1	E12	A1	knn_log_standard	final_test	0.9078		0.9112
E12-old_rule-A2-VPN	E12	A2-VPN	knn_log_standard	final_test	0.8444		0.8442

Every parameter, metric and tag of each of these runs, and every text file saved with a run, are kept with the project's files.